BESCHREIBUNG

Assemblerdokumentation

Cross-Assembler c82ass: Installation, Sprachdefinition, Anweisungen, Makros und Compilierungsbeispiel

Dieses PDF gibt die Quelle im Wortlaut wieder; Listen, Tabellen und Codeblöcke entsprechen der Quelle, weich umbrochene Zeilen sind zu Absätzen zusammengeführt.

Quelle	`C8205_Assembler_Dokumentation.md`	1.03-023
Stand	10.11.2026	
Umfang	96 Seiten, A4 hoch	
Erzeugt mit	`erzeuge_assemblerdokumentation_pdf.py`	

> Maßgeblich bleibt die Quelldatei. Dieses PDF wird nicht automatisch nachgeführt: Es ist aktuell, solange die Version oben mit der Versionszeile der Quelldatei übereinstimmt.

Inhaltsverzeichnis

Cross-Assembler für C8205 / C8205Z – Version 1.03-023 . . . 4
Inhaltsverzeichnis . . . 4
1. Installation . . . 5
1.1 Verzeichnisstruktur . . . 6
2. Überblick . . . 7
3. Der Assembler c82ass . . . 9
3.1 Programmstart . . . 9
3.1a Schalter --ABSPR (absolut adressierter KEP-Lochstreifen) . . . 12
3.1b Vorspann der .test-Datei (Testmarken im Quelltext) . . . 12
3.1c Nachspann der .test-Datei (Prüfungs-Testmarken im Quelltext) . . . 14
3.1d Schalter --pass0 (nur Pass 0, Ausgabe zur Weiterverarbeitung) . . . 14
3.1e Befehlsworte im Lochstreifen (;#PRECOMMAND, ;#POSTCOMMAND) . . . 15
3.2 Sprachdefinition . . . 17
3.3 Assembleranweisungen . . . 18
3.3.1 Definiere Bereich (db) . . . 19
3.3.2 Definiere Bereich auf Null (dbn) . . . 19
3.3.3 Definiere Wert (dw) . . . 19
3.3.4 Definiere Adresse (da) . . . 19
3.3.5 Definiere Text zur Ausgabe von Zeichen (dt, dto, dtm) . . . 20
3.3.6 Definiere Zeichen (dz) . . . 20
3.3.7 Beginn des Quellcodes (start) . . . 20
3.3.8 Definition Relativadresse (equ) . . . 21
3.3.9 Definition Absolutadresse (abs) . . . 21
3.3.10 Definition Absolute Sektoradresse (spa) . . . 21
3.3.11 Ende des verschieblichen Teils des Quelltextes (end) . . . 21
3.3.12 Beginn einer Folge zusammenhängender Anweisungen (begin) . . . 22
3.3.13 Ende einer zusammenhängenden Folge von Anweisungen (bend) . . . 22
3.3.14 Komprimierter Text: 4 Ausgabezeichen in einem Maschinenwort (kat4) . . . 22
3.3.15 Komprimierter Text: 5 Ausgabezeichen in einem Maschinenwort (kat5) . . . 23
3.3.16 Pragma LEITADRESSE . . . 24
3.3.17 opsym und Namensdatei: Namen für Operationscodes . . . 25
3.3.18 Komprimierter Text: 5 Zeichen im Eingabecode der Kodetabelle in einem Maschinenwort . . . 27
3.3.19 Definiere Wert binär (dwb) . . . 29
3.3.19a Definiere Wert hexadezimal (dwh) . . . 29
3.3.20 Definiere Festkommazahl (gebrochen) (dwf) . . . 30
3.3.21 Definiere Gleitkommazahl (dwg) . . . 31
Tabelle 2: Sonderzeichen für die dz-Anweisung . . . 32
Tabelle 3: Ausgabe-Umschlüsselung außer Sonderzeichen dz . . . 33
4. Der Makro-Präprozessor des Assemblers . . . 34
4.1 Makros im Quelltext . . . 36
4.2 Rechenausdrücke in Makros ($CALC, $CALCO, $TYPZAHL) . . . 38
4.3 Schleife im Makro ($REPT, $ENDREPT, $Z1 bis $Z4) . . . 40
4.4 Geschachtelte Bedingungen ($IF, $ELIF, $ELSE, $ENDIF) . . . 40
4.5 Typ $TYPSYM und Meldungen aus dem Makro ($ERROR, $WARN) . . . 41
4.6 Vorgabewerte und Parameteranzahl ($DEFAULT, $#) . . . 42
4.7 Makro-Variablen ($SET, $INC, $V) . . . 42
4.8 Erweiterte Rechenausdrücke (min, max, Vergleiche, $CALCOn) . . . 43
4.9 Projektspezifische Makrodateien (;#MACRODATEI1= bis ;#MACRODATEI9=) . . . 43
4.10 Zeichenketten-Parameter (Text in Anführungszeichen, $TYPTEXT, $LENn) . . . 45
4.11 Makroliste (--makros) . . . 45
4.12 Zeichenketten in Makro-Variablen ($SET name "text", $V(#name)) . . . 46
4.13 Variable Parameterzahl (MACRO n+, $P(i)) und Herkunft im Listing . . . 46
4.14 Makrodateien zusammensetzen ($INCLUDE) . . . 47
4.15 Reihenfolge der Auflösung, Rekursion und Operation aus Parameter . . . 48
4.16 Wiederholungsbefehle im Makro (!$wdhl und !$wdhl_adr_e) . . . 48

4.17 Felder ($DIM, $SET name(i), $V(name(i))) . . . 49
4.18 Symbole in Rechenausdrücken (abs, equ) . . . 50
4.19 Schleife mit Abbruchbedingung und vorzeitiges Verlassen ($WHILE, $ENDWHILE, $EXIT) . . . 51
4.20 Lokale Variablen und Felder ($LOCAL) . . . 53
4.21 Zeichenkettenfunktionen ($SUBSTR, $INDEX, $UPPER, $LOWER, $STRCMP, $ORD, $CHR, $VAL) . . . 54
4.22 Zahl und Text, dezimal und oktal ($VALO, $CALC im Text) . . . 56
4.23 Attributabfragen (def, art, zellen, adr, abstand, $PROBE) . . . 58
4.24 Hilfen zur Fehlersuche im Makro ($NOTE, $TRACE, $DUMP) . . . 67
4.25 Existenzabfrage für Variablen, Felder und Makros ($DEFINED, $MDEFINED) . . . 71
4.26 Benannte Parameter ($name statt $1 bis $9) . . . 73
4.27 Binärzahlen im Makroprozessor ($CALCB, $CALCBn, 0b, $VALB) . . . 75
4.28 Hexadezimalzahlen im Makroprozessor ($CALCH, $CALCHn, 0x, $VALH) . . . 77
4.29 Aufrufnummer im Makroprozessor ($AUFRUFNR) . . . 79
4.30 Schlüsselwortparameter im Makro (name=vorgabe) . . . 81
4.31 Makro-Variablen standardmäßig lokal (;#MAKROLOKAL=1, $GLOBAL) . . . 83
4.32 Spezifikationszeile und gemeinsame Blöcke im Makro ($SPEC, $BLOCK) . . . 85

5. Der Optimierer des Assemblers . . . 88
5.1 Freie Zellen . . . 88
5.2 Umordnung der Literale . . . 89
5.3 Wirkung auf Ausgaben und Laufzeit . . . 90
5.4 Blöcke begin – bend und Blockverschiebung (RELOC-EQU, RELOC-ALL, FILL) . . . 90
5.5 Noch nicht festgelegt . . . 92
5.6 Umordnung der Makro-Rümpfe (TYP=M, ;#MAKROUMORDNEN) . . . 92

6. Compilierungsbeispiel . . . 95
6.1 Quelltext bsp_5_16.asm und Assemblerlauf . . . 95
6.2 Durch c82ass erzeugter Objektcode (K1K2-Programmcode) für die Eingabe mit Kennzeichen (Format des Minimalprogramms) . . . 96

Version: 1.03-023

Diese Datei ist eine Textableitung von `C8205_Assembler_Dokumentation.odt` in der Version 1.03-023, gedacht zum Durchsuchen und Zitieren. **Maßgeblich bleibt die .odt** (siehe `C8205-Projektstruktur.md`, Abschnitte 1 und 3); bei Widersprüchen gilt die .odt. Diese Ableitung wird nicht automatisch aktualisiert, wenn die .odt sich ändert.

Cross-Assembler für C8205 / C8205Z – Version 1.03-023

Inhaltsverzeichnis

1. Installation
- 1.1 Verzeichnisstruktur

2. Überblick

3. Der Assembler c82ass
- 3.1 Programmstart
- 3.1a Schalter --ABSPR (absolut adressierter KEP-Lochstreifen)
- 3.1b Vorspann der .test-Datei (Testmarken im Quelltext)
- 3.1c Nachspann der .test-Datei (Prüfungs-Testmarken im Quelltext)
- 3.1d Schalter --pass0 (nur Pass 0, Ausgabe zur Weiterverarbeitung)
- 3.1e Befehlsworte im Lochstreifen (;#PRECOMMAND, ;#POSTCOMMAND)
- 3.2 Sprachdefinition
- 3.3 Assembleranweisungen
  - 3.3.1 Definiere Bereich (db)
  - 3.3.2 Definiere Bereich auf Null (dbn)
  - 3.3.3 Definiere Wert (dw)
  - 3.3.4 Definiere Adresse (da)
  - 3.3.5 Definiere Text zur Ausgabe von Zeichen (dt, dto, dtm)
  - 3.3.6 Definiere Zeichen (dz)
  - 3.3.7 Beginn des Quellcodes (start)
  - 3.3.8 Definition Relativadresse (equ)
  - 3.3.9 Definition Absolutadresse (abs)
  - 3.3.10 Definition Absolute Sektoradresse (spa)
  - 3.3.11 Ende des verschieblichen Teils des Quelltextes (end)
  - 3.3.12 Beginn einer Folge zusammenhängender Anweisungen (begin)
  - 3.3.13 Ende einer zusammenhängenden Folge von Anweisungen (bend)
  - 3.3.14 Komprimierter Text: 4 Ausgabezeichen in einem Maschinenwort (kat4)
  - 3.3.15 Komprimierter Text: 5 Ausgabezeichen in einem Maschinenwort (kat5)
  - 3.3.16 Pragma LEITADRESSE
  - 3.3.17 opsym und Namensdatei: Namen für Operationscodes
  - 3.3.18 Komprimierter Text: 5 Zeichen im Eingabecode der Kodetabelle in einem Maschinenwort
  - 3.3.19 Definiere Wert binär (dwb)
  - 3.3.19a Definiere Wert hexadezimal (dwh)
  - 3.3.20 Definiere Festkommazahl (gebrochen) (dwf)
  - 3.3.21 Definiere Gleitkommazahl (dwg)

4. Der Makro-Präprozessor des Assemblers
- 4.1 Makros im Quelltext
- 4.2 Rechenausdrücke in Makros ($CALC, $CALCO, $TYPZAHL)
- 4.3 Schleife im Makro ($REPT, $ENDREPT, $Z1 bis $Z4)
- 4.4 Geschachtelte Bedingungen ($IF, $ELIF, $ELSE, $ENDIF)

- 4.5 Typ $TYPSYM und Meldungen aus dem Makro ($ERROR, $WARN)
- 4.6 Vorgabewerte und Parameteranzahl ($DEFAULT, $#)
- 4.7 Makro-Variablen ($SET, $INC, $V)
- 4.8 Erweiterte Rechenausdrücke (min, max, Vergleiche, $CALCOn)
- 4.9 Projektspezifische Makrodateien
- 4.10 Zeichenketten-Parameter (Text in Anführungszeichen, $TYPTEXT, $LENn)
- 4.11 Makroliste (--makros)
- 4.12 Zeichenketten in Makro-Variablen ($SET name "text", $V(#name))
- 4.13 Variable Parameterzahl (MACRO n+, $P(i)) und Herkunft im Listing
- 4.14 Makrodateien zusammensetzen ($INCLUDE)
- 4.15 Reihenfolge der Auflösung, Rekursion und Operation aus Parameter
- 4.16 Wiederholungsbefehle im Makro (!$wdhl und !$wdhl_adr_e)
- 4.17 Felder ($DIM, $SET name(i), $V(name(i)))
- 4.18 Symbole in Rechenausdrücken (abs, equ)
- 4.19 Schleife mit Abbruchbedingung und vorzeitiges Verlassen ($WHILE, $ENDWHILE, $EXIT)
- 4.20 Lokale Variablen und Felder ($LOCAL)
- 4.21 Zeichenkettenfunktionen ($SUBSTR, $INDEX, $UPPER, $LOWER, $STRCMP, $ORD, $CHR, $VAL)
- 4.22 Zahl und Text, dezimal und oktal ($VALO, $CALC im Text)
- 4.23 Attributabfragen (def, art, zellen, adr, abstand, $PROBE)
- 4.24 Hilfen zur Fehlersuche im Makro ($NOTE, $TRACE, $DUMP)
- 4.25 Existenzabfrage für Variablen, Felder und Makros ($DEFINED, $MDEFINED)
- 4.26 Benannte Parameter ($name statt $1 bis $9)
- 4.27 Binärzahlen im Makroprozessor ($CALCB, $CALCBn, 0b, $VALB)
- 4.28 Hexadezimalzahlen im Makroprozessor ($CALCH, $CALCHn, 0x, $VALH)
- 4.29 Aufrufnummer im Makroprozessor ($AUFRUFNR)
- 4.30 Schlüsselwortparameter im Makro (name=vorgabe)
- 4.31 Makro-Variablen standardmäßig lokal (;#MAKROLOKAL, $GLOBAL)
- 4.32 Spezifikationszeile und gemeinsame Blöcke im Makro ($SPEC, $BLOCK)

5. Der Optimierer des Assemblers
- 5.1 Freie Zellen
- 5.2 Umordnung der Literale
- 5.3 Wirkung auf Ausgaben und Laufzeit
- 5.4 Blöcke begin – bend und Blockverschiebung (RELOC-EQU, RELOC-ALL, FILL)
- 5.5 Noch nicht festgelegt
- 5.6 Umordnung der Makro-Rümpfe (TYP=M, ;#MAKROUMORDNEN)

6. Compilierungsbeispiel
- 6.1 Quelltext bsp_5_16.asm und Assemblerlauf
- 6.2 Durch c82ass erzeugter Objektcode (K1K2-Programmcode) für die Eingabe mit Kennzeichen (Format des Minimalprogramms)

Die gesamte Software läuft sowohl unter Linux als auch Windows 10. Sie ist in C (gcc) programmiert.

1. Installation

Zum Zweck der Installation ist die Datei `C8205.zip` in einem geeigneten Verzeichnis zu entpacken. Beim Entpacken wird ein Unterverzeichnis angelegt, in dem sich sowohl Windows- als auch Linux-Binaries befinden.

1.1 Verzeichnisstruktur

Der Assembler liegt im Unterverzeichnis `Assembler` des Gesamtprojekts C8205. Es enthält folgende Struktur (Beispielprojekte gekürzt):

```
make_c82ass.sh                # Skript zum Erzeugen des Assemblers unter Linux
c82ass.sh                     # Ausführen des Assemblers
asm_alle.sh                   # assembliert alle .asm-Dateien unterhalb eines Verzeichnisses (meldet nur Fehler)
                                asm_alle.sh [Basisverzeichnis] assembliert alle .asm-Dateien
                                unterhalb eines Verzeichnisses (Vorgabe: ./) mit c82ass.sh. Bei
                                fehlerfreiem Lauf erscheint keine Meldung; bei einem Fehler wird die
                                Datei mit den Fehlerzeilen des Assemblers gemeldet.
                                Rückgabewert: 0 = alles fehlerfrei, 1 = Fehler
asm_aufraeumen.sh             # entfernt .log, .mqo, .obj und .sym (Trockenlauf; --loeschen löscht)
make_c82ass.bat               # Batch-Datei zum Erzeugen des Assemblers unter Windows (Pendant zu make_c82ass.sh)
Dokumentation                 # diese Dokumentation
_Linux                        # Linux-Binary des Assemblers
projekdetails                 # zuletzt benutzte Aufrufparameter
projekdetails.<Projekt>       # gesicherte Aufrufparameter je Projekt
Projekt_1                     # Projektverzeichnis (Beispiel, mit Unterverzeichnissen)
Sourcecode                    # C-Quellcode des Assemblers selbst
_Windows                      # Windows-Binary des Assemblers
```

```
./Assembler/Dokumentation:
C8205_Assembler_Dokumentation.md   C8205_Assembler_Dokumentation.odt   C8205_Assembler_Dokumentation.pdf

./Assembler/_Linux:
c82ass

./Assembler/Projekt_1:
… die Projekte, die zu einem Quelltext alle erzeugten Dateien zusammenhalten
```

Projektverzeichnisse können beliebig benannt sein, auch Unterverzeichnisse enthalten. In ihnen steht der Quelltext, die erzeugte Objektdatei (ein binär codierter R300-Lochstreifen, siehe 3.1) und weitere Hilfsdateien. Das Programm `c82ass` erwartet seine Dateien immer in einem Projektunterverzeichnis, das im gleichen Pfad wie es selbst liegen muss.

```
./Assembler/Sourcecode:
alle .c und .h Dateien des Assemblerquelltextes und die Makrodatei c82ass.makros

./Assembler/_Linux:
Der Pfad _Linux enthält alle Linux-Binaries und Libraries des Assemblers

./Assembler/_Windows:
Das Windows-Binary des Assemblers
```

Für die Prüfung von Assemblerergebnissen im Interpreter dient die Datei `.test` (s. `C8205-Interpreter-Durchführung_eines_Tests.md`, Abschnitt 2a).

`asm_aufraeumen.sh` entfernt die erzeugten Dateien `.log`, `.mqo`, `.obj` und `.sym` unterhalb von `Assembler` (nicht in `Dokumentation`, `Sourcecode`, `_Linux`, `_Windows`); ohne Parameter wird nur angezeigt, was gelöscht würde (Trockenlauf), mit `--loeschen` wird tatsächlich gelöscht. Beide Skripte (`asm_alle.sh` und `asm_aufraeumen.sh`) gehören in das Verzeichnis `Assembler`.

2. Überblick

Sowohl das Programm `c82ass.sh` (Linux) als auch das Windows-Binary `_Windows\c82ass.exe` (die Datei `make_c82ass.bat` erzeugt es nur) werden über Aufrufparameter versorgt. Bei jedem Start eines der Programme werden die jeweils letztbenutzten Parameter automatisch in der Datei `projekdetails` abgelegt, sodass ein Folgestart ohne Angabe von Parametern gelingt. Zur Änderung müssen die Parameter einfach mit veränderten Werten wieder angegeben werden.

```
$ c82ass.sh -h
C8205 Assembler C82aas Version 1.03-011
Usage: ./_Linux/c82ass -p Projektverzeichnis -s Quellcode-Datei [-o Objektdatei] [-t Testdatei] [-m Makrodatei]
    [-WARN=SYM,SPR,TOT,ALLE,NOSLIT] [--xref] [--makros] [--opsyms] [--ABSPR] [--no-opt] [--pass0] [--color |
    --no-color]
       ./_Linux/c82ass -h | --help | --version

  -p Projektverzeichnis  Verzeichnis mit dem Quelltext; dorthin werden auch die Ausgabedateien
                         (.log .obj .mqo .sym .dmp .test) geschrieben. Ohne -p gilt der zuletzt
                         benutzte Wert aus der Datei 'projekdetails' im aktuellen Verzeichnis.
  -s Quellcode-Datei     Assembler-Quelltext im Projektverzeichnis; die Endung .asm darf entfallen.
                         Ohne -s gilt ebenfalls der zuletzt benutzte Wert aus 'projekdetails'.
  -o Pfad | Datei        Ablageort der .obj-Datei statt des Projektverzeichnisses. Verzeichnis
                         (existiert, oder endet auf / bzw. \): Dateiname aus -s mit Endung .obj.
                         Sonst Dateiname mit oder ohne Pfad (relativ zum aktuellen Verzeichnis),
                         unveraendert uebernommen. Das Verzeichnis muss existieren.
  -t Pfad | Datei        wie -o, aber fuer die .test-Datei (Endung .test).
  -m Makrodatei          Pfad der Makrodatei statt 'c82ass.makros' (sonst: aktuelles Verzeichnis,
                         ersatzweise 'Sourcecode/'). Mit -m wird nur dieser Pfad benutzt.
  -WARN=liste            Zusaetzliche Warnungen; Werte durch Komma getrennt (Gross-/Kleinschreibung egal):
                           SYM  Symbole, die definiert, aber nirgends verwendet werden.
                           SPR  Sprungbefehle, deren Ziel ein Datenwort (dw, dwb, dwh, dwf, dwg, da) oder ein
    unbelegtes Wort ist.
                           TOT  Befehle hinter einem unbedingten Sprung, auf die nichts verweist.
                           ALLE alle drei.
                           NOSLIT schaltet die (sonst immer eingeschaltete) Warnung ab, dass ein Symbol-Literal !x
    in einem
                                Rechenausdruck steht; gehoert nicht zu ALLE.
                         Die Meldungen stehen auch im .log (hinter der Symboltabelle).
  --xref                 Gibt nach fehlerfreiem Lauf eine Querverweisliste aus: je Symbol Wert,
                         Definitionszeile und alle Zeilen des Quelltexts, die es verwenden. Sie steht auch im .log.
  --makros               Gibt nach fehlerfreiem Lauf die Makroliste aus: je aufgerufenem Makro die Zahl der
                         Aufrufe und die Herkunft (Quelltext, c82ass.makros oder Projektdatei mit Zeile);
                         dazu nicht aufgerufene Makros aus Quelltext und Projektdateien. Auch im .log.
  ;#MACRODATEI1= bis ;#MACRODATEI9=<Pfad>
                         Anweisung im Quelltext (keine Option): projektspezifische Makrodatei; Pfad relativ
                         zur Quelldatei. Suchreihenfolge: Quelltext, Datei 1 bis 9, c82ass.makros.
  ;#MAKROLOKAL=1         Anweisung im Quelltext (keine Option, nur einmal): $SET und $DIM in einem Makroaufruf
    legen lokale
                         Variablen und Felder an (wie beim IBM-Makroassembler); $GLOBAL <name> vereinbart global.
    Ohne die Zeile
                         (oder mit =0) sind sie global wie bisher.
  ;#MAKROUMORDNEN=NO     Anweisung im Quelltext (keine Option, nur einmal): schaltet die Umordnung der
    Makro-Rumpfbloecke ab.
                         Vorgabe ist an: der Optimierer verlegt die Rumpfbloecke der Makros mit $SPEC TYP=M hinter
    end und
                         packt sie dicht (groesste zuerst), wenn die Endadresse dadurch kleiner wird; =YES nennt
```

```
  die Vorgabe.
;#PRECOMMAND=<Wert>    Anweisung im Quelltext (keine Option, mehrfach erlaubt): <Wert> (Ziffern und Punkte,
  endet mit Punkt, z. B. 212000.) wird
                       als Lochbilder vor das erste Wort des .obj-Streifens geschrieben, mehrere in der
  Reihenfolge des Quelltexts.
;#POSTCOMMAND=<Wert>   wie PRECOMMAND, aber hinter das letzte Wort, noch vor das Endekennzeichen /. Bei --ABSPR
  ohne Wirkung.
--opsyms               Gibt die Standardtabelle der Namen fuer Operationscodes (c82ass.opsym) aus, ohne
  Assemblerlauf:
                       je Name Code, Triaden B-A-Z, Nummer der Einzelbefehlsuebersicht und Bedeutung.
                       Im Quelltext schreibt man einen Namen mit vorangestelltem Punkt im Operationscode-Feld
  (.LD).
;#OPSYM=<Pfad>         Anweisung im Quelltext (keine Option, bis 9 Zeilen): weitere Namensdatei; Pfad relativ
  zur Quelldatei.
                       Im Quelltext selbst vereinbart  NAME: opsym <code>  einen Namen ab dieser Zeile.
--ABSPR                Statt der .obj-Datei einen absolut adressierten, binaer im R300-Code codierten
                       Lochstreifen <name>.kep fuer das Kleine Eingabeprogramm (KEP) erzeugen:
                       Leitbefehl <start> 240: und Worte als Dualziffern (auch --ABSOLUTPROGRAMM, --ASBPR).
--no-opt, --NOOPT      Optimierer ausschalten: Literale bleiben hinter dem ersten end, dw-Zeilen an
                       ihrer Adresse (sonst werden Literale in freie Zellen vor dem Programmende verlegt);
                       keine Blockverschiebung (begin RELOC-EQU, RELOC-ALL), die Bloecke bleiben an ihrer Stelle.
--pass0                Nur Pass 0 (Makroaufloesung) ausfuehren, ohne Pass 1 bis 3, ohne Ausgabedateien
  (vorhandene bleiben
                       liegen), ohne Berichte (-WARN, --xref, --makros). Farblos; alles, was sonst auf STDERR
  steht, geht auf
                       STDOUT, die uebrige Ausgabe entfaellt. Es fehlen die Kopfmeldungen (Projektverzeichnis,
  Benutze fuer ...),
                       und bei $NOTE steht nur der Text, ohne "Hinweis: Makro <Name> (<Datei>:<Zeile>): ".
--probeausgabe         Zeigt die Ausgabe von $NOTE, $TRACE und $DUMP auch fuer die Probelaeufe von adr und
  abstand (sonst
                       erscheint nur die des eigentlichen Laufs): je Zeile "Probelauf <n>: " davor. Ohne
  adr/abstand ohne Wirkung.
--color, --no-color    Farbige Ausgabe erzwingen bzw. abschalten. Ohne Angabe nur, wenn STDOUT und
                       STDERR ein Terminal sind und die Umgebungsvariable NO_COLOR fehlt.
-h, --help             Diese Hilfe ausgeben.
--version              Die Versionsnummer ausgeben (steht auch in der ersten Ausgabezeile).
```

`c82ass` muss aus dem Verzeichnis `Assembler` gestartet werden; dort werden auch `projekdetails` und (zuerst) `c82ass.makros` gesucht (sofern nicht mit `-m` ein Pfad angegeben ist).

- `-p` nennt das Verzeichnis, in dem die `.asm`-Datei **direkt** liegt (z. B. `Projekt_1/bsp_5.16`). Unterverzeichnisse werden nicht durchsucht; ein Pfadanteil bei `-s` wird abgeschnitten.
- Hat der Name bei `-s` keine Endung aus genau drei Zeichen, wird `.asm` angehängt (`bsp_5_16` wird zu `bsp_5_16.asm`).
- `-o` und `-t` legen fest, wo die Objektdatei (`.obj`) bzw. die Testdatei (`.test`) abgelegt wird. Ohne diese Schalter liegen beide im Projektverzeichnis; der Dateiname wird aus `-s` gebildet. Ist ein Schalter angegeben, gilt statt des Standardorts der angegebene Pfad. Der Wert ist entweder ein Verzeichnis (es existiert bereits, oder der Wert endet auf `/` bzw. `\`) – dann wird der Dateiname aus `-s` gebildet (Name der Quellcode-Datei ohne `.asm`, dazu `.obj` bzw. `.test`) – oder ein Dateiname, mit oder ohne Pfad. Ein Dateiname wird unverändert übernommen (eine fehlende Endung wird nicht ergänzt); ohne Pfadanteil gilt er relativ zum aktuellen Verzeichnis, nicht zum Projektverzeichnis.
- Das Verzeichnis muss existieren; der Assembler legt es nicht an. Ein nicht vorhandenes Verzeichnis, ein Schalter ohne Wert (auch wenn der nächste Parameter mit `-` beginnt) und ein zu langer Pfad sind Fehler (Rückgabecode 16). `-o` und `-t` gelten nur für den einen Aufruf und werden nicht in `projekdetails` gespeichert; die übrigen Ausgabedateien (`.log`, `.mqo`, `.sym`, `.dmp`) bleiben im Projektverzeichnis. Beispiel: `$ c82ass.sh -p Projekt_1/bsp_5.16 -s bsp_5_16 -o ausgabe/ -t tests/bsp_5_16_neu.test`
- `-h` und `--help` geben den Usage-Text aus, `--version` nur die Versionszeile „C8205 Assembler C82aas Version 1.03-011“. Beide enden ohne Assemblerlauf mit Rückgabecode 0.
- `projekdetails` enthält die Zeilen `-p <Verzeichnis>` und `-s <Datei>`; fehlt einer der Parameter im Aufruf, wird der gespeicherte Wert benutzt.
- `-WARN=<liste>` schaltet zusätzliche Warnungen ein. Die Werte stehen durch Komma getrennt (z. B. `-WARN=SYM,SPR`); Groß-/Kleinschreibung ist egal, ein unbekannter Wert ergibt eine Warnung. Über `c82ass.sh` darf auch `--WARN=SYM` geschrieben werden. Alle Warnungen erscheinen erst nach fehlerfreiem Pass 3 und verändern weder `.obj` noch `.dmp`; sie lassen den Rückgabecode bei 0. In die `.log`-Datei werden sie hinter der Symboltabelle angehängt, ohne Farbcodes und mit denselben Überschriften wie auf STDOUT; ohne `-WARN=` und `--xref` bleibt die `.log` unverändert.
  - `SYM` meldet Symbole, die definiert, aber nirgends verwendet werden (vom Assembler erzeugte Symbole wie Literalsymbole und Makro-Labels werden nicht gemeldet).

  - `SPR` meldet Sprungbefehle (unbedingt, bedingt oder mit Rückkehrabsicht; Operationscodes 000–007, 024–027, 040–057, 064–077, 100–107, 124–127, 140–157, 170–177), deren Ziel kein Befehl ist, sondern ein Datenwort (`dw`, `dwb`, `dwh`, `dwf` oder `dwg` mit Wert ungleich 0, `da`) oder ein Wort, das der Assembler gar nicht belegt. Nicht gemeldet werden Ziele, die als Befehl dienen können: `dz`, `dt`, `dto` und `dtm` erzeugen Ausgabebefehle, `db`, `dbn`, `dw 0` sowie `dwb`, `dwh`, `dwf` und `dwg` mit Wert 0 reservieren Zellen, in die das Programm zur Laufzeit etwa den Rücksprungbefehl schreibt. Ebenfalls nicht gemeldet wird ein Sprungbefehl, den ein Lade- oder Speicherbefehl anspricht (Adresskonstante in Befehlsform wie `000 feld`): Er gilt als Datenzelle. Sprünge mit absoluter Adresse (`abs`-Symbol) werden nicht geprüft.
  - `TOT` meldet Befehle, die unmittelbar hinter einem unbedingten Sprung (000–007, 100–107) stehen und auf die nichts verweist: keine Marke, die irgendwo verwendet wird, kein Operand mit relativer Adresse, kein `da`-Wort, nicht die Startadresse und nicht Adresse 0000. Folgen mehrere solche Befehle aufeinander, wird nur der erste gemeldet, mit der Zahl der folgenden. Nicht gemeldet wird ein unbedingter Sprung hinter einem unbedingten Sprung (Sprungtabelle, die das Programm durch berechnete Adressen anspringt; die C8205 kennt keinen indirekten Sprung) und ein Befehl hinter einem Sprung, den ein Lade- oder Speicherbefehl anspricht (das Programm überschreibt ihn zur Laufzeit).
  - `ALLE` schaltet `SYM`, `SPR` und `TOT` ein.
  - `NOSLIT` schaltet die Warnung ab, dass ein Symbol-Literal `!x` in einem Rechenausdruck steht (3.2). Diese Warnung ist sonst immer eingeschaltet; `NOSLIT` gehört nicht zu `ALLE`.
  - `SPR` und `TOT` sind Hinweise und keine Fehler: Programme, die Befehle zur Laufzeit in Datenworte schreiben oder Sprungadressen berechnen, können sie zu Recht auslösen. Die Meldungen haben die Form `<datei>:<zeile>: Warnung: …` auf STDERR; auf STDOUT stehen eine Überschrift und die Anzahl.
- `-m <Makrodatei>` gibt den Pfad der Makrodatei an, statt des festen Namens `c82ass.makros`. Mit `-m` wird nur dieser Pfad benutzt (keine Ersatzsuche in `Sourcecode/`). Lässt sich die Datei nicht öffnen, endet der Lauf mit Rückgabecode 16 und einer Meldung mit Pfad und Ursache; `-m` ohne Wert ist ebenfalls ein Fehler (16).
- `--xref` gibt nach einem fehlerfreien Lauf eine Querverweisliste aus: je Symbol Wert, Definitionszeile (Name in Spalte 1) und alle Zeilen des Quelltextes, in denen der Name als ganzes Wort vorkommt (Kommentare ab `;` zählen nicht). Vom Assembler erzeugte Symbole fehlen. Die Operanden aus Makroexpansionen stehen im Listing und zählen mit: ein Symbol, das ein Makro verwendet, gilt als verwendet in der Zeile des Makroaufrufs (Definitionen im Quelltext und in der Makrodatei zählen nicht gesondert). Die Liste steht zusätzlich am Ende der `.log`-Datei. Ohne `--xref` entsteht keine Liste.
- `--makros` gibt nach einem fehlerfreien Lauf die Makroliste aus (Abschnitt 4.11) und hängt sie an die `.log`-Datei an. Ohne `--makros` entsteht keine Liste.
- `--opsyms` gibt die Standardtabelle der Namen für Operationscodes aus und endet mit Rückgabecode 0, ohne Assemblerlauf (Abschnitt 3.3.17).
- `--no-opt` (auch `--NOOPT`) schaltet den Optimierer aus (Abschnitt 5.3); Literale stehen dann hinter dem ersten `end`. `dw`-Zeilen verlegt der Optimierer nie, mit oder ohne Schalter. Auch die Blockverschiebung (`begin RELOC-EQU`, Abschnitt 5.4) findet dann nicht statt.
- `--pass0` führt nur Pass 0 aus und gibt die Meldungen farblos auf STDOUT aus, ohne Kopfmeldungen und ohne Ausgabedateien; bei `$NOTE` steht nur der Text (Abschnitt 3.1d).
- `--color` und `--no-color` erzwingen bzw. unterdrücken die farbige Ausgabe. Ohne Angabe wird nur farbig ausgegeben, wenn STDOUT und STDERR ein Terminal sind und die Umgebungsvariable `NO_COLOR` nicht gesetzt ist; bei Umleitung in eine Datei oder Pipe stehen keine Farbcodes (ESC-Folgen) in der Ausgabe.

3. Der Assembler c82ass

3.1 Programmstart

Der Assembler `c82ass` erwartet eine Eingabedatei mit `<quelltext>` unter einem Dateinamen `.asm`.

- **Pass 0:** liest die Makrodatei `c82ass.makros` (Vorgabe `c82ass.makros`; gesucht zuerst im Arbeitsverzeichnis des Aufrufs, sonst in `Sourcecode/`; mit `-m` ist ein anderer Pfad wählbar), sammelt die im Quelltext definierten Makros (Abschnitt 4.1), prüft die Makrodefinitionen und löst die Makroaufrufe im Quelltext auf (Abschnitt 4).
- **Pass 1:** ermittelt die Literaltabelle, ausgegeben im Logfile `.log`.
- **Pass 2:** ermittelt die Symboltabelle, ausgegeben im Logfile `.log`, und schreibt die Symboldatei `.sym`.

- **Optimierer (zwischen Pass 2 und Pass 3):** verlegt Literale in freie Zellen des Programms (Abschnitt 5); die Adressen der Literalsymbole werden dabei neu vergeben. Zeilen des Programmierers, auch `dw`-Zeilen, verlegt er nie.
- **Pass 3:** erzeugt aus dem aufgelösten Quelltext (Literale und Symbole eingesetzt) die Kennzeichenbefehle für die Eingabe mit Kennzeichen (Format des Minimalprogramms, nicht des KEP), ausgegeben im Objektfile `.obj`. Im selben Pass entstehen die Hilfsdatei `.mqo`, die formatierte Speicherbelegung `.dmp` und die Trommeldatei `.test` für den Interpreter.

Auf STDOUT gibt der Assembler den formatierten Quelltext (einschließlich der aufgelösten Literale) aus. Der Aufruf erfolgt zum Beispiel in der Form:

```
$ c82ass.sh -p PzBeispiele_4 -s bsp_5.17_Oktaldruck.asm
```

Meldungen mit `Fehler:`, `Warnung:` und `Abbruch` sowie die Statuszeilen `Projektverzeichnis:`, `Benutze für -s die Datei:`, `Leitadresse:` (nur wenn der Quelltext das Pragma `LEITADRESSE` enthält, siehe 3.3.16) und `Startadresse:` gehen auf STDERR (farbig hervorgehoben). Fehler werden je nach Pass in unterschiedlicher Form gemeldet (siehe die folgende Tabelle); jeder Fehler beendet den Lauf mit Rückgabecode 16. Fehlermeldungen mit Ort erscheinen in der Form `<datei>:<zeile>: Fehler: <text>`. Bei einem Fehler werden keine `.obj`-, `.mqo`- und `.test`-Datei hinterlassen (eine unvollständige Objektdatei könnte sonst versehentlich geladen werden); `.log`, `.sym` und `.dmp` bleiben zur Fehlersuche erhalten. Meldungen und Statuszeilen sind nur auf einem Terminal farbig (siehe `--color`).

Fehlerobergrenze und Zusammenfassung (ab Version 1.00-034): Der Assembler gibt höchstens 20 Fehlerzeilen aus, in allen Pässen und für alle Fehlerarten, auch für „bei Operand“, „bei Labeldefinition“, „bei Macrodefinition“ sowie für Fehler in `dt` und `dz`. Dieselbe Meldung (gleicher Text, ohne Zeilennummer) erscheint höchstens dreimal und nur mit verschiedenen Quellzeilen; jede weitere Wiederholung wird nur gezählt. Neue Meldungen jenseits der 20 Zeilen werden ebenfalls nur gezählt. Am Programmende, auch nach einem Abbruch, folgt für jede zusammengefasste Meldung eine Zeile „`<datei>: <n>-mal gleiche Fehlermeldung (Zeile <z>, davon <k> einzeln ausgegeben): <text>`“ (bei verschiedenen Zeilen „Zeilen `<a>` bis `<b>`“) und, wenn etwas zusammengefasst oder unterdrückt wurde, die Gesamtzeile „`<datei>: <n> Fehler insgesamt, <w> gleiche Wiederholungen zusammengefasst, <u> weitere Fehlermeldungen unterdrückt (Obergrenze 20 Fehlerzeilen).`“; die Teile für `w` und `u` entfallen, wenn sie 0 sind. Die Meldungen „Abbruch …“ sind nicht begrenzt. Der Rückgabecode bleibt 16.

Fehlerausgaben

Fehlerart	Ausgabe auf STDOUT (Listing)	Ausgabe auf STDERR	Ablauf danach
Pragma `LEITADRESSE` fehlerhaft (beim Einlesen des Quelltextes, vor Pass 0; siehe 3.3.16)	–	`Fehler: Zeile <n> der Quelldatei: Das Pragma LEITADRESSE …,` `Fehler: Abbruch.`	Abbruch, keine Ausgabedateien
Makrodefinition fehlerhaft (Pass 0)	–	`Fehler: Makrodefinition …, Fehler: N fehlerhafte Makrodefinition(en) …`	Abbruch nach Pass 0, keine Ausgabedateien
Makroaufruf fehlerhaft (Pass 0), z. B. Parameteranzahl oder -typ	`;*** Fehler: erwartet 2 Parameter, aber 1 wurden übergeben!` bzw. `;*** Fehler $1 Parameter $1: Wert "123" …`	`Warnung: Compilierung endete mit FEHLERn.`	Abbruch nach Pass 0, keine Ausgabedateien
Syntaxfehler in Pass 1 (z. B. `#` am Anfang eines `dt`-Textes)	`;*** 1 Fehler: …`	`Fehler: bei Operand: … in Zeile: …,` `Fehler: Abbruch wegen Syntaxfehler.`	Pass 2 läuft noch (`.log`, `.sym`), Pass 3 entfällt
Fehler in Pass 2 (z. B. Symbol doppelt, Programmzählerüberlauf über 7777)	`;*** (2) Fehler: …`	`Fehler: Abbruch wegen Syntaxfehler.`	Pass 3 entfällt

Fehlerart	Ausgabe auf STDOUT (Listing)	Ausgabe auf STDERR	Ablauf danach
Fehler in Pass 3 (Operand oder Operationscode ungültig)	Zeile `<adresse>: EE …` mit darunter stehender Meldungszeile `bei Operand: … ungültig`	`<datei>:<zeile>: Fehler: Operationscode … ist ungültig` bzw. `Symbol … ist nicht definiert`; `Fehler: Abbruch. Keine Objekt-, mqo- und Testdatei erzeugt …`	alle Pässe gelaufen; keine `.obj`, `.mqo`, `.test`; `.log`, `.sym`, `.dmp` bleiben
Belegungsfehler in Pass 3	`;*** 3 Fehler: Adresse … doppelt belegt: …`	`Fehler: Adresse … doppelt belegt: …`, `Fehler: Abbruch.`	wie Fehler in Pass 3

Belegungsprüfung (Pass 3): Jedes Wort der Trommel darf nur von einer Anweisung beschrieben werden. Geprüft werden Maschinenbefehle, `dw`, `dwb`, `dwh`, `dwf`, `dwg`, `da`, `dbn`, `dt`, `dto`, `dtm`, `dz` und die automatisch erzeugten Literale. Überschneidet sich die fortlaufende Speicherbelegung mit einem früher belegten Bereich, zum Beispiel weil ein `equ` zurückspringt oder das Programm in einen mit `equ` weiter hinten angelegten Datenbereich hineinläuft, meldet der Assembler `Adresse <oktal> doppelt belegt: "<neue Zeile>" ueberschneidet sich mit "<frühere Zeile>"`. und der Lauf endet mit Rückgabecode 16. Pro Zeile wird höchstens ein Konflikt gemeldet. Nicht geprüft werden `db` (reserviert nur Speicherplätze), `equ`, `start`, `abs` und `end`.

Die `<adresse>` der Befehle im Objektfile beginnt immer bei 0000. Ein Operand von `start` ändert die Adressen nicht; der Wert steht nur in der Kennzeichenzeile `210000<zahl>` (siehe 3.3.7).

Der Assembler erzeugt bzw. bearbeitet folgende Dateien im Verzeichnis der `.asm`-Datei (Beispiel `Projekt_1/bsp_5.16/`):

Endung	Inhalt
`.asm`	Quelltext (Eingabe)
`.dmp`	formatierte Speicherbelegung (Listing: Adresse, Maschinenwort, Label, Mnemonic, Operand, Kommentar); entspricht dem Listing auf STDOUT, jedoch ohne die Farbcodes der Fehlermeldungen; das Kennzeichen 01 kann durch das Pragma `LEITADRESSE` ersetzt sein (siehe 3.3.16)
`.log`	Abschnitte, soweit vorhanden: Literaltabelle, EQU-Definitionen, ABS-Definitionen, Symboltabelle (Symbol, RelAddr, Adresse, original Operation) und Adresstabelle (nach Adresse sortiert)
`.mqo`	aufbereiteter Quelltext. Gegenüber dem Quelltext fehlen `start`, `end`, Makrodefinitionen, reine Kommentarzeilen und `abs`-Anweisungen; Makroaufrufe sind durch ihre Ersetzung ersetzt, `equ`-Operanden aufgelöst, und die Literale stehen am Ende als `dw`-Zeilen (`l0001 dw 5`)
`.obj`	binär codierter R300-Lochstreifen mit dem Objektcode K1K2 (Eingabe mit Kennzeichen); es gibt nur noch das binäre Format: je Zeichen eine Zeile aus acht Ziffern 0/1 (Spurfolge w v u p 8 4 2 1, 1 = gelocht, ungerade Lochzahl, Kodetabelle 1, Klbuch). Der Text der Objektzeile steht ohne Leerzeichen und wird mit `.` abgeschlossen, zum Beispiel `010007000.` für das Listing `01 0007 000`; ein Zeilenende wird nicht gelocht (ein Lochstreifen ist endlos, SatzMarke/WRZL würde das Minimalprogramm v6 als zusätzliches Wort 0 speichern), die Wörter folgen unmittelbar aufeinander, das letzte Zeichen ist `/`. Die Datei kann unmittelbar in den Ordner des Lochbandlesers gelegt werden; zurück in Text wandelt `c8205_lochstreifen_umwandlung.py` mit `--IN-TXT`. Das Kennzeichen 01 kann mit dem Pragma `LEITADRESSE` durch 00 oder 02 … 09 ersetzt werden (siehe 3.3.16); bei 00 entfällt die Zeile `210000<zahl>`
`.kep`	nur mit dem Schalter `--ABSPR`, **statt** `.obj`: absolut adressierter, binär im R300-Code codierter Lochstreifen für das Kleine Eingabeprogramm (KEP); siehe 3.1a
`.sym`	Symboldatei. Je Zeile `<symbol>,<adresse>` (vierstellig oktal, in Quelltextreihenfolge, einschließlich Literalsymbolen wie `l0001`)
`.test`	Trommeldatei im Format des Interpreters (`Adresse: Wert # Quelltextzeile`) mit Kopf (Abschnitt `Trommeldatei:`) und Fuß (Abschnitt `Tastenfolge:` mit `H G R`). Der Kopf besteht aus den Vorspannzeilen des Quelltextes (siehe 3.1b); enthält der Quelltext keine, erzeugt der Assembler als Platzhalter `#DOK-0001`, `#SPEI-0001 0000-0000` und `#PROT STUFE=2`. Kopf und Fuß müssen für den jeweiligen Fall ergänzt bzw. erweitert werden. Zeilen gibt es nur für Wörter mit K1=00 oder 01 (Befehle, `dw`, `da`, `dz`, `dt`, `dto`, `dtm`, Nullwort); `db`, `equ`, `start` und `abs` erzeugen keine Zeile; Grundlage für Testaufträge, s. `C8205-Interpreter-Durchführung_eines_Tests.md`, Abschnitt 2a

Vor jedem Lauf löscht der Assembler die Dateien `.log`, `.obj`, `.mqo`, `.sym`, `.test` und `.dmp` des Projekts und erzeugt sie neu. Mit `-o` bzw. `-t` (Abschnitt 2) wird statt der `.obj`- bzw. `.test`-Datei im Projektverzeichnis die dort angegebene Datei gelöscht und neu erzeugt; eine ältere Datei im Projektverzeichnis bleibt unberührt. Die früher genannten Endungen `.mqt` (modifizierter Quelltext) und `.l82` (Objektcode für den `C8205Simulator`) erzeugt der aktuelle Assembler nicht mehr. Dasselbe gilt für `.mac`: Der Quellcode legt keine solche Datei an.

3.1a Schalter `--ABSPR` (absolut adressierter KEP-Lochstreifen)

Das KEP liest nur Dualziffern. Mit diesem Schalter kann der Assembler statt einer `.obj`-Datei im BIS-Minimalprogramm-Format eine binär codierte Datei im KEP-Format erzeugen.

Mit `--ABSPR` (gleichwertig `--ABSOLUTPROGRAMM` und `--ASBPR`, Groß-/Kleinschreibung egal) schreibt der Assembler deshalb statt der `.obj`-Datei die Datei `<name>.kep` (die mit `-o` wie bei `.obj`-Dateien auch in den gewünschten Pfad umlenkbar ist).

Sie ist eine binäre Lochstreifendatei, d. h. je Zeile wird ein einzelnes Zeichen durch die 8 Bit des R300-Codes dargestellt (in der Form der Lochstreifen-Hilfsprogramme: je Lochungszeile 8 Ziffern 0/1 (Spurfolge `w v u p 8 4 2 1`), nur die Zeichen `0 1 : .`). Eine solche Datei ist also ohne weitere Umwandlung in den Leser 1 einlegbar.

Der Aufbau wird aus den Zeilen, die sonst in eine `.obj`-Datei kämen, abgeleitet; die Kennziffern K1K2 entfallen, bestimmen aber die Aufbereitung.

Automatisch wird ein Leitbefehl vorangestellt: `<start> 240:`. Die Adresse kommt aus der `start`-Anweisung (oktal, ohne Vornullen, höchstens 8 Oktaltriaden), dann folgt der Operationscode 240.

Dabei werden die K1K2 wie folgt umgesetzt:

K1K2	Umsetzung
`00`	Rest der Zeile ohne Leerzeichen unverändert, mit `.` abgeschlossen
`01`	Adressstellen plus aktueller Wert `oo..oo` (Anfangswert = `start`), dann die 3 Oktalziffern des Operationscodes, ohne Vornullen, mit `.`
`20`	`oo..oo` wird durch den angegebenen Wert (ohne Leerzeichen) ersetzt und gilt für spätere Additionen; dazu ein neuer Leitbefehl `<wert> 240:`
`21`, Endezeichen `/`	entfallen

Jede Oktalziffer wird durch ihre 3 Bits ersetzt, führende Nullbits entfallen (Beispiele: `0040 240:` = `100000010100000:`, `0001 164.` = `1001110100.`). Das Pragma `LEITADRESSE` hat Vorrang und wird vor der Umsetzung ins binäre Format angewendet: Ersetzt es 01 durch 00, wird K1=00 behandelt (Rest unverändert, keine Addition). Ersetzt es 01 durch 02 … 09, werden diese Zeilen nicht umgesetzt (sie fehlen in der `.kep`); der Assembler gibt dazu eine Warnung mit der Anzahl aus. Gibt es danach keine Zeile mit K1=01 mehr, gibt er eine Warnung aus; ist die Ursache das Pragma `LEITADRESSE 00` (absolut geschriebener Quelltext, der Regelfall beim Minimalprogramm), ist es nur ein Hinweis „Der Quelltext gilt als absolut adressiert, es wird keine Adresse umgerechnet“ (ab Version 1.00-030). Dasselbe gilt für `LEITADRESSE 00` zusammen mit `start <adresse>` ungleich 0: Der erste Leitbefehl lautet dann `<adresse> 240`, die Operanden bleiben aber unverändert (die Startadresse wird nicht addiert); der Assembler warnt (ab Version 1.00-028). Regel: Relativ geschriebener Quelltext mit `start <adresse>` ohne Pragma; absolut geschriebener Quelltext mit Pragma `LEITADRESSE 00` und `start` ohne Operand. Wird ein Wort auf eine Zelle von 0002 bis 0037 geladen (dort liegt das KEP selbst), gibt der Assembler eine Warnung mit der Zelle aus (ab Version 1.00-029; vorher warnte er bei jedem Leitbefehl auf 0000 bis 0037). Die Zellen 0000 und 0001 sind erlaubt und warnen nicht: Dort trägt das geladene Programm, wie das Minimalprogramm, seinen Sprung zur Startadresse ein; der Bahnsperrenschalter muss beim Laden aus sein. Das KEP selbst ist nicht Teil des Lochstreifens.

3.1b Vorspann der `.test`-Datei (Testmarken im Quelltext)

Der Assembler erzeugt immer eine `.test`-Datei. Die „`#…`“-Makros zur Teststeuerung mit dem Abschnitt `Trommeldatei:` (Kopf), den Trommelzeilen und dem Abschnitt `Tastenfolge:` (Fuß) können schon im Assembler-Quelltext als Assembler-Kommentare angelegt werden.

Regel. Der Assembler liest den Quelltext Zeile für Zeile und behandelt jede Zeile so:

1. Er überspringt Leerzeichen und Tabulatoren am Zeilenanfang. Das erste übrige Zeichen muss ein Semikolon `;` sein, sonst ist die Zeile keine Vorspannzeile. Ein Kommentar hinter einem Befehl (`ADD 5 ;#DOK-0001 …`) wird deshalb nicht übernommen.
2. Er überspringt das `;` und den Leerraum dahinter.
3. Der Rest muss mit einer Testmarke beginnen (siehe Tabelle). Sonst bleibt die Zeile ein gewöhnlicher Kommentar.
4. Er schneidet Leerraum und Zeilenende am Ende der Zeile ab und schreibt den Rest unverändert in die `.test`-Datei. Das `;` und der Leerraum davor erscheinen dort nicht.

Die Zeilen kommen in der Reihenfolge des Quelltextes in den Abschnitt `Trommeldatei:`. Sie dürfen an beliebiger Stelle des Quelltextes stehen, üblicherweise am Anfang. Zeilen mit mehr als 2047 Zeichen werden ignoriert.

Erkannte Testmarken.

Marke	Erkennung	Bedeutung
`#DOK-xxxx`	als Präfix	Dokumentationstext zu einer Debug-Nummer
`#SPEI-xxxx`	als Präfix	Speicherbereich ssss-eeee zur Ausgabe
`#SPUR-xxxx`	als Präfix	Speicherausgabe nach Spuradresse
`#PROT`	ganzes Wort	Protokollstufe (`#PROT STUFE=n`)
`#AC`, `#BZ`	ganzes Wort	Anfangswerte von Akkumulator und Befehlszähler (`#AC=…`, `#BZ=…`)
`#STARTE`	ganzes Wort	Ansprung eines Speicherplatzes; ersetzt die Tastenfolge (siehe unten)
`#TROMMEL`	ganzes Wort	Trommel leeren oder mit Abbild belegen (`#TROMMEL=LEER`, `#TROMMEL=MP`)
`#ZTROMMEL`	ganzes Wort	Zusatztrommeln vor dem Test leeren oder unverändert lassen (`#ZTROMMEL=LEER`, `#ZTROMMEL=KEEP`; nur mit Interpreter-Schalter `--C8205Z` wirksam)
`--leser1`, `--leser2`, `--stanzer1`, `--sw-ausgabe`, `--k0`	ganzes Wort	Geräteschalter mit Pfad (`--k0 datei=PFAD`)

Ein Eintrag mit „ganzes Wort“ gilt nur, wenn hinter der Marke Zeilenende, Leerraum oder `=` folgt. `#PROTOKOLL` allein ist also keine Testmarke. Die Bedeutung der Marken beschreibt `C8205-Interpreter-Durchführung_eines_Tests.md` in den Abschnitten 2, 2b, 2e und 2g.

Nicht übernommen (die Zeile bleibt ein gewöhnlicher Kommentar):

Zeile	Grund
`--pruefe-…`	gehört in den Abschnitt `Prüfungen:`, nicht in den Kopf
`#INTERRUPT`	muss zwischen zwei Befehlszeilen der Trommeldatei stehen, nicht im Vorspann
`;#; LEITADRESSE xx`	Pragma für den Objektcode (3.3.16), keine Testmarke
jede andere Kommentarzeile	keine Testmarke

Sonderfälle.

- *Keine Vorspannzeile:* Enthält der Quelltext keine einzige übernommene Zeile, erzeugt der Assembler die Platzhalter `#DOK-0001`, `#SPEI-0001 0000-0000` und `#PROT STUFE=2`. Schon eine einzige Vorspannzeile unterdrückt alle drei Platzhalter. Der Kopf enthält dann nur Ihre Zeilen.
- `#STARTE`*:* Steht `#STARTE` im Vorspann, ersetzt es die Tastenfolge. Der Assembler erzeugt dann keinen Abschnitt `Tastenfolge:` (sonst steht dort `H G R`).

Beispiel. Im Quelltext:

```
;#DOK-0001 Druck von BZ und AC
; --sw-ausgabe virtuelle_Geraete/SW-Ausgabe
;#PROT STUFE=2
;#; LEITADRESSE 03
; gewöhnlicher Kommentar
```

ergibt in der `.test`-Datei:

```
Trommeldatei:
#DOK-0001 Druck von BZ und AC
--sw-ausgabe virtuelle_Geraete/SW-Ausgabe
#PROT STUFE=2
0000: ... (die Zeilen aus dem Quelltext)
Tastenfolge:
H G R
```

3.1c Nachspann der `.test`-Datei (Prüfungs-Testmarken im Quelltext)

Durch die Definition von:

```
;#PRUEFE-START
… beliebiger Text ..
;#PRUEFE-ENDE
```

können die in `C8205-Interpreter-Durchführung_eines_Tests.md` (Abschnitt 3a) beschriebenen Prüfungen schon im Assembler-Quelltext definiert werden. Die Zeilen zwischen den beiden Marken werden am Ende der `.test`-Datei unter einer Zeile „`Prüfungen:`“ angefügt; das führende Semikolon (und der Leerraum dahinter) wird bei jeder Zeile dazwischen entfernt.

`;#PRUEFE-START` und `;#PRUEFE-ENDE` selbst werden nicht übernommen; dafür steht vor den Prüfungen die Zeile:

```
Prüfungen:
```

vorangestellt.

Weitere Regeln: Der Abschnitt steht hinter `Tastenfolge:` (bei `#STARTE` hinter den Trommelzeilen). Mehrere Bereiche ergeben einen gemeinsamen Abschnitt. Zeilen im Bereich ohne führendes Semikolon und Zeilen, die danach leer sind, werden übergangen. Fehlt `;#PRUEFE-ENDE` oder steht es ohne `;#PRUEFE-START`, gibt der Assembler eine Warnung aus; ein Bereich ohne Ende wird nicht übernommen.

ZT-Zellen-Prüfung (Zusatztrommeln, Ergänzung 30.10.2026). Im Bereich darf der Unterabschnitt `ZT-Zellen-Pruefung:` mit Zeilen der Form `o-aaaa: ttttttttttt` stehen (alles oktal; `o` = Trommel 0 bis 3, `aaaa` = Zellenadresse, `ttttttttttt` = Sollwert; Einzelheiten `C8205-Interpreter-Durchführung_eines_Tests.md`, Abschnitt 3a.6). Der Assembler übernimmt diese Zeilen wie alle anderen Zeilen des Bereichs, ohne sie zu prüfen; die formale Prüfung macht der Interpreter, und zwar nur mit dem Schalter `--C8205Z` (ohne ihn werden die Zeilen kommentarlos ignoriert). Die Testmarke `;#ZTROMMEL=LEER` bzw. `;#ZTROMMEL=KEEP` (erkannt ab c82ass 1.00-056) gehört in den Vorspann (Tabelle „Erkannte Testmarken“) und wird in den Abschnitt `Trommeldatei:` übernommen.

Beispiel:

```
;#ZTROMMEL=LEER
;#PRUEFE-START
;BZ=2027
;ZT-Zellen-Pruefung:
;0-0000: 52525252525
;0-0020: 00000000001
;#PRUEFE-ENDE
```

Beispiel (`bsp_5_19_2_teilwoerter2.asm`). Im Quelltext:

```
;#PRUEFE-START
; BZ=3054 AC=77760000000 AR=0000 FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0 V2=0
; 3000: 52526317434
; 3001: 52520000000
;#PRUEFE-ENDE
```

ergibt am Ende der `.test`-Datei:

```
Tastenfolge:
H G R
Prüfungen:
BZ=3054 AC=77760000000 AR=0000 FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0 V2=0
3000: 52526317434
3001: 52520000000
```

3.1d Schalter `--pass0` (nur Pass 0, Ausgabe zur Weiterverarbeitung)

Mit `--pass0` (Groß-/Kleinschreibung egal; ab c82ass 1.01-017) führt der Assembler nur Pass 0 aus, also die Auflösung der Makros und die Auswertung der $-Direktiven des Quelltextes, und gibt die Meldungen dieses Passes so aus, dass man sie ohne Nachbearbeitung in eine Datei umleiten oder an ein anderes Programm weitergeben kann. Gedacht ist der Schalter für Makroprogramme, von denen man nur die Ausgabe von $NOTE, $DUMP, $TRACE, $WARN und $ERROR sehen will (Abschnitte 4.5 und 4.24), ohne den ganzen Lauf und ohne Ausgabedateien.

Aufruf:

```
c82ass -p <ordner> -s <name>.asm --pass0 > ergebnis.txt
```

Die Ausgabe ist dieselbe wie die der Shell-Kette, die man früher dafür brauchte (verglichen bei Claude/makrospur und Claude/makrospur_ohne, bei $NOTE im Quelltext und im Makro, bei $WARN und bei $ERROR mit Rückgabecode 16: Byte-gleich):

```
c82ass -p <ordner> -s <name>.asm --no-color 2>&1 >/dev/null \
 | grep -v -E '^(Projektverzeichnis:|Benutze für|Startadresse:|Warnung:  Das Programm)' \
 | sed 's/^Hinweis: [^(]*([^)]*): //' > ergebnis.txt
```

Wirkung:

- Es läuft nur Pass 0; Pass 1 bis 3 entfallen. Fehler, die erst dort entstehen (unbekannte Symbole, ungültige Operanden), meldet der Schalter nicht: Ein fehlerfreier Lauf mit --pass0 sagt nicht, dass sich das Programm assemblieren lässt.
- Alles, was sonst auf STDERR steht (Hinweise, Warnungen, Fehler, die Zeilen von $TRACE und $DUMP), geht auf STDOUT; die übrige Ausgabe auf STDOUT (Versionszeile, „Pass 0“, Listing) entfällt, STDERR bleibt leer. Der Assembler leitet dazu wie die Shell-Kette (2>&1 >/dev/null) STDERR auf STDOUT und STDOUT ins Null-Gerät um.
- Die Ausgabe ist farblos, auch mit --color.
- Die Kopfmeldungen „Projektverzeichnis:“, „Benutze für …“ und „Startadresse:“ entfallen.
- Bei $NOTE und $DUMP steht nur der Text; die Einleitung „Hinweis: Makro <Name> (<Datei>:<Zeile>): “ (4.24) und im Quelltext „Hinweis: Quelltext (<Datei>:<Zeile>): “ (4.1) entfallen. Warnungen und Fehlerzeilen stehen wie sonst da.
- Es entstehen keine Ausgabedateien (.log, .obj, .mqo, .sym, .dmp, .test, .kep); vorhandene Dateien dieser Namen werden weder gelöscht noch verändert, -o und -t bleiben ohne Wirkung. Berichte (-WARN, --xref, --makros) entfallen. Die Datei projekdetails wird wie sonst beschrieben.
- Rückgabecode 0 (kein Fehler in Pass 0) oder 16; bei 16 stehen die Fehlermeldungen und die Zeile „Warnung: Compilierung endete mit FEHLERn.“ in der Ausgabe.
- Mit -h, --help, --version und --opsyms hat der Schalter keine Wirkung; diese Auskünfte erscheinen wie gewohnt.
- Mit `adr` und `abstand` (4.23, ab c82ass 1.01-018) im Quelltext liefert `--pass0` für beide Funktionen den Wert 0, ohne Meldung, ohne Probeläufe und ohne die Zeile „Adressen fuer adr/abstand …“ (Annahme: Wert 0); die Argumente werden weiter auf Syntax geprüft, eine Marke, die es nicht gibt, ist deshalb kein Fehler. Dazu ist `$PROBE` dort 1 (4.23, ab c82ass 1.01-020), sodass ein Makro die fehlenden Adressen erkennen kann. Auch `--probeausgabe` (4.24) ist dort ohne Wirkung: Es gibt keine Probeläufe, die etwas zeigen könnten.

Beispiel: Das Programm Claude/makrospur (Ablaufspur mit $TRACE, 60 Ausgabezeilen) liefert mit

```
c82ass -p Claude/makrospur -s makrospur.asm --pass0 > ergebnis.txt
```

in ergebnis.txt unter anderem als erste Zeilen:

```
Spur SUMME 1: $LOCAL i s => lokal vereinbart: i s
Spur SUMME 2: $SET i 1 => i: 0 -> 1 (lokal)
Spur SUMME 3: $WHILE 1 <= 1 => wahr (Durchlauf 1)
Spur SUMME 4: $INC s 1 => s: 0 -> 1 (lokal)
i=1 s=1
```

Prüfung: Im Regressionstest stehen die Negativtests pass0_note ($NOTE im Quelltext und im Makro, $WARN, keine Kopfmeldungen, nichts auf STDERR, keine Ausgabedateien), pass0_fehler ($ERROR, Rückgabecode 16), pass0_gross (--PASS0), pass0_version (--pass0 mit --version) und pass0_berichte (--makros und -WARN=ALLE ohne Berichte); der Regressionstest mit c82ass 1.01-017 (268 Programme, 412 Negativtests) läuft ohne Abweichung, auch unter AddressSanitizer und UBSan. Unter Windows ist der Schalter nicht geprüft (die Umleitung benutzt dup, dup2 und open mit „NUL“).

3.1e Befehlsworte im Lochstreifen (`;#PRECOMMAND`, `;#POSTCOMMAND`)

Mit den Anweisungen `;#PRECOMMAND=<wert>` und `;#POSTCOMMAND=<wert>` (ab c82ass 1.03-009) legt der Quelltext Befehlsworte fest, die der Assembler zusätzlich in den .obj-Lochstreifen schreibt: alle PRECOMMAND vor das erste Wort des Programms, alle POSTCOMMAND hinter das letzte Wort, noch vor das Endekennzeichen /.

Syntax:

```
;#PRECOMMAND=212000.
;#POSTCOMMAND=102000000.
```

(212000. und 102000000. sind Beispielwerte.)

Regeln:

- <wert> besteht aus den Ziffern 0 bis 9 und Punkten und endet mit einem Punkt. Er wird unverändert, Zeichen für Zeichen, als Lochbilder in den Streifen geschrieben (jedes Zeichen als 8 Bit des R300-Codes, wie die übrigen Wörter des .obj). Der Assembler fügt nichts hinzu: kein Kennzeichen K1K2, kein Leerzeichen, keinen weiteren Punkt.
- Die Anweisung ist eine Kommentarzeile: Sie beginnt mit einem Semikolon (davor Leerraum, dahinter Leerraum erlaubt) und steht in der Hauptdatei oder in einer mit $INCLUDE eingebundenen Datei. Hinter dem Wert darf ein Kommentar mit Semikolon folgen. Der Name steht in Großbuchstaben.
- Beide Anweisungen dürfen mehrfach vorkommen, höchstens 64 je Art; ein Wert hat höchstens 80 Zeichen. Die Werte gelten in der Reihenfolge des Quelltextes (bei $INCLUDE an der Stelle der Einbindung).
- Fehler (Rückgabecode 16, keine Ausgabedateien): kein „=“, leerer Wert, ein anderes Zeichen als Ziffer oder Punkt (auch der Schrägstrich /, der das Streifenende kennzeichnet), ein Wert ohne Punkt am Ende, zu langer Wert, zu viele Anweisungen.
- Ohne diese Anweisungen bleibt der Lochstreifen unverändert. Listing, .dmp, .mqo, .sym und .test hängen von den Anweisungen nicht ab; nur die .obj-Datei ändert sich.
- Mit --ABSPR (3.1a) gibt es keinen .obj-Streifen, sondern die .kep-Datei: Die Anweisungen haben dann keine Wirkung, der Assembler warnt davor.

Beispiel (Claude/objbefehl, zwei PRECOMMAND 212000. und 77., zwei POSTCOMMAND 102000000. und 1.): Der Streifen enthält, als Zeichen dargestellt,

```
212000.77.010001000.010004211.010005240.010000020.0000000000123.0.102000000.1./
```

Prüfung: Programm Claude/objbefehl (Test wahr; die .obj-Datei wird mit der Referenz verglichen) und die Negativtests objbefehl_ohne_gleich, objbefehl_ohne_punkt, objbefehl_zeichen, objbefehl_leer und objbefehl_schraegstrich; der Regressionstest mit c82ass 1.03-009 (119 Programme, 346 Negativtests) läuft ohne Abweichung.

3.2 Sprachdefinition

```
<zahl>              ::= <oktalzahl> | <dezimalzahl>
<oktalzahl>          ::= oktalziffer==0 [ oktalziffer [ oktalziffer … ]]
<oktalziffer>        ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7
<vorzeichen>         ::= -
<dezimalziffer>      ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
<dezimalzahl>        ::= [<vorzeichen>] dezimalziffer<>0 [ dezimalziffer [ dezimalziffer … ]]
<binärziffer>        ::= 0 | 1
<binärzahl>          ::= binärziffer [ binärziffer [ binärziffer … ]]  (1 bis 33 Stellen, nur bei dwb, 3.3.19)
<hexziffer>          ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | A | B | C | D | E | F | a | b | c | d | e | f
<hexzahl>            ::= hexziffer [ hexziffer [ hexziffer … ]]  (1 bis 8 Stellen, nur bei dwh, 3.3.19a)
<festkommazahl>      ::= [+|-] dezimalziffer [ dezimalziffer … ] [ . dezimalziffer [ dezimalziffer … ] ]  (nur bei
    dwf, 3.3.20)
<gleitkommazahl>     ::= <festkommazahl> [ E [+|-] dezimalziffer [ dezimalziffer [ dezimalziffer ]]]  (nur bei
    dwg, 3.3.21)
<alphazeichen>       ::= abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ
<sonderzeichen>      ::= % : + - / ` , . < " [ > ' ~ ] # * ( = ; ) ? !
<wert>               ::= <oktalzahl> | <dezimalzahl> | <symbol>
<rechenzeichen>      ::= + | - | * | /
<speicherplatz>      ::= Speicherstelle auf dem Trommelspeicher, fortlaufend nummeriert durch
                         eine Oktalzahl mit dem Wertebereich 00000 bis 07777.
<symbol>             ::= <adresssymbol> | <literalsymbol> | (leer)
<adresssymbol>       ::= Zeichenfolge aus 1 bis 32 Zeichen:
                         <alphazeichen> [ <alphazeichen> | <dezimalziffer> [ … ]],
                         abgeschlossen durch Doppelpunkt.
                         Sie wird vom Programmierer frei definiert. Der Unterstrich `_` wird ebenfalls akzeptiert.
                         Das erste Zeichen ist ein Buchstabe oder `_`, keine Ziffer. Erlaubt sind nur die
    ASCII-Buchstaben
                         `a`–`z`, `A`–`Z`, die Ziffern `0`–`9` und `_`; Zeichen wie `-`, `.` oder `ä` sind
    unzulässig.
                         Doppelt definierte Symbole, Symbole über 32 Zeichen und unzulässige Symbole sind Fehler.
<literalsymbol>      ::= 'l' (kleines L) gefolgt von einer Oktalziffernfolge von genau 4
                         Oktalziffern, abgeschlossen durch Doppelpunkt.
                         Sie wird vom Assembler automatisch aus dem Inhalt des Literalzählers vergeben.
<ausdruck>           ::= <werteausdruck> | <literal>
<operand>            ::= <werteausdruck> | <literal> | (leer)
<adressausdruck>     ::= <wert><rechenzeichen>[<wert><rechenzeichen> … ]
                         Einschränkungen: kein negatives Vorzeichen;
                         Wertebereich nur von 00000 … 07777.
                         Ein Vorzeichen am Anfang (auch `+`) ist nicht zulässig.
                         Verstöße meldet der Assembler als Fehler in Pass 3.
<werteausdruck>      ::= <wert><rechenzeichen>[<wert><rechenzeichen> … ]
```

```
                          Einschränkung: Bei `dw` und in `#`-Literalen ist ein relatives Ergebnis (eine einzelne
    symbolische
                          Adresse, auch mit Zahl) nicht zulässig. Zulässig: Zahlen, `abs`-Symbole, die paarweisen
                          Differenzen symbolischer Adressen (auch mehrere). Wertebereich -2^32 … 2^33-1.
                          Adressen werden mit `da` oder `@`-Literalen geschrieben.
<literal>            ::= #<werteausdruck> | @<adressausdruck> | !<symbolname>
                          Literal bedeutet, dass der numerische Wert von <werteausdruck> oder
                          <adressausdruck> auf einem vom Assembler selbst festgelegten
                          <speicherplatz> gespeichert wird. Diesem Speicherplatz wird vom
                          Assembler ein <literalsymbol> zugeordnet.
                          Bei ! wird kein Wert gespeichert: Der Assembler legt eine Zelle mit dem Inhalt 0 an (wie
    dbn 1,
                          3.3.2) und gibt ihr den Namen <symbolname>. Im Operanden steht danach die Adresse dieser
    Zelle.
                          Das Symbol-Literal darf stehen, wo auch # und @ stehen dürfen (Operand eines Befehls,
    hinter da);
                          bei dw und spa ist es ein Fehler.
<symbolname>         ::= 1 bis 32 Zeichen: <alphazeichen> [ <alphazeichen> | <dezimalziffer> [ … ]], ohne
    Doppelpunkt
                          (wie <adresssymbol>; der Unterstrich ist erlaubt, eine Ziffer am Anfang nicht).
                          Der Name ist eine gewöhnliche Marke (Symboltabelle, .sym). Derselbe Name in mehreren
                          !<symbolname> meint dieselbe Zelle; sie entsteht beim ersten Auftreten im Quelltext.
    Gibt es
                          denselben Namen auch als Marke, equ- oder abs-Symbol, ist das der Fehler „doppelt
    definiert“.
                          Die Symbol-Literale zählen bei den Namen l0001, l0002, … nicht mit.
                          Der Optimierer darf die Zelle verlegen (5.2). Steht der Name in einem Rechenausdruck
    (zum Beispiel
                          da x+1, dw b-a), warnt der Assembler (Warnung mit Datei und Zeile auf STDERR,
    Rückgabecode
                          unverändert): Die Lage der Zelle bestimmt der Optimierer, Summen und Abstände mit diesem
    Symbol
                          verantwortet der Programmierer. Die Warnung ist standardmäßig eingeschaltet;
    -WARN=NOSLIT schaltet sie ab (NOSLIT gehört nicht zu ALLE). In einem Makro darf der Name aus Parametern und
    $V( ) gebildet werden (!$1, !z$V(n)); derselbe Name in mehreren Aufrufen meint dieselbe Zelle. Ein lokaler
    Makroname mit Prozentzeichen (!hilf%) ist kein gültiger Name und ein Fehler. In $IF steht !x nur als Text
    (Textvergleich, keine Zelle entsteht), in $CALC ist !x ein Fehler.
<adresse>            ::= numerischer Wert eines <symbol>, der einen <speicherplatz> adressiert.
<kommentar>          ::= ; (Semikolon) gefolgt von ASCII-Zeichen. | (leer)
                          Innerhalb der \-Zeichen von dt wird ; nicht als Kommentar betrachtet.
<blockkommentar>     ::= ```   als beginn auf einer Zeile für sich, ab Spalte 1
                          ```   als Ende auf einer Zeile für sich, ab Spalte 1
                          Dazwischen beliebiger Text
<trenner>            ::= Tabulator[<trenner>[...]] | Leertaste<trenner>[...]]
<befehl>             ::= [<symbol>] <trenner><operationscode> [<trenner><operand>]
                          [<trenner><kommentar>]
                          Alle Befehle müssen auf einer Zeile stehen; eine Zeile darf höchstens 255 Zeichen lang
    sein.
                          Ein <symbol> darf auch allein auf einer Zeile stehen (ohne <operationscode>). Die Zeile
    belegt keinen
                          Speicher; das Symbol erhält die Adresse der nächsten Zeile, die Speicher belegt, und der
    Befehlszähler
                          bleibt unverändert. Mehrere solche Symbole in Folge erhalten dieselbe Adresse. Vor start
    ist das ein
                          Fehler, wie ein Symbol auf der start-Zeile selbst.
<operationscode>     ::= <maschinenbefehl> | <assembleranweisung> | .<opname>   (Namen für Operationscodes, 3.3.17)
<assembleranweisung> ::= start | end | db | dbn | dw | dwb | dwh | dwf | dwg | da | dt | dto | dtm | dz | equ |
    equ10 | equ40 | equ100 | abs | spa | begin | bend | kat4 | kat5 | kat6
<maschinenbefehl>    ::= <oktalziffer><oktalziffer><oktalziffer>
<quelltext>          ::= <befehl>[<befehl> [ … ]]
```

Auswertung von Ausdrücken (`<werteausdruck>` und `<adressausdruck>`): `*` und `/` binden stärker als `+` und `-`; Operatoren gleichen Ranges werden von links nach rechts ausgewertet (`2+3*4` ergibt 14, `10-2-3` ergibt 5). `/` ist eine ganzzahlige Division, deren Ergebnis zur Null hin abgeschnitten wird (`7/2` ergibt 3, `-7/2` ergibt -3); eine Division durch 0 ist ein Fehler. Ein Vorzeichen ist nur am Anfang des Ausdrucks zulässig (`-7/2`, `+5`, nicht `7/-2` oder `--5`). Klammern und Leerzeichen innerhalb eines Ausdrucks sind nicht zulässig.

Keine mnemonischen Operationscodes: Seit 1.00-026 kennt der Assembler nur noch dreistellige Oktalcodes (000–777). Die frühere Ansatztabelle mit `add`, `sub`, `ld`, `jmp`, `stopp` und `nop` (vorläufige Platzhalterwerte) ist entfallen; solche Namen ergeben jetzt den Fehler „Operationscode ist ungültig“.

3.3 Assembleranweisungen
```

3.3.1 Definiere Bereich (db)

```
[<symbol>]  db  <zahl>
```

Hält <zahl> Speicherplätze frei, ohne ihren Inhalt zu definieren. <zahl> muss eine Oktalzahl (führende 0) oder eine Dezimalzahl sein, höchstens 010000. Negative Zahlen, Ausdrücke (z. B. 3+1), Symbole und ein fehlender Operand sind Fehler.

3.3.2 Definiere Bereich auf Null (dbn)

```
[<symbol>]  dbn  <zahl>
```

Hält <zahl> Speicherplätze frei, setzt ihren Inhalt auf 0. Für <zahl> gilt dasselbe wie bei db. Je Speicherplatz erzeugt dbn in der Objektdatei die Zeile 000..

3.3.3 Definiere Wert (dw)

```
[<symbol>]  dw  <werteausdruck>
```

- Speichert den numerischen Wert von <werteausdruck> auf dem <speicherplatz>. Zulässig ist der Bereich -2^32 bis 2^33-1 (33-Bit-Wort), sonst meldet der Assembler Operand ist ungültig. Negative Werte werden im Zweierkomplement gespeichert (dw -5 ergibt 77777777773). Ein Ergebnis, das ein relativer Wert ist, ist im <werteausdruck> von dw nicht zulässig; das meint eine einzelne symbolische Adresse. Dagegen sind paarweise Differenzen symbolischer Adressen zulässig, auch mehrfach. Auch absolute Symbole (abs) sind zulässig; Adressen schreibt man mit da.
- Es wird keine Hauptleitadresse addiert.
- In der Objektdatei steht ein Datenwort als 00<elf Oktalziffern>.; nur der Wert 0 wird kurz als 0. ausgegeben (Nullwort ohne Kennzeichen).
- Eine Bitfolge (Maske, Schalter) schreibt man mit dwb (3.3.19), eine Hexadezimalzahl mit bis zu 8 Ziffern mit dwh (3.3.19a).
- Einen Dezimalbruch als gebrochene Festkommazahl schreibt man mit dwf (3.3.20), eine Gleitkommazahl mit dwg (3.3.21).

3.3.4 Definiere Adresse (da)

```
[<symbol>]  da  <adressausdruck>
```

Speichert den numerischen Wert von <adressausdruck> auf dem <speicherplatz>, verschoben um «9 in den Adressstellen. Zum <adressausdruck> wird die 1. Hauptleitadresse addiert.

Das Ergebnis von <adressausdruck> muss bei da (und bei @-Literalen) immer eine **relative Adresse** sein. Jedes relative Symbol (Marke, equ-Symbol, Literalsymbol) zählt dabei +1, wenn es addiert wird, und −1, wenn es subtrahiert wird; Zahlen und abs-Symbole zählen 0. Es muss mindestens ein relativer Adressanteil übrig bleiben (Summe ≥ 1). Unzulässig sind daher:

- Konstanten und konstante Ausdrücke (da 5, da absx, da absx+2),
- paarweise Differenzen relativer Adressen, die sich aufheben (da a-b, da b-a+absx),
- relative Adressen in * oder / (da a*2).

Zulässig sind z. B. da a, da a+5, da a-b+c, da a+absx und da *. Verstöße meldet der Assembler in Pass 2 als Fehler mit Datei und Zeile (Operand: "a-b" ist bei da nicht zulässig: …). Konstante Werte schreibt man mit dw.

```
[<symbol>]  da  <literal>
```

- Erzeugt am Programmende automatisch aus <literal> eine Definition der Art <literalsymbol> da <adressausdruck>.
- Zum <adressausdruck> wird die 1. Hauptleitadresse addiert.
- Nimmt stattdessen die <adresse> von <literalsymbol> in den Befehl, der das <literal> enthält.
- Der Optimierer (Abschnitt 5) kann das Literal statt ans Programmende in eine freie Zelle davor verlegen.
- Bei !<symbolname> entsteht stattdessen die Zelle <symbolname> dbn 1 (3.2); der Operand erhält deren Adresse.
- da ist zwischen begin und bend erlaubt (3.3.12).

3.3.5 Definiere Text zur Ausgabe von Zeichen (dt, dto, dtm)

```
[<symbol>]  dt[o|m]  \<alphazeichen> | <leerzeichen> | <dezimalziffer>
             <sonderzeichen> [ <alphazeichen> | <leerzeichen> | <dezimalziffer> | <sonderzeichen>[...]]\
```

- Gespeichert wird ein Code gemäß R300-Code Ausgabeadresse C8205 (siehe R300-code.pdf). Die Abspeicherung erfolgt als Folge von Speicherplatzinhalten in der Form (oktal): `0000 xxxx 030`, wobei `xxxx` die Ausgabeadresse ist.
- Die Zeichen „\" am Anfang und Ende sind erforderlich, wenn der Text mit `#` beginnt, ein `;` enthält (sonst beginnt dort der Kommentar) oder führende bzw. abschließende Leerzeichen enthalten soll. Leerzeichen innerhalb des Textes (`dt a b`) sind auch ohne „\" zulässig; sonst werden führende und abschließende Leerzeichen entfernt. Ein führendes `#` kennzeichnet ein Literal (`#<werteausdruck>`), keinen Text; `dt #ab` und `dto #x` meldet der Assembler als Fehler. Richtig ist `dt \#ab\` bzw. `dto \#x\`. Ein `#` innerhalb des Textes (`dt a#b`) ist zulässig.
- `dt` ist für längere, zusammenhängende Textausgaben gedacht. Sein Vorteil: Der Text lässt sich ohne ausgeschriebene Umschaltzeichen definieren, der Quelltext bleibt gut lesbar. Für verstreute Einzelzeichen ist `dt` nicht geeignet, dafür nimmt man `dz` (3.3.6) und schreibt alle Umschaltzeichen explizit.
- Der Assembler setzt `dt` zur Assemblerzeit in Maschinenbefehle um und kennt den Zustand KB/GB zur Laufzeit nicht. Er führt den Zustand nur in der Reihenfolge des Quelltexts weiter. Beim allerersten `dt` des Quelltexts nimmt er KB an. Jedes weitere `dt` setzt den Zustand des vorangehenden `dt` fort und fügt nur dort ein Umschaltzeichen ein, wo sich der Zustand innerhalb des Textes ändert.
- Daraus folgt: Vor jeder zusammengehörenden Folge von `dt`-Anweisungen muss ein Umschaltbefehl durchlaufen werden, der exakt auf den Zustand des ersten Zeichens dieser Folge schaltet (`dz $inkb`, `dz $ingb` oder `dtm` als erste Anweisung der Folge). Das gilt für jede Folge neu. Es kann dabei ein eigentlich unnötiger Umschaltbefehl entstehen. Der ist beabsichtigt, weil sich Assemblerzeit und Laufzeit nicht synchronisieren lassen. Umschaltzeichen mit `dz` zwischen zusammengehörenden `dt`-Anweisungen sind nicht zulässig, sie bringen den Zustand des Assemblers durcheinander.
- Praktischer Hinweis dazu: `dtm` fügt immer vor dem ersten Zeichen ein Umschaltzeichen passend zum ersten Buchstaben ein. Damit ist es die natürliche erste Anweisung einer `dt`-Folge. Es ersetzt das von Hand gesetzte `dz $inkb` oder `dz $ingb` davor.
- `dto` verhält sich wie `dt`, außer dass generell KEINE Umschaltzeichen automatisch eingefügt werden. Das bedeutet, in einer `dto`-Anweisung kann man KEINE gemischte Groß- und Kleinschreibung definieren (auch keine Zahlen und Großbuchstaben usw.).
- `dtm` verhält sich wie `dt`, außer dass vor dem ersten Zeichen generell ein Umschaltzeichen, dem ersten Buchstaben entsprechend, automatisch eingefügt wird.
- `dt`, `dto` und `dtm` sind zwischen `begin` und `bend` erlaubt (3.3.12; Block mit RELOC-EQU: Abschnitt 5.4).

3.3.6 Definiere Zeichen (dz)

```
[<symbol>]  dz  $wrzl | $sm | $inkb | $ingb | $irrz | $irrs | $trans | $bm | $vorl |
                $irrb | $wr | $tab | $zr | % | : | + | ` | / | , | . | -
```

- Gespeichert wird ein Code gemäß R300-Code Ausgabeadresse C8205 (siehe Tabelle 2 unten). Die Abspeicherung erfolgt auf einem Speicherplatz in der Form (oktal): `0000 ssss 030`. Eine automatische Ergänzung von Umschaltzeichen wie bei `dt` erfolgt hier generell NICHT.
- Findet man z. B. `dz $wrzl` zu kompliziert, kann man das mit einem Makro vereinfachen:

  ```
  WRZL:  MACRO
             dz  $wrzl  ; Zeilenschaltung
         MEND
  ```

  (In der mitgelieferten `c82ass.makros` enthält `WRZL` zusätzlich die Anweisung `ANWAHL 4700`.)
- Das Zeichen `;` kann bei `dz` nicht angegeben werden, weil es einen Kommentar einleitet; der Operand für den Code 0150 ist `:` (vgl. Tabelle 2 und 3).
- `dz` ist zwischen `begin` und `bend` erlaubt (3.3.12).

3.3.7 Beginn des Quellcodes (start)

```
start  [<zahl>]
```

- Nur als 1. Anweisung im `<quelltext>` zulässig, und nur einmal; ein Label vor `start` ist ein Fehler. Der Operand muss eine Konstante sein. Der Befehlszähler beginnt unabhängig vom Operanden bei 0. Dasselbe gilt für ein Label allein auf einer Zeile vor `start`. Eine Maschinencodeausgabe erfolgt nicht; mit Operand wird im Objektcode die Kennzeichenzeile `210000<zahl>` ausgegeben.
- Ist `<zahl>` angegeben, wird die 1. Leitadresse in den „Lochstreifen" ausgegeben. Es wird dann auf diese 1. Leitadresse + Ladeadresse geladen. Die Ladeadresse existiert dabei nur beim Laden.
- Das Kennzeichen 01, das für die 1. Leitadresse steht, kann mit dem Pragma `;#; LEITADRESSE xx` durch ein anderes Kennzeichen ersetzt werden (Abschnitt 3.3.16). Das Pragma muss vor `start` stehen.
- Mit `;#; LEITADRESSE 00` wird dem Objektcode kein Leitbefehl vorangestellt: Die Kennzeichenzeile `210000<zahl>` entfällt (im Objektcode und im Listing); der Operand von `start` hat dann keine Wirkung auf Objektcode und Listing (Abschnitt 3.3.16).

3.3.8 Definition Relativadresse (equ)

```
[<symbol>]  equ  <werteausdruck> | *
```

- Definiert für das `<symbol>` einen `<werteausdruck>`. Der Befehlszähler wird auf diesen Wert gesetzt.
- Ist `*` angegeben, wird dem Symbol stattdessen der aktuelle Befehlszählerstand zugewiesen.
- Eine Maschinencodeausgabe erfolgt nicht. Im Objektcode (Eingabe mit Kennzeichen, Format des Minimalprogramms) wird der Kennzeichenbefehl für das Setzen der Relativadresse ausgegeben (KZ=20).
- Das weitere Laden erfolgt auf 1. Leitadresse + Ladeadresse + Relativadresse. Die Ladeadresse existiert dabei nur beim Laden des Objektcodes.
- Die Anweisungen `equ100`, `equ40` und `equ10` runden den Befehlszähler (Operand `*`) auf das nächste Vielfache von 0100, 040 bzw. 010 (oktal) auf. Der Operand muss `*` sein; eine Zahl oder ein Symbol als Operand ist unzulässig (Fehler; c82ass 1.00-040 nimmt es noch an, siehe Abschnitt 5). Wer einen bestimmten Wert braucht, schreibt `equ`.
- Als erste Anweisung zwischen `begin RELOC-EQU` oder `begin RELOC-ALL` und `bend` ist `equ100 *` bzw. `equ40 *` zulässig; der Assembler verschiebt dann den Block (3.3.12, Abschnitt 5.4). Sonst ist jedes `equ` in einem Block ein Fehler.

3.3.9 Definition Absolutadresse (abs)

```
[<symbol>]  abs  <oktalzahl> | <dezimalzahl>
```

- Definiert für das `<symbol>` eine absolute `<adresse>`. Weder Objektcode- noch Maschinencodeausgabe erfolgt.
- Beachte: Symbole, die durch `abs` definiert wurden, sind nicht verschieblich. Anweisungen, in denen ein solches Symbol im Adressausdruck oder Werteausdruck vorkommt, werden mit K1K2=00 generiert!
- `abs` kann mehrfach mit dem gleichen Wert definiert werden, ohne Fehler zu erzeugen, aber nicht mehrfach mit dem gleichen Symbol für unterschiedliche Werte.

3.3.10 Definition Absolute Sektoradresse (spa)

```
[<symbol>]  spa  <oktalzahl> | <dezimalzahl>      im Bereich 00..77 (oktal)
```

- Dieser Befehl verhält sich wie ein `equ *`, nur dass die rechten beiden Oktalziffern durch den Wert von `<oktalzahl> | <dezimalzahl>`, dargestellt in 2 Oktalziffern, ersetzt werden.
- Beispiel: `spa 020` macht, wenn die Anweisung auf `*` steht, mit dem Beispielwert `*==0256` das Gleiche wie `equ 0220`. Steht `spa 020` auf 0277, wirkt sie wie `equ 0220`. Steht `spa` auf 0301, wirkt sie wie 0320.

3.3.11 Ende des verschieblichen Teils des Quelltextes (end)

```
[<symbol>]  end
```

- Hinter dieser Anweisung sollten nur noch `abs`- oder `equ`-Anweisungen folgen. Sie werden auch dann ausgewertet, wenn das Programm Literale enthält. Makrodefinitionen (siehe Abschnitt 4.1) dürfen ebenfalls dort stehen. Der Assembler ordnet hinter dem ersten „end" die automatisch erzeugten Literaldefinitionen ein, sodass sie noch zum verschieblichen Teil hinzukommen. Ein nachfolgendes `equ` käme also HINTER die Literale. Ein weiteres „end" ändert daran nichts. Der Optimierer (Abschnitt 5) kann Literale in freie Zellen davor verlegen; der Literalbereich ist dann kürzer oder entfällt.
- Eine Maschinencodeausgabe erfolgt für „end" nicht.

- Code und Daten (Maschinenbefehle, `db`, `dw` usw.) hinter „end" sind nicht vorgesehen. Der Assembler lehnt sie nicht ab, sondern übersetzt sie hinter den Literalen (soweit vorhanden) mit.

3.3.12 Beginn einer Folge zusammenhängender Anweisungen (begin)

```
begin  [RELOC-EQU | RELOC-ALL] [FILL]
```

- Zwischen „Begin“ und „Bend“ sind alle Anweisungen erlaubt, die Speicher belegen (`db`, `dbn`, `dw`, `dwb`, `dwh`, `dwf`, `dwg`, `da`, `dt`, `dto`, `dtm`, `dz`, `kat4`, `kat5`, `kat6`), und Befehle (Befehle ab Version 1.00-041; `da`, `dt`, `dto`, `dtm`, `dz`, `kat4`, `kat5` und `kat6` ab Version 1.02-002). Andere Anweisungen (`equ`, `spa`, `abs`, `start`, `end` und so weiter), die nur Adressen oder den Befehlszähler festlegen, führen zu einem Fehler. Ein „Begin“ ohne folgendes „Bend“ ist ebenfalls ein Fehler. Eine Tabelle, die `dw`-Zeilen und Befehle mischt (zum Beispiel Sprungeinträge), steht so in einem einzigen Block.
- Schlüsselwörter : `RELOC-EQU` verlangt, den Block als Ganzes an das obere Ende des Programmbereichs zu verschieben (Abschnitt 5.4); `RELOC-ALL` (ab c82ass 1.03-011) verlangt dasselbe und verschiebt zusätzlich Blöcke ohne `equ100`/`equ40`; `FILL` erlaubt dem Optimierer, die Lücke hinter dem verschobenen Block mit Literalen zu füllen. `RELOC-EQU` und `RELOC-ALL` schließen einander aus; `FILL` steht höchstens einmal und hinter `RELOC-EQU` bzw. `RELOC-ALL`. Alle Wörter sind wahlfrei; ein anderes Wort, ein doppeltes Wort, beide Schlüsselwörter zusammen oder `FILL` davor ist ein Fehler. Der Bindestrich gehört zum Wort `RELOC-EQU` bzw. `RELOC-ALL` (der Operandenteil von `begin` ist eine Wortliste, kein Rechenausdruck). `FILL` ohne `RELOC-EQU` und `RELOC-ALL` ist zulässig und ohne Wirkung.
- Bei `RELOC-EQU` und `RELOC-ALL` darf die erste Anweisung des Blocks `equ100 *` (Gruppe 1) oder `equ40 *` (Gruppe 2) sein; sie bestimmt die Gruppe der Verschiebung. Kommentar- und Leerzeilen davor zählen nicht. Steht bei `RELOC-EQU` keine solche Zeile als erste Anweisung, bleibt der Block, wo er ist, ohne Meldung. Bei `RELOC-ALL` (ab c82ass 1.03-011) wird ein solcher Block trotzdem verschoben, und zwar ohne Ausrichtung (Gruppe 4, Abschnitt 5.4); er darf wie jeder Block kein `equ` und kein `spa` enthalten (beides bleibt ein Fehler). `equ100 *` oder `equ40 *` ohne `RELOC-EQU` oder `RELOC-ALL` ist ein Fehler, auch mit `--no-opt` (die Syntaxprüfung hängt nicht vom Schalter ab; mit `--no-opt` rundet die Zeile den Zähler wie sonst, und der Block bleibt an seiner Stelle). Jedes andere `equ` im Block, auch `equ100` oder `equ40` nicht als erste Anweisung, bleibt ein Fehler.
- Marken: Eine Marke auf der `begin`-Zeile eines Blocks mit `RELOC-EQU` und eine Marke allein auf einer Zeile vor der `equ100`/`equ40`-Zeile sind Fehler (sie bekämen nach der Verschiebung nicht die Blockadresse). Die Marke gehört auf die `equ`-Zeile; dort erhält sie die Adresse des gerundeten Zählers, nach der Verschiebung also die Anfangsadresse des Blocks. Das gilt auch bei RELOC-ALL mit equ100/equ40 vorn. In einem RELOC-ALL-Block ohne equ darf eine Marke auf der begin-Zeile oder vor der ersten Anweisung stehen und behält die Blockadresse, weil der Assembler das equ <Adresse> vor das begin setzt (5.4, Gruppe 4).

3.3.13 Ende einer zusammenhängenden Folge von Anweisungen (bend)

```
bend
```

- „Bend“ ohne vorangehendes „Begin“ führt zu einem Fehler.
- „Begin“ und „Bend“ belegen keinen Speicherplatz und verändern den Quelltext nicht. Sie fassen die dazwischenliegenden Zellen im Quelltext als Block zusammen (zum Beispiel eine Tabelle). „Begin“-Blöcke können mehrfach vorkommen. Ein Block bleibt an seiner Stelle und wird nur verschoben, wenn er mit `begin RELOC-EQU` beginnt und seine erste Anweisung `equ100 *` oder `equ40 *` ist (Abschnitt 5.4); einzelne Zeilen des Programmierers verlegt der Optimierer nie, auch keine dw-Zeilen (Abschnitt 5.2). Bei der Verschiebung bleiben die Zeilen des Blocks beieinander und in ihrer Reihenfolge; nur bei `FILL` können Literale in die Lücke hinter dem Block treten. Der Assembler prüft die Paarung von `begin` und `bend`, die erlaubten Anweisungen dazwischen (ab 1.00-041 auch Befehle, ab 1.02-002 alle Anweisungen, die Speicher belegen) und ab 1.01-030 die Schlüsselwörter. Ab 1.02-003 zählt der Assembler auch die Zellen von dt, dto, dtm, kat4, kat5 und kat6 selbst; ein Block mit RELOC-EQU darf daher auch solche Anweisungen enthalten und wird verschoben (Abschnitt 5.4).

3.3.14 Komprimierter Text: 4 Ausgabezeichen in einem Maschinenwort (kat4)

```
[<symbol:>]  kat4  \<alphazeichen> | <leerzeichen> | <dezimalziffer> |
                    <sonderzeichen> [ <alphazeichen> | <leerzeichen> |
                    <dezimalziffer> | <sonderzeichen>[...]]\
```

- In der Kodetabelle (Punkt 1.4), Spalte Ausgabeadresse, stehen höchstens 3 Ziffern, gefolgt von einem x. Jede dieser 3 Ziffern gibt eindeutig ein ausgabefähiges Zeichen an; 000 gibt nichts aus. Das Textformat `kat4` (komprimierter Alpha-Text, 4 Zeichen pro Maschinenwort) nimmt diese 3 Ziffern a 8 Bit, lässt das werthöchste Bit weg und speichert 4 Stück dieser Bitfolgen von rechts beginnend in einem Maschinenwort. Sind weniger als 4

Ausgabezeichen gespeichert, bleiben die Stellen links auf 0. Ist der Text komplett, wird das letzte `kat4`-Maschinenwort mit Bit z0=1 gekennzeichnet. Es ist das 33. Bit.

3.3.15 Komprimierter Text: 5 Ausgabezeichen in einem Maschinenwort (kat5)

```
[<symbol>:]  kat5  <kat5zeichen> [ <kat5zeichen> [...]]

<kat5zeichen>    ::= <kleinbuchstabe> | <dezimalziffer> | <leerzeichen>
                     | . | , | - | : | + | = | <marke>
<kleinbuchstabe> ::= a | b | c | ... | z
<marke>          ::= $wrzl | $inkb | $ingb
```

- Das Textformat `kat5` (komprimierter Text) speichert bis zu 5 Zeichen in genau einem Maschinenwort, je Zeichen einen 6-Bit-Code nach der Tabelle am Ende dieses Abschnitts. Die Codes sind frei numeriert und hängen nicht von der Ausgabeadresse der Kodetabelle (Punkt 1.4) ab. Sie sind so numeriert, dass die Reihenfolge der Codes die Sortierreihenfolge ist. Die Ausgabeadresse, die ein Programm beim Ausgeben eines Codes verwendet, steht in der Kodetabelle.
- Eine `kat5`-Anweisung ist immer genau ein Wort und belegt immer genau 1 Zelle. Der Operand hat 1 bis 5 Zeichen; jede `<marke>` zählt als ein Zeichen. Ein leerer Operand ist ein Fehler, ebenso ein Operand mit mehr als 5 Zeichen (die Meldung nennt die Zeichenzahl). Längere Texte stehen in mehreren `kat5`-Zeilen oder in `kat6`.
- Das Wort ist linksbündig: das erste Zeichen steht in z3 bis z8, das zweite in z9 bis z14, das dritte in z15 bis z20, das vierte in z21 bis z26, das fünfte in z27 bis z32. z0, z1 und z2 sind immer 0. Ein Endekennzeichen gibt es nicht (z0 entfällt).
- Sind weniger als 5 Zeichen angegeben, werden die Stellen rechts mit dem Code 00 (alle Bits 0) aufgefüllt. Der Code 00 gibt nichts aus. Der Assembler fügt keine Umschaltzeichen ein.
- Sortierung: Das Wort ist nie negativ, und die Codes steigen in der Reihenfolge Leerzeichen, Satzzeichen, Ziffern, Buchstaben, Marken (Tabelle unten). Zwei `kat5`-Worte lassen sich deshalb mit einem einzigen Vergleichsbefehl (330) ordnen. Ein kürzeres Wort steht vor einem längeren mit gleichem Anfang, weil die Füllung 00 kleiner ist als jedes Zeichen. Verglichen wird streng nach dem Code: Umschaltzeichen sind Zeichen mit eigenem Code, der Zustand KB/GB geht nicht in den Vergleich ein.
- Die Marken `$wrzl` (Zeilenschaltung, Satzmarke), `$inkb` (Umschalten auf Kleinbuchstaben) und `$ingb` (Umschalten auf Großbuchstaben) sind dieselben Namen wie bei `dz` (Abschnitt 3.3.6, Tabelle 2). Sie werden ohne Beachtung der Groß-/Kleinschreibung erkannt. Eine Marke endet nach ihrem Namen, das nächste Zeichen folgt unmittelbar. Ein `$` ohne gültigen Markennamen ist ein Fehler.
- Leerzeichen innerhalb des Textes (`kat5 a b`) sind auch ohne `\` zulässig. Führende und abschließende Leerzeichen werden sonst entfernt; sollen sie erhalten bleiben, steht der Text zwischen `\` und `\`, wie bei `kat4` und `dt`.
- Großbuchstaben und alle anderen Zeichen außerhalb des Vorrats sind ein Fehler. Die Meldung nennt die Position des Zeichens, wie bei `kat4`.
- `+` und `=` haben dieselbe Ausgabeadresse (0400) und deshalb denselben Code. Ob `+` oder `=` erscheint, bestimmt der Zustand KB/GB beim Ausgeben. Wer `=` ausgeben will, setzt `$ingb` davor und danach `$inkb`. Der Doppelpunkt hat die Ausgabeadresse 0150 und erscheint im Zustand KB; im Zustand GB erscheint dort die runde Klammer auf, die nicht zum Vorrat gehört.
- Warnung bei Zustandsunstimmigkeit: Der Assembler führt innerhalb des Operanden den Zustand KB/GB mit. Er ist am Textanfang unbekannt, `$inkb` setzt ihn auf KB, `$ingb` auf GB. Ist der Zustand bekannt, warnt der Assembler, wenn
  - im Zustand GB eine Ziffer, `.`, `,`, `-`, `:` oder `+` steht (die Ausgabe zeigt ein anderes Zeichen), oder
  - im Zustand KB ein `=` steht (die Ausgabe zeigt `+`).

  Buchstaben, Leerzeichen und `$wrzl` lösen keine Warnung aus. Die Warnung ist kein Fehler und ändert den Code nicht. Der Zustand wird nicht über das Ende der Anweisung hinaus verfolgt.
- `kat5` ist wie `kat4` eine Datenanweisung: Der Optimierer verlegt sie nicht, die Belegungsprüfung zählt ihre Zelle. Im Listing steht das Wort in der Quellzeile.
- Stand der Umsetzung: Diese Definition gilt seit der Doku 1.02-012 (09.11.2026) und ist im Assembler ab c82ass 1.03-001 umgesetzt (Doku 1.03-001). Die Unterversion steigt von 1.02 auf 1.03, weil das neue Format bestehende `kat5`-Programme ungültig macht: Ein Text mit mehr als 5 Zeichen ist jetzt ein Fehler, die Codes und die Lage der Zeichen im Wort haben sich geändert. Die Zählregeln (4.23) und der Optimierer (5.4) gelten: `kat5` belegt immer genau eine Zelle.

Code (oktal)	Zeichen
00	Füllung (gibt nichts aus)
01	Leerzeichen
02, 03, 04	`,` `-` `.`
05	`+` und `=` (Ausgabeadresse 0400)
06	`:` (Ausgabeadresse 0150)
07 bis 020	0 bis 9 (0 = 07, 1 = 010, ... 9 = 020)
021 bis 052	a bis z (a = 021, b = 022, ... z = 052)
053	`$wrzl` (Ausgabeadresse 2720)
054	`$inkb` (Ausgabeadresse 1170)
055	`$ingb` (Ausgabeadresse 0570)
056 bis 077	frei; solange nicht definiert, ein Fehler

Beispiele (Wörter oktal, erstes Zeichen links, z0 bis z2 = 0, Füllung 00 rechts):

```
        kat5    hallo               ; 1 Zelle: 03021343437
        kat5    ab                  ; 1 Zelle: 02122000000 (Füllung 00 rechts)
        kat5    abc                 ; 1 Zelle: 02122230000 (größer als ab)
        kat5    b                   ; 1 Zelle: 02200000000 (größer als abc)
        kat5    a:b                 ; 1 Zelle: 02106220000
        kat5    $ingb=1             ; Warnung: Ziffer 1 im Zustand GB
        kat5    hallo welt          ; Fehler: 10 Zeichen, erlaubt sind höchstens 5
```

Zum Vergleich: `kat4` braucht für den Text `hallo welt` 3 Zellen, `kat6` 2. `kat5` nimmt nur bis zu 5 Zeichen in einer Zelle auf; es ist für kurze Wörter gedacht, die sich als Zahl vergleichen und sortieren lassen.

3.3.16 Pragma LEITADRESSE

```
;#; LEITADRESSE <xx>
```

- Ein Pragma ist eine Kommentarzeile, die mit `;#;` beginnt und dem Assembler eine Anweisung gibt. Es ist keine Assembleranweisung im Sinne der Abschnitte 3.3.1 bis 3.3.15, erzeugt weder Maschinencode noch ein Wort auf der Trommel und steht auf einer eigenen Zeile (Einrückung mit Leerzeichen oder Tabulator ist zulässig). Ein anderes Kommentar, das mit `;#;` beginnt, bleibt ein gewöhnliches Kommentar.
- `<xx>` ist genau zweistellig und liegt zwischen 00 und 09 (`02`, nicht `2`; `10` ist ein Fehler). Danach folgt nur noch Leerraum oder ein Kommentar (`;`). Das Schlüsselwort `LEITADRESSE` wird ohne Beachtung der Groß-/Kleinschreibung erkannt und vom Wert durch Leerraum getrennt.
- Das Pragma darf höchstens einmal vorkommen und muss **vor** `start` stehen. Ohne Pragma gilt 01.
- Wirkung: Im Objektcode (`.obj`) und im Listing (STDOUT und `.dmp`) steht anstelle des Kennzeichens 01 (K1K2=01, bezogen auf die 1. Leitadresse, siehe 3.3.7 und 3.3.8) das Kennzeichen `<xx>`. Das betrifft alle Zeilen, die sonst mit 01 beginnen. Zeilen mit anderen Kennzeichen bleiben unverändert. Die Dateien `.test`, `.mqo`, `.sym` und `.log` enthalten kein Kennzeichen und ändern sich nicht.
- Sonderfall `;#; LEITADRESSE 00`: Der Code wird so als Objektcode (`.obj`) ausgegeben, wie ihn der Assembler erzeugt hat, aber mit dem Kennzeichen 00 statt 01. Das dient dazu, bereits absolut codierte Programme im Objektcode-Format auszugeben (K1K2=00: keine Adressänderung beim Laden, vgl. 3.3.9). Dem Objektcode wird dabei kein Leitbefehl vorangestellt: Die Kennzeichenzeile `210000<zahl>`, die `start` mit Operand sonst erzeugt (3.3.7), wird weder in die `.obj` noch in das Listing (STDOUT und `.dmp`) geschrieben. Die Kennzeichenzeilen `200000<adresse>` für `equ` bleiben erhalten. Im Objektcode sind die früheren 01-Wörter dann von den übrigen 00-Wörtern nicht mehr zu unterscheiden.
- Hat der Quelltext das Pragma, gibt der Assembler auf STDERR die Statuszeile `Leitadresse: <xx>` aus (nach `Benutze für -s die Datei:`). Das gilt auch für `;#; LEITADRESSE 01`.
- Fehler (jeweils `Fehler: Zeile <n> der Quelldatei: …`, danach `Fehler: Abbruch.`, Rückgabecode 16, keine Ausgabedateien):
  - Das Pragma steht hinter `start`: „Das Pragma LEITADRESSE muss vor `start` stehen.“
  - Das Pragma kommt ein zweites Mal vor: „Das Pragma LEITADRESSE darf nur einmal vorkommen.“

  - Der Wert fehlt, ist nicht zweistellig oder liegt außerhalb von 00 … 09: „Das Pragma LEITADRESSE erwartet einen zweistelligen Wert von 00 bis 09 (z. B. ;#; LEITADRESSE 02).“
- Achtung: Ein falsch geschriebenes Schlüsselwort (zum Beispiel `LEITADRESS`) oder ein Wert ohne Trennung vom Schlüsselwort (`LEITADRESSE02`) wird nicht gemeldet. Die Zeile gilt dann als gewöhnliches Kommentar, und das Kennzeichen bleibt 01. Ob das Pragma gewirkt hat, zeigt die Statuszeile `Leitadresse:`.

Beispiel (`Projekt_1/bsp_5.16/bsp_5_16.asm`, Quelltext beginnt mit der Zeile `;#; LEITADRESSE 03`):

Quelltextzeile	`.obj` **ohne Pragma**	`.obj` **mit** `;#; LEITADRESSE 03`
`000   go`	`012000000.`	`032000000.`
`equ   02000`	`2000002000.`	`2000002000.`
`aqv: da    qvektor`	`012027000.`	`032027000.`
`dqv: dw    05000`	`0000000005000.`	`0000000005000.`
`i: db    1`	`202007.`	`202007.`
`164   0001`	`010001164.`	`030001164.`

Im Listing (`.dmp` und STDOUT) steht in denselben Zeilen entsprechend `03` statt `01` in der Kennzeichenspalte, zum Beispiel `0000: 03 2000 000`.

Beispiel für `00` (kleines Programm, das nicht zum Projekt gehört; der Quelltext beginnt mit der Zeile `;#; LEITADRESSE 00`, danach folgt `start 0100`):

Quelltextzeile	`.obj` **ohne Pragma**	`.obj` **mit** `;#; LEITADRESSE 00`
`start 0100`	`2100000100.`	keine Zeile
`anf: 211 wert`	`010020211.`	`000020211.`
`240 ziel`	`010021240.`	`000021240.`
`000 anf`	`010000000.`	`000000000.`
`equ 020`	`2000000020.`	`2000000020.`
`wert: dw 5`	`0000000000005.`	`0000000000005.`
`ziel: da anf`	`010000.`	`000000.`
`020 0`	`010000020.`	`000000020.`

Im Listing (`.dmp` und STDOUT) steht entsprechend `00` statt `01` (zum Beispiel `0000: 00 0020 211`), und die Zeile `21 0000 0100` fehlt.

3.3.17 opsym und Namensdatei: Namen für Operationscodes

```
<operationscode>  ::= <maschinenbefehl> | <assembleranweisung> | .<opname>
<opname>          ::= <buchstabe oder _> [<buchstabe, ziffer oder _> [...]]   (höchstens 8 Zeichen)
<opsymzeile>      ::= <opname>: opsym <oktalcode>
<oktalcode>       ::= <oktalziffer><oktalziffer><oktalziffer>

Namensdatei:   <opname> <oktalcode> [; <kommentar>]
```

- Ein Name steht für genau einen dreistelligen Operationscode (000 bis 777). Er wird im Operationscode-Feld mit vorangestelltem Punkt geschrieben: `.LD a` ist dasselbe wie `211 a`. Der Punkt gehört zum Gebrauch, nicht zur Definition; in der Namensdatei und in der `opsym`-Zeile steht der Name ohne Punkt. Groß-/Kleinschreibung zählt nicht.
- Der Punkt ist in Symbolen nicht erlaubt (3.2). Deshalb sind die markierten Namen von Anweisungen, Makronamen und Symbolen grundsätzlich getrennt: Ein Makro `ADD` und der Name `.ADD` sind verschieden, und ein Symbol `ld` bleibt ein Symbol (`ld: .LD ld`). Ein Name wirkt nur im Operationscode-Feld, nicht im Operanden; ein Befehlswort als Datum schreibt man weiter mit dem Zahlencode (`dw 0240`).
- Die Namen werden nach Pass 0 (im Quelltext und in den Makrorümpfen) durch den Oktalcode ersetzt. Alle weiteren Pässe, die Warnungen und die Dateien `.obj`, `.kep`, `.sym`, `.log` und `.mqo` sehen nur den Code. Listing (STDOUT), `.dmp` und die Kommentare der `.test` zeigen den Namen so, wie er im Quelltext steht. Ein Programm mit Namen und seine Fassung in Oktalcodes liefern deshalb gleiche `.obj`, `.mqo` und `.sym`.

- Standardtabelle `c82ass.opsym`: Sie wird beim ersten markierten Namen im Quelltext geladen (gesucht wie `c82ass.makros`: aktuelles Verzeichnis, sonst `Sourcecode/`; ab c82ass 1.00-066 zuletzt neben dem Programm: Verzeichnis des Binarys, dann dessen `../Sourcecode/`, damit auch ein Aufruf aus einem anderen Verzeichnis die Tabelle findet; gilt ebenso für `--opsyms`). Ein Quelltext ohne markierten Namen verhält sich wie bisher; die Datei wird dann nicht geöffnet.
- Weitere Namensdateien bindet die Zeile `;#OPSYM=<Pfad>` ein (bis zu 9 Zeilen; Pfad relativ zur Quelldatei, sonst relativ zum Arbeitsverzeichnis oder absolut), wie `;#MACRODATEI1=` bis `9` (4.9). Eine Namensdatei besteht aus Zeilen `<NAME> <code> [; Kommentar]` mit einem dreistelligen Oktalcode; Leerzeilen und Kommentarzeilen mit `;` sind erlaubt. Mehrere Namen dürfen denselben Code tragen (Alias). Derselbe Name noch einmal mit demselben Code ist erlaubt, mit anderem Code ein Fehler; spätere Dateien dürfen frühere Namen also nicht umdefinieren. Der Kommentar jedes Eintrags beginnt mit der Nummer der Einzelbefehlsübersicht der Befehlsliste (`; Nr. 78 …`).
- Die Zeile `NAME: opsym <code>` vereinbart einen Namen ab dieser Zeile bis zum Ende des Quelltexts; sie ist für Einzelfälle gedacht, wiederkehrende Namen gehören in eine Namensdatei. Das Wort vor `opsym` ist kein Symbol: Es entsteht keine Adresse, kein Eintrag in `.sym` oder `.log`, und es kollidiert nicht mit einem gleichnamigen Label. Die Zeile belegt keinen Speicher und erscheint nicht im Listing.
- Option `--opsyms`: `c82ass --opsyms` gibt die Standardtabelle auf STDOUT aus und endet mit Rückgabecode 0, ohne Assemblerlauf (wie `-h`). Je Name eine Zeile: Name, Code, die Triaden B·A·Z, die Nummer in der Einzelbefehlsübersicht und die Kurzbeschreibung. Namen aus Projektdateien gibt die Option nicht aus.
- Fehler (jeweils mit Datei und Zeile, Rückgabecode 16, keine Ausgabedateien): markierter Name, der in keiner geladenen Tabelle steht (die Meldung nennt die geladenen Namensdateien); derselbe Name mit verschiedenem Code; Code, der kein dreistelliger Oktalcode ist; ungültiger Name; fehlende Namensdatei bei `;#OPSYM=`, fehlende Standardtabelle bei markiertem Namen; `;#OPSYM` ohne `=` oder ohne Pfad; mehr als 9 Zeilen `;#OPSYM=`; mehr als 1024 Namen; `opsym` ohne Namen vor dem Wort oder ohne gültigen Code. Ein Name ohne Punkt (`LD`) ist wie bisher ein ungültiger Operationscode.
- Unterschied zu IBM `OPSYN`: Dort wird ein vorhandener Operationscode zum Synonym eines anderen; hier wird ein Name einem dreistelligen Code zugeordnet. Die Standardtabelle enthält die folgenden 23 Namen (Codes und Nummern nach der Einzelbefehlsübersicht der Befehlsliste; maßgeblich ist die Einzelbefehlsübersicht, nicht die generische Tabelle 4.3 der Befehlsliste):

Name	Code	B·A·Z	Nr.	Bedeutung
JMP	000	0·0·0	1	unbedingter Sprung ohne Löschen, ohne Resultatbehandlung (Alias-Code 100)
STOP	020	0·2·0	14	unbedingter Stop 1. Art
JNEG	040	0·4·0	22	Negativsprung
JPOS	050	0·5·0	27	Positivsprung
JOVF	070	0·7·0	37	Überlaufsprung
JVG1	140	1·4·0	53	Vergleichersprung 1
JVG2	150	1·5·0	58	Vergleichersprung 2
JVG3	170	1·7·0	68	Vergleichersprung 3
NOP	164	1·6·4	66	Leer-Operation ohne Löschen
AND	200	2·0·0	73	Einzel-Konjunktion im Akkumulator
ADD	210	2·1·0	77	Einzel-Addition
LD	211	2·1·1	78	Einzel-Addition mit Löschen (ac := <a>)
SUB	220	2·2·0	85	Einzel-Subtraktion
LDN	221	2·2·1	86	Einzel-Subtraktion mit Löschen (ac := -<a>)
CADD	230	2·3·0	93	bedingte Addition/Subtraktion
STO	240	2·4·0	97	Einzel-Transport (<a> := <AC>)
STOC	241	2·4·1	98	Einzel-Transport mit Löschen
ANDS	250	2·5·0	102	Einzel-Konjunktion im Speicher
LDR	251	2·5·1	103	R-Lesebefehl (indirektes Lesen über berechnete Adresse)
ANDN	260	2·6·0	110	Konjunktion mit negiertem Akkumulator

Name	Code	B·A·Z	Nr.	Bedeutung
CLRP	270	2·7·0	114	bedingte Speicherlöschung (wenn <AC> >= 0: <a> := 0)
LDRN	271	2·7·1	115	R-Minuslesebefehl
CMP	330	3·3·0	142	reines Vergleichersetzen

Tests im Regressionstest: `Claude/opsymstandard` (mit Gegenstück `opsymstandard_okt` in Oktalcodes), `opsymmakro`, `opsymdatei`, `opsymzeile`, `opsymalias`; Negativtests `opsym_unbekannt`, `opsym_ohnepunkt`, `opsym_doppelt`, `opsym_code`, `opsym_datei`, `opsym_keinestandard` und `opsyms_ausgabe` (`--opsyms`). Das Skript `testscripte/opsym_tabelle.py [Namensdatei]` prüft jeden Eintrag einer Namensdatei (ohne Argument die Standardtabelle) gegen die Einzelbefehlsübersicht.

3.3.18 Komprimierter Text: 5 Zeichen im Eingabecode der Kodetabelle in einem Maschinenwort

```
[<symbol>:]  kat6  <kat6zeichen> [ <kat6zeichen> [...]]

<kat6zeichen>    ::= <kleinbuchstabe> | <dezimalziffer> | <leerzeichen>
                     | . | , | - | : | + | = | / | % | ` | <marke>
<kleinbuchstabe> ::= a | b | c | ... | z
<marke>          ::= $wrzl | $inkb | $ingb
```

- Das Textformat `kat6` (komprimierter Text, 5 Zeichen pro Maschinenwort) speichert je Zeichen den 6-Bit-Code, den die Eingabe der Maschine liefert: die sechs Bits `v u 8 4 2 1` des Lochbilds (v = 040, u = 020, 8 = 010, 4 = 04, 2 = 02, 1 = 01; Kodetabelle, Punkt 1.4, Spalte „Eingabe Adresse“). Anders als bei `kat5` sind die Codes weder frei numeriert noch alphabetisch; die Tabelle am Ende dieses Abschnitts zeigt sie.
- Ausgabe ohne Tabelle: Aus dem Code gewinnt ein Ausgabeprogramm die Ausgabeadresse durch Maskieren und Schieben: Zeichen (8 Bit) = (Code UND 017) ODER ((Code UND 060) mal 2), Ausgabeadresse = Zeichen mal 8, im Ausgabebefehl `0000 ssss 030` also ssss = Ausgabeadresse. Die Bits p und w bleiben 0, denn das Schreibwerk wertet nach der Kodetabelle nur die sechs Bits `v u 8 4 2 1` aus (C8205-Interpreter-Kodetabellen, Abschnitt 1.2). Beispiel: `a` hat den Code 021, das Zeichen 041 und die Ausgabeadresse 0410; die Kodetabelle nennt 0610, also dasselbe Zeichen mit gesetztem Bit p.
- Ein Wort enthält 5 Codes. Das erste Zeichen steht rechts (z27 bis z32), die weiteren folgen nach links: das zweite in z21 bis z26, das dritte in z15 bis z20, das vierte in z9 bis z14, das fünfte in z3 bis z8. z1 bleibt 0 (reserviert).
- Stoppbit statt Füllung: Der Code 00 ist die Ziffer 0; unbesetzte Stellen lassen sich deshalb nicht wie bei `kat5` mit 00 füllen. Stattdessen steht in jedem Wort unmittelbar links vom letzten Zeichen des Wortes ein Stoppbit 1: in einem vollen Wort in z2, in einem letzten Wort mit n Zeichen (n = 1, 2, 3, 4) in z26, z20, z14, z8. Alle Stellen links davon bleiben 0. Ein Ausgabeprogramm schiebt je Zeichen 6 Stellen nach rechts und ist mit dem Wort fertig, sobald nur noch das Stoppbit (Wert 1) übrig ist. Nullen im Text, auch am Ende, gehen so nie verloren.
- Ist der Text komplett, wird das letzte Wort mit z0 = 1 gekennzeichnet. Kein Wort ist 0, denn jedes enthält das Stoppbit. Ein Wort ohne Zeichen (nur z0 und das Stoppbit in z32) erzeugt der Assembler nicht; ein Ausgabeprogramm gibt dafür nichts aus. Der Assembler fügt keine Umschaltzeichen ein.
- Der Text belegt (n + 4) / 5 Zellen, ganzzahlig gerundet. n ist die Zahl der Zeichen, jede `<marke>` zählt als ein Zeichen. Ein leerer Operand ist ein Fehler.
- Die Marken `$wrzl` (Zeilenschaltung, Satzmarke, Code 032), `$inkb` (Umschalten auf Kleinbuchstaben, Code 057) und `$ingb` (Umschalten auf Großbuchstaben, Code 037) sind dieselben Namen wie bei `dz` (Abschnitt 3.3.6, Tabelle 2). Sie werden ohne Beachtung der Groß-/Kleinschreibung erkannt. Eine Marke endet nach ihrem Namen, das nächste Zeichen folgt unmittelbar. Ein `$` ohne gültigen Markennamen ist ein Fehler.
- Leerzeichen innerhalb des Textes (`kat6 a b`) sind auch ohne `\` zulässig. Führende und abschließende Leerzeichen werden sonst entfernt; sollen sie erhalten bleiben, steht der Text zwischen `\` und `\`, wie bei `kat4`, `kat5` und `dt`.
- Zeichenvorrat: Er umfasst alle Tasten der Kodetabelle, die ein Zeichen schreiben, mit Ausnahme von `$` (Taste NS): Kleinbuchstaben, Ziffern, Leerzeichen, `. , - : + / %`, der Akzent `` ` `` und `=`. Gegenüber kat5 kommen `/`, `%` und `` ` `` hinzu. Großbuchstaben und alle anderen Zeichen außerhalb des Vorrats sind ein Fehler; die Meldung nennt die Position des Zeichens, wie bei kat4`. Tasten ohne Schreibzeichen (Leserstopp, Tab, WR, ZL und weitere) gehören nicht zum Vorrat; ihre Codes sind in der Tabelle als frei bezeichnet.

- `+` und `=` liegen auf derselben Taste und haben deshalb denselben Code (020). Ob `+` oder `=` erscheint, bestimmt der Zustand KB/GB beim Ausgeben. Wer `=` ausgeben will, setzt `$ingb` davor und danach `$inkb`. Ebenso zeigt im Zustand GB jede andere Taste ihr GB-Zeichen: `:` erscheint als `(`, `/` als `)`, `%` als `*`, der Akzent als `;`, Buchstaben als Großbuchstaben (Spalte GB der Tabelle).
- Warnung bei Zustandsunstimmigkeit: Der Assembler führt innerhalb des Operanden den Zustand KB/GB mit. Er ist am Textanfang unbekannt, `$inkb` setzt ihn auf KB, `$ingb` auf GB. Ist der Zustand bekannt, warnt der Assembler, wenn
  - im Zustand GB eine Ziffer, `.`, `,`, `-`, `:`, `+`, `/`, `%` oder der Akzent steht (die Ausgabe zeigt ein anderes Zeichen), oder
  - im Zustand KB ein `=` steht (die Ausgabe zeigt `+`).

Buchstaben, Leerzeichen und `$wrzl` lösen keine Warnung aus. Die Warnung ist kein Fehler und ändert den Code nicht. Der Zustand wird nicht über das Ende der Anweisung hinaus verfolgt.

- `kat6` ist wie `kat4` und `kat5` eine Datenanweisung: Der Optimierer verlegt sie nicht, die Belegungsprüfung zählt ihre Zellen. Im Listing steht das erste Wort in der Quellzeile, jedes weitere Wort in einer Folgezeile.

Code (oktal)	KB	GB
00 bis 07	0 bis 7	# < " ≈ [> ' ≋
010, 011	8, 9	~]
012	Leerzeichen	Leerzeichen
015	:	(
020	+	=
021 bis 031	a bis i (a = 021, b = 022, … i = 031)	A bis I
032	`$wrzl` (Zeilenschaltung)	dasselbe
033	.	!
037	`$ingb` (Um. GB)	–
040	-	△
041 bis 051	j bis r (j = 041, k = 042, … r = 051)	J bis R
057	`$inkb` (Um. KB)	–
060	`` ` ` | ;` ``	
061	/	)
062 bis 071	s bis z (s = 062, t = 063, … x = 067, y = 070, z = 071)	S bis Z
073	,	?
074	%	*
013, 014, 016, 017, 034 bis 036, 052 bis 056, 072, 075 bis 077	frei (Funktionstasten und $); solange nicht definiert, ein Fehler	

Beispiele (Wörter oktal, erstes Zeichen rechts, Stoppbit über dem letzten Zeichen, z0 = 1 im letzten Wort):

```
        kat6    hallo welt          ; 2 Zellen: 14643432130 und 56343256612
        kat6    10                  ; 1 Zelle:  40000010001 (Ziffer 0 = Code 00, Stoppbit in z20)
        kat6    000                 ; 1 Zelle:  40001000000 (drei Nullen gehen nicht verloren)
        kat6    x$ingb=$inkb1$wrzl  ; 2 Zellen: 10157203767 und 40000000132
        kat6    a/b                 ; 1 Zelle:  40001226121
        kat6    $ingb=1             ; Warnung: Ziffer 1 im Zustand GB
```

Zum Vergleich: `kat4` braucht für den Text `hallo welt` 3 Zellen, `kat6` 2, `kat5` ebenfalls 2 (in zwei Anweisungen zu höchstens 5 Zeichen). `kat6` hat dieselbe Wortdichte wie `kat5` (20 % weniger Zellen als `kat4` bei langen Texten) und einen größeren Zeichenvorrat. Ein Ausgabeprogramm für `kat6` braucht keine Tabelle der Ausgabebefehle, die bei `kat5` 45 Zellen kostet (Stand c82ass 1.02-003).

3.3.19 Definiere Wert binär (dwb)

```
[<symbol>]  dwb  <binärzahl>

<binärziffer>  ::= 0 | 1
<binärzahl>    ::= binärziffer [ binärziffer [ binärziffer … ]]    (1 bis 33 Stellen)
```

- Speichert die Bitfolge von `<binärzahl>` als Wort auf dem `<speicherplatz>`: Jede Ziffer ist ein Bit des Wortes. Die letzte (rechte) Ziffer ist das niedrigste Bit z32; die erste Ziffer einer 33stelligen Zahl ist das höchste Bit z0 (das 33. Bit ganz links, bei `dw` das Vorzeichenbit). Hat die Zahl weniger als 33 Stellen, steht sie rechtsbündig im Wort, und die Stellen links davon sind 0. Führende Nullen sind erlaubt und zählen als Stellen: `dwb 101` und `dwb 0000101` ergeben beide das Wort 5.
- Der Operand besteht nur aus den Ziffern 0 und 1, mindestens einer und höchstens 33 Stellen, ohne Vorzeichen, Präfix, Symbol, Rechenzeichen, Klammer und Trenner. Eine führende 0 bedeutet hier nicht oktal: Die Ziffernfolge wird immer dual gelesen, so wie sie nach dem Auflösen der Makros im Operanden steht (`dwb 10` und `dwb 010` ergeben beide das Wort 2).
- `dwb` schreibt ein Bitmuster (Maske, Schalter, Befehlsbits), `dw` eine Zahl. Jedes der 2^33 Wörter lässt sich mit `dwb` schreiben; 33 Einsen ergeben dasselbe Wort wie `dw -1` und `dw 8589934591` (`77777777777`).
- Fehler: Ein leerer Operand, ein Zeichen außer 0 und 1 (zum Beispiel 2, `-`, `b`) und mehr als 33 Stellen meldet der Assembler wie bei `dw` in Pass 3 als `Operand ist ungültig`; die Meldungszeile nennt den Operanden (Tabelle der Fehlerausgaben in 3.1).
- Es wird keine Hauptleitadresse addiert. In der Objektdatei steht das Wort wie bei `dw` als `00<elf Oktalziffern>.`, nur das Wort 0 kurz als `0.`. Die 33 Bit sind 11 Oktalziffern zu je 3 Bit; die erste Oktalziffer enthält z0 bis z2, die letzte z30 bis z32.
- `dwb` belegt genau eine Zelle und verhält sich sonst wie `dw`: Die Belegungsprüfung (Pass 3) prüft sie, der Optimierer (Abschnitt 5) verlegt sie nie, sie ist zwischen `begin` und `bend` erlaubt, `zellen(marke)` ergibt 1 (4.23), und die Warnung `SPR` (3.1) zählt sie wie `dw` als Datenwort, wenn ihr Wert ungleich 0 ist. Eine Marke auf der Zeile ist die Adresse der Zelle.

Beispiele (Wörter oktal, 11 Ziffern zu je 3 Bit; die erste Ziffer enthält z0 bis z2):

```
        dwb     101                                 ; Wort 5: 00000000005
        dwb     0000101                             ; ebenso 5 (führende Nullen)
        dwb     1111                                ; 15: 00000000017
        dwb     001000000                           ; 9 Stellen, 64: 00000000100
        dwb     0                                   ; Wort 0, in der Objektdatei „0.“
        dwb     10000000000000000000000000000000    ; z0 = 1: 40000000000
        dwb     10101010101010101010101010101010    ; z0 = 1: 52525252525
        dwb     11111111111111111111111111111111    ; alle 33 Bit: 77777777777
        dwb     100000000000000000000000000000000   ; Fehler: 34 Stellen
        dwb     102                                 ; Fehler: Ziffer 2
        dwb                                         ; Fehler: Operand fehlt
```

3.3.19a Definiere Wert hexadezimal (dwh)

```
[<symbol>]  dwh  <hexzahl>

<hexziffer>  ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | A | B | C | D | E | F | a | b | c | d | e | f
<hexzahl>    ::= hexziffer [ hexziffer [ hexziffer … ]]    (1 bis 8 Stellen)
```

- Speichert die Hexadezimalzahl `<hexzahl>` als Wort auf dem `<speicherplatz>`: Jede Ziffer sind 4 Bit. Die letzte (rechte) Ziffer sind die niedrigsten Bits z29 bis z32; die Zahl steht rechtsbündig im Wort, und die Stellen links davon sind 0. Acht Ziffern füllen die Bits z1 bis z32; das höchste Bit z0 (bei `dw` das Vorzeichenbit) bleibt immer 0. Wörter mit z0 = 1 schreibt man mit `dwb` (3.3.19) oder `dw` (3.3.3). Führende Nullen sind erlaubt und zählen als Stellen: `dwh FF` und `dwh 000000FF` ergeben beide das Wort 0377.
- Der Operand besteht nur aus den Ziffern 0 bis 9 und den Buchstaben A bis F oder a bis f (Groß- und Kleinschreibung gelten gleich), mindestens einer und höchstens 8 Stellen, ohne Vorzeichen, Präfix `0x`, Symbol, Rechenzeichen, Klammer und Trenner. Die Ziffernfolge wird immer hexadezimal gelesen, so wie sie nach dem Auflösen der Makros im Operanden steht: `dwh 10` ergibt das Wort 16 (020), `dwh 010` ebenfalls. Eine Folge aus Hexziffern ist nie ein Symbol: `dwh cafe` ist das Wort 0xCAFE (0145376), auch wenn im Programm eine Marke `cafe` steht.
- `dwh` schreibt ein Bitmuster in Vierergruppen (Maske, Befehlsbits, Adresse), `dw` eine Zahl, `dwb` ein Bitmuster Bit für Bit. Das Wort ist die Zahl, die die Ziffern bedeuten: `dwh 7FFFFFFF` ergibt 2^31 - 1 (`17777777777`), `dwh FFFFFFFF` ergibt 2^32 - 1 (`37777777777`).

- Fehler: Ein leerer Operand, ein Zeichen außer 0 bis 9 und A bis F (zum Beispiel G, `-`, `x`, Punkt) und mehr als 8 Stellen meldet der Assembler in Pass 3 als `Operand '<operand>' ist ungültig (dwh erwartet 1 bis 8 Hexadezimalziffern 0-9 und A-F)`; die Meldungszeile nennt den Operanden (Tabelle der Fehlerausgaben in 3.1). Ein Präfix gibt es nicht: `dwh 0xFF` ist ein Fehler, und `dw 0xFF` meldet `Operand '0xFF' ist ungültig`; richtig ist `dwh FF` oder `dw $CALC(0xFF)` (4.28).
- Es wird keine Hauptleitadresse addiert. In der Objektdatei steht das Wort wie bei `dw` und `dwb` als `00<elf Oktalziffern>.`, nur das Wort 0 kurz als `0.`.
- `dwh` belegt genau eine Zelle und verhält sich sonst wie `dw` und `dwb`: Die Belegungsprüfung (Pass 3) prüft sie, der Optimierer (Abschnitt 5) verlegt sie nie, sie ist zwischen `begin` und `bend` erlaubt, `zellen(marke)` ergibt 1 (4.23), und die Warnung `SPR` (3.1) zählt sie wie `dw` als Datenwort, wenn ihr Wert ungleich 0 ist. Eine Marke auf der Zeile ist die Adresse der Zelle.
- Im Makro erzeugt `$CALCH(...)` (4.28) die Hexziffern: `dwh $CALCH(255)` ergibt das Wort 0377. Vorsicht: `dwh $CALC(255)` liest die Dezimalziffern 255 hexadezimal und ergibt das Wort 0x255 (01125); wer eine Rechnung als Hexzahl schreiben will, nimmt `$CALCH`, nicht `$CALC`.
- Gilt seit c82ass 1.01-023; ein älteres Binary meldet `Operationscode 'dwh' ist ungültig`.

Beispiele (Wörter oktal, 11 Ziffern zu je 3 Bit; die erste Ziffer enthält z0 bis z2):

```
        dwh     FF                  ; 255: 00000000377
        dwh     000000FF            ; ebenso 255 (führende Nullen)
        dwh     ff                  ; ebenso 255 (Kleinbuchstaben)
        dwh     0                   ; Wort 0, in der Objektdatei „0.“
        dwh     7FFFFFFF            ; 2^31 - 1: 17777777777
        dwh     80000000            ; 2^31: 20000000000
        dwh     DEADBEEF            ; 33653337357
        dwh     FFFFFFFF            ; 2^32 - 1: 37777777777
        dwh     cafe                ; 0xCAFE, kein Symbol: 00000145376
        dwh     123456789           ; Fehler: 9 Stellen
        dwh     12G4                ; Fehler: Ziffer G
        dwh     0xFF                ; Fehler: Präfix
        dwh     -1                  ; Fehler: Vorzeichen
        dwh                         ; Fehler: Operand fehlt
```

3.3.20 Definiere Festkommazahl (gebrochen) (dwf)

```
[<symbol>]  dwf  <festkommazahl>

<festkommazahl>  ::= [ + | - ] dezimalziffer [ dezimalziffer … ] [ . dezimalziffer [ dezimalziffer … ] ]
```

- Speichert den Dezimalbruch von `<festkommazahl>` als gebrochene Festkommazahl im Wort auf dem `<speicherplatz>`. Die Deutung des Wortes ist die der Dualzahl in der Interpreter-Datenstruktur (Abschnitt „Dualzahlen“; Programmieranleitung 6.1): Das Komma steht zwischen z0 und z1, z0 ist das Vorzeichen, z32 ist das niedrigste Bit mit dem Gewicht 2^-32. Eine positive Zahl x ist die Summe der zi · 2^-i (i = 1 bis 32) mit z0 = 0; eine negative Zahl steht im Zweierkomplement mit z0 = 1. Das Wort ist also x · 2^32, auf eine ganze Zahl gerundet und bei negativem x modulo 2^33.
- Zahlenbereich: -1 ≤ x ≤ 1 - 2^-32 (das ist 0.99999999976716935634613037109375). Das ergibt die Wörter `40000000000` (-1) bis `37777777777`; die kleinste positive Zahl ist 2^-32 (Wort 1). Die 1 und größere Zahlen sind ein Fehler, weil sie als gebrochene Zahl nicht darstellbar sind; die ganze Zahl 1 schreibt man `dw 1`.
- Rundung: Ein Dezimalbruch wie 0.1 ist als Dualbruch meist nicht genau darstellbar. Der Assembler wertet den Operanden exakt aus (ohne Zwischenrundung, nicht in Gleitkommaarithmetik), rundet x · 2^32 auf die nächste ganze Zahl (genau in der Mitte weg von 0) und prüft erst das gerundete Wort gegen den Zahlenbereich; `-1.0000000001` ergibt deshalb das Wort von -1. Eine Warnung bei Rundung gibt es nicht.
- Null: `0`, `0.0`, `-0` und jeder Wert, der zu 0 rundet, ergeben das Wort 0; das Zweierkomplement kennt keine negative Null.
- Operand: Ziffern mit höchstens einem Dezimalpunkt und einem Vorzeichen `+` oder `-` am Anfang. Vor dem Punkt steht mindestens eine Ziffer, nach dem Punkt, wenn er da ist, ebenfalls. Nicht zulässig sind ein Dezimalkomma (`0,5`), ein Exponent (`1E-3`; dafür gibt es `dwg`, 3.3.21), ein Symbol, ein Ausdruck, ein Literal (`#0.5`) und ein Trenner im Operanden. Die Ziffernfolge ist immer dezimal: Eine führende 0 bedeutet hier nicht oktal. Die Zahl der Stellen begrenzt nur die Zeilenlänge (3.2). Dezimalbrüche gibt es nur als Operand von `dwf` und `dwg`, nicht im `<werteausdruck>` (3.2).

- Ganze Festkommazahlen (Komma hinter z32; Programmieranleitung 6.2) schreibt man mit `dw` (3.3.3): Der Zahlenbereich der Interpreter-Datenstruktur, -2^32 bis 2^32 - 1, liegt im Bereich von `dw`. Eine Bitfolge schreibt man mit `dwb` (3.3.19), eine Hexadezimalzahl mit `dwh` (3.3.19a).
- Fehler: Ein leerer Operand, ein Zeichen außer Ziffern, Punkt und dem Vorzeichen am Anfang (zum Beispiel Komma, `E`, Buchstabe), ein zweiter Punkt, ein Punkt ohne Ziffer davor oder danach und ein Wert, dessen gerundetes Wort außerhalb von -1 bis 1 - 2^-32 liegt, meldet der Assembler wie bei `dwb` in Pass 3 als `Operand ist ungültig`; die Meldungszeile nennt den Operanden und in Klammern den Grund (Aufbau oder Zahlenbereich; Tabelle der Fehlerausgaben in 3.1).
- Es wird keine Hauptleitadresse addiert. In der Objektdatei steht das Wort wie bei `dw` als `00<elf Oktalziffern>.`, nur das Wort 0 kurz als `0.`. `dwf` belegt genau eine Zelle und verhält sich sonst wie `dw`: Die Belegungsprüfung (Pass 3) prüft sie, der Optimierer (Abschnitt 5) verlegt sie nie, sie ist zwischen `begin` und `bend` erlaubt, `zellen(marke)` ergibt 1 (4.23), und die Warnung `SPR` (3.1) zählt sie wie `dw` als Datenwort, wenn ihr Wort ungleich 0 ist. Eine Marke auf der Zeile ist die Adresse der Zelle.

Beispiele (Wörter oktal, 11 Ziffern zu je 3 Bit; die erste Ziffer enthält z0 bis z2):

```
        dwf     0.5                                 ; 20000000000: z1 = 1
        dwf     0.25                                ; 10000000000: z2 = 1
        dwf     0.75                                ; 30000000000: z1 und z2 = 1
        dwf     -0.5                                ; 60000000000: Zweierkomplement, z0 = 1
        dwf     -1                                  ; 40000000000: kleinster Wert, nur z0 gesetzt
        dwf     0                                   ; Wort 0, in der Objektdatei „0.“
        dwf     0.1                                 ; 03146314632: 0.1 · 2^32 = 429496729.6, gerundet 429496730
        dwf     -0.1                                ; 74631463146: Zweierkomplement von 03146314632
        dwf     0.00000000023283064365386962890625  ; 00000000001: 2^-32, kleinste positive Zahl
        dwf     0.99999999976716935634613037109375  ; 37777777777: größter Wert, 1 - 2^-32
        dwf     00.50                               ; 20000000000: führende 0 ist dezimal, wie 0.5
        dwf     1                                   ; Fehler: nicht kleiner als 1 (ganze Eins: dw 1)
        dwf     0,5                                 ; Fehler: Komma statt Punkt
        dwf     .5                                  ; Fehler: Ziffer vor dem Punkt fehlt
        dwf     1E-3                                ; Fehler: Exponent nur bei dwg
        dwf                                         ; Fehler: Operand fehlt
```

3.3.21 Definiere Gleitkommazahl (dwg)

```
[<symbol>]  dwg  <gleitkommazahl>

<gleitkommazahl>  ::= <festkommazahl> [ E [ + | - ] dezimalziffer [ dezimalziffer [ dezimalziffer ]]]
                      (<festkommazahl> wie in 3.3.20; E auch als e; Exponent höchstens 3 Ziffern)
```

- Speichert die Dezimalzahl von `<gleitkommazahl>` als Gleitkommazahl in der internen Darstellung auf dem `<speicherplatz>` (Interpreter-Datenstruktur, Abschnitt „Gleitkommazahlen, interne Darstellung“; Programmieranleitung 6.3, Abb. 28): z = ± m · 2^e mit normalisierter Mantisse 0,5 ≤ |m| < 1 und dem um 32 erhöhten Exponenten e' = e + 32. Das Wort besteht aus z0 (Vorzeichen der Mantisse, 0 = +, 1 = -, Vorzeichen-Betrags-Darstellung, kein Zweierkomplement), z1 bis z6 (Maschinenexponent e', z1 höchstes Bit, z6 niedrigstes) und z7 bis z32 (Mantissenbetrag m1 bis m26; z7 = m1 ist stets 1). Der Rechner kennt keine Gleitkommabefehle; das Wort ist eine Programmkonvention.
- Zahlenbereich: Exponent 0 ≤ e' ≤ 63 (-32 ≤ e ≤ 31), 2^-33 ≤ |z| ≤ (1 - 2^-26) · 2^31 = 2^31 - 2^5 = 2147483616, und z = 0. Die kleinste positive Zahl ungleich 0 ist 2^-33 (1.16415321826934814453125E-10, Wort `00200000000`), die größte ist 2147483616 (Wort `37777777777`).
- Schreibweise: `E` (auch `e`) leitet einen Zehnerexponenten ein, z = Mantisse · 10^E (`1.5E3` ist 1500, `-2.5E-3` ist -0.0025). Die Mantisse der Schreibweise ist nicht normalisiert und nicht auf acht Stellen beschränkt (anders als die externe Gleitkommadarstellung des BIS, BIS-1.1); der Exponent hat höchstens drei Ziffern. Eine Zahl ohne Punkt und ohne Exponent ist erlaubt (`dwg 16`). Für Mantisse, Dezimalpunkt und Vorzeichen gilt dasselbe wie bei `dwf` (3.3.20): Dezimalpunkt statt Komma, Vorzeichen nur am Anfang, keine Symbole, Ausdrücke, Literale und Trenner, Ziffern immer dezimal.
- Umrechnung: Der Assembler bildet den Wert exakt (ohne Zwischenrundung, nicht in Gleitkommaarithmetik), findet e mit 0,5 ≤ |z| / 2^e < 1 und rundet die Mantisse auf 26 Bit (zur nächsten Zahl, genau in der Mitte weg von 0). Ergibt die Rundung 1, wird m = 0,5 und e um 1 größer. Erst danach prüft er e' = e + 32 gegen 0 bis 63. 26 Bit entsprechen etwa 7 Dezimalstellen; weitere Stellen der Eingabe beeinflussen nur die Rundung. Eine Warnung bei Rundung gibt es nicht.
- Null: `0`, `0.0`, `0E5` und `-0` ergeben das Wort 0 (e' = 0, Mantisse 0). Die „negative Null“ (nur z0 gesetzt) ist keine gültige Darstellung und wird nie erzeugt. Nur ein Wert, der genau 0 ist, ergibt die Null; ein Betrag ungleich 0, der auch nach der Rundung unter 2^-33 liegt, wird nicht zu 0, sondern ist ein Fehler.

- Fehler: wie bei `dwf` (leerer Operand, unzulässiges Zeichen, Komma statt Punkt, zweiter Punkt, Punkt ohne Ziffer davor oder danach), außerdem ein `E` ohne Ziffern, ein Exponent mit mehr als drei Ziffern, ein Betrag, dessen gerundete Zahl größer als 2^31 - 2^5 wäre (e' > 63; auch 2147483632 und 2147483647 runden auf 2^31), und ein Betrag ungleich 0, dessen gerundete Zahl kleiner als 2^-33 wäre (e' < 0); ein Betrag knapp unter 2^-33, der auf 2^-33 rundet, ist gültig. Der Assembler meldet sie wie bei `dwb` in Pass 3 als `Operand ist ungültig`; die Meldungszeile nennt den Operanden und in Klammern den Grund.
- Objektdatei und Verhalten wie bei `dwf` und `dw`: keine Hauptleitadresse, das Wort steht als `00<elf Oktalziffern>.`, nur das Wort 0 kurz als `0.`; genau eine Zelle, Belegungsprüfung (Pass 3), der Optimierer verlegt sie nie, erlaubt zwischen `begin` und `bend`, `zellen(marke)` ergibt 1 (4.23), `SPR` (3.1) zählt sie als Datenwort, wenn das Wort ungleich 0 ist. Eine Marke auf der Zeile ist die Adresse der Zelle.

Beispiele (Wörter oktal, 11 Ziffern zu je 3 Bit; die erste Ziffer enthält z0 bis z2):

```
        dwg     16                                  ; 22600000000: 0.5 · 2^5, e' = 37
        dwg     -16                                 ; 62600000000: wie das Beispiel der Interpreter-Datenstruktur,
    z0 = 1
        dwg     0.5                                 ; 20200000000: e = 0, e' = 32, m = 0.5
        dwg     1                                   ; 20600000000: 0.5 · 2^1
        dwg     -1                                  ; 60600000000
        dwg     0.75                                ; 20300000000: m = 0.75, e = 0
        dwg     12.5                                ; 22310000000: 0.78125 · 2^4
        dwg     0.1                                 ; 16714631463: 0.8 · 2^-3, Mantisse auf 26 Bit gerundet
        dwg     1E3                                 ; 25372000000: 1000 = 0.9765625 · 2^10
        dwg     1.5E3                               ; 25673400000: 1500
        dwg     -2.5E-3                             ; 54243656051: -0.0025
        dwg     0                                   ; Wort 0, in der Objektdatei „0.“
        dwg     -0.0                                ; Wort 0 (keine negative Null)
        dwg     2147483616                          ; 37777777777: größte Zahl, 2^31 - 2^5
        dwg     1.16415321826934814453125E-10       ; 00200000000: kleinste Zahl, 2^-33
        dwg     1.1641532182693481 4E-10            ; 00200000000: knapp unter 2^-33, rundet auf 2^-33 (gültig)
        dwg     2147483632                          ; Fehler: rundet auf 2^31, e' wäre 64
        dwg     1E10                                ; Fehler: größer als 2^31 - 2^5
        dwg     1E-11                               ; Fehler: kleiner als 2^-33
        dwg     1,5                                 ; Fehler: Komma statt Punkt
        dwg     1E                                  ; Fehler: Exponent ohne Ziffern
        dwg                                         ; Fehler: Operand fehlt
```

Tabelle 2: Sonderzeichen für die dz-Anweisung

Spalte 1: String, der als Operand bei `dz` anzugeben ist. Spalte 2: 4 Oktalziffern, die die 8 Bit des 030-Ausgabecodes enthalten.

Sonderzeichen-String	Oktalcode (030-Ausgabe)
`$wrzl`	02720
`$sm`	02720
`$inkb`	01170
`$ingb`	0570
`$irrz`	03770
`$irrs`	0160
`$trans`	01770
`$bm`	03520
`$vorl`	0
`$irrb`	0760
`$zr`	0320
`$tab`	01550
`$wr`	01120
`%`	01740
`:`	0150
`+`	0400
`` ` ``	01600

Sonderzeichen-String	Oktalcode (030-Ausgabe)
/	01410
,	01530
.	0730
-	01000

Tabelle 3: Ausgabe-Umschlüsselung außer Sonderzeichen dz

Spalten: ASCII-Kleinbuchstabe, ASCII-Großbuchstabe, Oktalcode (8 Bit des 030-Ausgabecodes), ASCII-Code Großbuchstabe (dez.), ASCII-Code Kleinbuchstabe (dez.).

Klein	Groß	Oktalcode	ASCII Groß (dez)	ASCII Klein (dez)
1	<	0010	60	49
2	"	0020	34	50
3	≈	0230	0	51
4	[	0040	91	52
5	>	0250	62	53
6	'	0260	39	54
7	≋	0070	0	55
8	~	0100	126	56
9	]	0310	93	57
0	#	0200	35	48
%	*	1740	42	37
Tabulator	Tabulator	1550	9	9
q	Q	1300	81	113
w	W	1660	87	119
e	E	0450	69	101
r	R	1110	82	114
t	T	1630	84	116
z	Z	1710	90	122
u	U	1440	85	117
i	I	0510	73	105
o	O	1060	79	111
p	P	1270	80	112
:	(	0150	40	58
+	=	0400	61	43
a	A	0610	65	97
s	S	1420	83	115
d	D	0640	68	100
f	F	0460	70	102
g	G	0670	71	103
h	H	0700	72	104
j	J	1210	74	106
k	K	1220	75	107
l	L	1030	76	108
` | ; | 1600 | 59 | 96				

Klein	Groß	Oktalcode	ASCII Groß (dez)	ASCII Klein (dez)
/	)	1410	41	47
SatzMarke	WRZL	2720	10	10
y	Y	1500	89	121
x	X	1470	88	120
c	C	0430	67	99
v	V	1650	86	118
b	B	0620	66	98
n	N	1050	78	110
m	M	1240	77	109
,	?	1530	63	44
.	!	0730	33	46
-	—	1000	—	45
Leertaste	ZwR	0320	32	32

4. Der Makro-Präprozessor des Assemblers

Der Präprozessor ist sehr einfach gehalten und hat eine starre Syntax. Er prüft, ob eine Operation ein Makroname ist. Die Prüfung erfolgt in der Textdatei `c82ass.makros`, die erweitert werden darf. Der Assembler sucht sie unter diesem festen Namen zuerst im Arbeitsverzeichnis des Aufrufs, sonst unter `Sourcecode/c82ass.makros`; fehlt sie an beiden Orten, bricht er mit einer Fehlermeldung ab. Zusätzlich können Makros direkt im Quelltext definiert werden (Abschnitt 4.1). Mit `$INCLUDE <Pfad>` liest der Assembler ab c82ass 1.01-009 außerdem eine Datei an dieser Stelle in den Quelltext ein (Abschnitt 4.14).

- Jedes dort definierte Makro beginnt mit dem Makronamen, unmittelbar gefolgt von einem Doppelpunkt und danach (getrennt durch Tab oder Leerzeichen) dem Schlüsselwort `MACRO`.
- Müssen dem Makro Parameter übergeben werden, folgt nach `MACRO`, getrennt durch Leerzeichen, die Anzahl der Parameter. Es sind maximal 9 Parameter zulässig. Ab c82ass 1.01-014 dürfen statt der Anzahl Namen stehen (bis zu 32 Parameter, Abschnitt 4.26); im Rumpf steht dann `$name` statt `$1` bis `$9`.
- Danach kommen die Definitionsanweisungen des Makros. Sie können die formalen Parameter `$x` mit x=1..9 enthalten, die durch die Übergabeparameter ggf. mehrfach ersetzt werden; in einem Makro mit Namen (4.26) steht `$name` an dieser Stelle.
- Jedes Makro wird durch das Schlüsselwort `MEND` abgeschlossen (Leerraum davor ist beliebig).

Im Definitionsteil kann für jeden Parameter eine Prüfliste angegeben werden. Ist sie angegeben, darf der betreffende Parameter nur die Werte der Liste annehmen. Eine Prüfliste wird durch `$PRx` mit x=1..9 definiert (in einem Makro mit Namen auch durch `$PR(name)`, 4.26); sie muss auf einer Zeile für sich stehen.

Statt einer Werteliste kann hinter `$PRx` eine Typangabe stehen (z. B. `$PR1: $TYPADR`). Dann prüft der Assembler beim Makroaufruf, ob der übergebene Wert zum Typ passt; sonst meldet er einen Fehler `Parameter $x: Wert "<wert>" …`:

Typ	Zulässiger Wert beim Aufruf
`$TYPADR`	eine symbolische Adresse (ein Label, z. B. `marke`) oder eine Sternadresse (`*`, `*+3`); keine Zahl
`$TYPKON`	eine Oktalkonstante: nur Ziffern 0–7, beginnt mit `0` (z. B. `0100`); `100` oder `09` sind Fehler
`$TYPABS`	ein Symbol, das im Quelltext mit `abs` definiert ist (z. B. `absx: abs 05`)

Die Prüfung erfolgt vor der eigentlichen Übersetzung und ist deshalb nur syntaktisch: Bei `$TYPADR` wird nicht geprüft, ob das Label im Programm vorkommt (das meldet die spätere Symbolauflösung).

Die Definition der zu generierenden Anweisungen kann durch eine `$IF`-`$ELIF`-`$ELSE`-`$ENDIF`-Struktur gesteuert werden (ausführlich in 4.4). Sie kann mehrfach auftreten und beliebig geschachtelt werden (bis zur Tiefe 16). Die Schlüsselworte `$IF`, `$ELIF`, `$ELSE`, `$ENDIF` müssen jeweils auf einer Zeile für sich stehen. `$ELIF` und `$ELSE` sind wahlfrei; `$ELSE` steht höchstens einmal je `$IF` und hinter allen `$ELIF`. Wiederholungen erzeugt die Schleife `$REPT` … `$ENDREPT` (4.3). Diese Zeilen gelten im Rumpf eines Makros; ab c82ass 1.01-008 wirken sie ebenso im normalen Quelltext (4.1).

Hinter `$IF` und `$ELIF` muss mindestens einer, können bis zu 8 Bedingungsblöcke angegeben werden. Ein Bedingungsblock besteht aus 3 Strings: `<wert1> <vergleichszeichen> <wert2>`. Als `<vergleichszeichen>` sind `>`, `<`, `=`, `>=`, `<=`, `!=` und `<>` zugelassen. Jeder folgende Bedingungsblock ist durch `|` (logisch ODER) oder `&` (logisch UND) zu verknüpfen.

Vorrang (ab c82ass 1.01-031): Ohne Klammern bindet `&` stärker als `|`, wie in C; gleiche Verknüpfungszeichen werden von links nach rechts ausgewertet. Die Bedingungsblöcke werden also zuerst durch `&` zu Gruppen zusammengefasst, und `|` verknüpft die Gruppen; das entspricht einer impliziten Klammerung der Art:

```
<bedingungsblock1> | <bedingungsblock2> & <bedingungsblock3> | <bedingungsblock4> & <bedingungsblock5>
gilt als: <bedingungsblock1> | ( <bedingungsblock2> & <bedingungsblock3> ) | ( <bedingungsblock4> &
    <bedingungsblock5> )
```

Bis c82ass 1.01-030 wurden die Bedingungsblöcke ohne Klammern von links nach rechts verknüpft (kein Vorrang von `&` vor `|`); `a | b & c` galt dort als `(a | b) & c` und gilt jetzt als `a | (b & c)`. Ein Quelltext oder eine Makrodatei, die sich auf die alte Reihenfolge verlassen hat, muss die Klammern selbst setzen. Die Programme und Makrodateien des Projekts und des Regressionstests verhalten sich unverändert; der Test `Assembler/Claude/makro_stolperfallen` (Fall 4) liefert mit dem alten Verhalten ein anderes Ergebnis.

Soll anders gruppiert werden, stehen die Wörter `(` und `)`, jeweils durch Leerzeichen getrennt, um Bedingungsblöcke, z. B. `$IF ( $1 = 1 | $2 = 1 ) & $3 = 1`. Klammern dürfen geschachtelt werden und zählen nicht als Parameter.

Innerhalb eines Definitionsblockes sind Kommentare zugelassen: `#` leitet einen Kommentar ein, der nur in der Makrodefinition gilt und auf einer eigenen Zeile stehen muss; `;` leitet einen Kommentar ein, der auch generiert wird — er kann auf einer eigenen Zeile oder in der Anweisung vorkommen.

Beispiel:

```
ANWAHL:  MACRO 1
  $PR1 4000 4100 4200 4300 4400 4500 4600 4700 K0 K1 K2 K3 K4 K5 K6 K7 LSL1 LSL2 LKL1 LKS1 LSS1 SW1I SW1O SW2O KONI
  # Generieren einer Anwahl für Zeichenkanäle
      $IF  $1  =  4000 | $1 = K7
         030 04000  ; 4000 oder K7
      $ELIF  $1  =  4100 | $1 = K6
         030 04100  ; 4100 oder K6
      $ELIF  $1  =  4200 | $1 = K5
         030 04200  ; 4200 oder K5
      $ELIF  $1  =  4300 | $1 = K4 | $1 = LKS1
         030 04300  ; 4300 oder K4 oder LKS1
      $ELIF  $1  =  4400 | $1 = K3 | $1 = LKL1 | $1 = SW2O
         030 04400  ; 4400 oder K3 oder LKL1 oder SW2 Output
      $ELIF  $1  =  4500 | $1 = K2 | $1 = LSL2 | $1 = SW1O
         030 04500  ; 4500 oder K2 oder LSL2 oder SW1 Output
      $ELIF  $1  =  4600 | $1 = K1 | $1 = LSL1 | $1 = LSS1
         030 04600  ; 4600 oder K1 oder LSL1 oder LSS1
      $ELIF  $1  =  4700 | $1 = K0 | $1 = SW1I | $1 = KONI
         030 04700  ; 4700 oder K0 oder SW1 Input oder KON Input
      $ENDIF
         MEND
```

Aufruf: `ANWAHL LSS1` erzeugt: `030  04600  ; 4600 oder K1 oder LSL1 oder LSS1`

Der Makroaufruf im Assemblerprogramm darf ein Label haben, z. B.:

```
input_loop:  ANWAHL  LSL1
             …
```

In einem solchen Fall wird generiert:

```
input_loop:  030  04600  ; 4600 oder K1 oder LSL1 oder LSS1
```

Der Makroaufruf selbst wird nicht als Kommentarzeile (`;  ANWAHL  LSL1`) ausgegeben; er erscheint nicht im Listing und nicht in `.mqo`, `.dmp` oder `.test`. Ausnahme: Mit `--makros` steht im Listing und in `.dmp` vor jeder Expansion eine Kommentarzeile mit Makroname und Herkunft (4.13).

Enthält das Makro selbst eine Schleife mit einem Label, z. B.:

```
BSP1:  MACRO
# Makro enthält eine Schleife
ML0001%: 211  …
         …
         000  ML0001%
         MEND
```

so wird bei einem Makroaufruf `loop: BSP1` folgende Anweisungsfolge generiert, wenn die erste zu generierende Anweisung ein eigenes Label hat:

```
loop      equ  *
ML0001DKR 211  …
          …
          000  ML0001DKR
```

Hat die erste Anweisung kein eigenes Label, entfällt die `equ`-Anweisung und die erste Anweisung erhält das Label des Makroaufrufs. Ausnahme: Ist die erste Anweisung `equ`, `abs`, `start` oder `end`, bleibt die `equ`-Anweisung bestehen.

Ein Makro SOLLTE kein „normales" Label der Assemblersprache enthalten, sonst kann es nur einmalig aufgerufen werden. Stattdessen muss ein lokales Label verwendet werden: ein Name, danach `%`, ein optionaler Zusatz und der Doppelpunkt. Der Name beginnt mit einem Buchstaben oder _, danach folgen Buchstaben, Ziffern und _ (höchstens 24 Zeichen). Das kann ein generisches Label sein (`ML`, vier Ziffern und `%`, z. B. `ML0004%:`) oder, ab c82ass 1.00-064, ein lesbares Label (z. B. `schleife%:` oder `ende%:`); im Operanden steht derselbe Name mit `%`, z. B. `000  schleife%`. Bei jedem Aufruf wird das `%` durch drei Großbuchstaben ersetzt (z. B. `ML0001DKR`, `schleifeDKR`), sodass jeder Aufruf ein eigenes Label erhält; ein Zusatz hinter dem `%` bleibt erhalten. Ein Name wird nur als ganzes Wort ersetzt (`ende%` ist nicht das Ende von `weiterende%`). Heißt ein Label des Programms zufällig wie das erzeugte (`schleifeDKR`), meldet der Assembler „Symbol existiert bereits“. Ein lesbares Label, das nirgends verwendet wird, steht nicht in der Meldung „nie verwendet“ (wie die ML-Labels). Jedes andere Label mit `%` (Name beginnt mit einer Ziffer, mehr als 24 Zeichen, unzulässiges Zeichen) meldet den Fehler `Macrolabel muß aus einem Namen … bestehen`. Test `Assembler/Claude/makrolabelname` und Negativtests `makro_label_name_lang`, `makro_label_name_ziffer` und `makro_label_name_doppelt`.

Makro-Aufrufparameter, die exakt 4 Ziffern enthalten und nur diese, werden ggf. als 4-Ziffern-String geprüft (durch `$PRx`); sie erhalten bei der Benutzung (Ersetzung des formalen Parameters) automatisch eine Vornull. Beispiel: `ANWAHL 4600` erzeugt:

```
     030    04600  ; 4600 oder K1 oder LSL1 oder LSS1
```

4.1 Makros im Quelltext

Makros können nicht nur in der Makrodatei `c82ass.makros`, sondern auch direkt im Quelltext (`.asm`) definiert werden. Die Definition hat dieselbe Syntax wie in der Makrodatei: Sie beginnt mit dem Makronamen, unmittelbar gefolgt von einem Doppelpunkt, danach folgt das Schlüsselwort `MACRO` (bei Bedarf mit der Anzahl der Parameter). Abgeschlossen wird sie durch `MEND` (zulässig sind auch `MACROENDE` und `macroende`). Der Doppelpunkt hinter dem Makronamen ist im Quelltext Pflicht.

- **Gültigkeit:** Eine Definition darf an beliebiger Stelle im Quelltext stehen, auch vor `start`. Das Makro gilt im gesamten Quelltext, auch für Aufrufe, die vor der Definition stehen. Die Definition selbst erzeugt keinen Code.
- **Definitionsteil:** Es gelten alle Regeln dieses Abschnitts (formale Parameter `$x`, Prüflisten `$PRx`, `$IF-$ELIF-$ELSE-$ENDIF`, `$REPT-$ENDREPT`, Kommentare mit # und ;, generische Labels mit `%`). Ein Makro darf andere Makros aufrufen, gleichgültig ob sie in der Makrodatei oder im Quelltext definiert sind, aber nicht sich selbst (auch nicht über andere Makros, 4.15).
- **Gleicher Name wie in der Makrodatei:** Das Makro im Quelltext ersetzt das gleichnamige Makro der Makrodatei, jedoch nur für diesen Assemblerlauf; die Makrodatei selbst bleibt unverändert. Der Assembler gibt dazu eine Warnung aus. Die Ersetzung wirkt auch dort, wo ein anderes Makro das ersetzte Makro aufruft.
- **Fehler:** Ein Makroname darf im Quelltext nur einmal definiert werden. Weitere Fehler sind eine `MACRO`-Zeile ohne Namen, eine Definition innerhalb einer anderen Definition (Schachtelung ist nicht erlaubt), ein fehlendes `MEND` und ein `MEND` ohne zugehöriges `MACRO`. Der Assembler meldet den Fehler und bricht nach Pass 0 ab.

Beispiel:

```
ZEILE2:  MACRO
         dz  $wrzl  ; Zeilenschaltung
         dz  $wrzl  ; Zeilenschaltung
         MEND
```

Der Aufruf `ZEILE2` erzeugt die beiden Anweisungen `dz $wrzl`.

Soll das Makro `WRZL` der Makrodatei nur für ein Programm anders arbeiten, wird es im Quelltext neu definiert:

```
WRZL:    MACRO
         dz  $wrzl  ; Zeilenschaltung
         dz  $wrzl  ; Zeilenschaltung
         MEND
```

Der Assembler meldet dazu:

```
Warnung: Makro WRZL aus der Makrodatei wird für diesen Lauf durch die Definition im Quelltext ersetzt.
```

`$`-Direktiven im Quelltext : `$IF`, `$ELIF`, `$ELSE`, `$ENDIF`, `$REPT`, `$ENDREPT`, `$WHILE`, `$ENDWHILE`, `$SET`, `$INC`, `$DIM`, `$WARN`, `$ERROR`, `$NOTE` und `$DUMP`, die Zeichenkettenfunktionen (4.21) sowie `$V`, `$CALC`, `$DEFINED` und `$MDEFINED` wirken im normalen Quelltext wie im Rumpf eines Makros, nur ohne Aufrufparameter. Makroaufrufe im Quelltext werden an ihrer Stelle aufgelöst. Die Direktivzeilen erzeugen selbst nichts und fehlen im Listing, in `.dmp` und `.mqo`; Kommentarzeilen und das Kommentarfeld einer Zeile bleiben unverändert (ein `$V(n)` im Kommentar bleibt Text). Bis c82ass 1.01-007 wurden alle diese Zeilen ohne Meldung übergangen.

Auslösung: Der Assembler wertet den Quelltext nur aus, wenn mindestens eine Zeile mit `$` beginnt (eine Einrückung davor zählt nicht). Ohne eine solche Zeile bleibt `dw $CALC(def(x))` Text und endet mit „Operand '$CALC(def(x))' ist ungültig“ (Rückgabecode 16); schon eine einzige `$SET`-Zeile im Quelltext genügt, damit das `$CALC` ausgewertet wird. Ein unbekanntes Wort wie `$FOO` an erster Stelle einer Zeile wird weiterhin ohne Meldung übergangen und die Zeile entfällt; das gilt im Quelltext wie im Rumpf eines Makros.

Marke vor der Direktive: Steht vor der Direktive eine Marke, ist die Zeile eine normale Zeile und keine Direktive: `lab: $SET n 1` endet mit „Operationscode '$SET' ist ungültig“ (Rückgabecode 16), und bei `lab: $IF 1 = 1` wird das `$IF` nicht erkannt, gemeldet wird zuerst das zugehörige `$ENDIF` („$ENDIF ohne vorheriges $IF“). Eine Marke mit `%` bleibt im normalen Quelltext ein Fehler; lokale Marken gelten nur im Rumpf eines Makros (Abschnitt 4).

Fehler und Meldungen: `$EXIT`, `$LOCAL` und `$GLOBAL` gibt es nur im Makroaufruf; im Quelltext sind sie ein Fehler („$EXIT steht außerhalb eines Makroaufrufs“, ebenso `$LOCAL` und `$GLOBAL`; Rückgabecode 16). `$SET`, `$INC` und `$DIM` im Quelltext sind immer global, auch mit der Anweisung `;#MAKROLOKAL=1`. Ebenso ist `$AUFRUFNR` im Quelltext ein Fehler („$AUFRUFNR steht außerhalb eines Makroaufrufs“, 4.29); die Auswertung des Quelltexts läuft dafür auch ohne eine Zeile mit `$` am Anfang. `$TRACE ON` und `$TRACE OFF` bleiben im Quelltext ohne Wirkung (keine Ausgabe, die Zeile entfällt); `$TRACE` ohne `ON` oder `OFF` ist ein Fehler („$TRACE: ON oder OFF erwartet“). Fehler der Quelltext-Auswertung tragen den Zusatz „(Quelltext)“ und nennen die Quelltextzeile der Direktive, zum Beispiel `quelltext_else_ohne_if.asm:2: Fehler: (Quelltext) $ELSE ohne vorheriges $IF`. Im Listing und in `.dmp` des Fehlerfalls steht die fehlerhafte Direktivzeile dabei als Kommentarzeile vor der Meldung. Fehler in einem nicht erzeugten Zweig bleiben stumm. `$NOTE` und `$DUMP` melden „Hinweis: Quelltext (<Datei>:<Zeile>): …“, `$WARN` „Warnung: Quelltext (Zeile <n>): $WARN …“ (4.24). Die Grenzen sind die des Makros: `$REPT` 0 bis 4096 und höchstens 4096 Zeilen je `$REPT`, `$WHILE` höchstens 4096 Durchläufe, insgesamt höchstens 6144 Zeilen („Zu viele Zeilen“).

Querverweise: Bei `--xref` sind die Zeilennummern („Def.“ und „Verwendet in Zeile“) die des Original-Quelltexts; ein Symbol, dessen Marke erst aus `$V` entsteht (zum Beispiel `m4` aus `m$V(n):`), steht mit „Def.“ `-`, seine Verwendungen werden gezählt. Der Test `Assembler/Claude/quelltextdirektiven` prüft die Direktiven im Quelltext, 23 Negativtests `quelltext_*` im Regressionstest die Fehlerfälle.

Bedingte Assemblierung: Ein Programmteil, der nur mit einem Schalter wie `DEBUG` entstehen soll, lässt sich ab c82ass 1.01-008 unmittelbar im Quelltext mit `$IF` und `$ENDIF` einklammern. Der Schalter ist ein Symbol, nach dem `def` fragt (4.23). Im Beispiel sichert der eingeklammerte Teil den Akkumulator, erhöht einen Zähler um 1 und stellt den Akkumulator wieder her, aber nur, wenn der Quelltext `DEBUG` vereinbart (hier hinter `end`):

```
           start
           000 haupt                        ; Sprung zum Hauptprogramm
wert:      dw 017                           ; Konstante
merk:      dw 0                             ; Merkzelle
zaehler:   dw 0                             ; Zähler
eins:      dw 1                             ; Konstante 1
haupt:     211 wert                         ; Akkumulator laden
           $IF $CALC(def(DEBUG)) = 1
           240 merk                         ; Akkumulator sichern
           211 zaehler                      ; Zähler laden
           210 eins                         ; um 1 erhöhen
           240 zaehler                      ; Zähler zurückspeichern
           211 merk                         ; Akkumulator wiederherstellen
           $ENDIF
           020 0                            ; Halt
ende:      end
DEBUG:     abs 1                            ; Schalter; ohne diese Zeile entfällt der Teil
```

Mit der Zeile `DEBUG` entstehen die fünf Anweisungen: Der Lauf endet mit BZ 0014 und AC 017, `merk` (Zelle 0002) steht auf 017 und `zaehler` (Zelle 0003) auf 1. Ohne die Zeile `DEBUG` entsteht nichts: Der Lauf hält mit BZ 0007 und AC 017 auf der Zelle 0006, `merk` und `zaehler` bleiben 0. Beide Läufe sind mit c82ass 1.01-008 assembliert und im Interpreter gelaufen (Test wahr; die Sollwerte waren vorher von Hand hergeleitet). Das `$CALC` in der Bedingung wird ausgewertet, weil der Quelltext mit `$IF` selbst eine Zeile mit `$` enthält (Auslösung, oben).

Der Weg über ein Hilfsmakro bleibt gültig und war bis c82ass 1.01-007 der einzige, weil ein `$IF` im Quelltext damals ohne Meldung übergangen wurde und seine Zeilen immer entstanden. Der Teil gehört dann in den Rumpf eines Hilfsmakros, das die Bedingung enthält; an seiner Stelle steht der Aufruf. Der Schalter ist ein Symbol, nach dem `def` fragt (4.23), oder ein Parameter des Hilfsmakros. Im Beispiel macht `TESTTEIL` dasselbe wie der eingeklammerte Teil oben:

```
TESTTEIL:  MACRO
           $IF $CALC(def(DEBUG)) = 1
           240 merk                         ; Akkumulator sichern
           211 zaehler                      ; Zähler laden
           210 eins                         ; um 1 erhöhen
           240 zaehler                      ; Zähler zurückspeichern
           211 merk                         ; Akkumulator wiederherstellen
           $ENDIF
           MEND
           start
           000 haupt                        ; Sprung zum Hauptprogramm
wert:      dw 017                           ; Konstante
merk:      dw 0                             ; Merkzelle
zaehler:   dw 0                             ; Zähler
eins:      dw 1                             ; Konstante 1
haupt:     211 wert                         ; Akkumulator laden
           TESTTEIL                         ; Programmteil nur mit Schalter
           020 0                            ; Halt
ende:      end
DEBUG:     abs 1                            ; Schalter; ohne diese Zeile entfällt der Teil
```

Mit der Zeile `DEBUG` erzeugt der Aufruf die fünf Anweisungen, und der Zähler steht nach dem Lauf auf 1; ohne die Zeile entsteht nichts, und der Zähler bleibt 0. Mit einem Parameter statt `def` (`TESTTEIL: MACRO 1`, `$IF $1 = 1`, Aufruf `TESTTEIL 1` oder `TESTTEIL 0`) ergibt sich dasselbe. Alle vier Fälle sind mit c82ass 1.01-007 assembliert und im Interpreter gelaufen (Test wahr; die Sollwerte waren vorher von Hand hergeleitet).

Zum Hilfsmakro: Eine Marke auf der Aufrufzeile bleibt auch bei ausgeschaltetem Schalter erhalten; die `.mqo` zeigt sie dann als `equ` mit dem Kommentar „automatisch generiert“ und der Adresse der nächsten erzeugten Zelle. Eine normale Marke im Rumpf ist erlaubt, wenn das Makro nur einmal aufgerufen wird (sonst lokale Marken mit `%`, Abschnitt 4). Springt das übrige Programm auf eine Marke im Rumpf, fehlt sie bei ausgeschaltetem Schalter, und der Assembler meldet „Symbol 'innen' ist nicht definiert“ (Rückgabecode 16).

4.2 Rechenausdrücke in Makros (`$CALC`, `$CALC0`, `$TYPZAHL`)

In der Makrodefinition darf an jeder Stelle einer Anweisung (Marke, Operation, Operand, Kommentar) und in den Bedingungen von `$IF` und `$ELIF` ein Rechenausdruck stehen:

- `$CALC(<ausdruck>)` wird durch das Ergebnis als **Dezimalzahl** ersetzt.
- `$CALCO(<ausdruck>)` wird durch das Ergebnis als **Oktalzahl mit führender 0** ersetzt (Ergebnis 0 ergibt `0`; ein negatives Ergebnis ist ein Fehler).
- `$CALCB(<ausdruck>)` wird durch das Ergebnis als Folge von **Binärziffern** ohne Präfix ersetzt (Ergebnis 0 ergibt `0`; ein negatives Ergebnis ist ein Fehler; seit c82ass 1.01-022, Abschnitt 4.27).
- `$CALCH(<ausdruck>)` wird durch das Ergebnis als Folge von **Hexadezimalziffern** (0-9, A-F in Großbuchstaben) ohne Präfix ersetzt (Ergebnis 0 ergibt `0`; ein negatives Ergebnis ist ein Fehler; seit c82ass 1.01-023, Abschnitt 4.28).

Ablauf beim Makroaufruf: Zuerst werden die Schleifen `$REPT` aufgelöst (4.3), dann die Rechenausdrücke ausgewertet; dabei wird jedes `$1` bis `$9` im Ausdruck durch den **eingeklammerten** Aufrufparameter ersetzt (der Parameter `3+4` ergibt in `$1*2` den Ausdruck `(3+4)*2`). Danach werden die übrigen `$x` ersetzt und die `$IF`-Bedingungen ausgewertet. Ein `$CALC` kann deshalb in einer Bedingung stehen (`$IF $CALC($1%2) = 0`), und der Aufrufparameter darf selbst ein Ausdruck sein (`SCHIEBE_LI 2+1`). Ein Ausdruck in einer Makrodefinition wirkt in der Makrodatei und in Makros im Quelltext gleich.

Ausdruck: Er enthält **keine Leerzeichen** und kein `;`. Erlaubt sind Zahlen (führende 0 = oktal, sonst dezimal, `0b` = binär seit c82ass 1.01-022 (4.27), `0x` = hexadezimal seit c82ass 1.01-023 (4.28); `09` ist ein Fehler), die Aufrufparameter `$1` bis `$9` und die folgenden Operatoren, hier von der stärksten Bindung zur schwächsten (links nach rechts, Rangfolge wie in C):

Operatoren	Bedeutung
`( )`	Klammern
`- + ~` (einstellig)	Vorzeichen, bitweises Komplement
`* / %`	Mal, ganzzahlige Division, Rest
`+ -`	Plus, Minus
`<< >>`	Schieben (Schiebezahl 0 bis 62)
`&`	bitweises UND
`^`	bitweises ausschließendes ODER
`|`	bitweises ODER

Gerechnet wird mit 64-Bit-Ganzzahlen mit Vorzeichen. Division und Rest durch 0, nichtnumerische Zeichen (Buchstaben und Marken, soweit sie nicht ein abs- oder equ-Symbol mit Zahlenwert sind, 4.18), fehlende Klammern und Zahlen über dem Bereich sind Fehler: Der Assembler meldet `$CALC(...): <Grund>` (bei einem Fehler des Aufrufparameters zusätzlich die Meldung der Typprüfung) und bricht wie bei anderen Makrofehlern nach Pass 0 ab.

Rechenausdruck im Text: Seit c82ass 1.00-070 darf ein Ausdruck auch im Text von `$SET name "text"` und in den Textargumenten der Zeichenkettenfunktionen stehen; er wird nach dem Auflösen von `$V(...)` ausgewertet und durch das Ergebnis ersetzt (4.22). So lässt sich ein Oktaltext aus einer Variablen bilden: `$SET t "$CALCO($V(n))"`.

Attributabfragen: Seit c82ass 1.01-001 stehen im Rechenausdruck auch die Funktionen `def(name)`, `art(name)` und `zellen(name)`; sie fragen, ob ein Symbol im Quelltext vorkommt, welche Art es hat und wie viele Zellen seine Anweisung belegt (4.23). Ab c82ass 1.01-012 ergibt zellen(von,bis) die Zellenzahl eines Blocks zwischen zwei Marken (4.23). Ein `$1` im Argument wird wie überall eingeklammert eingesetzt (`def($1)` wird `def((tab))`).

Typprüfung `$PRx $TYPZAHL <min> <max>`**:** Der Aufrufparameter `$x` muss ein ganzzahliger Ausdruck der oben beschriebenen Art (Zahlen und abs- oder equ-Symbole mit Zahlenwert, 4.18) sein, dessen Wert zwischen `<min>` und `<max>` einschließlich liegt (auch die Grenzen sind Ausdrücke ohne Leerzeichen). Sonst meldet der Assembler z. B. `Parameter $1: Wert "33" (= 33) liegt nicht im Bereich 1 bis 32`. `$TYPZAHL` steht wie `$TYPADR` auf einer Zeile für sich und ersetzt eine Werteliste `$PRx`.

Beispiel: `SCHIEBE_LI n` und `SCHIEBE_RE n` der Makrodatei verschieben den AC n-fach (Wiederholungsbefehl 644 bzw. 642 auf der Arbeitsspur 7640). Der Operand ist 07640 + (19·n mod 32) (Festlegung 026). Früher stand dafür für jedes n von 1 bis 32 ein eigener `$IF`-Zweig in der Makrodatei, jetzt genügen zwei Zeilen:

```
SCHIEBE_LI: MACRO 1
$PR1 $TYPZAHL 1 32
         644 $CALCO(07640+19*$1%32)  ; $CALC($1) mal Linksverschiebung (Operand nach Festlegung 026)
         MEND
```

`SCHIEBE_LI 3` erzeugt `644 07671  ; 3 mal Linksverschiebung (Operand nach Festlegung 026)`; `SCHIEBE_LI 2+1` erzeugt dasselbe. Der Test `Assembler/Claude/makrocalc` prüft die Rechenausdrücke, `Assembler/Claude/makros` alle 64 Schiebefälle.

4.3 Schleife im Makro (`$REPT`, `$ENDREPT`, `$Z1` bis `$Z4`)

`$REPT <anzahl>` wiederholt die Zeilen bis zum zugehörigen `$ENDREPT` `<anzahl>`-mal. Beide Schlüsselworte stehen auf einer Zeile für sich (ein Kommentar mit ; dahinter ist erlaubt). Der Rumpf besteht aus beliebigen Zeilen der Makrodefinition, auch aus `$IF`-Strukturen und weiteren Schleifen. Ab c82ass 1.01-008 gilt die Schleife auch unmittelbar im Quelltext (4.1).

`<anzahl>` ist eine Zahl oder ein Rechenausdruck ohne Leerzeichen wie in 4.2 (auch `$CALC(...)` und die Aufrufparameter `$1` bis `$9`, z. B. `$REPT $1` oder `$REPT 2*$1`) im Bereich 0 bis 4096; bei 0 entsteht nichts. Je Makroaufruf dürfen insgesamt höchstens 4096 Zeilen entstehen.

Zähler: Im Rumpf steht `$Z1` für die Nummer des Durchlaufs der äußersten Schleife (0, 1, 2, ...), `$Z2` für die der zweiten Schleife usw. bis `$Z4` (höchstens 4 geschachtelte Schleifen). Der Zähler wird in Marke, Operation, Operand und Kommentar ersetzt und ist auch in `$CALC(...)` und in Bedingungen zulässig, z. B. `$CALC($Z1*8+$Z2)` oder `$IF $CALC($Z1%2) = 0`. Außerhalb einer Schleife ist `$Z1` bis `$Z4` ein Fehler.

Ablauf beim Makroaufruf: Zuerst werden die Schleifen aufgelöst, dann die Rechenausdrücke (4.2) und die Aufrufparameter ersetzt, zuletzt die Bedingungen (4.4) ausgewertet. Eine Schleife, deren Ende erst im Lauf feststeht (Abbruchbedingung statt fester Anzahl): 4.19.

Marken: Eine Marke im Rumpf entsteht in jedem Durchlauf neu. Eine normale Marke muss deshalb vom Zähler abhängen, z. B. `w$Z1:` (ergibt `w0`, `w1`, `w2`, ...). Ein lokales Label `<Name>%` (Abschnitt 4, z. B. `ML0001%` oder `schleife%`) darf im Rumpf stehen: Ist es im Rumpf definiert, erhält es in jedem Durchlauf den Zusatz `_<Zähler>` (`ML0001%_0`, `ML0001%_1` oder `schleife%_0`, `schleife%_1`, ... bei geschachtelten Schleifen `ML0001%_0_1`) und der Assembler ersetzt wie gewohnt das `%` durch drei Buchstaben; ebenso wird jede Verwendung im Rumpf angepasst. Ein Label außerhalb des Rumpfes, auf das der Rumpf springt, bleibt unverändert.

Fehler (der Assembler bricht wie bei anderen Makrofehlern nach Pass 0 ab): `$REPT` ohne `$ENDREPT` und umgekehrt, mehr als 4 geschachtelte Schleifen, eine Anzahl, die sich nicht auswerten lässt oder außerhalb von 0 bis 4096 liegt, mehr als 4096 erzeugte Zeilen, ein Zähler außerhalb einer Schleife.

Beispiel: `MACHE_ADRESSEINHEIT` der Makrodatei hatte acht gleiche Zeilen `004 *+1` (Verschiebung des AC um 1 Bit nach links). Jetzt genügt die Schleife, der erzeugte Code ist unverändert:

```
MACHE_ADRESSEINHEIT:  MACRO 2
             215 $1            ; Lade $1 in AC und 1x Linksverschiebung
             $REPT 8
             004 *+1           ; Verschiebe AC 1x links ($CALC($Z1+2))
             $ENDREPT
             240 $2
             WDHL JA LI KON 9   004000
         MEND
```

Beispiel mit zwei geschachtelten Schleifen: `MATRIX` erzeugt die Worte 0, 1, 10, 11, 20, 21 (Zeile `$Z1`, Spalte `$Z2`):

```
MATRIX:  MACRO
         $REPT 3
         $REPT 2
         dw $CALC($Z1*10+$Z2)   ; Zeile $Z1, Spalte $Z2
         $ENDREPT
         $ENDREPT
         MEND
```

Der Test `Assembler/Claude/makroschleife` prüft Schleifen, Zähler, Marken mit Zähler und den Code aus einer Schleife.

4.4 Geschachtelte Bedingungen (`$IF`, `$ELIF`, `$ELSE`, `$ENDIF`)

Die `$IF`-Struktur darf geschachtelt werden (bis zur Tiefe 16). Jedes `$IF` braucht sein eigenes `$ENDIF`; `$ELIF` und `$ELSE` gehören zum innersten offenen `$IF`. `$ELSE` steht höchstens einmal je `$IF` und hinter allen `$ELIF`. Erzeugt wird der Zweig mit der ersten wahren Bedingung, sonst der `$ELSE`-Zweig; die Bedingungen eines nicht erzeugten Zweiges werden nur auf Schreibfehler geprüft. Ein `$IF` in einem nicht erzeugten Zweig erzeugt ebenfalls nichts. Ab c82ass 1.01-008 gilt die Struktur auch unmittelbar im Quelltext (4.1).

Bedingung: bis zu 8 Bedingungsblöcke (3, 7, 11, ... 31 Wörter ohne die Klammerwörter), Vergleichszeichen > < = >= <= != <>. Sind beide Werte Zahlen (auch negativ, z. B. ein Ergebnis von `$CALC`), wird numerisch verglichen, sonst alphabetisch. Ein unbekanntes Vergleichszeichen ist ein Fehler (früher galt die Bedingung dann als wahr). Mehrere Bedingungsblöcke verknüpfen `|` (ODER) und `&` (UND); ohne Klammern bindet `&` stärker als `|` (ab c82ass 1.01-031, Einleitung von Abschnitt 4), die Wörter `(` und `)` ändern die Gruppierung.

Fehler: fehlendes `$ENDIF`, `$ELIF`, `$ELSE` oder `$ENDIF` ohne `$IF`, ein zweites `$ELSE` im selben `$IF`, `$ELIF` hinter `$ELSE`, mehr als 16 geschachtelte `$IF` (zusammen mit `$WHILE`, 4.19), eine Wörterzahl, die nicht 3, 7, 11, ... ist, ein Verknüpfungszeichen an der 4., 8., 12. ... Stelle, das nicht `|` oder `&` ist, fehlende oder überzählige Klammer.

Beispiel: Vorzeichen zweier Parameter (1 = beide positiv, 2 = nur der erste, 3 = nur der zweite, 4 = keiner):

```
IFN:     MACRO 2
         $IF $1 > 0
           $IF $2 > 0
         dw 1   ; beide positiv
           $ELSE
         dw 2   ; nur der erste positiv
           $ENDIF
         $ELSE
           $IF $2 > 0
         dw 3   ; nur der zweite positiv
           $ELSE
         dw 4   ; keiner positiv
           $ENDIF
         $ENDIF
         MEND
```

Der Test `Assembler/Claude/makroschleife` prüft außerdem `$IF-$ELIF-$ELSE`, `$IF` in einer Schleife, Klammern, sechs Bedingungsblöcke und das Vergleichszeichen `!=`. Der Test `Assembler/Claude/makro_stolperfallen` prüft außerdem den Vorrang von `&` vor `|` ohne Klammern (`$V(a) = 1 | $V(b) = 1 & $V(c) = 1` mit a = 1, b = 0, c = 0 ist wahr).

4.5 Typ `$TYPSYM` und Meldungen aus dem Makro (`$ERROR`, `$WARN`)

Typprüfung `$PRx $TYPSYM`: Der Aufrufparameter `$x` muss ein Symbol sein, also aus Buchstaben, Ziffern und _ bestehen, nicht mit einer Ziffer beginnen und höchstens 32 Zeichen lang sein. Anders als bei `$TYPADR` sind eine Sternadresse (`*`, `*+3`) und jede Zahl ein Fehler. Der Assembler meldet z. B. `Parameter $1: Wert "07" ist kein Symbol (Buchstaben, Ziffern und _, nicht mit einer Ziffer beginnend)`. Die Prüfung ist wie bei `$TYPADR` nur syntaktisch: ob das Symbol im Programm vorkommt, meldet die spätere Symbolauflösung. `$TYPSYM` steht auf einer Zeile für sich und ersetzt eine Werteliste `$PRx`.

`$ERROR <text>` meldet einen Fehler aus der Makrodefinition. Die Zeile steht für sich; der Text reicht bis zum Zeilenende (auch ein `;` gehört zum Text). In ihm werden `$1` bis `$9`, `$Z1` bis `$Z4` und `$CALC(...)` wie in jeder Anweisung ersetzt. Die Zeile wirkt nur, wenn sie im erzeugten Zweig liegt: Sie steht meist in einem `$IF` und prüft Kombinationen von Parametern, die `$PRx` allein nicht erfasst. Im nicht erzeugten Zweig bleibt sie wirkungslos. Die Meldung lautet `<Makro> $ERROR: <text>` (Text höchstens 300 Zeichen) und nennt die Zeile des Aufrufs; der Assembler bricht wie bei anderen Makrofehlern nach Pass 0 ab (Rückgabecode 16). In einer Schleife wird die Meldung je Durchlauf ausgegeben.

`$WARN <text>` gilt wie `$ERROR` (Schreibweise, Ersetzung, nur im erzeugten Zweig), gibt aber nur eine Warnung aus: `Warnung: Makro <Name>: $WARN <text>`. Der Lauf geht weiter, der Rückgabecode bleibt 0. Beide Schlüsselwörter erzeugen keinen Code. Eine Ausgabe, die weder Warnung noch Fehler ist, und weitere Hilfen zur Fehlersuche im Makro (`$NOTE`, `$TRACE`, `$DUMP`): Abschnitt 4.24. Ab c82ass 1.01-005 ersetzen `$WARN` und `$ERROR` auch `$V(...)` im Text (Einzelheiten in 4.24).

Beispiel: `KOPIERE` verlangt zwei Symbole, weist gleiche Quelle und Ziel zurück und warnt bei der Quelle `quelle`:

```
KOPIERE:        MACRO 2
                $PR1 $TYPSYM
                $PR2 $TYPSYM
                $IF $1 = $2
                $ERROR Quelle $1 und Ziel $2 sind dieselbe Zelle
                $ENDIF
                $IF $1 = quelle
                $WARN Die Zelle $1 wird nach $2 kopiert
                $ENDIF
                211     $1              ; AC := Zelle $1
                240     $2              ; Zelle $2 := AC
                MEND
```

`KOPIERE quelle ziel` erzeugt `211 quelle` und `240 ziel` und gibt die Warnung aus; `KOPIERE 07 ziel` und `KOPIERE * ziel` scheitern an `$TYPSYM`, `KOPIERE ziel ziel` am `$ERROR`. Der Test `Assembler/Claude/makroprueft` prüft den Erfolgsfall, die Negativtests `makro_error`, `makro_error_inaktiv`, `makro_warn`, `makro_typsym_zahl` und `makro_typsym_stern` liegen im Regressionstest.

4.6 Vorgabewerte und Parameteranzahl (`$DEFAULT`, `$#`)

`$DEFAULT <nr> <wert>` gibt dem Parameter `$nr` einen Vorgabewert. Die Zeile steht für sich in der Makrodefinition; die Nummer reicht von 1 bis zur vereinbarten Parameterzahl (in einem Makro mit Namen auch ab 10, bis 32), der Wert ist ein Wort ohne Leerzeichen. In einem Makro mit Namen (4.26) steht statt der Nummer auch der Name: `$DEFAULT zweiter 9`; die Nummern 11 und 12 gehen ebenso (`$DEFAULT 11 7`). Ruft ein Aufruf weniger Parameter auf, als `MACRO` vereinbart, ergänzt der Assembler die fehlenden am Ende aus den `$DEFAULT`-Zeilen; für jeden fehlenden Parameter muss es eine geben, sonst gilt die Meldung `erwartet n Parameter, aber m wurden übergeben!` mit dem Zusatz, dass ein `$DEFAULT` fehlt. Zu viele Parameter, eine ungültige Nummer, ein Wert mit Leerzeichen und ein zweites `$DEFAULT` für denselben Parameter sind Fehler. Der Vorgabewert wird wie ein übergebener Parameter behandelt, auch von einer Prüfliste `$PRx`.

`$#` steht für die Zahl der tatsächlich übergebenen Parameter (ohne die ergänzten Vorgabewerte). Es darf überall stehen, auch in `$CALC(...)`, in Bedingungen und in `$REPT $#`. Damit lassen sich optionale Parameter erkennen: `$IF $# = 1` prüft, ob nur der erste Parameter übergeben wurde.

Beispiel: `LADE` hat drei Parameter, `$2` hat die Vorgabe 5, `$3` die Vorgabe 0:

```
LADE:           MACRO 3
                $DEFAULT 2 5
                $DEFAULT 3 0
                dw      $#              ; Zahl der übergebenen Parameter
                dw      $2              ; zweiter Parameter oder Vorgabe
                dw      $CALC($1+$#)    ; erster Parameter plus Anzahl
                MEND
```

`LADE 10` erzeugt die Worte 1, 5, 11; `LADE 10 6` erzeugt 2, 6, 12. Die Reihenfolge beim Aufruf bleibt: Vorgabewerte und `$#` sind vor `$REPT`, `$CALC` und `$IF` ersetzt. Der Test `Assembler/Claude/makrovorgabe` prüft Vorgabewerte und `$#`, die Negativtests `makro_default_fehlt`, `makro_default_viele`, `makro_default_syntax` und `makro_default_doppelt` liegen im Regressionstest.

4.7 Makro-Variablen (`$SET`, `$INC`, `$V`)

Makro-Variablen sind ganzzahlige Werte, die über alle Makroaufrufe eines Assemblerlaufs erhalten bleiben (bis zu 64, Name aus Buchstaben, Ziffern und _, beginnt mit einem Buchstaben). So lassen sich fortlaufende Nummern für Tabellen oder Marken erzeugen. Die laufende Nummer des Makroaufrufs selbst liefert `$AUFRUFNR` ohne Variable (4.29). Lokale Variablen, die nur in einem Makroaufruf gelten: 4.20. Mit der Quelltext-Anweisung `;#MAKROLOKAL=1` ist eine in einem Makroaufruf mit `$SET` angelegte Variable lokal, und `$GLOBAL` vereinbart eine Variable ausdrücklich als global; ohne die Anweisung bleibt es bei den globalen Variablen dieses Abschnitts.

`$SET <name> <ausdruck>` setzt die Variable (legt sie an, falls nötig); `$INC <name> [<ausdruck>]` erhöht sie um den Ausdruck, ohne Angabe um 1. Der Ausdruck ist wie in 4.2 ein Rechenausdruck ohne Leerzeichen (er darf `$1` bis `$9`, `$Z1` bis `$Z4` und `$V(name)` enthalten). Beide Zeilen stehen für sich, wirken nur im erzeugten Zweig, erzeugen keinen Code und werden in der Reihenfolge der erzeugten Zeilen ausgeführt. `$INC` einer nicht gesetzten Variable ist ein Fehler. Ein Kommentar mit `;` hinter `$INC <name>` ist auch ohne Ausdruck erlaubt, wenn ein Leerzeichen vor dem `;` steht: `$INC n ; Kommentar` erhöht `n` um 1 (ab c82ass 1.01-031; bis 1.01-030 endete diese Zeile mit „$INC erwartet <Name> [<Ausdruck>] (ohne Leerzeichen)“), und mit Ausdruck (`$INC n 2 ; Kommentar`) war der Kommentar schon vorher erlaubt. Ohne Leerzeichen (`$INC n;Kommentar`) gehört das `;` zum Namen, und die Zeile endet mit „Variablenname n;Kommentar ungültig“. Das gilt im Makro wie im Quelltext; der Test `Assembler/Claude/makro_stolperfallen` prüft es im Makro. Ab c82ass 1.01-008 gelten `$SET`, `$INC` und `$V` auch im normalen Quelltext (4.1); dort sind die Variablen immer global, auch mit `;#MAKROLOKAL=1` (4.31).

`$V(name)` steht für den aktuellen Wert (dezimal) und darf in Marke, Operation, Operand und Kommentar der erzeugten Zeilen stehen, auch innerhalb von `$CALC(...)` (z. B. `$CALC($V(zahl)*10)`). Der Wert ist der Stand an der Stelle der Zeile. In den Bedingungen von `$IF` und `$ELIF` darf `$V(name)` ebenfalls stehen (auch in `$CALC(...)` dort, z. B. `$IF $CALC($V(n)*2) > 6`); es gilt der Stand an der Stelle der Bedingung . In `$REPT` darf `$V(name)` in der Anzahl stehen (auch in `$CALC(...)`, z. B. `$REPT $V(n)` oder `$REPT` $CALC(`$V(n)`-3);es gilt der Stand zu Beginn des Makroaufrufs, weil die Schleifen vor der Ausführung von `$SET` und `$INC` dieses Aufrufs aufgelöst werden. Im Rumpf der Schleife bleibt `$V(name)` wie bisher der Stand an der jeweiligen Zeile. Eine nicht gesetzte Variable in `$V(name)` ist ein Fehler. Ob eine Variable oder ein Feld vorhanden ist, fragt `$DEFINED` (4.25, ab c82ass 1.01-006).

Beispiel: `ZINIT` setzt `zahl` auf 0, `NUMMER` erzeugt den Stand, erhöht ihn und erzeugt das Zehnfache des neuen Standes. Drei Aufrufe von `NUMMER` erzeugen 0, 10, 1, 20, 2, 30:

```
ZINIT:          MACRO
                $SET zahl 0
                MEND
NUMMER:         MACRO
                dw      $V(zahl)        ; Stand vor der Erhöhung
                $INC zahl
                dw      $CALC($V(zahl)*10)   ; Zehnfaches des neuen Standes
                MEND
```

Der Test `Assembler/Claude/makrovariable` prüft `$SET`, `$INC` (auch mit Schrittweite) und `$V` in `$CALC`; die Negativtests `makro_var_undefiniert`, `makro_var_inc_undefiniert` und `makro_var_set_syntax` liegen im Regressionstest.

Felder (mehrere nummerierte Werte mit berechnetem Index): Abschnitt 4.17. Schleife mit Abbruchbedingung und vorzeitiges Verlassen eines Makros (`$WHILE`, `$EXIT`): Abschnitt 4.19. Ausgabe von Werten und Ablaufspur zur Fehlersuche (`$NOTE`, `$TRACE`, `$DUMP`): Abschnitt 4.24.

4.8 Erweiterte Rechenausdrücke (`min`, `max`, Vergleiche, `$CALCOn`)

Die Rechenausdrücke aus 4.2 kennen zusätzlich:

`min(a,b)` **und** `max(a,b)` liefern den kleineren bzw. größeren von zwei Ausdrücken (die Teilausdrücke dürfen selbst `min`, `max` und Klammern enthalten, ohne Leerzeichen), z. B. `$CALC(min($1,31))`.

Vergleiche `<` `>` `<=` `>=` `==` `!=` liefern 1 (wahr) oder 0 (falsch). Sie stehen in der Rangfolge wie in C: schwächer als `+` `-` und `<<` `>>`, stärker als `&`; zuerst `<` `>` `<=` `>=`, dann `==` und `!=`. Beispiel: `$CALC($1>=$2)` ergibt 1, wenn `$1` mindestens so groß ist wie `$2`. Ein einzelnes `=` ist kein Operator.

`$CALCOn(ausdruck)` mit n = 1 bis 9 gibt das Ergebnis oktal mit führender 0 und genau n Ziffern aus (mit führenden Nullen aufgefüllt): `$CALCO4(100)` ergibt `00144`, `$CALCO4(2432)` ergibt `04600`. Braucht das Ergebnis mehr als n Oktalziffern oder ist es negativ, meldet der Assembler einen Fehler. `$CALCO(...)` ohne Zahl bleibt unverändert (keine feste Breite). Im Text von `$SET` darf der Ausdruck auch `$V(...)` enthalten (`$SET nm "$CALCO2($V(i))"`, Abschnitt 4.22).

`$CALCBn(ausdruck)` mit n = 1 bis 64 gibt das Ergebnis binär mit genau n Ziffern aus (mit führenden Nullen aufgefüllt, ohne Präfix): `$CALCB8(5)` ergibt `00000101`; zu schmal, negativ oder n über 64 ist ein Fehler (seit c82ass 1.01-022, Abschnitt 4.27).

`$CALCHn(ausdruck)` mit n = 1 bis 16 gibt das Ergebnis hexadezimal mit genau n Ziffern aus (Großbuchstaben, mit führenden Nullen aufgefüllt, ohne Präfix): `$CALCH8(255)` ergibt `000000FF`; zu schmal, negativ oder n über 16 ist ein Fehler (seit c82ass 1.01-023, Abschnitt 4.28).

Vereinheitlichung auf 64 Bit: Die Zahlen in den Vergleichen von `$IF` und `$ELIF` werden jetzt wie in `$CALC` mit 64 Bit verglichen (früher 32 Bit). Ziffernfolgen mit mehr als 18 Stellen gelten nicht mehr als Zahl und werden alphabetisch verglichen.

Der Test `Assembler/Claude/makrocalc2` prüft `min`, `max`, die Vergleiche, `$CALCO4` und den 64-Bit-Vergleich; die Negativtests `makro_calc_min_komma` und `makro_calc_breite` liegen im Regressionstest.

Neben `min` und `max` kennt der Rechner seit c82ass 1.01-001 die Funktionen `def(name)`, `art(name)` und `zellen(name)` (Abschnitt 4.23). Ab 1.01-018 kommen `adr(name)` und `abstand(von,bis)` hinzu (Adresse einer Marke und Abstand zweier Marken, 4.23). Ab 1.01-012 kennt `zellen` auch zwei Namen: `zellen(von,bis)` ergibt die Zellenzahl eines Blocks (4.23).

4.9 Projektspezifische Makrodateien (`;#MACRODATEI1=` bis `;#MACRODATEI9=`)

Neben `c82ass.makros` kann ein Programm bis zu neun projektspezifische Makrodateien benutzen. Sie werden im Quelltext durch Kommentarzeilen der Form `;#MACRODATEI1=<Pfad>` bis `;#MACRODATEI9=<Pfad>` zugeordnet (vor oder hinter `start`, an beliebiger Stelle). Der Pfad ist der Rest der Zeile ohne Randleerzeichen. Ein relativer Pfad wird zuerst im Verzeichnis der Quelldatei gesucht, dann im Arbeitsverzeichnis des Aufrufs; ein absoluter Pfad gilt, wie er ist. Eine Nummer darf nur einmal vorkommen. Dateiformat und Makrosyntax sind die der Makrodatei `c82ass.makros`.

Kommentar hinter `MACRO` (ab c82ass 1.01-007): Die Zeile `name: MACRO <Parameter>` einer Makrodatei endet beim ersten `;`, wie im Quelltext (4.1); Leerraum vor dem `;` und am Zeilenende zählt nicht. `MKA: MACRO ; Kommentar` ist ein Makro ohne Parameterangabe, `MKB: MACRO 2 ; Kommentar` und `MKC: MACRO 2;Kommentar` haben genau zwei Parameter, `MKD: MACRO 1+ ; Kommentar` mindestens einen (4.13). Bis c82ass 1.01-006 zählte in einer Makrodatei jedes Wort hinter `MACRO` in derselben Zeile zur Parameterangabe, auch ein Kommentar; im Quelltext störte derselbe

Kommentar nie. Beispiele der Meldungen (Rückgabecode 16; die Zahl hängt von den Wörtern des Kommentars ab, die Makros wurden mit der richtigen Zahl von Parametern aufgerufen): `MACRO ; Kommentar` meldet „erwartet 2 Parameter, aber 0 wurden übergeben“, `MACRO 2 ; Kommentar` meldet „erwartet 3 Parameter, aber 2 wurden übergeben“, `MACRO 2;Kommentar` meldet „erwartet 1 Parameter, aber 2 wurden übergeben“, und ein Kommentar mit mehr als neun Wörtern meldet „Makro MKE: 18 Parameter angegeben, zulaessig sind maximal 9“. Mit einem älteren c82ass steht der Kommentar deshalb in einer Makrodatei in einer eigenen Zeile.

Steht das ; ohne Leerzeichen unmittelbar hinter `MACRO` und folgt keine Parameterangabe (`name: MACRO;Kommentar`), ist die Zeile ab c82ass 1.01-031 eine Makrodefinition ohne Parameterangabe (wie `name: MACRO ; Kommentar`), in der Makrodatei ebenso wie im Quelltext; der Assembler trennt den Kommentar ab. Mit Parameterangabe galt das ; ohne Leerzeichen schon früher (`MACRO 2;Kommentar`, siehe oben). Bis c82ass 1.01-030 zählte `MACRO;Kommentar` ohne Leerzeichen als ein einziges Wort und damit als Operationscode, und die Zeile war keine Makrodefinition: Im Quelltext endete der Lauf mit „Makrodefinition im Quelltext: MEND ohne zugehöriges MACRO“, der Aufruf eines solchen Makros aus einer Makrodatei mit „Operationscode 'name' ist ungültig“ (jeweils Rückgabecode 16). Die Meldung „MEND ohne zugehöriges MACRO“ bleibt für andere fehlerhafte Zeilen vor `MEND` unverändert. Mit einem älteren c82ass gehört vor das ; ein Leerzeichen. Die Positivtests zum neuen Verhalten sind `Assembler/Claude/makro_stolperfallen` (Makro im Quelltext) und `Assembler/Claude/makrokommentar_dicht` (Makrodatei, siehe unten).

Suchreihenfolge eines Makros: 1. im Quelltext (Makrodefinition, 4.1), 2. in den projektspezifischen Makrodateien in der Reihenfolge `MACRODATEI1` bis `MACRODATEI9`, 3. in `c82ass.makros` (oder der mit `-m` angegebenen Makrodatei). Es gilt die erste Fundstelle; die übrigen gleichnamigen Makros bleiben für diesen Lauf unwirksam. Dazu gibt der Assembler je eine Warnung aus: „Makro X aus c82ass.makros wird für diesen Lauf durch die Definition in der Makrodatei <Datei> (Zeile n) ersetzt“, „Makro X ist schon in der Makrodatei <Datei> definiert; die Definition in <Datei2> (Zeile n) wird ignoriert“ und, wenn der Quelltext ein Makro einer projektspezifischen Datei ersetzt, „Makro X aus der Makrodatei <Datei> wird für diesen Lauf durch die Definition im Quelltext ersetzt“. Ein doppelter Name innerhalb derselben Datei bleibt wie bei `c82ass.makros` eine Fehlermeldung, das zweite Makro wird übersprungen. Die Dateien selbst bleiben unverändert.

Fehler (Rückgabecode 16, Abbruch nach Pass 0): Nummer außerhalb 1 bis 9, fehlendes `=`, leerer Pfad, doppelte Nummer, Datei nicht lesbar. Fehler in einem Makro einer projektspezifischen Datei nennen deren Namen und die Zeile der Makrodefinition. Die Zeilen `;#MACRODATEIn=` gelangen nicht in die `.test`-Datei.

Beispiel (Programm `Assembler/Projekt_ZT_LOADER/Unterprojekt_Allgemein/pname`, Datei `Assembler/Projekt_ZT_LOADER/macro_zt_loader.macro` mit dem Makro `PNAME`, dem Unterprogramm „Programmname eintragen“ des Urladers der Zusatztrommel):

```
;#MACRODATEI1=../../macro_zt_loader.macro
        start
...
pname:          PNAME               ; Rumpf des Unterprogramms (Aufruf 025 pname)
```

Der Test `Assembler/Projekt_ZT_LOADER/Unterprojekt_Allgemein/pname` benutzt das Makro; die Negativtests `makro_projekt_fehlt`, `makro_projekt_nummer`, `makro_projekt_leer`, `makro_projekt_doppelt`, `makro_projekt_ohne_gleich` und der Test `makro_projekt_rang` (Rangfolge) liegen im Regressionstest.

Prüfung des Kommentars hinter `MACRO`: Der Positivtest `Assembler/Claude/makrokommentar` (die Makrodatei `makrokommentar.macro` kommt über `;#MACRODATEI1=` hinzu) endet im Interpreter mit „Test wahr“; die acht Sollwerte sind von Hand hergeleitet und standen vor dem ersten Lauf im Prüfblock. In der Makrodatei stehen `MKNULL` (`MACRO ; Kommentar`), `MKZWEI` (`MACRO 2` mit einem Kommentar aus mehr als neun Wörtern), `MKDICHT` (`MACRO 2;Kommentar`), `MKVAR` (`MACRO 1+ ; Kommentar`, aufgerufen mit einem und mit drei Parametern) und `MKTAB` (Tabulatoren um die Parameterzahl), im Quelltext zum Vergleich `QNULL` und `QZWEI`. Mit c82ass bis 1.01-006 scheitert das Programm („Makro MKZWEI: 19 Parameter angegeben, zulaessig sind maximal 9“ und weitere Meldungen), der Test greift also. Ein Negativtest liegt im Regressionstest: `makro_datei_kommentar_zahl` (Makro mit `MACRO 2 ; Kommentar`, aufgerufen mit einem Parameter: Rückgabecode 16, „erwartet 2 Parameter, aber 1 wurden übergeben“; der Kommentar zählt nicht mit). Der frühere Negativtest `makro_datei_kommentar_dicht` (`MACRO;Kommentar` ohne Leerzeichen: „Operationscode 'MKDICHT' ist ungültig“) ist mit c82ass 1.01-031 entfallen, weil dieser Fall jetzt funktioniert; an seine Stelle tritt der Positivtest `Assembler/Claude/makrokommentar_dicht` (eine eigene Makrodatei mit `MKDICHT: MACRO;Kommentar ohne Leerzeichen` und einem `dw 1`; Zelle 0001 enthält 1, Test wahr). Der Lauf durch `pruefe_alles.sh` mit c82ass 1.01-007 (Übersetzung mit `-Werror` ohne Warnung, danach auch unter AddressSanitizer und UBSan): 259 Programme ohne Abweichung, 323 Negativtests, ALLES IN ORDNUNG; mit c82ass 1.01-031 (ebenso, auch unter AddressSanitizer und UBSan): 283 Programme ohne Abweichung, 508 Negativtests, ALLES IN ORDNUNG.

4.10 Zeichenketten-Parameter (Text in Anführungszeichen, `$TYPTEXT`, `$LENn`)

Ein Aufrufparameter in Anführungszeichen ("..." oder '...') ist ein einziger Parameter und darf Leerzeichen enthalten. Die Anführungszeichen werden entfernt. Das öffnende Zeichen steht am Wortanfang, das schließende gleiche Zeichen unmittelbar vor Leerraum oder Zeilenende; sonst gilt das Wort wie bisher bis zum nächsten Leerraum. Im Text darf das jeweils andere Anführungszeichen stehen ("it's"). Nicht möglich sind das Begrenzungszeichen selbst und das Semikolon (es leitet den Kommentar der Aufrufzeile ein). Ein Parameter hat höchstens 255 Zeichen; "" ergibt einen leeren Parameter.

Der Text wird wie jeder Parameter an der Stelle von $1 bis $9 eingesetzt, z. B. dto $1 oder dt $1. In den Wortvergleichen von $IF und $ELIF trennen Leerzeichen die Wörter; ein Text mit Leerzeichen lässt sich dort nicht vergleichen, wohl aber seine Länge (siehe $LENn).

$PRx $TYPTEXT [maxlänge]: Prüfung eines Zeichenketten-Parameters. Der Text darf nicht leer sein, keine Steuerzeichen enthalten und höchstens maxlänge Zeichen haben (maxlänge 1 bis 200, ohne Angabe 200). Fehler (Rückgabecode 16): „Parameter $1: Text ist leer“, „Text enthält Steuerzeichen“, „Text hat n Zeichen, erlaubt sind höchstens m“ und „$TYPTEXT erwartet höchstens eine Längenangabe (1 bis 200)“.

$LEN1 bis $LEN9: Anzahl der Zeichen des Aufrufparameters 1 bis 9 als Dezimalzahl, nach dem Entfernen der Anführungszeichen. $LENn gilt im Operanden, im Kommentar, in $CALC(...), in $IF und $ELIF und in $REPT (z. B. $REPT $LEN1 oder $CALC($LEN1+2)). Wird der Parameter n nicht übergeben, bleibt $LENn im Text stehen und in $CALC und $REPT meldet der Assembler „Parameter $n wird nicht übergeben“.

$LEN(ausdruck) : Wie $LENn, aber die Nummer des Parameters ist ein Ausdruck, ein Parametername oder eine Zahl, auch über 9. $LEN(name) liefert die Zeichenzahl des Parameters name (4.26). $LEN(ausdruck) verhält sich wie $P(ausdruck) (4.13): Es gilt im Operanden, im Kommentar, in $CALC(...), in $REPT, $IF und $ELIF, auch in Makros ohne Namen ($LEN(1)); $V(name) ist im Ausdruck nur innerhalb von $CALC erlaubt ($CALC($LEN($V(n))) geht, dw $LEN($V(n)) meldet „Operand '$LEN(1)' ist ungültig“). Fehler (Rückgabecode 16): „$LEN: $LEN(5) = 5: Parameter 5 wird nicht übergeben (übergeben: 2)“, „$LEN: Klammer) fehlt“, „$LEN: $LEN(dritter): Symbol "dritter" ist im Quelltext weder mit abs noch mit equ <Zahl> definiert …“ (ein Wort, das kein Parametername ist, wird als Ausdruck ausgewertet), „$LEN: $LEN( ): unerlaubtes Zeichen (erlaubt: Zahlen, abs- und equ-Symbole mit Zahlenwert, + - * / % << >> < > <= >= == != & ^ | ~, min(a,b), max(a,b), def(s), art(s), zellen(s), adr(s), abstand(s,t) und Klammern)“. $DEFINED im Ausdruck von $LEN ist nicht geprüft.

Beispiel:

```
TXT:      MACRO 2
          $PR1 $TYPTEXT 20
          dw  $LEN1             ; Zeichenzahl des Textes
          dw  $CALC($LEN1+$2)   ; Zeichenzahl plus Parameter 2
          MEND
          TXT "ab cd e" 3         ; erzeugt dw 7 und dw 10
```

Der Test Assembler/Claude/makrotext prüft Text mit Leerzeichen, Apostrophe, $LENn im Operanden, in $CALC, $IF und $REPT; die Negativtests makro_text_zu_lang, makro_text_leer und makro_text_laenge liegen im Regressionstest.

Texte lassen sich seit c82ass 1.00-068 auch zerlegen, durchsuchen und vergleichen: Die Zeichenkettenfunktionen `$SUBSTR`, `$INDEX`, `$UPPER`, `$LOWER`, `$STRCMP`, `$ORD`, `$CHR` und `$VAL` stehen in Abschnitt 4.21, `$VALO` (oktal) in 4.22, `$VALB` (binär) in 4.27, `$VALH` (hexadezimal) in 4.28. Der Wortvergleich von `$IF` bleibt unverändert; ein Text mit Leerzeichen lässt sich mit `$STRCMP` vergleichen.

4.11 Makroliste (`--makros`)

Mit der Option `--makros` gibt der Assembler nach einem fehlerfreien Lauf eine Makroliste aus und hängt sie an die `.log`-Datei an. Sie nennt je Makro die Zahl der Aufrufe in diesem Lauf (verschachtelte Aufrufe zählen mit) und die Herkunft: „Quelltext, Zeile n“ (MACRO-Anweisung im `.asm`), „Projektdatei <Pfad>, Zeile n“ (`;#MACRODATEIn=`, Abschnitt 4.9) oder „<Pfad von c82ass.makros>, Zeile n“. Aufrufnummern nennt die Liste nicht; die Nummer eines Aufrufs liefert `$AUFRUFNR` (4.29).

Zuerst stehen die aufgerufenen Makros in der Reihenfolge der Makrotabelle (c82ass.makros, Projektdateien, Quelltext), dann die nicht aufgerufenen Makros aus Quelltext und Projektdateien mit 0 Aufrufen. Nicht aufgerufene Makros aus c82ass.makros fehlen, sonst wäre die Liste jedes Mal lang. Ruft das Programm kein Makro auf, steht „(keine Makros aufgerufen)“. Die Liste verändert weder `.obj` noch `.dmp`, der Rückgabecode bleibt 0. Bei einem Fehler in Pass 0 oder später entsteht keine Liste.

Beispiel (Aufruf mit `--makros`):

```
Makro                   | Aufrufe | Herkunft
----------------------- | ------- | --------
ZINIT                   |       1 | Quelltext, Zeile 4
ZAEHL                   |       5 | Quelltext, Zeile 8
```

Die Tests `makroliste` (mit `--makros`) und `makroliste_aus` (ohne Option, keine Liste) liegen im Regressionstest. Der Usage-Text (`-h`) nennt jetzt `--makros` und die Anweisung `;#MACRODATEI1=` bis `;#MACRODATEI9=`.

4.12 Zeichenketten in Makro-Variablen (`$SET name "text"`, `$V(#name)`)

Mit `$SET name "text"` (oder `'text'`) hält eine Makro-Variable eine Zeichenkette . Der Text steht in der $SET-Zeile selbst, darf Leerzeichen und Parameter ($1 bis $9) enthalten, hat höchstens 200 Zeichen und darf kein Semikolon enthalten; hinter dem schließenden Anführungszeichen darf nur ein Kommentar stehen. `$V(name)` liefert den Text ohne Anführungszeichen, im Operanden, in `$IF` und `$ELIF` (dort gilt der Text als Wort, ein Text mit Leerzeichen lässt sich nicht vergleichen) und in `$REPT`; `$V(#name)` liefert die Zeichenzahl. Ein späteres $SET name mit einer Zahl macht die Variable wieder zur Zahl. Fehler (Rückgabecode 16): `$INC` auf einer Textvariablen, `$V(#name)` auf einer Zahl, ein Text ohne schließendes Anführungszeichen, ein Text über 200 Zeichen und ein Text mit Semikolon ; geprüft wird der Text nach dem Auflösen von `$V(...)`; vorher wurde er angenommen und erst bei der Verwendung als Fehler gemeldet; ein Kommentar hinter dem schließenden Anführungszeichen bleibt erlaubt; Negativtest `makro_str_set_semikolon`). Die Meldung nennt das Makro, z. B. „M1 $SET a: Der Text enthält ein Semikolon, ein Text enthält kein Semikolon“. Über einen Aufrufparameter kommt kein Semikolon in den Text, denn die Aufrufzeile wird am Semikolon als Kommentar abgeschnitten (`M1 ab;cd` übergibt `ab`).

Beispiel: `$SET t "ab $1 e"` mit dem Aufruf T1 "cd fg" ergibt den Text ab cd fg e; `dw $V(#t)` erzeugt das Wort 10 (012 oktal). Der Test Assembler/Claude/makrovartext prüft das, dazu die Negativtests makro_set_text_inc, makro_set_text_ende und makro_vlen_zahl.

Seit c82ass 1.00-068 löst `$SET name "text"` die Ausdrücke `$V(...)` im Text sofort beim Setzen auf: `$SET c "$V(a)$V(b)"` setzt den zusammengesetzten Text, und `$V(#c)` liefert dessen wahre Länge (zuvor blieb `$V(a)$V(b)` stehen und wurde erst bei der Verwendung aufgelöst). Teilstring, Suchen, Groß- und Kleinschreibung, Vergleich und Zeichencode: Abschnitt 4.21. Seit c82ass 1.00-070 wertet `$SET name "text"` auch `$CALC(...)`, `$CALC0(...)` und `$CALC0n(...)` im Text aus (Abschnitt 4.22); die Prüfungen auf 200 Zeichen und auf ein Semikolon gelten für das Ergebnis.

4.13 Variable Parameterzahl (MACRO n+, $P(i)) und Herkunft im Listing

Variable Parameterzahl: Die Definition MACRO n+ legt fest, dass der Aufruf mindestens n und höchstens 32 Parameter hat; MACRO + (ohne Zahl) bedeutet mindestens 0. Ohne das Pluszeichen bleibt die Zahl die genaue Parameterzahl, mehr Parameter sind wie bisher ein Fehler. $1 bis $9 und $LEN1 bis $LEN9 gelten weiter für die ersten neun Parameter; $# liefert die Zahl der übergebenen Parameter. In einem Makro mit Namen stehen die Namen hinter MACRO und das + als letztes Wort (MACRO von bis +); der Aufruf braucht dann mindestens so viele Parameter, wie Namen stehen, und die weiteren erreicht nur $P(i). Ein Name ab dem zehnten Parameter wird im Rumpf zu $P(k); $P(name) und $LEN(name) (4.10) nehmen einen vereinbarten Namen als Nummer. Ein Aufruf mit zu wenigen Parametern meldet „erwartet mindestens 2 Parameter, aber 1 wurden übergeben! (für $ende gibt es kein $DEFAULT)“.

$P(ausdruck) liefert den Parameter mit der Nummer, die der Ausdruck ergibt. Der Ausdruck darf Zahlen, den Schleifenzähler $Z1 bis $Z4, $# und die Rechenzeichen von $CALC enthalten (z. B. $P($Z1+1) oder $P($#)); $V(name) ist darin nicht erlaubt. $P(...) steht im Operanden, in $CALC und in $CALC-Teilen von $IF, $ELIF und $REPT; in $CALC wird der Parameter geklammert eingesetzt. Fehler (Rückgabecode 16): Nummer kleiner als 1 (z. B. $P(0)), Nummer größer als die Zahl der übergebenen Parameter, fehlende schließende Klammer, ein Aufruf mit weniger als n Parametern ohne $DEFAULT, ein Aufruf mit mehr als 32 Parametern.

Beispiel: LISTE: MACRO 1+ mit den Zeilen dw $# und $REPT $# / dw $P($Z1+1) / $ENDREPT erzeugt zuerst die Parameterzahl und dann alle Parameter, auch den zehnten und elften, die $1 bis $9 nicht erreichen. Der Test Assembler/Claude/makroparamvar prüft das (Sollwerte von Hand hergeleitet); die fünf Negativtests makro_param_* decken die Fehlerfälle ab.

Herkunft im Listing: Mit der Option --makros (4.11) steht im Listing und in der .dmp vor jeder Makroexpansion eine Kommentarzeile ;+++ Makro NAME aus <Herkunft>, aufgerufen in Quelltextzeile N. Die Herkunft hat die Form wie in der Makroliste („Quelltext, Zeile n“, „Projektdatei <Pfad>, Zeile n“ oder Pfad von c82ass.makros mit Zeile). Ohne

--makros bleiben Listing und .dmp unverändert, die Ausgabe ist bitgleich. Die Tests makroliste und makroliste_aus prüfen jetzt auch die .dmp; der Regressionstest kennt dafür die Direktiven dmp-da und dmp-weg.

4.14 Makrodateien zusammensetzen ($INCLUDE)

Mit $INCLUDE <Pfad> im Rumpf eines Makros übernimmt der Assembler die Zeilen der genannten Datei an dieser Stelle in den Rumpf . Die Anweisung steht auf einer eigenen Zeile zwischen MACRO und MEND einer Makrodatei (c82ass.makros oder Projektdatei, 4.9); im normalen Quelltext und im Rumpf eines dort definierten Makros wirkt sie ab c82ass 1.01-009 ebenfalls (siehe unten). Der Pfad ist relativ zu der Datei, die das $INCLUDE enthält; findet sich dort nichts, gilt er wie angegeben (Arbeitsverzeichnis oder absolut). Er darf in Anführungszeichen stehen und von einem Kommentar mit Semikolon gefolgt werden.

In einer Makrodatei enthält die eingebundene Datei nur Rumpfzeilen (kein MACRO, kein MEND) und darf selbst $INCLUDE enthalten; die Verschachtelung darf höchstens 4 Ebenen tief sein. Fehler (Rückgabecode 16): Datei nicht lesbar, Verschachtelung tiefer als 4 (das fängt auch einen Kreis ab). Ein $INCLUDE ohne Pfad wird in einer Makrodatei nicht erkannt und als ungültiges Makro-Schlüsselwort gemeldet. Die Fehlermeldung nennt Datei und Zeile der $INCLUDE-Anweisung.

Beispiel: INCTEST: MACRO 1 mit der Zeile $INCLUDE teil/a.inc, die ihrerseits $INCLUDE b.inc enthält (teil/b.inc). Der Test Assembler/Claude/makroinclude prüft das (Sollwerte von Hand hergeleitet).

`$INCLUDE` im Quelltext: Eine Zeile `$INCLUDE <Pfad>` im Quelltext (.asm) wird durch die Zeilen der genannten Datei ersetzt, als stünden sie an dieser Stelle im Quelltext; so lassen sich Datenteil, Unterprogramm-Bibliothek oder Makrosammlung eines Programms auf Dateien verteilen. `$INCLUDE` steht als ganzes Wort am Zeilenanfang (Leerzeichen oder Tabulator davor sind erlaubt). Der Pfad darf in Anführungszeichen (" oder ') stehen und von einem Kommentar gefolgt werden, der mit einem Semikolon nach einem Leerzeichen oder Tabulator beginnt; ein `;K` ohne Leerzeichen davor gehört zum Pfad und ergibt „... konnte nicht geöffnet werden“. Der Pfad gilt relativ zu der Datei, die das `$INCLUDE` enthält; findet sich dort nichts, gilt er wie angegeben (Arbeitsverzeichnis oder absolut). Die Verschachtelung darf 4 Ebenen tief sein: die Hauptdatei bindet eine Datei ein, diese eine zweite, diese eine dritte und diese eine vierte; ein `$INCLUDE` in der vierten Datei, und damit auch ein Kreis, ist ein Fehler („$INCLUDE '<Pfad>' ist zu tief verschachtelt (höchstens 4; ein Kreis wird so abgefangen)“). Weitere Fehler (Rückgabecode 16): kein Pfad („$INCLUDE braucht einen Pfad.“), eine Datei, die sich nicht öffnen lässt („$INCLUDE '<Pfad>' konnte nicht geöffnet werden: <Ursache>“), und eine Marke vor `$INCLUDE`: Die Zeile ist dann eine normale Zeile und endet mit „Operationscode '$INCLUDE' ist ungültig“ (wie bei den anderen Direktiven, 4.1).

Eigenschaften: Die eingebundenen Zeilen sind gewöhnlicher Quelltext. Anweisungen, Marken, Kommentare, `$`-Direktiven (4.1) und Makrodefinitionen sind erlaubt, und ein in einer eingebundenen Datei definiertes Makro lässt sich im Haupttext aufrufen. Das Einbinden geschieht beim Einlesen des Quelltextes, also vor der Auswertung von `$IF` und `$REPT`: Ein `$INCLUDE` in einem Zweig, der nicht erzeugt wird, wird trotzdem gelesen, und eine fehlende Datei ist dort ein Fehler; zwischen `$REPT` und `$ENDREPT` werden die eingebundenen Zeilen mitwiederholt. Im Rumpf eines im Quelltext definierten Makros gehören die eingebundenen Zeilen zum Rumpf (bis c82ass 1.01-008 wurde die Zeile dort übergangen; in einer Makrodatei gilt weiter die Regel vom Anfang dieses Abschnitts). Die Auslösung der `$`-Auswertung (4.1) zählt die eingebundenen Zeilen mit, die `$INCLUDE`-Zeile selbst löst sie nicht aus: Hat weder die Hauptdatei noch eine eingebundene Datei eine andere Zeile mit `$` am Anfang, bleibt ein `$CALC` im Operanden Text und ist ein Fehler. Die Pragmas `LEITADRESSE` (3.3.16), `;#MACRODATEI1=` bis `;#MACRODATEI9=` (4.9) und `;#OPSYM=` gelten nur in der Hauptdatei und sind in einer eingebundenen Datei ein Fehler (`;#MAKROLOKAL=` dagegen gilt auch in einer eingebundenen Datei, 4.31); ein `;#PRUEFE`-Block und die Testmarken (`;#DOK` usw.) werden nur aus der Hauptdatei gelesen und bleiben in einer eingebundenen Datei ohne Wirkung und ohne Meldung.

Fehlerorte: Meldungen nennen die Datei, in der die Zeile steht, und die Zeile in dieser Datei, nicht eine Zählung über alle Dateien: `<Pfad wie geöffnet>:<Zeile>: Fehler: …` (zum Beispiel `quelltext_include_fehler_1.inc:3: Fehler: Operationscode 'foo' ist ungültig`). Die Meldungen der Einlesephase nennen „Zeile n der Quelldatei“ für die Hauptdatei und „Zeile n der eingebundenen Datei '<Pfad>'“ für eine eingebundene Datei. Ein Fehler in der Hauptdatei hinter einem `$INCLUDE` trägt seine eigene Zeilennummer (Negativtest `quelltext_include_danach`). Die Warnung „Symbol 'nix' (0001, quelltext_include_warn_1.inc Zeile 1) ist definiert, wird aber nie verwendet“ nennt Datei und Zeile des Symbols; die Querverweisliste (`--xref`) gibt für die Hauptdatei wie bisher die Zeilennummer und für eine eingebundene Datei `<Datei>:<Zeile>` an.

Beispiel und Prüfung: Das Programm Assembler/Claude/quelltextinclude bindet teile/daten.inc (Datenteil, darin `$SET n 3` und `dw $V(n)`) und teile/bibl.inc (Unterprogramm verdopple, das seinerseits teile/bibl2.inc mit plus1 und plus3 einbindet) in die Hauptdatei ein. Der Test prüft 28 Zellen; die Sollwerte sind von Hand hergeleitet (erg = 017, dann 036, 037 und 042; Halt auf 0014, AC = 042). Dazu kommen im Regressionstest 14 Negativtests quelltext_include_*: Datei fehlt, ohne Pfad, Kreis, zu tief, Grenze bei 4 Ebenen (geht), Fehler in der eingebundenen

Datei, Fehler dahinter in der Hauptdatei, verschachtelte Datei, `$V`-Fehler in der eingebundenen Datei, die drei Pragmas, Include in einem inaktiven `$IF` und die Warnung mit Datei und Zeile. Stand 04.11.2026: Der vollständige Regressionstest mit c82ass 1.01-009 ergibt 262 Programme ohne Abweichung und 360 Negativtests ohne Fehler, auch unter AddressSanitizer und UBSan.

4.15 Reihenfolge der Auflösung, Rekursion und Operation aus Parameter

Verschachtelte Aufrufe in Quelltextreihenfolge: Ruft ein Makro ein anderes auf, wird der innere Aufruf an der Stelle seiner Zeile aufgelöst, bevor die folgenden Zeilen des äußeren Makros und die folgenden Aufrufe im Quelltext bearbeitet werden. $SET, $INC und $V(name) (4.7) wirken deshalb in der Reihenfolge der Zeilen, auch über Aufrufgrenzen hinweg. Beispiel: OUT1 mit den Zeilen $INC n und INNER, INNER mit dw $V(n); drei Aufrufe von OUT1 erzeugen 1, 2, 3 (früher dreimal den Endstand 3, weil zuerst alle äußeren und danach alle inneren Aufrufe aufgelöst wurden). Ruft ein Makro erst INNER auf und führt danach $INC n aus, sieht INNER den Stand vor dem Erhöhen. Test Assembler/Claude/makroverschachtelt.

Generische Labels: Die drei Buchstaben hinter ML<4 Ziffern> hängen von der laufenden Nummer des Aufrufs ab (Reihenfolge der Auflösung, der äußere Aufruf vor den inneren). Es gibt 17576 verschiedene Suffixe; früher waren es nur 26, sodass ab dem 27. Makroaufruf eines Laufs (über alle Makros gezählt) Labels gleicher Nummer kollidierten (Meldung „Symbol existiert bereits“). Ein generisches Label darf im Operanden mehrfach vorkommen (000 ML0001%+ML0001%; früher wurde nur das erste ersetzt). Ein Makro darf sein eigenes generisches Label einem verschachtelten Makro als Parameter übergeben, auch wenn dieses ein Label gleicher Nummer hat. Mehr als 17000 Makroaufrufe in einem Lauf bricht der Assembler ab. Diese laufende Nummer liefert `$AUFRUFNR` als Dezimalzahl (4.29). Tests makrolabel, makroverschachtelt und Negativtest makro_label_doppelt.

Operation aus einem Parameter: Eine Zeile der Makrodefinition darf mit einem Parameter oder Rechenausdruck beginnen ($1 bis $9, $P(..), $V(..), $CALC(..), $LENn). Die Zeile ist dann eine Anweisung mit Operation, Operand und Kommentar, z. B. $1 $2 im Makro OP; der Aufruf OP 211 zahl erzeugt 211 zahl. Früher wurde eine solche Zeile ohne Meldung verworfen. Test Assembler/Claude/makroopparam. Auch die Marke darf aus einem Parameter stammen (`$1: abs 5`, ab c82ass 1.01-031; Einzelheiten in 4.18, Test `Assembler/Claude/makro_stolperfallen`).

Ende der Makrodefinition: In einer Makrodatei gilt MEND (auch MACROENDE und macroende) nur als ganzes Wort in der Operationsspalte; ein Label davor ist erlaubt. Kommentarzeilen (; und #) und Wörter, die MEND nur enthalten (z. B. AMEND, RECOMMEND), beenden das Makro nicht mehr (früher beendete jedes Vorkommen der Zeichenfolge in der Zeile das Makro). Fehlt das Ende, lautet die Meldung „Fehlendes MEND in MACRO <Name>“ (früher mit Tippfehler). Test Assembler/Claude/makromend und Negativtest makro_mend_fehlt.

Rekursion und Schachtelungstiefe: Ein Makro darf sich weder selbst noch über andere Makros aufrufen. Bei mittelbarer Rekursion meldet der Assembler die Aufrufkette, z. B. „Rekursiver Macro-Aufruf von: A1 [Aufrufkette: A1 -> B1 -> A1]“ (Rückgabecode 16; früher endete der Lauf erst an der Zeilengrenze mit „Zu viele Zeilen“). Makroaufrufe dürfen höchstens 32 Ebenen tief verschachtelt sein. Negativtests makro_rekursion_indirekt und makro_kette_tief.

Leerzeilen: Behoben ist ein Fehler der Makroauflösung, bei dem eine Leerzeile des Quelltextes Reste einer früheren Zeile übernehmen und so eine Anweisung ohne Quelltextzeile erzeugen konnte (Beispiel: Proj_FeldSortieren/code hatte eine überzählige Zelle 240 min_index; die Referenz des Regressionstests ist neu geschrieben).

4.16 Wiederholungsbefehle im Makro (!$wdhl und !$wdhl_adr_e)

Zweck: Die Anweisungen !$wdhl und !$wdhl_adr_e stehen im Rumpf eines Makros (Makrodatei oder Quelltext). Beim Auflösen des Makroaufrufs setzt der Assembler jede solche Zeile in eine Befehlszeile des Wiederholungsbefehls um, also in einen Operationscode (604 bis 631 und 644, siehe unten) mit einem Adressteil. Der Adressteil ist der Anfang der Spur plus der Startsektor für n Wiederholungen. Der Startsektor ist $13 \cdot (32 - n)$ modulo 32 (n = 1 bis 32, oktal ausgegeben; n = 1 ergibt Sektor 023, n = 32 Sektor 0). Die Zelle mit diesem Sektor ist die erste der n Zellen, die der Wiederholungsbefehl nacheinander verarbeitet; die folgenden Zellen liegen je 13 Sektoren weiter und enden mit Sektor 023. Beispiel: !$wdhl JA LI KON 3 004000 wird zu 605 04031.

Neue Form (ab 1.00-059): Fünf Operanden stehen auf der Zeile selbst: !$wdhl <Löschen> <Resultat> <Operation> <Anzahl> <Adresse>, zum Beispiel !$wdhl JA LI KON 3 004000. Die Operanden dürfen Makroparameter ($1 bis $9 oder, in einem Makro mit Namen, $name; 4.26) sein. Löschen ist JA oder NE (der AC wird vor der Operation gelöscht oder nicht), Resultat ist LI, RE, ZL oder OHNE, Operation ist KON, ADD, SUB oder BADD, Anzahl ist eine Zahl von 1 bis 32, Adresse ist der Anfang der Spur als Zahl (oktal mit führender 0, sonst dezimal; kein Symbol).

Alte Form (unverändert): !$wdhl mit einem oder ohne Operanden, zum Beispiel !$wdhl $5. Löschen, Resultat, Operation, Anzahl und Adresse kommen dann aus den Parametern $1 bis $5 des Makros, in dieser Reihenfolge. Der

Operand der Zeile wird nicht ausgewertet, und die Werte werden nicht geprüft (eine Anzahl außerhalb von 1 bis 32 wird auf 1 oder 32 begrenzt). So arbeitet das Makro WDHL der Makrodatei c82ass.makros (ab c82ass 1.01-014 mit benannten Parametern: WDHL: MACRO loeschen resultat operation anzahl spur, Rumpfzeile !$wdhl $spur; Aufruf und Ergebnis sind unverändert, 4.26). Die Fünf-Operanden-Form mit Namen (!$wdhl $loeschen $resultat $operation $anzahl $spur) ist möglich, aber strenger: sie verlangt eine Zahl als Adresse und prüft die Werte; mit dem Platzhalter $ADRESSE meldet sie „!$wdhl: Adresse "$ADRESSE" ungültig (Zahl, oktal mit führender 0 oder dezimal; kein Symbol)“. Zwei bis vier Operanden oder mehr als fünf sind ein Fehler.

Befehlscodes: Der Code ergibt sich nur aus Löschen und Operation. KON ergibt 604 (Löschen NE) und 605 (JA), ADD 612 und 613, SUB 626 und 627, BADD 630 und 631. Die letzte Ziffer legt die Verschiebung fest: KON verschiebt nach links (W-Konjunktion mit Linksverschiebung), ADD nach rechts, SUB zyklisch nach links, BADD gar nicht; die um 1 größere Ziffer bedeutet mit Löschen. Der Operand Resultat (LI, RE, ZL, OHNE) wird in der neuen Form nur auf gültige Werte geprüft; er ändert den Befehlscode nicht.

!$wdhl_adr_e <Anzahl> <Spuranfang> erzeugt den Befehl 644 (W-Transport ohne Löschen, mit Linksverschiebung) mit derselben Adressberechnung. Der zweite Operand ist der Anfang der Spur, die das Programm selbst bereitstellt (Handbuch, Festlegung 026); der Assembler setzt nirgends eine Nullspur voraus. Beispiel: !$wdhl_adr_e 3 004000 wird zu 644 04031.

Mehrere Zeilen und Labels: Jede !$wdhl- und !$wdhl_adr_e-Zeile eines Makros wird aufgelöst. Früher wurde nur die erste aufgelöst, und die weiteren blieben unverändert stehen. Ein Label auf der Zeile, auch ein generisches Label wie ML0001%, bleibt erhalten. Beispiel:

```
ZWEISCHIEB:  MACRO 4
ML0001%:     !$wdhl NE LI KON $1 $3   ; erster Wiederholungsbefehl
             240    $4                ; Zwischenergebnis sichern
             !$wdhl NE LI KON $2 $3   ; zweiter Wiederholungsbefehl
             MEND
```

Der Aufruf ZWEISCHIEB 3 2 02000 zw erzeugt 604 02031, 240 zw und 604 02006. Test Assembler/Claude/makrowdhl (Makros ZWEISCHIEB und ZWEIALT in macro_wdhl.macro; Sollwerte von Hand hergeleitet).

Fehler (Rückgabecode 16; die Meldung nennt Quelltextzeile und Makro): In der neuen Form sind Löschen, Resultat, Operation oder Anzahl außerhalb der oben genannten Werte, die Adresse keine Zahl oder die Operandenzahl falsch. Beispiele der Meldungen:

```
!$wdhl: Resultatsbehandlung "XX" ungültig (LI, RE, ZL oder OHNE)
!$wdhl: Anzahl "33" ungültig (1 bis 32)
!$wdhl erwartet fünf Operanden (Löschen Resultat Operation Anzahl Adresse), nicht 4
```

Negativtests makro_wdhl_ungueltig, makro_wdhl_zahl und makro_wdhl_anzahl.

4.17 Felder (`$DIM`, `$SET name(i)`, `$V(name(i))`)

Zweck: Ein Feld ist eine Reihe nummerierter Makro-Variablen. Das Element wird über einen Index gewählt, der zur Assemblerzeit berechnet wird, zum Beispiel mit dem Schleifenzähler $Z1 (4.3). Wer mehrere Werte zwischenspeichern und später gezielt, mehrfach oder in anderer Reihenfolge verwenden will, braucht damit keine eigene Variable je Wert. Ein Feld gibt es nur zur Assemblerzeit: Im erzeugten Quelltext (Listing, .dmp) steht nur der ausgewertete Wert, zum Beispiel dw 9; es entsteht keine Speicherzelle und keine Adressrechnung im C8205-Programm. Der Index muss zur Assemblerzeit feststehen. Es gibt keine mehrdimensionalen Felder und keine Felder von Feldern. Quelltexte ohne Felder liefern unverändert dieselben Ergebnisse.

Vereinbarung: $DIM <name> <anzahl> vereinbart ein Feld mit <anzahl> Elementen; alle Elemente haben anfangs den Zahlenwert 0. Die Anzahl ist ein Rechenausdruck ohne Leerzeichen (4.2, 4.8, auch mit $V(...) und als ganzer Wert $CALC(...)) und muss 1 bis 256 ergeben; höchstens 16 Felder sind möglich. Der Name folgt den Regeln der Variablennamen (Buchstabe zuerst, dann Buchstaben, Ziffern und _, höchstens 39 Zeichen); ein Name bezeichnet entweder eine einfache Variable (4.7) oder ein Feld, nie beides. Eine zweite Vereinbarung desselben Namens mit gleicher Anzahl setzt alle Elemente auf 0 zurück, mit anderer Anzahl ist sie ein Fehler. Mit `;#MAKROLOKAL=1` (4.31) legt `$DIM` in einem Makroaufruf ein lokales Feld an (höchstens 16 je Aufruf), es sei denn, der Name ist mit `$GLOBAL` vereinbart; `$GLOBAL name` vor `$DIM name n` macht das Feld global, und die Elemente eines nicht lokal vereinbarten Feldes bezeichnen das globale Feld. Das Feld muss vor der ersten Verwendung vereinbart sein, gerechnet in der Reihenfolge der Auflösung (4.15), nicht des Quelltextes.

Zugriff: Die Indizes zählen von 0 bis Anzahl−1, wie der Zähler $Z1. Schreiben: $SET name(index) ausdruck und $SET name(index) "text" (Text wie in 4.12, höchstens 200 Zeichen); $INC name(index) [ausdruck] erhöht ein

Zahlenelement wie $INC bei einer einfachen Variablen. Lesen: $V(name(index)) liefert den Wert (Zahl oder Text ohne Anführungszeichen), $V(#name(index)) die Zeichenzahl eines Textelements, $V(@name) die Anzahl der Elemente. Ein Element kann zwischen Zahl und Text wechseln. Bei $SET und $INC darf der Wert ganz aus $CALC(...) bestehen, auch mit $V(...) darin (also $SET a(0) $CALC($V(a(1))+1)).

Der Index ist ein Rechenausdruck ohne Leerzeichen (4.2, 4.8). Erlaubt sind Zahlen, $Z1 bis $Z4, $# und $P(..), $1 bis $9, $V(name) einer einfachen Variablen und $V(feld(index)), also auch ein Feldzugriff im Index, beliebig geschachtelt. Parameter werden im Index unverändert eingesetzt; steht in $1 ein Ausdruck mit Operator, setzt man ihn in Klammern, z. B. name(($1)*2). $V(name(index)) darf überall stehen, wo $V(name) erlaubt ist: im Operanden, in $CALC, in $IF und $ELIF und in der Anzahl von $REPT; für den Zeitpunkt gelten dieselben Regeln wie in 4.7 (in $IF und $ELIF der Stand an der Stelle der Bedingung, in $REPT der Stand zu Beginn des Makroaufrufs). In $P(..) bleibt $V weiterhin nicht erlaubt. Ein Textelement mit Leerzeichen lässt sich in $IF wie bisher nicht vergleichen (4.12).

Lebensdauer: Felder verhalten sich wie einfache Variablen (4.7): Ihre Werte bleiben über alle Makroaufrufe eines Laufs erhalten und gelten für alle Makros (aus Quelltext, c82ass.makros und Projektdateien). Mit dem Ende des Laufs sind sie verworfen; nichts davon steht in projekdetails, .log, .sym oder .test. --makros (4.11) und die Herkunftszeile (4.13) bleiben unverändert. Lokale Felder, die nur in einem Makroaufruf gelten: 4.20.

Fehler (Rückgabecode 16, Meldung mit Datei und Zeile wie die übrigen Makrofehler): Feld nicht vereinbart; Index kleiner als 0 oder nicht kleiner als die Anzahl (die Meldung nennt Feldname, Index und Anzahl); $DIM mit Anzahl unter 1, über 256 oder ohne Wert, mit anderer Anzahl als bei der früheren Vereinbarung, mit einem Namen, der schon eine einfache Variable ist (und umgekehrt), oder als 17. Feld; Klammer fehlt oder ist überzählig; leerer Index; Feldname ohne Index bei $SET, $INC und $V (außer $V(@name)); $INC auf einem Textelement; $V(#name(index)) auf einem Zahlenelement.

Beispiel: QUADRATE berechnet acht Quadrate und gibt sie in umgekehrter Reihenfolge aus; der Aufruf erzeugt 49, 36, 25, 16, 9, 4, 1, 0:

```
QUADRATE: MACRO
          $DIM q 8
          $REPT 8
          $SET q($Z1) $CALC($Z1*$Z1)
          $ENDREPT
          $REPT 8
          dw $V(q(7-$Z1))
          $ENDREPT
          MEND
```

Die Tests Assembler/Claude/makrofeld (das Beispiel), makrofeldtext (Textelemente, $V(#..), $V(@..)), makrofeldindex (Feldzugriff, $P, $# und Parameter im Index), makrofeldif, makrofeldrept und makrofeldglobal prüfen das (Sollwerte von Hand hergeleitet). Die Negativtests makro_feld_unbekannt, makro_feld_index_neg, makro_feld_index_gross, makro_feld_index_leer, makro_feld_dim_null, makro_feld_dim_gross, makro_feld_dim_ohne_wert, makro_feld_dim_anders, makro_feld_name_doppelt, makro_feld_klammer, makro_feld_ohne_index, makro_feld_set_ohne_index, makro_feld_inc_text, makro_feld_vlen_zahl und makro_feld_zuviele liegen im Regressionstest. Entschieden: Index ab 0; Elementzahl mit $V(@name); echte Felder statt zusammengesetzter Namen; Obergrenzen 256 Elemente und 16 Felder; Textvergleich in $IF unverändert.

4.18 Symbole in Rechenausdrücken (abs, equ)

Zweck: In einem Rechenausdruck des Makro-Präprozessors (4.2) dürfen Symbole stehen, die im Quelltext mit abs oder equ einen festen Zahlenwert haben. Ein Maß, eine Spurlage oder eine Schrittweite wird dann einmal mit abs vereinbart und im Makro und im Programm benutzt, ohne den Zahlenwert in jedem Aufruf zu wiederholen. Beispiel: breite: abs 12 und hoehe: abs 010 vereinbaren zwei Zahlen; im Makro erzeugt dw $CALC(breite*hoehe) das Wort 96, und der Aufruf MAL breite mit der Zeile dw $CALC($1*3) das Wort 36.

Wo Symbole gelten: in $CALC, $CALCO und $CALCOn, in einem $CALC innerhalb von $IF und $ELIF, in der Anzahl hinter $REPT, in $SET und $INC, in $P(...) und in den Grenzen und im Wert der Typprüfung $PRx $TYPZAHL (die dort bisher genannte Einschränkung „nur Zahlen, keine Symbole“ entfällt für abs- und equ-Symbole). Ein Aufrufparameter darf selbst ein solches Symbol oder ein Ausdruck mit Symbolen sein (MAL breite+1). Ein Vergleich in $IF ohne $CALC vergleicht weiter die Zeichen der Parameter; den Wert eines Symbols liefert dort $CALC(symbol).

Wertermittlung: Pass 0 läuft vor der Symboltabelle. Der Assembler sucht deshalb im Quelltext (ohne die Makrodefinitionen) eine Zeile <symbol>: abs <ausdruck> oder <symbol>: equ <ausdruck> und wertet den Operanden mit denselben Regeln aus wie in 4.2: Zahlen, weitere abs- und equ-Symbole und Operatoren (equ lage+040 mit lage: equ 07640 hat den Wert 07700). Die Definition darf hinter dem Aufruf stehen, auch hinter end. Bei equ ist der Wert die Zahl der Relativadresse, ohne Ladeadresse. Gibt es mehrere Definitionen eines Namens,

gilt die erste, deren Operand sich auswerten lässt. Verweise von einem Symbol auf ein anderes dürfen höchstens 16 Stufen tief sein.

Grenzen: Nicht auswertbar sind equ *, equ mit Labels sowie Labels von Befehlen und von db-, dw- und ähnlichen Anweisungen (ihr Wert hängt von der Zellenzahl ab, die Pass 0 noch nicht kennt). Ein equ im Programm setzt den Befehlszähler (3.3.8); für reine Zahlen ist abs vorzuziehen. Labels lassen sich nicht auswerten, fragen lassen sie sich aber mit def, art und zellen (Abschnitt 4.23): art(name) ergibt 1 oder 2 genau dann, wenn $CALC(name) gelingt. Auch die Zellenzahl eines mehrzeiligen Blocks (die Länge einer Tabelle aus mehreren Zeilen) lässt sich fragen: zellen(von,bis) (Abschnitt 4.23, ab c82ass 1.01-012). Adressen, Abstände von Labels im Allgemeinen und Sprungweiten bleiben in Pass 0 selbst nicht bestimmbar; ab c82ass 1.01-018 liefern sie die Funktionen `adr` und `abstand` (Abschnitt 4.23), die die Adressen aus Probeläufen holen.

Symbole aus Makros : Auch ein abs oder equ, das erst beim Auflösen eines Makros entsteht, ist für $CALC und die übrigen Stellen dieses Abschnitts sichtbar. Ein Makro kann damit eine berechnete Konstante (etwa breite2: abs $1 im Makro BREITE) als Symbol an ein anderes Makro weitergeben, ohne den Umweg über Variablen. Der Name der Marke steht im Rumpf; er darf aus $V(...) zusammengesetzt sein (q$V(i): abs $CALC($V(i)*$V(i)) in einer $WHILE-Schleife erzeugt q1, q2, q3), und der Operand darf Parameter, $V(...) und $CALC(...) enthalten. Eine Marke, die erst ein Aufrufparameter liefert ($1: abs 5), erkennt der Assembler ab c82ass 1.01-031: Er zerlegt die Zeile beim Aufruf in Marke, Operation, Operand und Kommentar und vermerkt das Symbol wie jedes andere abs oder equ aus einem Makro; der Aufruf m3 x1 eines Makros mit der Zeile $1: abs 5 erzeugt das Symbol x1 mit dem Wert 5, und x1 ist im Programm benutzbar (dw x1). Auch die Operation darf ein Parameter sein ($1: $2 5). Es muss eine Operation folgen: eine Marke allein ($1:) bleibt ein Fehler („$1: Ungültiges Makro-Schlüsselwort“). Bis c82ass 1.01-030 kannte der Assembler eine solche Marke nicht (Fehler „Operationscode 'y:' ist ungültig“ beim Aufruf mit dem Parameter y). Das Symbol gilt ab der Zeile, die es erzeugt, nicht früher: Ein $CALC im selben Makro sieht es nicht (die $CALC eines Rumpfs werden zu Beginn des Aufrufs ausgewertet), ebenso wenig ein Makro, das vor dem erzeugenden Makro aufgerufen wird. Ein später aufgelöstes Makro sieht es, auch ein verschachtelt aufgerufenes (das äußere Makro erzeugt das Symbol, das innere liest es). Der Quelltext hat Vorrang, und es gilt die erste Definition. Vermerkt wird ein Symbol nur, wenn sich sein Operand zu diesem Zeitpunkt als Zahl auswerten lässt (Zahlen, andere abs- und equ-Symbole, Operatoren); ein Operand mit einem Label (abs ziel+1), equ * und ein unbekanntes Symbol bleiben unsichtbar wie im Quelltext. Ein $CALC in einer Zeile des Quelltexts selbst (in einer $-Direktive oder in einer Anweisung, 4.1) wird vorab ausgewertet, bevor ein Makro aufgelöst ist, und sieht deshalb kein Symbol aus einem Makro; umgekehrt sieht ein Makro auch die Symbole, die ein $REPT im Quelltext erzeugt (sq$Z1: abs ...). Auch def und art (4.23) sehen das Symbol: def ergibt 1, art ergibt 1 für ein abs und 2 für ein equ; zellen meldet „Symbol "x": abs definiert einen Wert und belegt keine Zelle“. Die Zeile abs oder equ bleibt im erzeugten Quelltext und wird wie jede andere Zeile übersetzt (ein equ setzt den Befehlszähler, 3.3.8), das Symbol ist also auch im Programm benutzbar.

Fehler (Rückgabecode 16, die Meldung nennt die Quelltextzeile): „Symbol "x" ist im Quelltext weder mit abs noch mit equ <Zahl> definiert“ (es fehlt oder ist nur ein Label), „Symbol "x" hat keinen festen Zahlenwert (equ * ist eine Adresse, die erst nach Pass 0 feststeht)“ und „Symbol "x": Verweis zu tief oder zyklisch (mehr als 16 Stufen)“ (a: equ b und b: equ a). Liegt ein Symbol als Parameter von $TYPZAHL außerhalb des Bereichs, nennt die Meldung wie bisher den Wert, z. B. „Parameter $1: Wert "breite" (= 12) liegt nicht im Bereich 1 bis 10“.

Test Assembler/Claude/makrosymbol (abs vor start und hinter end, equ mit Verweis auf ein equ, Symbol als Aufrufparameter, in $CALC, $CALCO4, $IF und $REPT; Sollwerte von Hand hergeleitet) und die Negativtests makro_symbol_undef, makro_symbol_stern, makro_symbol_zyklus und makro_symbol_bereich. Symbole aus Makros: Test Assembler/Claude/makrosymbol2 (Quelltext ohne $-Zeile: Makros erzeugen ein abs, eine equ-Kette und q1 bis q3 in einer $WHILE-Schleife, ein anderes Makro liest sie in $CALC, $CALCO4, $REPT, art() und def(), auch aus einem verschachtelten Aufruf; 12 Zellen, Sollwerte von Hand hergeleitet) und Assembler/Claude/makrosymbol3 (Quelltext mit $-Zeilen: auch die Symbole, die ein $REPT im Quelltext erzeugt, in $CALC, $IF und $REPT; 5 Zellen) sowie die Negativtests makro_symbol_selbst (im selben Makro nicht sichtbar), makro_symbol_vorgriff (Aufruf vor dem erzeugenden Makro) und makro_symbol_label (Operand mit Label bleibt unsichtbar).

4.19 Schleife mit Abbruchbedingung und vorzeitiges Verlassen (`$WHILE`, `$ENDWHILE`, `$EXIT`)

Zweck: `$REPT` (4.3) wiederholt eine feste Anzahl, die zu Beginn des Makroaufrufs feststeht. Hängt die Zahl der Durchläufe von einem im Lauf berechneten Wert ab (eine Tabelle bis zu einer Obergrenze, die Suche nach einem Wert), schreibt man `$WHILE <bedingung>` ... `$ENDWHILE`. `$EXIT` verlässt das Makro vorzeitig. Beides entspricht dem, was beim IBM-Makroassembler `AIF`/`AGO` mit Sequence Symbols und `MEXIT` leisten.

Ablauf: Die Bedingung von `$WHILE` hat dieselbe Form wie die von `$IF` (4.4: bis zu 8 Bedingungsblöcke, Klammerwörter, `$V(name)`, `$CALC(...)`, `$1` bis `$9`). Ist sie wahr, wird der Rumpf bis zum zugehörigen `$ENDWHILE`

erzeugt. An `$ENDWHILE` wird die Bedingung mit dem jetzigen Stand der Variablen erneut ausgewertet: Ist sie noch wahr, beginnt der nächste Durchlauf, sonst geht es hinter `$ENDWHILE` weiter. Ist sie schon beim ersten Mal falsch, entfällt der Rumpf. Der Rumpf muss deshalb eine Variable ändern (`$INC` oder `$SET`), sonst endet die Schleife nie. `$WHILE` und `$ENDWHILE` stehen je auf einer Zeile für sich (ein Kommentar mit ; dahinter ist erlaubt). Der Rumpf besteht aus beliebigen Zeilen: `$IF`-Strukturen, weitere `$WHILE`-Schleifen, `$REPT`, `$SET`, `$INC`, `$DIM` und Aufrufe anderer Makros (der Aufruf geschieht in jedem Durchlauf neu).

Reihenfolge beim Makroaufruf: `$REPT` wird zuerst als Text aufgelöst (4.3). `$WHILE` läuft erst in der Auswertung der Bedingungen, zusammen mit `$IF`, `$SET` und `$INC` (4.7), in der Reihenfolge der Zeilen. Darum sieht `$WHILE` den laufenden Stand der Variablen, die Anzahl hinter `$REPT` dagegen nur den Stand zu Beginn des Aufrufs. Deshalb ist für eine Schleife, deren Ende erst im Lauf feststeht, `$WHILE` nötig.

`$EXIT`: steht für sich auf einer Zeile und wirkt nur im erzeugten Zweig. Es beendet das Makro sofort: Die Zeilen bis dahin bleiben, alle weiteren Zeilen (auch innerhalb offener `$IF` und `$WHILE`) erzeugen nichts mehr und führen nichts mehr aus (`$SET`, `$INC`, `$ERROR`). Die Klammerung von `$IF`/`$ENDIF` und `$WHILE`/`$ENDWHILE` wird auch dahinter noch geprüft. Es endet nur das Makro, in dem `$EXIT` steht; ein aufrufendes Makro läuft weiter.

Grenzen: Je Eintritt in eine `$WHILE`-Schleife höchstens 4096 Durchläufe (mehr gilt als Endlosschleife und ist ein Fehler); je Makroaufruf höchstens so viele erzeugte Zeilen, wie der Assembler überhaupt zulässt (6144). `$IF` und `$WHILE` zusammen dürfen bis zur Tiefe 16 geschachtelt werden.

Marken: Eine normale Marke im Rumpf muss in jedem Durchlauf anders heißen, z. B. `w$V(i):` (ergibt `w0`, `w1`, …). Ein lokales Label `<name>%` (4.3) darf im Rumpf einer `$WHILE`-Schleife definiert werden : Es erhält in jedem Durchlauf den Zusatz `_<Durchlauf, ab 0>` (`schleife%_0`, `schleife%_1`, …), und der Assembler ersetzt wie gewohnt das `%` durch drei Buchstaben; ebenso wird jede Verwendung im Rumpf angepasst, auch ein Label, das als Parameter an ein im Rumpf aufgerufenes Makro übergeben wird. Bei geschachtelten Schleifen steht je umschließender Schleife ein Zusatz, die äußere zuerst (`innen%_0_1`); steht der Rumpf außerdem in einem `$REPT`, steht dessen Zusatz davor. Das Label gilt nur im Rumpf der Schleife, die es definiert: Ein Sprung auf ein solches Label hinter der Schleife endet mit „Adresse … ist ungültig“, wie bei einem nie definierten lokalen Label. Ein Label außerhalb des Rumpfes, auf das der Rumpf springt, ist erlaubt. Im normalen Quelltext außerhalb eines Makros (4.1) bleibt ein lokales Label im Rumpf von `$WHILE` ein Fehler, weil es lokale Labels nur im Makro gibt; dort `$V(...)` im Namen benutzen.

Fehler (Rückgabecode 16, die Meldung nennt das Makro und die Quelltextzeile des Aufrufs): `$WHILE` ohne `$ENDWHILE` und umgekehrt, `$ENDWHILE` bei noch offenem `$IF`, `$ENDIF` bei noch offenem `$WHILE`, `$ELIF` oder `$ELSE` unmittelbar in einem `$WHILE` ohne eigenes `$IF`, eine Bedingung, die sich nicht auswerten lässt, mehr als 4096 Durchläufe, mehr als 6144 erzeugte Zeilen, mehr als 16 geschachtelte `$IF`/`$WHILE`, im normalen Quelltext ein lokales Label mit `%` im Rumpf von `$WHILE` („Im Rumpf von $WHILE darf kein lokales Label mit % definiert werden“), ein Sprung auf ein Label des Rumpfes hinter der Schleife.

Beispiel: `QUADRATE` erzeugt die Quadrate 1, 4, 9, … solange sie höchstens so groß sind wie der Parameter. `QUADRATE 30` erzeugt 1, 4, 9, 16, 25; die Zahl der Durchläufe steht erst im Lauf fest:

```
QUADRATE:  MACRO 1
           $SET i 1
           $WHILE $CALC($V(i)*$V(i)) <= $1
           dw $CALC($V(i)*$V(i))     ; Quadrat des Zählers i
           $INC i
           $ENDWHILE
           MEND
```

Beispiel für `$EXIT`: `FRUEH 0` erzeugt nur das Wort 1, `FRUEH 1` erzeugt die Wörter 1 und 2:

```
FRUEH:     MACRO 1
           dw 1                      ; immer
           $IF $1 = 0
           $EXIT
           $ENDIF
           dw 2                      ; nur bei $1 ungleich 0
           MEND
```

Mit `;#MAKROLOKAL=1` (4.31): Eine im Rumpf mit `$SET` angelegte lokale Variable bleibt über die Durchläufe desselben Aufrufs erhalten (anders als `$LOCAL` im Rumpf, das in jedem Durchlauf neu vereinbart und die Werte wieder auf 0 setzt; `$LOCAL` ändert sich nicht). Das Beispiel `QUADRATE` zählt dann ohne `$LOCAL` lokal.

Test Assembler/Claude/makrowhile (Quadrate bis zu einer Grenze, Suche mit `$EXIT` mitten in der Schleife, zwei geschachtelte Schleifen, Schleife ohne Durchlauf, `$EXIT` in einem `$IF`; Sollwerte von Hand hergeleitet) und die Negativtests makro_while_ohne_endwhile, makro_endwhile_ohne_while, makro_while_endlos, makro_while_label,

makro_while_if_offen, makro_while_zeilen und makro_while_bedingung. Lokale Labels im Rumpf: Test Assembler/Claude/makrowhilelabel (acht Makros: Label und Sprung im Rumpf, Sprung vorwärts, zwei geschachtelte Schleifen, `$REPT` im Rumpf, Label als Parameter an ein verschachteltes Makro, lokale Variable und Felder im Rumpf; Sollwerte von Hand hergeleitet) und die Negativtests makro_while_label (Sprung auf ein Rumpf-Label hinter der Schleife), makro_while_label_ohne_endwhile (fehlendes `$ENDWHILE`, genau eine Meldung) und quelltext_while_label (Label mit `%` im `$WHILE`-Rumpf des normalen Quelltexts).

4.20 Lokale Variablen und Felder (`$LOCAL`)

Zweck: Makro-Variablen und Felder (4.7, 4.17) sind global: Ein Name gilt in allen Makros eines Laufs, der Wert bleibt über alle Aufrufe erhalten. Für Hilfsgrößen ist das gefährlich: Ein Makro, das in der Schleife eines anderen Makros aufgerufen wird und denselben Zähler benutzt, zerstört dessen Wert. `$LOCAL` vereinbart Variablen und Felder, die nur in dem Makroaufruf gelten, der sie vereinbart, und mit dessen Ende verschwinden. Das entspricht `LCLA`/`LCLC` beim IBM-Makroassembler (global ist, was nicht lokal vereinbart ist). Mit der Quelltext-Anweisung `;#MAKROLOKAL=1` und der Direktive `$GLOBAL` gilt wahlweise das Gegenteil (lokal ist, was nicht mit `$GLOBAL` vereinbart ist).

Schreibweise: `$LOCAL <name> ...` vereinbart einen oder mehrere Namen, durch Leerzeichen getrennt. `<name>(<anzahl>)` vereinbart ein lokales Feld mit so vielen Elementen (Rechenausdruck wie bei `$DIM`, 4.17, erlaubt sind 1 bis 256; `$1` bis `$9` und `$V(...)` sind möglich). Alle lokalen Variablen und Feldelemente haben nach der Vereinbarung den Wert 0. Die Zeile steht für sich (ein Kommentar mit ; dahinter ist erlaubt), erzeugt keinen Code und wirkt nur im erzeugten Zweig (in einem nicht erzeugten `$IF`-Zweig nicht). Sie darf an jeder Stelle im Rumpf stehen, auch in einem `$IF`; der Bereich beginnt dort und endet mit dem Makroaufruf (`MEND` oder `$EXIT`).

Sichtbarkeit: Ab `$LOCAL` bezeichnet der Name im laufenden Aufruf die lokale Variable oder das lokale Feld, für alle Verwendungen (`$SET`, `$INC`, `$V(name)`, `$V(#name)`, `$V(name(index))`, `$V(@name)`, `$CALC`, `$IF`, `$WHILE`). Sie verdeckt eine globale Variable oder ein globales Feld gleichen Namens; die globale bleibt unberührt. Vor `$LOCAL` bezeichnet der Name die globale. Ein Makro, das aus diesem Aufruf heraus aufgerufen wird, sieht die lokalen Variablen des aufrufenden Makros nicht (nur die globalen und seine eigenen); nach der Rückkehr gelten wieder die lokalen des aufrufenden Makros mit unverändertem Stand. Werte gehen über Parameter (`$1` bis `$9`) hinein und über globale Variablen heraus. Jeder Aufruf beginnt ohne lokale Variablen; mit seinem Ende (auch bei einem Fehler oder `$EXIT`) sind sie verworfen.

Grenzen: Je Makroaufruf höchstens 64 lokale einfache Variablen und 16 lokale Felder (je Feld bis 256 Elemente; bis c82ass 1.01-031 waren es 16 und 4). Die Grenzen der globalen Variablen (64) und Felder (16) bleiben davon unabhängig. Mit `;#MAKROLOKAL=1` zählen auch die mit `$SET` und `$DIM` angelegten lokalen mit (4.31).

Besonderheiten: `$REPT` wird vor der Auswertung des Makros als Text aufgelöst (4.3); eine lokale Variable in der Anzahl von `$REPT` gibt es dann noch nicht (sie ist „nicht definiert“, und eine gleichnamige globale Variable würde stattdessen gelten). Für Schleifen mit lokalem Zähler gilt `$WHILE` (4.19). `$LOCAL` im Rumpf einer `$WHILE`-Schleife : Dieselbe Zeile vereinbart ihre Namen in jedem Durchlauf neu, die Werte sind wieder 0; ein Feld mit anderer Anzahl wird neu angelegt, bei gleicher Anzahl werden die Elemente auf 0 gesetzt. Derselbe Name in einer anderen Zeile oder zweimal in derselben Zeile bleibt ein Fehler. Im Quelltext außerhalb eines Makros ist `$LOCAL` ein Fehler („$LOCAL steht außerhalb eines Makroaufrufs“, Rückgabecode 16); `$SET` und `$DIM` gelten dort ab c82ass 1.01-008 (4.1).

Fehler (Rückgabecode 16, die Meldung nennt das Makro und die Quelltextzeile des Aufrufs): `$LOCAL` ohne Namen, ein ungültiger Name (Regeln wie in 4.7), ein Name, der im selben Aufruf schon lokal vereinbart ist (als Variable oder Feld), eine ungültige Feldanzahl (kleiner 1, größer 256, ohne Wert) oder eine fehlende Klammer, mehr als 64 lokale Variablen oder mehr als 16 lokale Felder in einem Aufruf (die Meldung nennt, dass auch `$SET` und `$DIM` mitzählen), ein Name, der im selben Aufruf schon mit `$GLOBAL` vereinbart ist (4.31). Liest ein verschachtelt aufgerufenes Makro eine lokale Variable des aufrufenden Makros, so ist sie dort „nicht definiert“ wie jede nicht gesetzte Variable.

Beispiel: SUMME bildet die Summe 1 bis $1 in den lokalen Variablen i und s (s beginnt mit 0) und erzeugt sie als Wort. TABELLE ruft SUMME in seiner eigenen Schleife über das lokale i auf und behält dabei seinen Zähler. TABELLE erzeugt die Wörter 1, 3, 6, 10. Mit globalem i ($SET statt $LOCAL) zerstört SUMME den Zähler von TABELLE, und es entstünden nur die Wörter 1 und 6:

```
SUMME:     MACRO 1
           $LOCAL i s
           $SET i 1
           $WHILE $V(i) <= $1
           $INC s $V(i)
           $INC i
           $ENDWHILE
           dw $V(s)                  ; Summe 1 bis $1
           MEND
TABELLE:   MACRO
           $LOCAL i
           $SET i 1
           $WHILE $V(i) <= 4
           SUMME $V(i)               ; Summe 1 bis i
           $INC i
           $ENDWHILE
           MEND
```

Test Assembler/Claude/makrolokal (das Beispiel, Überdeckung einer globalen Variablen, Neubeginn mit 0 bei jedem Aufruf, $EXIT mit gleichnamiger globaler Variablen, $LOCAL in einem $IF-Zweig, globale Werte nach allen Aufrufen unverändert), Test Assembler/Claude/makrolokalfeld (lokales Feld mit der Anzahl als Parameter, zwei Aufrufe mit verschiedener Größe, globales Feld gleichen Namens bleibt unverändert; Sollwerte von Hand hergeleitet) und die Negativtests makro_local_doppelt, makro_local_ohne_name, makro_local_name_ungueltig, makro_local_feld_anzahl, makro_local_feld_klammer, makro_local_zuviele, makro_local_felder_zuviele, makro_local_rept, makro_local_nicht_sichtbar, makro_local_while (derselbe Name in zwei Zeilen im Rumpf einer $WHILE-Schleife) und makro_local_while_doppelt (zweimal in einer Zeile); das Vereinbaren im Rumpf selbst deckt Test Assembler/Claude/makrowhilelabel ab (Neubeginn mit 0, Feld mit je Durchlauf anderer Anzahl). Entschieden: Schreibweise $LOCAL; mehrere Namen auf einer Zeile; eine lokale Variable darf eine globale überdecken; Anfangswert 0; Grenzen 16 und 4 je Aufruf (seit c82ass 1.02-001: 64 und 16); $REPT bleibt unverändert.

4.21 Zeichenkettenfunktionen ($SUBSTR, $INDEX, $UPPER, $LOWER, $STRCMP, $ORD, $CHR, $VAL)

Zweck: Eine Textvariable (4.12) hielt bisher nur einen Text. Zerlegen, durchsuchen, zusammensetzen oder vergleichen ließ er sich nicht: Ein Parameter name=wert war nicht in Namen und Wert zu trennen, und zwei Texte mit Leerzeichen waren nicht zu vergleichen, weil $IF Wörter vergleicht (4.10). Die Zeichenkettenfunktionen schließen diese Lücke. Sie wirken nur zur Assemblerzeit; im erzeugten Quelltext steht allein das Ergebnis. Das entspricht SUBSTR, INDEX, UPPER/LOWER, der Verkettung und den Zeichen-Zahl-Umwandlungen beim IBM-Makroassembler und ist die vierte Lücke gegenüber ihm (nach Schleife und MEXIT, 4.19, und lokalen Variablen, 4.20).

Form und Ziel: Die Funktionen sind Direktiven wie $SET und $INC. Die Zeile steht für sich (ein Kommentar mit ; dahinter ist erlaubt), erzeugt keinen Code, wirkt nur im erzeugten Zweig (in einem nicht erzeugten $IF-Zweig wird sie nicht ausgeführt) und wird in der Reihenfolge der erzeugten Zeilen ausgeführt. Das Ziel steht an erster Stelle: eine Variable oder ein Feldelement name(index) wie bei $SET (4.7, 4.17), auch eine lokale Variable oder ein lokales Feldelement (4.20). Eine nicht vorhandene Variable wird angelegt. Ein Textergebnis macht das Ziel zur Textvariablen, ein Zahlenergebnis zur Zahlenvariablen; ein früherer Wert der anderen Art ist dann verworfen: Nach einem Zahlenergebnis ist $V(#name) ein Fehler (4.12), nach einem Textergebnis $INC.

Argumente: Textargumente stehen in Anführungszeichen ("..." oder '...', wie bei $SET, 4.12). Zahlenargumente sind Rechenausdrücke ohne Leerzeichen wie bei $SET (4.7, 4.8); sie dürfen $1 bis $9, $V(...) und $CALC(...) enthalten. Ein Text ohne Anführungszeichen ist nicht erlaubt, weil er sich von einem Variablennamen oder einer Zahl nicht trennen ließe; ebenso ist ein Variablenname ohne $V(...) als Quelle nicht möglich. Der Assembler zerlegt die Zeile zuerst in ihre Argumente und ersetzt danach in jedem Argument $1 bis $9 und $V(...) durch den Stand an dieser Stelle. Ein aufgelöster Text darf deshalb Leerzeichen enthalten, ohne die Zerlegung zu stören. Enthält der aufgelöste Text aber das Anführungszeichen, mit dem er selbst begrenzt ist, meldet der Assembler einen Fehler mit dem Hinweis, das andere Anführungszeichen zu nehmen. Seit c82ass 1.00-070 werden in jedem Textargument nach $V(...) auch $CALC(...), $CALCO(...) und $CALCOn(...) ausgewertet (4.22).

Grenzen: Texte sind Bytefolgen. Ein UTF-8-Mehrbytezeichen zählt byteweise (wie bei $V(#name), 4.12). Ein Ergebnis hat höchstens 200 Zeichen. Eine Zeile darf nach dem Einsetzen der Parameter und Variablen höchstens 255 Zeichen haben (sonst „$V(...): Text passt nicht in die Zeile“). Ein Text enthält kein Semikolon (4.12): Enthält ein

Textargument nach dem Auflösen von `$1` bis `$9` und `$V(...)` ein Semikolon, meldet der Assembler einen Fehler, auch beim Suchtext von `$INDEX`. Ein Ergebnis enthält deshalb nie ein Semikolon; `$CHR` schließt den Code 59 aus.

`$SUBSTR <ziel> "<text>" <start> [<anzahl>]` bildet einen Teilstring (Ergebnis Text). `start` zählt ab 1; ohne `anzahl` gilt der Rest des Textes ab `start`. `$SUBSTR z "BANANE" 2 3` ergibt `ANA`, `$SUBSTR z "BANANE" 5` ergibt `NE`.

`$INDEX <ziel> "<text>" "<suchtext>"` liefert die Position (ab 1) des ersten Vorkommens von `suchtext` in `text`, 0 wenn es nicht vorkommt (Ergebnis Zahl). Groß- und Kleinschreibung werden unterschieden; wer das nicht will, wandelt vorher mit `$UPPER` oder `$LOWER`. `$INDEX p "abc=42" "="` ergibt 4, `$INDEX p "abc" "x"` ergibt 0.

`$UPPER <ziel> "<text>"` und `$LOWER <ziel> "<text>"` wandeln die Buchstaben `a` bis `z` in Großbuchstaben bzw. `A` bis `Z` in Kleinbuchstaben (Ergebnis Text). Alle anderen Bytes bleiben unverändert, auch Umlaute.

Verkettung hat keine eigene Direktive: `$SET <name> "<text>"` löst `$V(...)` im Text sofort beim Setzen auf. Mit `a` = `ab` und `b` = `cd` setzt `$SET c "$V(a)$V(b)-$1"` die Variable `c` auf `abcd-` gefolgt vom Text des Parameters 1, und `$V(#c)` liefert dessen wahre Länge. Eine spätere Änderung von `a` oder `b` ändert `c` nicht mehr. Ein Selbstverweis wie `$SET a "$V(a)"` setzt `a` auf den eigenen bisherigen Wert; ist `a` nicht gesetzt, meldet der Assembler „nicht definiert“ (in c82ass 1.00-067 lief er bei einem solchen Selbstverweis endlos).

`$STRCMP <ziel> "<a>" "<b>"` vergleicht die beiden Texte byteweise ohne Rücksicht auf Wortgrenzen. Das Ergebnis (Zahl) ist -1, wenn a kleiner ist, 0 bei Gleichheit und 1, wenn a größer ist. So lässt sich ein Text mit Leerzeichen vergleichen: `$STRCMP r "$V(x)" "ein text"` und danach `$IF $V(r) = 0`.

`$ORD <ziel> "<text>" [<pos>]` liefert den Wert des Bytes an der Position `pos` (ab 1, ohne Angabe 1) als Zahl. Für ASCII-Zeichen ist das der ASCII-Code (`$ORD n "A"` ergibt 65). Ein UTF-8-Mehrbytezeichen liefert den Wert seines Bytes: Bei `ä` (zwei Bytes) ergibt Position 1 den Wert 195.

`$CHR <ziel> <code>` erzeugt ein Zeichen aus dem ASCII-Code `code` (32 bis 126, Zahlenargument); das Ergebnis ist ein Text mit einem Zeichen. Das Semikolon (59) ist ausgeschlossen. `$CHR z 65` ergibt `A`.

`$VAL <ziel> "<text>"` wandelt den Text in eine dezimale ganze Zahl (Ergebnis Zahl). Erlaubt sind nur Ziffern, davor ein einzelnes `-`. Keine Zahl sind `12a`, `-`, der leere Text, `+5` und ein Text mit führendem Leerzeichen. Die Umkehrung (Zahl nach Text) geht über `$SET t "$V(n)"`; eine führende 0 liest `$VAL` dezimal. Oktal liest `$VALO`, und oktal schreibt `$SET t "$CALCO($V(n))"` (4.22). Binär liest `$VALB` und schreibt `$CALCB` (4.27). Hexadezimal liest `$VALH` und schreibt `$CALCH` (4.28).

Fehler (Rückgabecode 16, die Meldung nennt das Makro und die Quelltextzeile des Aufrufs): eine falsche Argumentzahl, ein Text ohne Anführungszeichen, ein fehlendes schließendes Anführungszeichen, Text hinter dem letzten Argument, der kein Kommentar ist, ein ungültiges Ziel, ein Ziel, das ein Feld ohne Index ist, ein Feldindex außerhalb des Feldes; bei `$SUBSTR` ein `start` kleiner 1 oder hinter dem Textende, eine `anzahl` kleiner 1, ein Ende (`start+anzahl-1`) hinter dem Textende und ein leerer Quelltext (ein leerer Text lässt sich nicht erzeugen); bei `$INDEX` ein leerer Suchtext; bei `$ORD` eine Position außerhalb des Textes; bei `$CHR` ein Code kleiner 32, größer 126 oder 59; bei `$VAL` ein Text, der keine ganze Zahl ist, und eine Zahl außerhalb des Zahlenbereichs (wie bei `$SET`); ferner ein Ergebnis über 200 Zeichen, eine Zeile über 255 Zeichen nach dem Auflösen, ein aufgelöster Text mit dem eigenen Begrenzungszeichen, ein aufgelöstes Textargument mit Semikolon (ab 1.00-069; die Meldung nennt die Nummer des Arguments, z. B. „M1 $UPPER z: Argument 1: Der Text enthält ein Semikolon, ein Text enthält kein Semikolon“) und `$V(name)` einer nicht gesetzten Variablen („nicht definiert“).

Sonstiges: Die Zeilen der Direktiven fehlen im erzeugten Quelltext (auch in der `.mqo`-Datei); der Kommentar der erzeugten Zeilen bleibt. Ab c82ass 1.01-008 wirken sie auch im Quelltext außerhalb eines Makros (4.1). `--makros` (4.11) und die Herkunftszeile (4.13) bleiben unverändert, ebenso der Usage-Text (`-h`).

Beispiel 1: `SPLIT` trennt einen Parameter `schlüssel=wert` und erzeugt die Länge des Schlüssels und den Wert als Zahl. `SPLIT abc=42` erzeugt die Wörter 3 und 42 (das `=` steht an Position 4, der Schlüssel `abc` hat 3 Zeichen, der Rest ab Position 5 ist `42`). Ohne Gleichheitszeichen ist `p` gleich 0, und `$SUBSTR` meldet für die Anzahl -1 einen Fehler:

```
SPLIT:     MACRO 1
           $INDEX p "$1" "="                ; Position des Gleichheitszeichens
           $SUBSTR k "$1" 1 $CALC($V(p)-1)  ; Schlüssel
           $SUBSTR w "$1" $CALC($V(p)+1)    ; Wert, bis zum Ende
           dw $V(#k)                        ; Länge des Schlüssels
           dw $V(w)                         ; Wert als Zahl
           MEND
```

Beispiel 2: `ANZAHL` zählt mit `$WHILE` und `$LOCAL` (4.19, 4.20) die Vorkommen des Buchstabens `A` im Parameter, ohne Rücksicht auf Groß- und Kleinschreibung. `ANZAHL "Banane"` erzeugt das Wort 2 (in BANANE steht A an den Positionen 2 und 4; mit `"Ananas"` wären es 3):

```
ANZAHL:    MACRO 1
           $LOCAL i n t z
           $UPPER t "$1"                    ; t = BANANE
           $SET i 1
           $WHILE $V(i) <= $V(#t)
           $SUBSTR z "$V(t)" $V(i) 1        ; ein Zeichen
           $IF $V(z) = A
           $INC n
           $ENDIF
           $INC i
           $ENDWHILE
           dw $V(n)
           MEND
```

Stand der Prüfung: Beide Beispiele, alle acht Funktionen und die Fehlerfälle sind mit c82ass 1.00-069 geprüft (Sollwerte von Hand hergeleitet). Im Regressionstest stehen vier Programme (`Claude/makrostrfkt`, `makrostrsplit`, `makrostrzeichen` und `makrostrcode`) und fünfunddreißig Negativtests `makro_str_*`, für jeden Fehlerfall dieses Abschnitts einer (seit 1.00-069 auch `makro_str_set_semikolon`, `makro_str_upper_semikolon` und `makro_str_index_semikolon`); der Regressionstest einschließlich AddressSanitizer und UBSan läuft ohne Abweichung (236 Programme, 201 Negativtests). Entschieden: Direktiven mit Texten in Anführungszeichen; Verkettung durch sofortiges Auflösen im `$SET`; Kern und Erweiterung zusammen; strenge Bereichsprüfung; ASCII für `$ORD` und `$CHR`; Groß- und Kleinschreibung werden unterschieden; Ziel ist jede Variable und jedes Feldelement.

4.22 Zahl und Text, dezimal und oktal (`$VALO`, `$CALC` im Text)

Zweck: Der C 8205 wird oktal geschrieben: Adressen, Befehlscodes und Wörter stehen im Quelltext als `dw 017`, `0001`, `211`. Der Makro-Präprozessor gibt Zahlen oktal aus (`$CALCO(...)` und `$CALCOn(...)`, 4.2 und 4.8) und liest eine Zahl mit führender 0 oktal. Die Umwandlung zwischen einer Zahl und ihrem Text lief aber über drei Wege, die dieselbe Ziffernfolge verschieden lesen, und ein Oktaltext ließ sich aus einer Variablen nicht erzeugen: `$SET b "$CALCO($V(n))"` setzte den Text wörtlich als `$CALCO(15)`, weil `$V(...)` erst beim Ausführen des `$SET` aufgelöst wurde, nach der Auswertung der Rechenausdrücke. Eine Marke mit der laufenden Nummer in Oktalschreibung (`z00144`) ließ sich aus einem Zähler deshalb nicht bilden, und ein Parameter `"17 21"` aus zwei Oktalzahlen ließ sich nicht zerlegen und lesen (`$VAL` liest `17`, nicht 15). Seit c82ass 1.00-070 gibt es zwei Ergänzungen: Rechenausdrücke im Text von `$SET` und in den Textargumenten der Zeichenkettenfunktionen, und die Direktive `$VALO`, die einen Text oktal liest. Alles wirkt nur zur Assemblerzeit; im erzeugten Quelltext steht allein das Ergebnis. Was bisher ging, bleibt bitgleich.

Die Zahlenwege: Dieselbe Ziffernfolge `017` (Text einer Variablen `c`) wird an drei Stellen verschieden gelesen; `$VALO` kommt als vierter Weg hinzu.

Stelle	liest `017` als	Folge
`$CALC(017)`, Ausdruck von `$SET` und `$INC` (auch `$SET m $V(c)`), Operand (`dw $V(c)`, `dw 017`)	oktal, 15	`$SET m $V(c)` ergibt 15; `dw $V(c)` erzeugt 017
`$IF` und `$ELIF` (Zahlenvergleich, 4.4)	dezimal, 17	`$IF $V(c) = 17` ist wahr, `$IF $V(c) = 15` unwahr
`$VAL v "017"` (4.21)	dezimal, 17	`dw $V(v)` erzeugt 17
`$VALO v "017"` (dieser Abschnitt)	oktal, 15	`dw $V(v)` erzeugt 15

Eine Ziffernfolge ohne führende 0 (`17`) ist in den ersten drei Wegen überall dezimal; `$VALO` liest sie oktal. Ein Text mit der Ziffer 8 oder 9 und führender 0 ist beim oktalen Lesen im Ausdruck ein Fehler („ungültige Oktalzahl“). Die größte oktal lesbare Zahl im Ausdruck ist `0777777777777777777777` (2^63 - 1); `0100000000000000000000` meldet „Zahl zu groß“. `$VAL` liest dezimal bis 9223372036854775807. `$VALB` (4.27) liest dieselbe Ziffernfolge dual; `017` ist dort wegen der Ziffer 7 ein Fehler. `$VALH` (4.28) liest sie hexadezimal; `017` ergibt dort 23. An `$VAL`, `$IF`, `$ELIF`, `$CALC`, `$CALCO`, `$CALCOn` und am Operanden ändert sich nichts: Der Bestandstest `makrostrzeichen` verlangt `$VAL "-0012"` = -12.

Rechenausdrücke im Text: Im Text von `$SET name "text"` werden nach dem Einsetzen von `$1` bis `$9` und dem Auflösen von `$V(...)` die Ausdrücke `$CALC(...)`, `$CALCO(...)` und `$CALCOn(...)` ausgewertet und durch das Ergebnis ersetzt (Regeln und Fehler wie in 4.2 und 4.8). Der Text der Variablen enthält danach das Ergebnis; erst danach gelten die Prüfungen auf 200 Zeichen und auf ein Semikolon (4.12). Das gilt auch für ein Ziel als Feldelement und

als lokale Variable (4.17, 4.20). Mit `n` = 15 setzt `$SET t "$CALCO($V(n))"` den Text `017`, und `$V(#t)` liefert 3; `$SET nm "x$CALCO4($V(n))"` setzt `x00017`.

Jedes Textargument der Zeichenkettenfunktionen (`$SUBSTR`, `$INDEX`, `$UPPER`, `$LOWER`, `$STRCMP`, `$ORD`, `$VAL` und `$VALO`, 4.21) wird ebenso behandelt: erst `$1` bis `$9` und `$V(...)`, dann `$CALC...`, dann die Prüfungen auf das Anführungszeichen und das Semikolon. So ergibt `$STRCMP r "$CALCO($V(n))" "017"` mit `n` = 15 den Wert 0. Die Zahlenargumente werteten `$CALC` mit `$V(...)` schon vorher aus.

Die Auflösung geschieht genau einmal: Ein Ergebnis (nur Ziffern und Vorzeichen) wird nicht erneut aufgelöst, und ein Text ohne `$CALC` bleibt unverändert. Zwei Folgen: Ein wörtliches `$CALC(` lässt sich im Text nicht mehr schreiben, und ein Aufrufparameter mit Ausdruck wird im Text ausgewertet: `M1 $CALC(2+5)` ergibt in `$SET t "$1"` den Text `7` (früher `$CALC(2+5)`, zehn Zeichen). Im Bestand an Tests und Programmen steht kein `$CALC` innerhalb von Anführungszeichen; es ist kein vorhandener Test und kein vorhandenes Programm betroffen.

`$VALO <ziel> "<text>"` wandelt den Text in eine **oktale** ganze Zahl (Ergebnis Zahl). Form, Ziel und Argument sind wie bei `$VAL` (4.21): Die Zeile steht für sich (ein Kommentar mit ; dahinter ist erlaubt), erzeugt keinen Code und wirkt nur im erzeugten Zweig; das Ziel an erster Stelle ist eine Variable oder ein Feldelement `name(index)`, auch lokal; das Textargument steht in Anführungszeichen, und `$1` bis `$9`, `$V(...)` und `$CALC...` werden vorher aufgelöst. Erlaubt sind die Ziffern 0 bis 7, davor ein einzelnes `-`. Führende Nullen sind erlaubt, aber nicht nötig: `17`, `017` und `00017` ergeben 15, `-17` ergibt -15, `0` ergibt 0. Keine Zahl sind `18` (Ziffer 8), `12a`, `-`, der leere Text, `+5` und ein Text mit Leerzeichen. Der Bereich ist der des oktalen Lesens im Ausdruck: bis 2^63 - 1, das sind 21 Siebenen (`7777777777777777777`); führende Nullen davor sind beliebig viele erlaubt. Darüber meldet der Assembler „Die Zahl ... liegt außerhalb des Zahlenbereichs“, wie bei `$VAL`.

Rundreise: `$SET t "$CALCO($V(n))"` und `$VALO r "$V(t)"` ergeben `r` = `n` für jedes `n` von 0 bis 2^63 - 1, auch mit fester Breite (`$CALCOn`). `$VAL` liest denselben Text dezimal; aus 15 wird über `017` die Zahl 17. Die Zeile von `$VALO` fehlt wie die von `$SET` im erzeugten Quelltext (auch in der `.mqo`-Datei); `--makros` (4.11), die Herkunftszeile (4.13) und der Usage-Text (`-h`) bleiben unverändert.

Fehler (Rückgabecode 16, die Meldung nennt das Makro und die Quelltextzeile des Aufrufs): bei `$VALO` eine falsche Argumentzahl (genau das Ziel und ein Text), ein Text ohne Anführungszeichen, ein fehlendes schließendes Anführungszeichen, Text hinter dem Argument, der kein Kommentar ist, ein ungültiges Ziel, ein Ziel, das ein Feld ohne Index ist, ein Text, der keine oktale ganze Zahl ist (Ziffer 8 oder 9, Buchstabe, leer, nur `-`, `+`, Leerzeichen), eine Zahl außerhalb des Bereichs und ein Semikolon im Text. Die Meldung zu einem Text lautet z. B. „M1 $VALO n: Der Text "18" ist keine oktale ganze Zahl (erlaubt sind die Ziffern 0 bis 7, davor ein einzelnes -)“.

Ein Fehler im Rechenausdruck im Text von `$SET` oder in einem Textargument (Division durch 0, negatives Ergebnis bei `$CALCO`, mehr Ziffern als die Breite von `$CALCOn`, nichtnumerisches Zeichen, fehlende Klammer) ist ein Fehler des `$SET` bzw. der Funktion und wird beim `$SET` gemeldet, nicht erst bei der Verwendung der Variablen. Die Meldung nennt das Makro, die Direktive, das Ziel und den Grund, z. B. „M1 $SET t: $CALC(1/0): Division durch 0 (bei "")“ und „M1 $STRCMP r: Argument 1: $CALC(1/0): Division durch 0 (bei "")“.

Eine Einschränkung gilt für die fehlende Klammer. Fehlt sie schon im Quelltext der Zeile (`$SET t "$CALC((1+2)"`), meldet die vorhandene Zeilenprüfung „$CALC: Klammer) fehlt“ mit der Quelltextzeile, aber ohne Direktive und Ziel, und zwar zweimal (der Assembler fasst sie zu „2-mal gleiche Fehlermeldung“ zusammen); das gilt für jede Zeile mit `$CALC`, auch für `dw`, und ist unverändert geblieben. Fehlt sie erst nach dem Auflösen von `$V(...)` (`c` = `(1` und der Text `$CALC($V(c)+2)`), arbeitet der neue Weg und meldet „M1 $SET t: $CALC: Klammer) fehlt“ mit Makro, Direktive und Ziel (Negativtest `makro_zahl_set_klammer`).

Grenzen: Der Text einer Variablen hat höchstens 200 Zeichen und enthält kein Semikolon (4.12, 4.21); beides wird nach der Auflösung von `$CALC...` geprüft. Binär kommt mit 4.27 hinzu (`$CALCB`, `$CALCBn`, `0b`, `$VALB`), hexadezimal mit 4.28 (`$CALCH`, `$CALCHn`, `0x`, `$VALH`); einen Dezimaltext mit fester Breite gibt es nicht. Der Wortvergleich von `$IF` bleibt unverändert (4.4, 4.10); wer einen Oktaltext als Zahl vergleichen will, liest ihn mit `$VALO` und vergleicht die Zahl.

Beispiel 1: `TAB3` erzeugt drei Wörter und gibt jedem eine Marke mit der Nummer in Oktalschreibung. `TAB3` erzeugt die Wörter 7, 8 und 9 in den Zellen 0000 bis 0002; `$CALCO2(7)` ergibt `007`, `$CALCO2(8)` ergibt `010` und `$CALCO2(9)` ergibt `011`, die Marken heißen `z007`, `z010` und `z011`:

```
TAB3:      MACRO
           $LOCAL i nm
           $SET i 7
           $WHILE $V(i) <= 9
           $SET nm "$CALCO2($V(i))"          ; Oktaltext mit der Breite 2
z$V(nm):   dw $V(i)                          ; Marke z007, z010, z011
           $INC i
           $ENDWHILE
           MEND
```

Beispiel 2: `OKTSUMME "17 21"` addiert zwei oktal geschriebene Zahlen, die ohne führende 0 in einem Parameter stehen. Das Leerzeichen steht an Position 3, `a` = `17` und `b` = `21`; 17 oktal ist 15, 21 oktal ist 17, die Summe 32 (040 oktal) wird als Wort erzeugt. Mit `$VAL` statt `$VALO` entstünde 38 (17 + 21):

```
OKTSUMME:  MACRO 1
           $INDEX p "$1" " "                 ; Position des Leerzeichens
           $SUBSTR a "$1" 1 $CALC($V(p)-1)   ; erste Zahl als Text
           $SUBSTR b "$1" $CALC($V(p)+1)     ; zweite Zahl als Text
           $VALO x "$V(a)"                   ; oktal gelesen
           $VALO y "$V(b)"
           dw $CALC($V(x)+$V(y))             ; Summe
           MEND
```

Beispiel 3: `RUNDE` bildet aus 15 den Oktaltext `017` (3 Zeichen) und liest ihn oktal (15) und dezimal (17) zurück; erzeugt werden die Wörter 3, 15 und 17:

```
RUNDE:     MACRO
           $SET n 15
           $SET t "$CALCO($V(n))"            ; Text 017
           $VALO r "$V(t)"                   ; oktal zurück
           $VAL s "$V(t)"                    ; dezimal (bisheriges Verhalten)
           dw $V(#t)                         ; 3
           dw $V(r)                          ; 15
           dw $V(s)                          ; 17
           MEND
```

Stand der Prüfung: Die drei Beispiele, die Messfälle dieses Abschnitts und alle Fehlerfälle sind mit c82ass 1.00-070 geprüft (Sollwerte vor dem Lauf von Hand hergeleitet). Im Regressionstest stehen zwei Programme und siebzehn Negativtests: `Claude/makrozahl` (35 Prüfzellen: Rechenausdrücke im Text mit Konstante, Variable, Parameter und gemischt, `$CALCOn` mit Breite, Text ohne `$CALC` unverändert, negatives Ergebnis, Textargumente der Zeichenkettenfunktionen, Ziel als Feldelement und als lokale Variable, Marken mit Oktalnummer), `Claude/makrozahlval` (21 Prüfzellen: `$VALO` mit `17`, `017`, `00017`, `0`, `-17`, `7`, `777` und der größten Zahl, die Beispiele 2 und 3, die Rundreise für 0 bis 100 mit `$CALCO` und `$CALCO5` und für 2^63 - 1, `$VAL`, `$SET` und `$IF` bei `017` unverändert) und die Negativtests `makro_zahl_*` (fünf zu Rechenausdrücken im Text, zwölf zu `$VALO`, für jeden Fehlerfall dieses Abschnitts einer). Alle bisherigen Tests laufen bitgleich; der Regressionstest einschließlich AddressSanitizer und UBSan läuft ohne Abweichung (238 Programme und 218 Negativtests). Entschieden: neue Direktive `$VALO` neben `$VAL`; `$CALC...` im Text von `$SET` und in den Textargumenten wird aufgelöst; Bereich wie das oktale Lesen im Ausdruck (bis 2^63 - 1, beliebig viele führende Nullen); ein einzelnes `-`, kein `+`; Name `$VALO`; `$VAL`, `$IF` und `$ELIF` bleiben dezimal; eigener Abschnitt 4.22. Zu Z-03 (fehlende Klammer) gilt die Einschränkung oben, der Code blieb unverändert.

4.23 Attributabfragen (`def`, `art`, `zellen`, `adr`, `abstand`, `$PROBE`)

Zweck: Ein Makro soll beim Erzeugen von Code fragen können, ob ein Symbol im Quelltext vorkommt, was für eine Art Symbol es ist und wie viele Zellen der Bereich hat, den es benennt. Damit erzeugt ein Makro zusätzlichen Prüfcode nur, wenn der Quelltext einen Schalter `DEBUG` vereinbart hat; ein Makro füllt einen Puffer, dessen Größe es dem `db`- oder `dbn`-Operanden entnimmt, statt dass der Aufrufer sie ein zweites Mal als Parameter nennt; ein Makro prüft, ob sein Parameter ein Zahlensymbol oder ein Label ist, und verzweigt entsprechend. Bisher ging das nicht: `$CALC(symbol)` (4.18) liefert nur den Wert eines `abs`- oder `equ`-Symbols mit Zahlenwert, und ein Label und ein Name, den es nirgends gibt, endeten in derselben Fehlermeldung, die den Lauf beendet. Seit c82ass 1.01-001 gibt es drei Funktionen im Rechenausdruck; ab 1.01-018 kommen `adr` und `abstand` hinzu (siehe „Adressen und Abstände“ unten). Alles wirkt nur zur Assemblerzeit; im erzeugten Quelltext steht allein das Ergebnis. Was bisher ging, bleibt bitgleich: Ein Name vor einer Klammer war bisher ein Fehler.

Die Funktionen: `def(name)`, `art(name)` und `zellen(name)` stehen wie `min` und `max` (4.8) im Rechenausdruck, also überall, wo ein Ausdruck stehen darf: in `$CALC`, `$CALC0` und `$CALC0n`, in einem `$CALC` innerhalb von `$IF`, `$ELIF` und `$WHILE`, in der Anzahl hinter `$REPT`, in `$SET`, `$INC`, `$P(...)` und im Argument von `$TYPZAHL`. Die Ergebnisse sind ganze Zahlen und rechnen wie jede andere Zahl (`$CALC(def(a)+def(b))`, `$CALC(zellen(puffer)*2)`). Die Namen der Funktionen unterscheiden Groß- und Kleinschreibung: `DEF(tab)` und `Art(tab)` sind keine Funktionen, sondern Symbole (Meldung wie in 4.18). Ohne Klammer bleiben `def`, `art` und `zellen` gewöhnliche Symbolnamen (ein Symbol `art: equ 4` rechnet in `$CALC(art+1)` als 5). In den Operanden eines `abs`- oder `equ`-Symbols, den 4.18 auswertet, gelten die Funktionen nicht; sie gelten im Ausdruck des Makros selbst.

Das Argument ist ein Symbolname: Buchstaben, Ziffern und _, nicht mit einer Ziffer beginnend, höchstens 32 Zeichen (3.2). Es darf von einem Paar Klammern umgeben sein, weil ein `$1` im Ausdruck eingeklammert eingesetzt wird (4.2): `def($1)` mit dem Parameter `tab` wird `def((tab))`. Ein Ausdruck, ein Operator, ein Komma, ein Leerraum und ein zweites Klammerpaar im Argument sind ein Fehler. `$V(...)` im Argument wird vor der Auswertung aufgelöst wie überall: Eine Zahlvariable darf im Namensteil stehen (`def(sym$V(i))` mit `i` = 2 fragt nach `sym2`; `zellen(tab$V(k))` nach `tab3`), als ganzes Argument ergibt sie eine Zahl und damit einen Fehler (`$SET n 5`, dann `art($V(n))`: „5“ ist kein Symbolname). Eine Textvariable mit einem Symbolnamen als ganzem Argument geht (`$SET nm "a_db"`, dann `zellen($V(nm))`).

Suchbasis: Gesucht wird derselbe Quelltext wie in 4.18: die Zeilen der Quelldatei ohne Makrodefinitionen, auch hinter dem Aufruf und hinter `end`. Ein Symbol gilt als vorhanden, wenn eine dieser Zeilen es als Marke trägt, auf einem Maschinenbefehl, einer Datenanweisung (`db`, `dbn`, `dw`, `dwb`, `dwh`, `dwf`, `dwg`, `da`, `dt`, `dto`, `dtm`, `dz`, `kat4`, `kat5`, `kat6`), `abs`, `equ`, `equ10`, `equ40`, `equ100`, `spa`, `end`, einem Makroaufruf, einem Namen mit Punkt (3.3.17) oder allein auf der Zeile. Das Ergebnis hängt nicht von der Stelle des Aufrufs ab. Nicht gefunden werden Symbole, die ein Makro erzeugt, bis auf die Ausnahme am Ende dieses Absatzes (auch lokale Marken `name%:` und die Marken der Form `ML...`): Sie sind in dem Makro, das fragt, unbekannt, auch wenn das erzeugende Makro vorher aufgerufen wurde. Das gilt gleich für Makros im Quelltext, aus einer Projektmakrodatei (4.9) und aus `c82ass.makros`: Alle fragen die Symbole des Quelltexts, und die Marke aus einem Makro einer Makrodatei ist kein Symbol des Quelltexts (`def` ergibt 0). Die Marke einer `opsym`-Zeile (3.3.17) ist kein Symbol. Literalsymbole (`l0001`) gibt es in Pass 0 noch nicht: `def(l0001)` ergibt 0, auch wenn der Quelltext Literale benutzt; ebenso ergibt `def` für den Namen eines Symbol-Literals (`!<symbolname>`, 3.2) 0, und `adr` meldet für ihn, die Marke komme nach Pass 2 nicht in der Symboltabelle vor. Symbole hinter `end` (nach dem Programm und vor dem Literalbereich) werden gefunden, mit und ohne `--no-opt`. Ausnahme: Ein `abs` oder `equ`, das ein Makro (auch aus einer Makrodatei) mit auswertbarem Zahlenwert erzeugt hat, sehen `def` und `art` von der erzeugenden Zeile an (`def` ergibt 1, `art` ergibt 1 für `abs` und 2 für `equ`); `zellen` meldet dafür einen Fehler, weil ein `abs` oder `equ` keine Zelle belegt. Test `makrosymbol2` (`art(lagem)`, `art(breite2)`, `def(gibtsnicht)`).

`def(name)` ergibt 1, wenn das Symbol nach der Suchbasis vorhanden ist, sonst 0. Ein unbekanntes Symbol ist kein Fehler; dafür ist die Funktion da. Ob eine Makro-Variable, ein Feld oder ein Makro vorhanden ist, fragen `$DEFINED` und `$MDEFINED` (4.25, ab c82ass 1.01-006); `def` kennt nur Symbole des Quelltexts.

`art(name)` ergibt die Art des Symbols als Zahl:

Wert	Bedeutung	`$CALC(name)`
0	nicht im Quelltext vorhanden	Fehler
1	`abs`-Symbol mit auswertbarem Operanden	Wert
2	`equ` mit festem Zahlenwert (auswertbar nach 4.18)	Wert
3	Adresse oder Wert, der erst nach Pass 0 feststeht: `equ *`, `equ` mit Labels im Ausdruck, `equ10`, `equ40`, `equ100`, `spa`, und jedes `abs` oder `equ`, dessen Operand sich nicht auswerten lässt (Zyklus, Kette tiefer als 16, Operand fehlt)	Fehler
4	Marke einer anderen Zeile: Befehl, Datenanweisung, Makroaufruf, `end`, Marke allein	Fehler

Damit gilt: `$CALC(name)` gelingt genau dann, wenn `art(name)` 1 oder 2 ist; `art` und `$CALC` stimmen immer überein. Gibt es mehrere Zeilen mit derselben Marke, gilt wie in 4.18 die erste, deren Operand sich auswerten lässt; gibt es keine, entscheidet die erste Zeile mit der Marke. Gemessen: (`equ *`, dann `abs 7`) ergibt 1; (`equ 5`, dann `abs 6`) ergibt 2; (`db 4`, dann `equ 5`) ergibt 2, aber `zellen` 4 (die erste Zeile mit der Marke); (`db 4`, dann `equ *`) ergibt 4; (`equ *`, dann `db 4`) ergibt 3. Der Assembler meldet doppelte Symbole selbst (Rückgabecode 16); die Fälle sind deshalb nur im Listing zu sehen.

Ketten und Zyklen: Verweise von einem `equ` auf ein anderes dürfen höchstens 16 Stufen tief sein (4.18). Ein `abs`-Symbol mit 15 `equ`-Stufen darüber (`ka0: abs 3`, `ka1: equ ka0` bis `ka15`) ergibt für `ka15` die Art 2 und `$CALC(ka15)` =

3; mit 16 Stufen (`ka16`) ergibt `art` 3, und `$CALC(ka16)` meldet „Symbol "ka0": Verweis zu tief oder zyklisch (mehr als 16 Stufen)“; die Meldung nennt das Ende der Kette, nicht das gefragte Symbol. Eine Kette, die in `equ *` endet, ergibt 3, und `$CALC` meldet „Symbol "kb0" hat keinen festen Zahlenwert (equ * ist eine Adresse, die erst nach Pass 0 feststeht)“. Ein Zyklus (`a: equ b`, `b: equ a`, auch `a: equ a`) ergibt `art` 3 und `def` 1. Ein `equ` oder `abs` ohne Operand ergibt `art` 3 und `def` 1; `$CALC` meldet „Symbol "eo" hat keinen festen Zahlenwert (der Operand fehlt)“. Zyklen und fehlende Operanden lehnt der Assembler selbst ab (Rückgabecode 16), kein gültiges Programm enthält sie; die Fälle sind nur im Listing sichtbar.

`zellen(name)` ergibt die Zahl der Speicherzellen der Anweisung, die das Symbol als Marke trägt. Bestimmbar sind nur Eigenschaften, die in der Zeile des Symbols selbst stehen; Pass 0 läuft vor der Symboltabelle (4.18):

Anweisung der Marke	Zellen	Bemerkung
`db n`, `dbn n` (Zahl, dezimal oder oktal)	n	Regeln von 3.3.1; `db 010000` ergibt 4096, `db 0` und `dbn 00` ergeben 0
Maschinenbefehl (Oktalcode mit drei Ziffern), `dw`, `dwb`, `dwh`, `dwf`, `dwg`, `da`, `dz`, `kat5` (3.3.15: immer ein Wort, ab c82ass 1.03-001)	1	
Name mit Punkt (3.3.17), zum Beispiel `.LD`	1	vor Pass 0 noch nicht ersetzt, zählt eine Zelle

Für alles andere ist `zellen` ein Fehler mit dem Grund: `dt`, `dto`, `dtm` (die Zellenzahl hängt vom Zustand KB/GB ab, 3.3.5), `kat4`, `kat6` (sie hängt vom Text ab; eine zweite Zählung wie in den späteren Pässen würde bei einer Abweichung einen falschen Wert statt eines Fehlers liefern), `abs`, `equ`, `equ10`, `equ40`, `equ100`, `spa` (ein Wert belegt keine Zelle), `start`, `end`, `begin`, `bend`, eine Marke allein auf der Zeile, eine Marke auf einem Makroaufruf (die erste erzeugte Zelle steht erst nach der Auflösung fest), ein Symbol, das es nicht gibt, ein Operationscode, der weder Befehl noch Anweisung ist, und `db` oder `dbn` mit einem Operanden, der keine gültige Zahl ist (3.3.1). Der Operand von `db` und `dbn` muss eine Zahl sein; ein Ausdruck (`db 3+1`) ist ein Fehler, so dass `zellen` nie einen geratenen Wert liefert.

Zellenzahl eines Blocks: Mit zwei Namen, durch ein Komma getrennt, ergibt `zellen` die Zahl der Speicherzellen eines mehrzeiligen Blocks im Quelltext: `zellen(von,bis)`. Gezählt werden die Zellen aller Zeilen ab der ersten Zeile mit der Marke `von` bis unmittelbar vor der ersten Zeile mit der Marke `bis`. Die Zeile von `von` zählt mit, die Zeile von `bis` nicht; `zellen(a,b)` ist also der Abstand `b-a` der beiden Marken in Zellen. Damit lässt sich die Länge einer Tabelle aus mehreren Zeilen schon in Pass 0 erfragen, etwa als Anzahl hinter `$REPT` oder in einem `$IF`, ohne dass der Aufrufer sie ein zweites Mal als Parameter nennt (`$REPT $CALC(zellen($1,$2))`). Der Wert ist der echte Abstand der Marken nach Pass 2, weil der Optimierer Zeilen des Programmierers nie verlegt (Abschnitt 5); der Test `makrozellen` rechnet beides nach: Das zweite Wort jedes `PAAR` ist `$2-$1`.

Zählregeln für `zellen(von,bis)`: Gezählt wird nur, was in Pass 0 sicher feststeht, nach denselben Regeln wie bei `zellen(name)`: `db` und `dbn` mit ihrer Zahl (3.3.1); Maschinenbefehl, `dw`, `dwb`, `dwh`, `dwf`, `dwg`, `da`, `dz`, `kat5` (3.3.15: immer ein Wort, ab c82ass 1.03-001) und Name mit Punkt je eine Zelle. Keine Zelle belegen und zählen 0: Kommentarzeilen (`;`, `#` oder `&` am Zeilenanfang), eine Marke allein auf der Zeile, `abs` und `opsym`. Anders als bei `zellen(name)`, wo eine Marke auf `abs` ein Fehler ist, sind diese Zeilen im Block erlaubt, weil sie keine Zelle belegen (Annahme der Umsetzung). Ein Block, der nur aus solchen Zeilen besteht, ergibt 0 (Test `makrozellen`, Block C). Makrodefinitionen im Block gehören nicht zur Suchbasis und zählen nicht. Ein Fehler mit Quelltextzeile und Grund, nie ein falscher Wert, ist jede Zeile im Block mit `dt`, `dto`, `dtm` (die Zellenzahl hängt vom Zustand KB/GB ab), `kat4`, `kat6` (sie hängt vom Text ab), `equ`, `equ10`, `equ40`, `equ100`, `spa`, `start`, `end`, `begin`, `bend` (sie setzen oder verschieben den Befehlszähler, ein Abstand ist so nicht bestimmbar), einem Makroaufruf (die Zellenzahl steht erst nach dem Auflösen fest), einer Zeile, die mit `$` beginnt (sie steht erst nach ihrer Auswertung fest), und einem Operationscode, der weder Befehl noch Anweisung ist; ebenso `db` oder `dbn` mit einem Operanden, der keine gültige Zahl ist (3.3.1).

Marken und Argumente von `zellen(von,bis)`: Beide Namen müssen Marken im Quelltext sein (Suchbasis wie oben); die Marke einer `opsym`-Zeile zählt nicht. Trägt eine Marke mehrere Zeilen, gilt die erste. `bis` muss im Quelltext hinter `von` stehen; die umgekehrte Reihenfolge und dieselbe Marke zweimal sind ein Fehler, ebenso eine Marke, die es nicht gibt, und eine Marke, die erst ein Makro erzeugt hat (auch ein von einem Makro erzeugtes `abs` oder `equ`): Ein Block braucht Marken im Quelltext. Jedes der beiden Argumente ist ein einzelner Symbolname wie bei `def(name)` (höchstens 32 Zeichen, nicht mit einer Ziffer beginnend) und darf von einem Paar Klammern umgeben sein, damit `zellen($1,$2)` mit eingeklammert eingesetzten Parametern geht (`zellen((tab_a),(tab_e))`). Ein Ausdruck, ein Leerraum (auch hinter dem Komma: `zellen(a, b)`), ein dritter Name und ein fehlender zweiter Name sind ein Fehler; `def` und `art` nehmen weiter nur einen Namen (`def(a,b)` ist ein Fehler). Beim Aufruf des Makros trennt man die Parameter mit Leerraum, nicht mit Komma (`PAAR tab_a tab_e`).

Fehler bei `zellen(von,bis)`: Die Meldung hat die Form „$CALC(zellen($1,$2)): zellen(a,b): <Grund> (bei "zellen((a),(b))")“ und kommt zweimal, einmal mit dem Ausdruck im Rumpf und einmal mit dem eingesetzten (die beiden Texte unterscheiden sich und werden nicht zusammengefasst). Gründe für eine Zeile im Block, jeweils mit der Quelltextzeile: „Quelltextzeile 9: "dt" belegt eine Zellenzahl, die vom Zustand KB/GB abhängt“ (`dto`, `dtm` ebenso; bei `kat4`, `kat6`: „… die vom Text abhängt“), „Quelltextzeile 9: "equ" setzt oder verschiebt den Befehlszähler, ein Abstand ist so nicht bestimmbar“ (`equ10`, `equ40`, `equ100`, `spa`, `start`, `end`, `begin`, `bend` ebenso), „Quelltextzeile 9: "M" ist ein Makroaufruf, die Zellenzahl steht erst nach dem Auflösen fest“, „Quelltextzeile 9: "FOO" ist weder Befehl noch Anweisung“, „Quelltextzeile 10 ist eine $-Zeile, die Zellenzahl steht erst nach ihrer Auswertung fest“ und für `db` und `dbn` die Meldung der Zahlenprüfung von 3.3.1 („Quelltextzeile 9: db: Operand "3+1" ist keine gültige Zahl (Oktalzahl mit führender 0 oder Dezimalzahl)“). Gründe für die Marken: „Symbol "nix" ist nicht im Quelltext vorhanden“, „Symbol "erz" hat ein Makro erzeugt und steht nicht im Quelltext (ein Block braucht Marken im Quelltext)“ und „Die Marke "a" muss im Quelltext hinter der Marke "b" stehen“ (auch bei derselben Marke zweimal). Nur ein von einem Makro erzeugtes `abs` oder `equ` wird so gemeldet; eine Marke auf einem `dw` oder einem Befehl in einem Makro meldet „Symbol "erz" ist nicht im Quelltext vorhanden“. Fehler im Argument: „zellen(von,bis): Der zweite Symbolname fehlt (erlaubt: Buchstaben, Ziffern und _, nicht mit einer Ziffer beginnend)“ (`zellen(a,)`), „zellen(von,bis): Hinter dem Komma darf kein Leerraum stehen (zellen(a,b), nicht zellen(a, b))“ (`zellen(a, b)`; diese Meldung gibt es ab c82ass 1.01-013, vorher kam hier irreführend „Der zweite Symbolname fehlt“), „zellen: Das Argument ist kein einzelner Symbolname (Operator, Komma, Leerraum und weitere Klammern sind nicht erlaubt; zellen nimmt höchstens zwei Namen)“ (`zellen(a,b,c)`, `zellen(a,b+1)`) und für `def(a,b)` und `art(a,b)` die bisherige Meldung „… kein einzelner Symbolname“.

Namenslänge: Ein Name mit 32 Zeichen geht überall (Marke auf `equ` und auf `db`, `def`, `art`, `zellen`, `$CALC(name)`). Bei 33 Zeichen melden `def`, `art` und `zellen` „<f>: Symbolname zu lang (höchstens 32 Zeichen)“ mit dem Namen der Funktion; `$CALC(name33)` ohne Funktion meldet wie bisher nur „Symbolname zu lang“.

Parameter als Ausdruck: Ein `$1` im Ausdruck wird eingeklammert eingesetzt (4.2). `DOPPELT art(a_equ)+1` mit `dw $CALC(2*$1)` ergibt daher 2*(2+1) = 6, `DOPPELT zellen(a_db)+art(a_abs)` ergibt 2*(5+1) = 12. Als Argument von `art`, `def` oder `zellen` ist ein Ausdruck dagegen ein Fehler: `FRAGE_ART a_abs+1` und `FRAGE_ART art(a_abs)` melden „art: Das Argument ist kein einzelner Symbolname …“, und zwar zweimal mit verschiedenem Text, einmal mit „$CALC(art($1))“ und einmal mit dem eingesetzten Argument („$CALC(art(a_abs+1))“). Die beiden Meldungen werden nicht zusammengefasst.

Im Quelltext: `$CALC` wird im Rumpf eines Makros und in Textargumenten (4.22) ausgewertet, ab c82ass 1.01-008 auch im normalen Quelltext, dort aber nur, wenn mindestens eine Zeile des Quelltexts mit `$` beginnt (4.1). Ohne eine solche Zeile bleibt `dw $CALC(def(x))` Text, und der Assembler meldet „Operand '$CALC(def(x))' ist ungültig“ (Rückgabecode 16).

Fehler (Rückgabecode 16, die Meldung beginnt wie die übrigen Rechenfehler mit „$CALC(...): “ und nennt die Quelltextzeile des Aufrufs; jede Meldung kommt zweimal und wird vom Assembler zu „2-mal gleiche Fehlermeldung“ zusammengefasst):

```
<f>(): Der Symbolname fehlt                              def(), art(), zellen()
<f>: "5" ist kein Symbolname (er darf nicht mit einer Ziffer beginnen)
<f>: Das Argument ist kein einzelner Symbolname (Operator, Komma, Leerraum und weitere Klammern sind nicht erlaubt)
                                                        def(a+b), def(a,b), def(a b), def(erz%), def(((tab)))
<f>(...): Das Argument ist kein Symbolname (erlaubt: Buchstaben, Ziffern und _, nicht mit einer Ziffer beginnend)
                                                        def(.LD)
<f>: Symbolname zu lang (höchstens 32 Zeichen)
$CALC: Klammer ) fehlt                                   def((tab) und def(tab
```

Die fehlende Klammer erkennt die vorhandene Zeilenprüfung (wie Z-03 in 4.22) ohne Direktive und Ziel. Für `zellen` kommen die Gründe hinzu, jeweils mit dem Namen des Symbols: „Symbol "x": dt belegt eine Zellenzahl, die vom Zustand KB/GB abhängt“ (`dto`, `dtm` ebenso), „… kat4 belegt eine Zellenzahl, die vom Text abhängt“ (`kat6` ebenso), „… abs definiert einen Wert und belegt keine Zelle“ (`equ`, `equ10`, `equ40`, `equ100`, `spa` ebenso mit dem Operationscode), „… steht allein auf einer Zeile und belegt keine Zelle“, „… Marke auf dem Aufruf des Makros M2, die Zellenzahl steht erst nach dem Auflösen fest“, „Symbol "nix": nicht im Quelltext vorhanden“ (auch für die Marke einer `opsym`-Zeile), „… start belegt keine Zelle“ (`end`, `begin`, `bend` ebenso), „… Operationscode "foo" ist weder Befehl noch Anweisung“, und für `db` und `dbn` die Meldung der Zahlenprüfung von 3.3.1: „Symbol "tab": db: Operand "3+1" ist keine gültige Zahl (Oktalzahl mit führender 0 oder Dezimalzahl)“, „db: Operand fehlt (erwartet wird eine Zahl)“, „db: Operand "-3" darf nicht negativ sein“, „db: Operand "010001" ist zu groß (die Trommel hat nur 010000 Speicherplätze)“. Die Meldung „unerlaubtes Zeichen“ des Rechners nennt die neuen Funktionen in der Liste der erlaubten Zeichen („… min(a,b), max(a,b), def(s), art(s), zellen(s), adr(s), abstand(s,t) und Klammern“); die

Meldung für ein unbekanntes Symbol bleibt wörtlich („Symbol "x" ist im Quelltext weder mit abs noch mit equ <Zahl> definiert“, 4.18).

Grenzen: Adressen, Abstände von Labels und Sprungweiten lassen sich in Pass 0 nicht bestimmen; das ist die Grenze von 4.18; Pass 0 selbst kennt sie weiterhin nicht, `adr` und `abstand` (unten, ab c82ass 1.01-018) holen sie aus Probeläufen. Eine Ausnahme ist die Zellenzahl eines Blocks aus Zeilen mit fester Zellenzahl: `zellen(von,bis)` (oben, ab c82ass 1.01-012). Enthält der Block eine Zeile, deren Zellenzahl in Pass 0 nicht feststeht (`dt`, `kat4`, ein Makroaufruf, `equ`, `spa` und so weiter), bleibt der Abstand nicht bestimmbar, und `zellen(von,bis)` meldet einen Fehler. Gefragt werden kann nur nach Symbolen des Quelltexts, nicht nach Makronamen, Variablen oder Feldern (dafür gibt es `$V(...)`, `$#`). Ein Name aus einer Zahlvariablen ist kein Symbolname (siehe oben). Nach den Labels in 4.18 gilt jetzt: Sie lassen sich nicht auswerten, aber fragen lassen sie sich mit `def`, `art`, `zellen` und, für die Adresse, mit `adr` und `abstand`.

Beispiel 1: Prüfcode nur auf Wunsch. `PROBE` erzeugt das Wort 63 nur, wenn der Quelltext einen Schalter `DEBUG` vereinbart (hier `DEBUG: abs 1` hinter `end`); ohne diese Zeile erzeugt `PROBE` nichts. `PROBE2` fragt nach einem Schalter `NODEBUG`, den es nicht gibt, und erzeugt das Wort 78 im `$ELSE`-Zweig.

```
PROBE:     MACRO
           $IF $CALC(def(DEBUG)) = 1
           dw 63                          ; Probemarke
           $ENDIF
           MEND
PROBE2:    MACRO
           $IF $CALC(def(NODEBUG)) = 1
           dw 77                          ; wird nicht erzeugt
           $ELSE
           dw 78                          ; wird erzeugt
           $ENDIF
           MEND
```

Beispiel 2: Art und Größe. `INFO` erzeugt die Art eines Symbols, `GROESSE` die Zellenzahl seiner Anweisung. Das Programm erzeugt die Wörter 4, 1, 0, 4, 2, 3 (`INFO`) und 8, 1 (`GROESSE`). Das Argument `puffer` wird als `(puffer)` eingesetzt; das ist nach den Regeln oben gültig.

```
INFO:      MACRO 1
           dw $CALC(art($1))              ; Art des Symbols
           MEND
GROESSE:   MACRO 1
           dw $CALC(zellen($1))           ; Zellen der Anweisung
           MEND
           start
           000 haupt
           INFO puffer                    ; 4: Marke einer Datenanweisung
           INFO maximum                   ; 1: abs
           INFO fehlt                     ; 0: nicht vorhanden
           INFO wort                      ; 4
           INFO lage                      ; 2: equ mit Zahl
           INFO hier                      ; 3: equ *
           GROESSE puffer                 ; 8
           GROESSE wort                   ; 1
haupt:     164 0001
           020 0
puffer:    dbn 010                        ; 8 Zellen (oktal 010)
wort:      dw 5                           ; 1 Zelle
           end
maximum:   abs 5
lage:      equ 07640
hier:      equ *
```

Beispiel 3: Eine Schleife über Namen mit Nummer. `SUCHE` fragt, welche der Symbole `sym1`, `sym2`, `sym3` vereinbart sind, und erzeugt für `sym1` und `sym3` (die hinter `end` stehen) das Wort 1, für `sym2` das Wort 0; danach die Zellenzahl von `tab3` (`tab3: db 3`).

```
SUCHE:     MACRO
           $SET i 1
           $WHILE $V(i) <= 3
           dw $CALC(def(sym$V(i)))        ; sym1: 1, sym2: 0, sym3: 1
           $INC i
           $ENDWHILE
           $SET k 3
           dw $CALC(zellen(tab$V(k)))     ; tab3 hat 3 Zellen
           MEND
```

Beispiel 4: Länge eines Blocks aus mehreren Zeilen. PAAR erzeugt die Zellenzahl des Blocks tab_a bis tab_e und den echten Abstand der beiden Marken; beide Wörter sind 9 (oktal 011). Der Block besteht aus db 5 (5 Zellen), zwei dw und zwei Befehlen (je eine Zelle); die Marke allein und das abs zählen nicht, und die Zeile von tab_e gehört nicht mehr dazu. Der Block enthält keine $-Zeile.

```
PAAR:       MACRO 2
            dw $CALC(zellen($1,$2))      ; Zellenzahl des Blocks in Pass 0
            dw $2-$1                     ; echter Abstand der Marken nach Pass 2
            MEND
            start
            000 haupt
            PAAR tab_a tab_e             ; Zelle 1 und 2: 9 und 9
haupt:      020 0
tab_a:      db 5                         ; 5 Zellen
            dw 1                         ; 1 Zelle
            dw 2                         ; 1 Zelle
            211 0                        ; 1 Zelle
            240 0                        ; 1 Zelle
marke_allein:                            ; Marke allein: null Zellen
w_abs:      abs 7                        ; abs: null Zellen
tab_e:      dw 3                         ; hier endet der Block, diese Zeile zählt nicht mehr
            end
```

Festlegungen: Funktionen im Rechenausdruck statt einer Direktive; die Namen def, art, zellen; alle drei Funktionen zusammen; die fünf Werte 0 bis 4 für art; dieselbe Suchbasis wie 4.18 (Marken aus Makros sind unbekannt); zellen für dt, dto, dtm, kat4, kat6 ein Fehler (kat5 zählt nach 3.3.15 eine Zelle); ein unbekanntes Symbol ergibt bei def und art 0, bei zellen einen Fehler; die Meldung für ein unbekanntes Symbol aus 4.18 bleibt, nur die Liste der erlaubten Zeichen nennt die Funktionen. Annahmen der Umsetzung: (1) abs und equ, deren Operand sich nicht auswerten lässt, ergeben art 3 (auch Zyklus und zu tiefe Kette); (2) bei mehreren Zeilen mit derselben Marke entscheidet die erste mit auswertbarem Operanden; (3) die Marke einer opsym-Zeile ist kein Symbol; (4) ein Name mit Punkt zählt eine Zelle; (5) start, end, begin, bend sind bei zellen ein Fehler; (6) höchstens ein Klammerpaar um den Namen, ein zweites meldet „kein einzelner Symbolname“; (7) die Meldungstexte wie oben.

Festlegungen zu zellen(von,bis) (Freigabe 04.11.2026): zellen(von,bis) ergibt die Zellen ab der Marke von bis unmittelbar vor der Marke bis (bis zählt nicht mit); gezählt werden nur Zeilen mit fester Zellenzahl, alles andere ist ein Fehler mit Grund, nie ein falscher Wert. Annahmen der Umsetzung: (1) abs und opsym sind im Block erlaubt, weil sie keine Zelle belegen (bei zellen(name) bleibt die Marke auf abs ein Fehler); (2) die Marken müssen im Quelltext stehen, eine von einem Makro erzeugte Marke ist ein Fehler; (3) trägt eine Marke mehrere Zeilen, gilt die erste; (4) eingeklammerte Argumente werden akzeptiert (zellen((von),(bis)) durch $1 und $2); (5) def und art mit zwei Argumenten bleiben Fehler.

Stand der Prüfung: Die Beispiele 1 bis 3, die Messfälle dieses Abschnitts (Ketten bis 15 und 16 Stufen, Zyklen, equ und abs ohne Operand, doppelte Marken, Namenslänge 32 und 33, Parameter als Ausdruck, Zahlvariable im Namensteil, Symbole hinter end nach Literalen, Makros aus einer Projektmakrodatei und aus c82ass.makros, --makros) und alle Fehlerfälle sind mit c82ass 1.01-001 geprüft (Sollwerte und Sollmeldungen vor dem Lauf von Hand hergeleitet). Im Regressionstest stehen drei Programme (Claude/makroattr mit 63 Wörtern, Claude/makroattr2 mit den Randfällen und Claude/makroattr3 mit einer Makrodatei) und zwanzig Negativtests makro_attr_* (leer, name_ziffer, name_ausdruck, klammer, lang, lang_art, lang_zellen, namenslaenge, zellen_dt, zellen_abs, zellen_allein, zellen_aufruf, zellen_unbekannt, zellen_db_operand, zyklus, kette, kette_stern, equ_leer, parameter_ausdruck, parameter_funktion). Zyklen, equ und abs ohne Operand und doppelte Marken, die der Assembler selbst ablehnt, wurden am Listing gelesen (Assembler/Regressionstest/proben_makroattr/, nicht Teil des Regressionstests). Alle bisherigen Tests laufen bitgleich; der Regressionstest einschließlich AddressSanitizer und UBSan läuft ohne Abweichung (241 Programme und 238 Negativtests).

Stand der Prüfung von zellen(von,bis): Das Programm Claude/makrozellen (Blöcke von 9, 11 und 0 Zellen, eine Marke allein als von, $REPT, $IF und die Differenz zweier Blöcke) läuft mit c82ass 1.01-012 beim ersten Lauf mit „Test wahr“; die Sollwerte sind vor dem Lauf von Hand hergeleitet, und das zweite Wort jedes PAAR ist der echte Abstand der Marken nach Pass 2 und stimmt mit zellen(von,bis) überein. Beispiel 4 und die Fehlerfälle dieses Abschnitts (dt, kat4, equ, spa, end, begin, Makroaufruf, unbekannter Operationscode, $-Zeile, db mit einem Ausdruck, fehlende Marke, Marke aus einem Makro, vertauschte und gleiche Marken, fehlender zweiter Name, dritter Name, Leerraum hinter dem Komma, def(a,b)) sind einzeln mit c82ass 1.01-012 gelaufen (nur die Meldung bei Leerraum hinter dem Komma ist ab 1.01-013 neu); die Meldungen stehen wörtlich so in den Läufen. Teil des Regressionstests (Stand 04.11.2026): das Programm makrozellen (Referenz neu) und zwanzig Negativtests makro_zellen_dt, _kat4, _equ, _begin, _aufruf, _unbekannt, _dollar, _db_ausdruck, _marke_fehlt, _marke_makro, _vertauscht, _gleich, _zweiter_name, _leerraum, _dritter_name, _ausdruck, _spa, _end, _def_zwei und _marke_dw. Der Negativtest

`makro_zellen_leerraum` hält die Meldung bei Leerraum hinter dem Komma fest (bis c82ass 1.01-012 kam dort irreführend „Der zweite Symbolname fehlt“). Der Regressionstest einschließlich AddressSanitizer und UBSan läuft mit c82ass 1.01-013 ohne Abweichung (266 Programme und 386 Negativtests).

Adressen und Abstände (`adr`, `abstand`: `adr(name)` ergibt die Adresse der Marke name, `abstand(von,bis)` ergibt adr(bis) - adr(von), mit Vorzeichen. Eine Sprungweite ist abstand(Marke der Sprungzeile, Ziel). Beide stehen wie `def`, `art` und `zellen` im Rechenausdruck, also überall, wo ein Ausdruck stehen darf, aber nicht im Operanden eines `abs`- oder `equ`-Symbols. Die Adresse ist die der Spalte address der Symboltabelle, also wie „name,oktal“ in der `.sym`-Datei: relativ zum Programmanfang, ohne Ladeadresse (ein `start 020` ändert sie nicht: in einem Probeprogramm blieb die Marke haupt bei 3). Gefragt werden kann nach jeder Marke der Symboltabelle, auch nach einer, die erst ein Makro erzeugt (anders als bei `def` und `zellen`), aber nicht nach einer Literalmarke (l0001 und so weiter) und nicht nach einem `abs`- oder `equ`-Namen: er ist keine Marke der Symboltabelle und ergibt den Fehler „kommt nach Pass 2 nicht in der Symboltabelle vor“. Die Argumente werden wie bei `zellen(von,bis)` geprüft (ein Symbolname je Argument, kein Leerraum hinter dem Komma, höchstens ein Klammerpaar; `adr` mit zwei Namen und `abstand` mit einem Namen sind Fehler). Hinweis: `$CALC` kennt kein „/“ und keine Leerzeichen im Ausdruck.

Ablauf: Pass 0 läuft vor der Symboltabelle und kennt keine Adressen. Der Assembler bestimmt sie deshalb in Probeläufen. Der erste Gebrauch von `adr` oder `abstand` in Pass 0 startet sie: Der Assembler ruft sich selbst mit denselben Argumenten auf und hängt zwei versteckte Optionen an, `--adrein=<Datei>` und `--adraus=<Datei>` (nur für diesen Selbstaufruf, nicht für den Aufruf von Hand). Mit `--probeausgabe` kommt als dritte versteckte Option `--adrdbg=<Datei>` hinzu (4.24). STDOUT des Probelaufs geht ins Null-Gerät, STDERR nach `<n>.adrlog`; ein Probelauf endet nach Pass 2 ohne Ergebnisdateien und schreibt die Adressen aller Marken (ohne Literalmarken) in die Datei hinter `--adraus`. Probelauf 1 läuft ohne Tabelle: `adr` und `abstand` rechnen mit 0 (eine Marke, die es nicht gibt, ergibt dort 0 ohne Meldung). Probelauf z liest die Tabelle von Probelauf z-1. Konvergenz ist erreicht, wenn die Tabelle eines Laufs der Eingabetabelle gleicht; das sind mindestens 2 und höchstens 8 Probeläufe. Danach läuft der eigentliche Lauf mit der fertigen Tabelle (eine Marke, die in ihr fehlt, ist ein Fehler mit dem Namen). Am Ende von Pass 2 prüft er, dass jede abgefragte Marke die Adresse hat, mit der gerechnet wurde (sonst Fehler, Rückgabecode 16). Bei Erfolg steht auf STDOUT die Zeile „Adressen fuer adr/abstand: <n> Probelaeufe, <m> Marken.“. Hinweise, `$NOTE`, `$TRACE` und `$DUMP` der Probeläufe erscheinen nicht, nur die des eigentlichen Laufs, es sei denn, `--probeausgabe` ist gesetzt (4.24); `<n>.adr1`, `<n>.adr2`, `<n>.adrlog` und `<n>.adrdbg` werden wieder gelöscht. Ohne `adr` und `abstand` im Quelltext gibt es keine Probeläufe, und alles bleibt bitgleich wie bisher; mit ihnen kommen bis zu 8 Läufe hinzu.

Blockverschiebung und Probeläufe: Die Verschiebung von Blöcken mit `begin RELOC-EQU` oder `begin RELOC-ALL` (5.4) läuft in jedem Probelauf, auch in denen für `adr` und `abstand`, nach Pass 1 und vor Pass 2. `adr` und `abstand` liefern deshalb die Adressen nach der Verschiebung, so wie sie im eigentlichen Lauf gelten (Test `begin_reloc_adr`: ein Block mit fünf Zellen, `abstand` zwischen erster und letzter Marke im Block 4, `adr` der letzten Marke 07504). Mit `--no-opt` findet keine Verschiebung statt; beide liefern dann die unverschobenen Adressen. `zellen(von,bis)` und `zellen(name)` bleiben unverändert: `begin`, `bend` und `equ` im Bereich sind dort weiter ein Fehler.

Zyklen und Fehler: Die Adresse einer Marke hängt von der Zellenzahl davor ab, und diese kann vom Ergebnis von `adr` oder `abstand` abhängen (ein Makro wählt eine kurze oder eine lange Form nach dem Abstand). Die Probeläufe suchen den Punkt, an dem beides zusammenpasst. Gibt es ihn nach 8 Probeläufen nicht, ist das ein Fehler (Rückgabecode 16) mit der ersten abweichenden Marke und beiden Adressen (vierstellig oktal): „Die Adressen fuer adr/abstand konvergieren nach 8 Probelaeufen nicht: Marke "ende": Adresse 0007 im vorigen Probelauf, 0006 im letzten“. Darunter steht der Verlauf: eine Zeile je Marke, deren Adresse über die Probeläufe wechselt (höchstens 20 Marken, dann „... und <k> weitere Marken“), mit den Adressen aller Probeläufe von 1 bis 8, etwa „Marke "ende": 0007 0006 0007 0006 0007 0006 0007 0006“ (fehlt die Marke in einem Lauf, steht dort „fehlt“); danach entweder „Die Tabelle von Probelauf 8 gleicht der von Probelauf 6: die Adressen wiederholen sich im Zyklus der Laenge 2.“ oder „Die Tabellen wiederholen sich nicht innerhalb von 8 Probelaeufen (kein Zyklus erkannt).“. Ein Zyklus zeigt, dass das Makro die Form nach dem Abstand wählt, den es selbst ändert (Beispiel 6, aber mit „höchstens 6“ statt „größer als 5“); die Zeilen nennen die Marken, bei denen man ansetzt. Schlägt ein Probelauf fehl, meldet der Assembler „Probelauf <z> fuer adr/abstand fehlgeschlagen (Rueckgabewert <rc>).“, die Kommandozeile und den Inhalt von `<n>.adrlog` (höchstens 100 Zeilen, unverändert, mit „datei:zeile:“), dann „Abbruch.“. Mit `--probeausgabe` stehen die Debugzeilen des fehlgeschlagenen Probelaufs vor dieser Meldung und nicht noch einmal im Auszug von `<n>.adrlog` (4.24). Folge von Probelauf 1 (alle Werte 0): Ein Makro, das mit 0 einen Fehler meldet (Division durch 0, `$ERROR`), beendet den Probelauf und damit den Lauf; das Makro fragt dazu `$PROBE` (siehe unten) und meldet den Fehler nur im eigentlichen Lauf. Fehler der Abfrage erscheinen als „adr(<name>): <Grund>“ und „abstand(<von>,<bis>): <Grund>“, zum Beispiel „adr(gibtsnicht): die Marke "gibtsnicht" kommt nach Pass 2 nicht in der Symboltabelle vor“. Nach Pass 2 melden „adr/abstand: Die Adresse der Marke "x" ist nach Pass 2 <o>, abgefragt und verwendet wurde <o2>.“ und „adr/abstand: Die Marke "x" kommt nach Pass 2 nicht in der

Symboltabelle vor.“ eine Abweichung. Mit `--pass0` liefern `adr` und `abstand` den Wert 0 ohne Meldung und ohne Probeläufe, und `$PROBE` ist dort 1 (3.1d). Windows: Die Kommandozeile der Probeläufe (Anführungszeichen, system()) ist nur unter Linux geprüft.

Beispiel 5: Abstand und Adresse. ABST legt den Abstand zweier Marken ab, ADRW die Adresse einer Marke, ZWEI erzeugt zwei Wörter (die Marken und Zellen eines Makros zählen mit). Adressen dezimal: 0 Sprung, 1 ABST, 2 ADRW, 3 haupt, 4 Halt, 5 bis 9 tab_a (db 5), 10 und 11 ZWEI, 12 dw 3, 13 tab_e, 14 ADRW. Also ist abstand(tab_a,tab_e) = 13 - 5 = 8, adr(tab_e) = 13 (oktal 015, ein Vorwärtsbezug) und adr(haupt) = 3; zwei Probeläufe (Lauf 2 liefert dieselben Adressen wie Lauf 1).

```
ABST: MACRO 2
          dw $CALC(abstand($1,$2)) ; Abstand der beiden Marken
          MEND
ADRW: MACRO 1
          dw $CALC(adr($1))       ; Adresse der Marke
          MEND
ZWEI: MACRO 0
          dw 1                    ; erstes Wort des Makros
          dw 2                    ; zweites Wort des Makros
          MEND
          start
          000 haupt               ; Zelle 0000: Sprung zum Hauptprogramm
          ABST tab_a tab_e        ; Zelle 0001: 8
          ADRW tab_e              ; Zelle 0002: 015
haupt:    164 0001                ; Zelle 0003: Diagnoseausgabe (nur im Test)
          020 0                   ; Zelle 0004: Halt
tab_a:    db 5                    ; Zellen 0005 bis 0011 (fünf Zellen)
          ZWEI                    ; Zellen 0012 und 0013
          dw 3                    ; Zelle 0014
tab_e:    dw 4                    ; Zelle 0015
          ADRW haupt              ; Zelle 0016: adr(haupt) = 3
          end
```

Beispiel 6: Ein Zyklus. WEIT wählt die lange Form (zwei Wörter), wenn der Abstand der beiden Marken größer als 5 ist, sonst die kurze (ein Wort); der Abstand hängt aber von der Wahl ab. Probelauf 1 (Wert 0): kurze Form, Abstand anf bis ende = 6. Probelauf 2 (6 > 5): lange Form, Abstand 7. Probelauf 3 (7 > 5): lange Form, Tabelle gleich der von Probelauf 2, also Konvergenz nach 3 Probeläufen. Ergebnis: Die Zellen 4 und 5 enthalten 021 und 022, ende liegt auf der Adresse 012 oktal (dezimal 10) und enthält 011.

```
WEIT: MACRO 2
          $IF $CALC(abstand($1,$2)) > 5
          dw 021                  ; lange Form, erstes Wort
          dw 022                  ; lange Form, zweites Wort
          $ELSE
          dw 011                  ; kurze Form
          $ENDIF
          MEND
          start
          000 haupt               ; Zelle 0000: Sprung zum Hauptprogramm
haupt:    164 0001                ; Zelle 0001: Diagnoseausgabe (nur im Test)
          020 0                   ; Zelle 0002: Halt
anf:      dw 0                    ; Zelle 0003
          WEIT anf ende           ; Zellen 0004 und 0005 (lange Form)
          db 4                    ; vier Zellen, dezimal 6 bis 9
ende:     dw 9                    ; oktal 012 (dezimal 10)
          end
```

Probelauf erkennen (`$PROBE`: `$PROBE` ergibt 1, solange `adr` und `abstand` noch keine endgültigen Werte liefern, und sonst 0. Es steht ohne Klammern und ohne Argument und wird wie `$V(...)` (4.7) und `$DEFINED` (4.25) durch die Zahl ersetzt, überall und zum selben Zeitpunkt wie diese. Der Wert ist 1 in einem Probelauf für `adr` und `abstand` (Lauf 1 bis 8; Lauf 1 rechnet mit dem Abstand 0, die späteren mit der Tabelle des vorigen Laufs) und bei `--pass0`, und 0 im eigentlichen Lauf mit der fertigen Tabelle, auch in einem Quelltext, der `adr` und `abstand` nicht benutzt. Es gibt nur 0 oder 1, keine Nummer des Probelaufs. Damit braucht ein Makro für den Wert 0 keinen Sonderfall im Rechenausdruck: Eine Prüfung, die nur mit echten Adressen gilt, steht in `$IF $PROBE = 0` (Beispiel 7), und `$IF $PROBE = 1` wählt, was nur ein Probelauf erzeugt.

Regeln zu `$PROBE`: Hinter dem Wort darf kein Buchstabe, keine Ziffer und kein _ stehen; `$PROBEX`, `$PROBE1` und `$PROBE_` bleiben unersetzt stehen. `$PROBE(` mit Klammer ist ein Fehler („$PROBE steht ohne Klammern und ohne Argument“, Rückgabecode 16). PROBE ist ein festes Wort (4.26) und als Parametername verboten, auch klein geschrieben; ein Makro mit dem Namen PROBE (Beispiel 1) bleibt möglich, sein Name hat mit `$PROBE` nichts zu tun. Folge für Makros: Die Zahl der erzeugten Zellen darf nicht von `$PROBE` abhängen, nur der Inhalt. Sonst verschieben sich die Marken

zwischen Probelauf und eigentlichem Lauf, und die Prüfung nach Pass 2 meldet die Adressabweichung (Negativtest `makro_probe_zellenzahl`).

Beispiel 7: Prüfung nur im eigentlichen Lauf. CHK verlangt einen Abstand von mindestens 1 zwischen zwei Marken, prüft das aber nur, wenn `$PROBE` 0 ist; im Probelauf 1 ist der Abstand 0, und ohne diese Unterscheidung bräche der Probelauf mit `$ERROR` ab („Folge von Probelauf 1“ oben). PW legt den Wert von `$PROBE` selbst ab, WAHL erzeugt in beiden Läufen genau ein Wort (im Probelauf 111, im eigentlichen Lauf 222, oktal 336), SUMME rechnet `$CALC($PROBE+5)`. Adressen dezimal: 0 Sprung, 1 a, 2 CHK, 3 b, 4 Halt, 5 PW, 6 WAHL, 7 SUMME. Im eigentlichen Lauf ist abstand(a,b) = 3 - 1 = 2, PW = 0, WAHL = 336 oktal und SUMME = 5; zwei Probeläufe (Lauf 2 liefert dieselben Adressen wie Lauf 1).

```
CHK:     MACRO 2
          $IF $PROBE = 0
          $IF $CALC(abstand($1,$2)) < 1
          $ERROR Abstand kleiner 1
          $ENDIF
          $ENDIF
          dw $CALC(abstand($1,$2))            ; Abstand der beiden Marken
          MEND
PW:      MACRO 0
          dw $PROBE                           ; im eigentlichen Lauf 0
          MEND
WAHL:    MACRO 0
          $IF $PROBE = 1
          dw 111                              ; nur im Probelauf
          $ELSE
          dw 222                              ; im eigentlichen Lauf (oktal 336)
          $ENDIF
          MEND
SUMME:   MACRO 0
          dw $CALC($PROBE+5)                  ; im eigentlichen Lauf 5
          MEND
          start
          000 haupt                           ; Zelle 0000: Sprung zum Hauptprogramm
a:       dw 0                                ; Zelle 0001
          CHK a b                             ; Zelle 0002: abstand(a,b) = 2
b:       dw 9                                ; Zelle 0003
haupt:   020 0                               ; Zelle 0004: Halt
          PW                                  ; Zelle 0005: 0
          WAHL                                ; Zelle 0006: 336 oktal
          SUMME                               ; Zelle 0007: 5
          end
```

Festlegungen zu `adr` und `abstand`: Pass 0 soll Labeladressen, Labelabstände und Sprungweiten erfahren (Lesart A: Probeläufe Pass 0 bis 2, danach Pass 0 erneut mit den Adressen); höchstens 8 Zyklen, keine Konvergenz ist ein Fehler; mit `--pass0` wird die Funktion ohne Meldung ignoriert (Wert 0). Entwurf B (Probeläufe als Kindprozesse mit demselben Binary) statt eines Zurücksetzens aller Zustände im selben Prozess: Der frische Zustand jedes Laufs ist garantiert, ein vergessenes Zurücksetzen kann nichts falsch rechnen. Annahmen der Umsetzung: (1) `adr` liefert die Adresse wie die `.sym`-Datei (relativ, ohne Ladeadresse); (2) `abs`- und `equ`-Namen sind keine Marken (Fehler); (3) Literalmarken sind ausgeschlossen; (4) die Tabelle der Probeläufe enthält jede Marke der Symboltabelle, auch die von Makros erzeugten; (5) die Konvergenz ist die Gleichheit zweier aufeinanderfolgender Tabellen.

Stand der Prüfung von `adr` und `abstand`: Die Programme `Claude/makroadr` (Beispiel 5, zwei Probeläufe) und `Claude/makroadr_zyklus` (Beispiel 6, drei Probeläufe, Zyklus kurz zu lang) laufen mit c82ass 1.01-018 mit „Test wahr“; die Sollwerte sind vor dem Lauf von Hand hergeleitet. Im Regressionstest stehen beide Programme (Referenz neu) und sechs Negativtests: `makro_adr_fehlt`, `makro_adr_abstand_ein`, `makro_adr_zwei_namen`, `makro_adr_keine_konvergenz` (Abstand wechselt 7, 6, 7, 6 …: Fehler nach 8 Probeläufen), `makro_adr_probelauf_fehler` (`$ERROR` im Probelauf 1) und `makro_adr_pass0` (Wert 0 ohne Meldung). Der Regressionstest einschließlich AddressSanitizer und UBSan läuft mit c82ass 1.01-019 ohne Abweichung (270 Programme und 418 Negativtests); der Negativtest makro_adr_keine_konvergenz hält auch den Verlauf und den Zyklus fest. Die Adressart wurde mit einem Probeprogramm (`start 020`) geprüft. Offen: Das Windows-Verhalten (Anführungszeichen der Kommandozeile, system()) ist nicht geprüft.

Stand der Prüfung von `$PROBE`: Das Programm `Claude/makroprobe` (Beispiel 7, zwei Probeläufe) läuft mit c82ass 1.01-020 mit „Test wahr“; die Sollwerte sind vor dem Lauf von Hand hergeleitet (BZ 0005; Zellen 0002, 0003 und 0005 bis 0007). Im Regressionstest stehen das Programm (Referenz neu) und sechs Negativtests: `makro_probe_klammern` (`$PROBE()`), `makro_probe_wortgrenze` (`$PROBEX`, `$PROBE1` und `$PROBE_` bleiben stehen), `makro_probe_pass0` (mit `--pass0` ist `$PROBE` gleich 1), `makro_probe_zellenzahl` (Zellenzahl hängt von `$PROBE` ab: Adressfehler nach Pass 2), `makro_probe_echt_fehler` (`$ERROR` aus dem eigentlichen Lauf, nicht aus einem Probelauf)

und `makro_probe_parametername` (festes Wort). Der Regressionstest einschließlich AddressSanitizer und UBSan läuft mit c82ass 1.01-020 ohne Abweichung (271 Programme und 424 Negativtests). Offen: Das Windows-Verhalten der Probeläufe (Anführungszeichen der Kommandozeile, system()) ist nicht geprüft.

4.24 Hilfen zur Fehlersuche im Makro (`$NOTE`, `$TRACE`, `$DUMP`)

Zweck: In `$WHILE`-Schleifen (4.19) mit lokalen Variablen (4.20), Feldern (4.17) und Texten (4.12, 4.21) ließ sich der Stand einer Variablen an einer bestimmten Stelle bisher nur nachrechnen oder raten. `$ERROR` (4.5) bricht den Lauf ab, `$WARN` ist eine Warnung; eine Ausgabe, die weder das eine noch das andere ist, gab es nicht. Drei Anweisungen schließen die Lücke: `$NOTE` gibt einen Text aus, `$DUMP` gibt den Stand von Variablen und Feldern aus, `$TRACE` protokolliert den Ablauf eines Makroaufrufs Zeile für Zeile mit den Werten, die tatsächlich gerechnet wurden. Alle drei beobachten nur: Sie erzeugen keinen Code, ändern keine Variable, zählen nicht als Warnung und lassen den Rückgabecode unberührt. Objektdatei, `.mqo`, `.sym`, `.log`, `.test`, `.dmp` und Listing bleiben bitgleich zu einem Lauf ohne diese Zeilen, bis auf die Quelltextzeilennummern, die sich mit den zusätzlichen Zeilen verschieben: Am Beispiel `SUMME` (mit `$TRACE ON`, `$NOTE` und `$DUMP` und ohne diese Zeilen) unterscheidet sich allein die Zeile des Aufrufs im Kommentar `;+++ Makro <Name> aus Quelltext, Zeile <n>, aufgerufen in Quelltextzeile <m>` der `.dmp` und des Listings; `.obj`, `.mqo`, `.sym`, `.log` und `.test` sind dort gleich.

Umsetzungsstand: Umgesetzt ab c82ass 1.01-005 (Quelltext `ca82_makros.c`). c82ass bis 1.01-004 überspringt die drei Anweisungen in einem Makro ohne Meldung. Ab 1.01-005 ersetzen auch `$WARN` und `$ERROR` das `$V(...)` im Text (4.5). Beide Beispiele dieses Abschnitts sind mit 1.01-005 gelaufen; die Ausgabe stimmt Zeichen für Zeichen mit dem Text überein. Den Stand der Prüfung im Regressionstest nennt der Absatz „Prüfung“ weiter unten.

Gemeinsame Regeln: Jede der drei Anweisungen steht auf einer Zeile für sich, wirkt nur im erzeugten Zweig (nicht in einem nicht erzeugten `$IF`-Zweig und nicht hinter `$EXIT`) und wird in der Reihenfolge der erzeugten Zeilen ausgeführt, auch über Aufrufgrenzen hinweg (4.15). In einer Schleife (`$REPT`, `$WHILE`) erscheint die Ausgabe je Durchlauf; in einem `$REPT`-Rumpf sind `$Z1` bis `$Z4` ersetzt wie in jeder Zeile. Ab c82ass 1.01-008 gelten sie auch im Quelltext außerhalb eines Makros (4.1): `$NOTE` und `$DUMP` melden dort „Hinweis: Quelltext (<Datei>:<Zeile>): ...“, `$WARN` „Warnung: Quelltext (Zeile <n>): $WARN ...“; `$TRACE ON` und `$TRACE OFF` bleiben ohne Wirkung. Jede Ausgabe erscheint genau einmal je Ausführung der Zeile, auch wenn der Assembler den Makrotext für andere Zwecke mehrfach durchläuft.

Ausgabe: Sie geht auf STDERR, also in den Strom der Warnungen und Fehlermeldungen, und steht mit ihnen in der Reihenfolge der Ausführung. Die Farbe ist die der übrigen Meldungen (Hinweise grün); mit `--no-color` entfallen die Farbcodes. Die Ausgabe steht nicht im Listing, nicht in `.dmp`, `.log` und `.test`. Die Zeilen von `$NOTE` und `$DUMP` beginnen mit `Hinweis: Makro <Name> (<Datei>:<Zeile>): ` ; `<Name>` ist das Makro, in dem die Zeile steht, `<Datei>` der Name der Quelltextdatei wie in den Fehlermeldungen und `<Zeile>` die Quelltextzeile des äußersten Aufrufs (bei geschachtelten Aufrufen nicht die des inneren), wie bei den Makrofehlern (4.5). Die Warnung von `$WARN` hat wie bisher keine Quelltextzeile (`Warnung: Makro <Name>: $WARN <text>`, 4.5). Die Zeilen von `$TRACE` beginnen mit `Spur` (siehe unten). In den Probeläufen für `adr` und `abstand` (4.23) erscheint sie nur mit `--probeausgabe` (Absatz „Probeläufe“ unten).

Probeläufe (`adr`, `abstand`; Schalter `--probeausgabe`): `adr` und `abstand` (4.23) lassen den Assembler Probeläufe als Kindprozesse starten. Deren `$NOTE`, `$TRACE` und `$DUMP` erscheinen normalerweise nicht, nur die des eigentlichen Laufs; ein Makro, das man im Probelauf verfolgen will, bleibt dort stumm. Mit `--probeausgabe` (Groß- und Kleinschreibung egal, auch `--PROBEAUSGABE`) werden sie sichtbar. Anzeige: Jede Zeile steht auf STDERR mit „Probelauf <n>: “ davor (`<n>` ist die Nummer des Probelaufs), in der Reihenfolge der Läufe, also erst alle Zeilen von Probelauf 1, dann die von Probelauf 2 und so weiter, danach die des eigentlichen Laufs ohne Vorsatz. Mit Farbe sind die Zeilen grün wie die übrigen Hinweise, mit `--no-color` ohne Farbcodes; mehrzeilige Ausgaben wie ein `$DUMP` tragen den Vorsatz an jeder Zeile. Fehlerfall: Schlägt ein Probelauf fehl, stehen seine Debugzeilen vor der Meldung „Probelauf <z> fuer adr/abstand fehlgeschlagen (Rueckgabewert <rc>).“ und erscheinen im Auszug des Fehlerprotokolls nicht noch einmal. Technik: Der Aufrufer hängt an jeden Probelauf die versteckte Option `--adrdbg=<n>.adrdbg` an (nur für diesen Selbstaufruf, nicht für den Aufruf von Hand; sie gilt nur zusammen mit `--adraus=`, sonst Fehler „Die Option --adrdbg= gilt nur zusammen mit --adraus=.“, Rückgabecode 16). Der Probelauf legt die Zeilen von `$NOTE`, `$TRACE` und `$DUMP` in dieser Zwischendatei ab, statt sie auf STDERR zu schreiben, und der Aufrufer zeigt die Datei nach dem Lauf; `<n>.adrdbg` wird vor jedem Lauf und am Ende gelöscht. Mit `--pass0` gibt es keine Probeläufe; der Schalter ist dort ohne Wirkung (3.1d). Ohne `--probeausgabe` bleibt alles wie bisher. `--probeausgabe` schaltet keine Spur ein: Spurzeilen gibt es weiter nur durch `$TRACE ON` im Makro.

`$NOTE <text>` gibt den Text aus. Schreibweise und Ersetzung wie bei `$WARN` (4.5): Der Text reicht bis zum Zeilenende (auch ein ; gehört dazu), `$1` bis `$9`, `$Z1` bis `$Z4` und `$CALC(...)` sind ersetzt. Zusätzlich ist `$V(...)` in allen Formen ersetzt (4.7, 4.12, 4.17, 4.20), mit dem Stand an der Stelle der Zeile, zum Beispiel

`$NOTE i=$V(i) s=$V(s) q3=$V(q(3))`; das ergibt `Hinweis: Makro SUMME (t.asm:12): i=3 s=6 q3=9`. Ein leerer Text ist erlaubt. Wirksam sind höchstens 255 Zeichen Text (nicht 300): Eine Quelltextzeile über 255 Zeichen lehnt der Assembler schon beim Lesen ab. Wird die Zeile erst durch die Ersetzung von `$1` bis `$9`, `$Z1` bis `$Z4` und `$CALC(...)` länger, wird sie ohne Fehler bei 255 Zeichen abgeschnitten; vom Text bleiben dann 249 Zeichen, weil die Zeile mit `$NOTE ` beginnt. Wird der Text erst durch `$V(...)` länger als 255 Zeichen, ist das ein Fehler: `$NOTE: $V(t): Text passt nicht in die Zeile` (Rückgabecode 16). Eine `$V(...)`-Angabe, die sich nicht auswerten lässt (nicht gesetzte Variable, Index außerhalb des Feldes), ist wie überall ein Fehler (Rückgabecode 16). `$NOTE` zählt nicht als Warnung und lässt den Rückgabecode bei 0. Zugleich ersetzen `$WARN` und `$ERROR` ab c82ass 1.01-005 `$V(...)` im Text auf dieselbe Weise (bis 1.01-004 blieb `$V(i)` dort wörtlich stehen); nur Texte, die `$V(` enthalten, ändern sich. Ist die Angabe nicht auswertbar oder der Text zu lang, lautet der Fehler `$WARN: <grund>` beziehungsweise `$ERROR: <grund>` (Rückgabecode 16).

`$DUMP [<ziel> ...]` gibt den Stand von Variablen und Feldern aus. Ein Ziel ist ein Name (eine einfache Variable oder ein ganzes Feld) oder ein Feldelement `name(index)` (Index wie in 4.17, ein Rechenausdruck ohne Leerzeichen); die Ziele stehen durch Leerzeichen oder Tabulatoren getrennt, ein Kommentar mit ; dahinter ist erlaubt. Ein Feldelement erscheint mit dem ausgewerteten Index: `$DUMP q(1+1)` gibt `q(2) = ...` aus. Gibt es einen Namen lokal und global, zeigt ein Ziel die lokale, also die sichtbare Variable. Ohne Ziel gibt `$DUMP` alles aus, was an dieser Stelle sichtbar ist: zuerst die lokalen Variablen und Felder des laufenden Aufrufs in der Reihenfolge ihrer Vereinbarung (4.20), dann die globalen in der Reihenfolge ihrer Anlage (Variablen und Felder gemischt, wie sie vereinbart wurden); eine globale, die eine lokale gleichen Namens verdeckt, erscheint mit dem Zusatz `verdeckt`, damit sich Verwechslungen finden lassen. Die Kopfzeile lautet `Hinweis: Makro <Name> (<Datei>:<Zeile>): $DUMP`, darunter steht je Variable eine Zeile `  <name> = <wert>  (<art>)` (zwei Leerzeichen davor und zwischen Wert und Klammer). Die Art ist `lokal`, `global` oder `global, verdeckt`; bei einer einfachen Variablen und bei einem einzelnen Feldelement, das einen Text enthält, steht zusätzlich `Text` (zum Beispiel `global, Text`); ein Textwert steht in Anführungszeichen. Ein ganzes Feld erscheint als `<name>(<anzahl>) = <e0> <e1> ...` mit höchstens 16 Elementen je Zeile; ein Text als Element steht in Anführungszeichen, ohne den Zusatz `Text`. Die Folgezeilen beginnen mit `<name>(16..31) =` und so weiter (hat die letzte Zeile nur ein Element, steht dort `<name>(32..32)`), die Art steht nur in der letzten Zeile des Feldes. Ein Name, den es nicht gibt, ist kein Fehler: Die Zeile lautet `  <name> = <nicht definiert>`. Gibt es ohne Ziel nichts auszugeben, lautet die Zeile darunter `  (keine Variablen)`.

`$TRACE ON` und `$TRACE OFF` schalten die Ablaufspur ein und aus. Sie gilt ab dieser Zeile für den laufenden Makroaufruf und für alle Makros, die aus ihm heraus aufgerufen werden, und endet mit `$TRACE OFF` oder mit dem Ende des Aufrufs, der `$TRACE ON` ausgeführt hat (`MEND`, `$EXIT`, Fehler). Eine vergessene Spur bleibt also nicht für den Rest des Programms eingeschaltet. Ein `$TRACE OFF` in einem aufgerufenen Makro schaltet die Spur insgesamt aus, also auch für den Rest des aufrufenden Makros. `$TRACE ON` bei eingeschalteter und `$TRACE OFF` bei ausgeschalteter Spur bleiben ohne Wirkung; geschachtelt gezählt wird nicht. `$TRACE` und `ON` oder `OFF` schreibt man in Großbuchstaben; dahinter darf nur ein Kommentar mit ; stehen.

Spurzeilen: Für jede ausgeführte Zeile eines verfolgten Aufrufs erscheint eine Zeile `Spur <Makro> <nr>: <zeile> => <wirkung>`. `<nr>` zählt die gespurten Zeilen des Aufrufs ab 1 (die Zeile `$TRACE ON` zählt nicht mit). `<zeile>` ist die Zeile des Makros nach der Ersetzung von `$1` bis `$9`, `$Z1` bis `$Z4`, `$CALC(...)` und `$V(...)`, ohne Kommentar und mit einem Leerzeichen als Trenner; so stehen dort die Werte, mit denen wirklich gerechnet wurde (höchstens 255 Zeichen, bei `erzeugt` höchstens 300). Aufrufe verschachtelter Makros erscheinen um zwei Leerzeichen je Aufruftiefe eingerückt (der äußerste verfolgte Aufruf ohne Einrückung). Zeilen in nicht erzeugten Zweigen und `$ENDIF` erscheinen nicht, ebenso `$NOTE`, `$DUMP`, `$WARN` und `$ERROR`, die ihre eigene Ausgabe haben, sowie Kommentarzeilen, Leerzeilen und die `MEND`-Zeile. Schlägt eine Anweisung fehl, erscheint für sie keine Spurzeile, nur die Fehlermeldung; sie steht wie alle Makrofehler erst nach den Spurzeilen des Aufrufs. Die Wirkung je Zeile:

`$SET`, `$INC` und die Anweisungen aus 4.21 und 4.22, die ein Ziel setzen: `<ziel>: <alt> -> <neu>`, bei einer lokalen Variablen mit dem Zusatz `  (lokal)`; ein Text steht in Anführungszeichen, ein bisher nicht vorhandenes Ziel hat den alten Wert `(neu)`. `$LOCAL`: `lokal vereinbart: <namen>` (ein Feld als `name(anzahl)`). `$DIM`: `Feld <name>(<anzahl>) vereinbart`. `$IF` und `$ELIF`: `wahr` oder `falsch` (`$ELIF` nur, wenn seine Bedingung ausgewertet wurde); `$ELSE`: `betreten`, wenn der Zweig erzeugt wird, sonst keine Zeile. `$WHILE`: `wahr (Durchlauf <n>)` oder `falsch`. `$ENDWHILE`: Die Zeile zeigt die erneut ausgewertete Bedingung des `$WHILE` mit den jetzigen Werten und die Wirkung `wahr (Durchlauf <n>)` oder `falsch (Ende nach <n> Durchläufen)`. `$EXIT`: `Makro beendet`. Eine erzeugte Anweisung steht ohne `=>` als `Spur <Makro> <nr>: erzeugt <zeile>` (die Zeile, wie sie in den Quelltext kommt, mit Marke); der Aufruf eines Makros als `ruft <Name> <parameter>` (nur für ein vorhandenes Makro, die Zeile zählt zum Aufrufer), danach folgt dessen Spur eine Stufe eingerückt.

Grenzen: Je Lauf erscheinen höchstens 10000 Spurzeilen, gezählt über alle Aufrufe; fällt eine 10001. Spurzeile an, folgt statt ihrer einmal die Zeile `Hinweis: Ablaufspur nach 10000 Zeilen abgebrochen, weitere Spurzeilen entfallen,`

und der Lauf geht ohne Fehler weiter. `$NOTE` und `$DUMP` sind davon nicht betroffen. Ein `$DUMP`-Textelement und ein Text in der Wirkung einer Spurzeile haben höchstens 200 Zeichen (4.17).

Fehler (Rückgabecode 16, die Meldung nennt das Makro und die Quelltextzeile des Aufrufs): `$TRACE` ohne zweites Wort, mit einem anderen Wort als `ON` oder `OFF` (auch in Kleinbuchstaben) oder mit etwas anderem als einem Kommentar dahinter (Meldung `$TRACE: ON oder OFF erwartet`); in `$DUMP` ein ungültiger Name (Regeln wie in 4.7; Meldung zum Beispiel `$DUMP: Variablenname 1abc ungültig`), eine fehlende oder überzählige Klammer (`$DUMP: q(: Klammer ) fehlt`), ein leerer Index, ein Index kleiner als 0 oder nicht kleiner als die Anzahl, ein Index bei einer einfachen Variablen (`$DUMP: a ist eine einfache Variable, ein Index ist nicht erlaubt`); in `$NOTE` eine `$V(...)`-Angabe, die sich nicht auswerten lässt (`$NOTE: <grund>`), und ein durch `$V(...)` zu langer Text. Bei einem Fehler in `$DUMP` erscheint keine Zeile des Standes, denn alle Ziele werden vor der Ausgabe geprüft.

Beispiel zu `$TRACE`: `SUMME` bildet die Summe 1 bis `$1` (das Makro aus 4.20, hier mit eingeschalteter Spur). Der Aufruf `SUMME 2` erzeugt das Wort 3 und gibt auf STDERR zehn Spurzeilen aus:

```
SUMME:     MACRO 1
           $TRACE ON
           $LOCAL i s
           $SET i 1
           $WHILE $V(i) <= $1
           $INC s $V(i)
           $INC i
           $ENDWHILE
           dw $V(s)                  ; Summe 1 bis $1
           MEND
```

```
Spur SUMME 1: $LOCAL i s => lokal vereinbart: i s
Spur SUMME 2: $SET i 1 => i: 0 -> 1 (lokal)
Spur SUMME 3: $WHILE 1 <= 2 => wahr (Durchlauf 1)
Spur SUMME 4: $INC s 1 => s: 0 -> 1 (lokal)
Spur SUMME 5: $INC i => i: 1 -> 2 (lokal)
Spur SUMME 6: $ENDWHILE 2 <= 2 => wahr (Durchlauf 2)
Spur SUMME 7: $INC s 2 => s: 1 -> 3 (lokal)
Spur SUMME 8: $INC i => i: 2 -> 3 (lokal)
Spur SUMME 9: $ENDWHILE 3 <= 2 => falsch (Ende nach 2 Durchläufen)
Spur SUMME 10: erzeugt dw 3
```

Beispiel zu `$DUMP` und `$NOTE`: `STAND` legt eine globale Variable `zahl`, ein globales Feld `q` und einen globalen Text `t` an, vereinbart dann lokal `zahl` und `i` und gibt zweimal den Stand aus. Wird `STAND` in Quelltextzeile 20 von `t.asm` aufgerufen, erscheint auf STDERR die Ausgabe darunter (die erste `$DUMP`-Zeile ohne Ziel, die zweite mit den Zielen `q(2)` und `x`, dann die `$NOTE`-Zeile):

```
STAND:     MACRO
           $SET zahl 12
           $DIM q 4
           $SET q(2) 9
           $SET t "abc"
           $LOCAL zahl i
           $SET zahl 5
           $SET i 3
           $DUMP
           $DUMP q(2) x
           $NOTE zahl=$V(zahl) i=$V(i) q2=$V(q(2))
           MEND
```

```
Hinweis: Makro STAND (t.asm:20): $DUMP
  zahl = 5  (lokal)
  i = 3  (lokal)
  zahl = 12  (global, verdeckt)
  q(4) = 0 0 9 0  (global)
  t = "abc"  (global, Text)
Hinweis: Makro STAND (t.asm:20): $DUMP
  q(2) = 9  (global)
  x = <nicht definiert>
Hinweis: Makro STAND (t.asm:20): zahl=5 i=3 q2=9
```

Prüfung: Die Ausgabe prüfen Tests im Stil der Negativtests des Regressionstests (`.erwartet` mit `rc` und den Zeilen `stderr`, `stderr-reihe`, `stderr-einrueckung`, `anzahl-stderr` und `nicht-stderr`; `stderr-reihe` verlangt, dass die Texte in dieser Reihenfolge in STDERR stehen, `stderr-einrueckung <n> <text>` eine ganze STDERR-Zeile aus n Leerzeichen und dem Text, damit auch die Einrückung der Spurzeilen geprüft wird, siehe `Regressionstest/LIESMICH.md`; Sollwerte aus den Beispielen dieses Abschnitts und von Hand hergeleitet). Vorhanden sind 25 Tests. Achtzehn davon betreffen `$NOTE`, `$DUMP`, `$V(...)` in `$WARN` und `$ERROR` und die Fehlerfälle von `$TRACE`: `makro_note` (mit `$REPT`, `$EXIT`, einem

verschachtelten Aufruf und einem durch `$1$1` zu langen Text), `makro_warn_var` und `makro_error_var` (`$V(...)` in `$WARN` und `$ERROR`), `makro_dump`, `makro_dump_feld` (Feld über 16 Elemente) und `makro_dump_leer` (ohne Variablen) sowie die Fehlerfälle (Rückgabecode 16) `makro_note_var`, `makro_dump_name`, `makro_dump_klammer`, `makro_dump_klammer_zu`, `makro_dump_index`, `makro_dump_index_neg`, `makro_dump_index_leer`, `makro_dump_index_einfach`, `makro_trace_ohne_wort`, `makro_trace_wort`, `makro_trace_klein` und `makro_trace_rest`. Sieben weitere prüfen die Ablaufspur von `$TRACE`: `makro_trace` (das Beispiel `SUMME` dieses Abschnitts mit seinen zehn Spurzeilen, geprüft mit `stderr-reihe`), `makro_trace_schachtel` (verschachtelter Aufruf: Einrückung um zwei Leerzeichen, geprüft mit `stderr-einrueckung`, `ruft` vor der Spur des Aufgerufenen, die Spur endet am Ende des Aufrufs, der sie eingeschaltet hat), `makro_trace_aus` (`$TRACE OFF` in einem aufgerufenen Makro schaltet die Spur insgesamt aus, auch für den Rest des aufrufenden Makros; die Zeile des Aufrufs steht noch in der Spur), `makro_trace_doppelt` (`$TRACE ON` bei eingeschalteter und `$TRACE OFF` bei ausgeschalteter Spur bleiben ohne Wirkung), `makro_trace_fehler` (für eine fehlgeschlagene Anweisung gibt es keine Spurzeile, die Fehlermeldung steht nach den Spurzeilen des Aufrufs, die Zeilen dahinter werden weiter gespurt; Rückgabecode 16), `makro_trace_grenze` (vier Aufrufe zu je 3000 gespurten Zeilen: genau 10000 Spurzeilen und danach einmal ein Hinweis, der Lauf geht ohne Fehler weiter) und `makro_trace_zweige` (Spurzeilen von `$IF`, `$ELIF`, `$ELSE`, `$LOCAL` mit Feld, `$DIM`, `$UPPER`, `$SUBSTR`, `$EXIT` und einer erzeugten Zeile mit Marke). Das Programm `Claude/makrospur` mit dem Gegenstück `makrospur_ohne` prüft dieselben Anweisungen im Zusammenspiel mit dem Assembler: Es ruft die Makros `SUMME` (mit `$TRACE ON`, `$NOTE` und `$DUMP`), `TABELLE` (ruft `SUMME`), `FELDTEXT` und `ZWEIG` auf; `makrospur_ohne` hat dieselben 66 Zeilen, nur sind die Zeilen mit `$TRACE`, `$NOTE` und `$DUMP` Kommentarzeilen. Beide Programme enden im Interpreter mit „Test wahr“ (Sollwerte von Hand hergeleitet: Zellen 1 bis 3 gleich 1, 3, 6, Zelle 4 gleich 9, Zelle 5 gleich 5, BZ gleich 0010 oktal). Unter demselben Programmnamen in zwei gleich aufgebauten Ordnern assembliert, sind `.log`, `.mqo`, `.obj`, `.sym`, `.dmp` und das Listing auf STDOUT bitgleich; verschieden ist allein die `.test`, und zwar nur durch ihre eigene Zeile `#DOK-0001`. Die Gleichheit von `.dmp` und Listing gilt, solange die Zeilenzahl gleich bleibt (Zeilen auskommentieren statt löschen); würden die Zeilen gelöscht, verschöbe sich nur die Zeilennummer des Aufrufs im Kommentar `;+++ Makro ...` (siehe oben). Der Lauf durch `regressionstest.sh` (1.00-068) mit c82ass 1.01-005: Die 255 bisherigen Programme blieben bitgleich zu ihrer Referenz; mit der Referenz für `makrospur` und `makrospur_ohne` sind es 257 Programme ohne Abweichung und 306 Negativtests ohne Fehler. `pruefe_alles.sh` einschließlich der Läufe unter AddressSanitizer und UBSan endet mit „ALLES IN ORDNUNG“.

Entschieden: Namen `$NOTE`, `$TRACE`, `$DUMP`; Ausgabe auf STDERR wie `$WARN` und nicht in `.log`, Listing und `.dmp`; `$NOTE` ohne Wirkung auf Warnungen und Rückgabecode; Spur nur durch `$TRACE ON` im Quelltext (kein Kommandozeilenschalter, kein Filter nach Namen) und begrenzt auf den Aufruf, der sie einschaltet; ein unbekannter Name in `$DUMP` ist kein Fehler; `$V(...)` wird ab 1.01-005 auch in `$WARN` und `$ERROR` ersetzt; wirksam höchstens 255 Zeichen je Text (am 04.11.2026 entschieden: die Dokumentation folgt dem Code, ein Abschneiden bei 300 Zeichen wird nicht eingebaut). Nicht vorgesehen: das Ausgeben der Aufrufparameter durch `$DUMP` (sie stehen in `$NOTE` und in der Spur schon ersetzt) und das Anhalten an einer Stelle. Entschieden: Ein Schalter `--probeausgabe` statt einer Anweisung im Quelltext; Ausgabe auf STDERR mit „Probelauf <n>: “ vor jeder Zeile; ohne den Schalter bleibt alles wie bisher. Annahmen der Umsetzung: (1) der Schalter gilt für alle Probeläufe eines Laufs; (2) die Zwischendatei `<n>.adrdbg` statt einer Pipe, damit der Aufrufer die Zeilen erst nach dem Lauf und in der Reihenfolge der Läufe zeigt; (3) mit `--pass0` ohne Wirkung.

Stand der Prüfung von `--probeausgabe` (c82ass 1.01-021): Im Regressionstest stehen sieben Negativtests. `makro_probeausgabe` prüft mit `$NOTE`, `$TRACE` und `$DUMP` in einem Makro, dass die Zeilen beider Probeläufe mit „Probelauf <n>: “ davor in der Reihenfolge der Läufe stehen, dass der eigentliche Lauf seine Zeilen ohne Vorsatz zeigt und dass `<n>.adr1`, `<n>.adr2`, `<n>.adrlog` und `<n>.adrdbg` danach gelöscht sind. `makro_probeausgabe_aus` hält fest, dass die Probeläufe ohne den Schalter unsichtbar bleiben. `makro_probeausgabe_gross` prüft `--PROBEAUSGABE`. `makro_probeausgabe_farbe` prüft, dass die Zeile mit Farbe ein Escape-Zeichen enthält (es stammt vom Aufrufer, die Datei des Probelaufs ist farblos). `makro_probeausgabe_fehler` lässt Probelauf 1 durch `$ERROR` scheitern: Die Debugzeile steht vor der Fehlermeldung und genau einmal (Rückgabecode 16). `makro_probeausgabe_pass0` prüft, dass der Schalter mit `--pass0` ohne Wirkung ist (`$PROBE` bleibt 1, der Text steht ohne Kopf auf STDOUT). `makro_probeausgabe_adrdbg` prüft, dass `--adrdbg=` ohne `--adraus=` ein Fehler ist (Rückgabecode 16). Die Erwartung der Versionstests `version.erwartet` und `pass0_version.erwartet` ist auf 1.01-021 gesetzt. `pruefe_alles.sh --ohne-asan` meldet 271 Programme und 431 Negativtests ohne Abweichung. Der Regressionstest einschließlich AddressSanitizer und UBSan läuft mit c82ass 1.01-021 ohne Abweichung (271 Programme und 431 Negativtests; `pruefe_alles.sh` endet mit „ALLES IN ORDNUNG“). Offen: Das Windows-Verhalten der Probeläufe (Anführungszeichen der Kommandozeile, auch für `--adrdbg=`, system()) ist nicht geprüft.

4.25 Existenzabfrage für Variablen, Felder und Makros (`$DEFINED`, `$MDEFINED`)

Zweck: Eine nicht gesetzte Variable in `$V(name)` ist ein Fehler (4.7), ebenso ein nicht vereinbartes Feld (4.17), und eine Zeile, die ein Makro aufruft, das es nicht gibt, ist es auch. Ein Makro kann deshalb bisher nicht „setze nur, wenn noch nicht gesetzt“ ausdrücken, nicht auf einen Wert reagieren, den der Aufrufer nur manchmal setzt, und nicht ein Makro aufrufen, das nur manche Makrodateien (4.9) liefern. `def(name)` (4.23) hilft dabei nicht: Es fragt nur Symbole des Quelltexts (Marken, `abs`, `equ`), weder Makro-Variablen noch Felder noch Makros. Zwei Funktionen schließen die Lücke: `$DEFINED(...)` fragt Variablen und Felder, `$MDEFINED(name)` fragt Makros. Beide wirken nur zur Assemblerzeit, erzeugen keinen Code und ändern nichts; im erzeugten Quelltext steht allein das Ergebnis. Quelltexte, die sie nicht benutzen, liefern unverändert dieselben Ergebnisse.

Umsetzungsstand: Umgesetzt ab c82ass 1.01-006 (festgelegt am 04.11.2026). Ein älteres c82ass kennt `$DEFINED` und `$MDEFINED` nicht, und das fällt nicht immer auf: Mit 1.01-005 endet `dw $DEFINED(x)` mit Rückgabecode 16 und der Meldung „Operand '$DEFINED(x)' ist ungültig“, aber `$IF $DEFINED(x) = 1` bleibt ohne Meldung, die Bedingung ist falsch, und der `$ELSE`-Zweig entsteht (Rückgabecode 0). Beide Beispiele unten sind mit 1.01-006 gelaufen (Programm Assembler/Claude/makrodefined, siehe Prüfung).

Form und Ergebnis: `$DEFINED(...)` und `$MDEFINED(...)` werden wie `$V(...)` (4.7) durch eine Zahl ersetzt: 1, wenn es das Gefragte gibt, sonst 0. Sie stehen überall, wo `$V(...)` ersetzt wird: in Marke, Operation, Operand und Kommentar der erzeugten Zeilen, in `$CALC(...)` (zum Beispiel `$CALC($DEFINED(a)+$DEFINED(b))`), in den Bedingungen von `$IF`, `$ELIF` und `$WHILE`, in der Anzahl hinter `$REPT`, in den Ausdrücken von `$SET` und `$INC`, in den Argumenten der Anweisungen aus 4.21 und 4.22, im Index eines Feldzugriffs (4.17) und im Text von `$NOTE`, `$WARN` und `$ERROR` (4.5, 4.24); `$TRACE` zeigt in der Spurzeile das ersetzte Ergebnis. In `$P(...)` sind sie wie `$V` nicht erlaubt. Auch der Zeitpunkt ist der von `$V`: Es gilt der Stand an der Stelle der Zeile, in `$IF`, `$ELIF` und `$WHILE` der Stand an der Stelle der Bedingung, in der Anzahl von `$REPT` der Stand zu Beginn des Makroaufrufs (4.7). Außerhalb eines Makros wird nichts ersetzt.

Die Bedingung von `$IF` bleibt dreiteilig (4.4): `$IF $DEFINED(n) = 0` fragt, ob `n` fehlt, `$IF $DEFINED(n) = 1`, ob es da ist. Eine Bedingung aus dem einen Wort `$DEFINED(n)` ist weiterhin ein Fehler.

`$DEFINED(name)` ergibt 1, wenn an dieser Stelle eine einfache Variable oder ein Feld dieses Namens sichtbar ist, sonst 0. Der Name folgt den Regeln von 4.7 (Buchstabe zuerst, dann Buchstaben, Ziffern und _, höchstens 39 Zeichen; Groß- und Kleinschreibung zählen). Sichtbar ist, was auch `$V` fände (4.20): Eine lokale Variable oder ein lokales Feld des laufenden Aufrufs verdeckt die globale gleichen Namens (auch eine Variable ein Feld: Eine lokale einfache Variable verdeckt ein globales Feld gleichen Namens, dann ergibt `$DEFINED(@name)` 0), und eine lokal vereinbarte Variable ist vorhanden (mit dem Wert 0), auch wenn noch kein `$SET` lief. Ein Makro, das aus dem Aufruf heraus aufgerufen wird, sieht die lokalen Variablen des aufrufenden Makros nicht: Dort ergibt `$DEFINED` für sie 0, wo `$V` „nicht definiert“ meldet. Ob die Variable eine Zahl oder einen Text hält, sagt das Ergebnis nicht.

`$DEFINED(@name)` ergibt 1 nur, wenn ein Feld dieses Namens sichtbar ist, und 0 bei einer einfachen Variablen und bei einem unbekannten Namen. Die Schreibweise `@name` ist die von `$V(@name)` (4.17).

`$DEFINED(name(index))` ergibt 1, wenn ein Feld `name` sichtbar ist und der Index von 0 bis Anzahl−1 reicht, sonst 0: auch bei einem Index außerhalb des Feldes, wo `$V(name(index))` ein Fehler wäre, und bei einer einfachen Variablen dieses Namens. Der Index ist ein Rechenausdruck ohne Leerzeichen wie in 4.17, auch mit `$V(...)`: `$DEFINED(q($V(i)))` fragt, ob das Element `i` des Feldes `q` vorhanden ist. Damit liest ein Makro ein Element nur, wenn es das gibt.

`$MDEFINED(name)` ergibt 1, wenn ein Makro dieses Namens bekannt ist, sonst 0. Gesucht wird wie beim Aufruf (4.9): im Quelltext (4.1), in den projektspezifischen Makrodateien und in `c82ass.makros`; auf die Fundstelle kommt es nicht an, und ein Makro des Quelltexts, das ein gleichnamiges Makro einer Datei ersetzt, zählt einmal. Das Ergebnis hängt nicht von der Stelle der Abfrage ab, weil alle Makros vor der Auflösung bekannt sind und eine Definition auch hinter dem Aufruf stehen darf (4.1). Maschinenbefehle, Anweisungen wie `dw` und `equ` und Symbole des Quelltexts sind keine Makros: Ergebnis 0. Der Name steht wie im Aufruf geschrieben (Groß- und Kleinschreibung zählen) und ohne Doppelpunkt. Die Abfrage ist kein Aufruf und erscheint nicht in der Makroliste (4.11).

Argument: `$V(...)` im Argument wird vor der Auswertung aufgelöst, wie bei `def` (4.23): `$DEFINED(a$V(i))` fragt mit `i` gleich 2 nach `a2`; ist `i` selbst nicht gesetzt, ist das ein Fehler und nicht 0. Ein Paar Klammern um das ganze Argument ist erlaubt, weil ein `$1` in `$CALC(...)` eingeklammert eingesetzt wird (4.2): `$DEFINED((x))` ist `$DEFINED(x)`. Ein Name, den es nicht gibt, ist kein Fehler, auch ein Tippfehler im Namen nicht: Das Ergebnis ist 0. `$NOTE` und `$TRACE` (4.24) zeigen das ersetzte Ergebnis.

Welche Frage wohin gehört: Symbole des Quelltexts (Marken, `abs`, `equ`) fragt `def(name)` (4.23), Makro-Variablen und Felder `$DEFINED(...)`, Makros `$MDEFINED(name)`, und ob ein Parameter angegeben wurde, `$#` (4.6, 4.13). Variablen und

Makros haben getrennte Namen: Ein Makro `ZAEHL` und eine Variable `ZAEHL` dürfen nebeneinander bestehen. Deshalb gibt es zwei Funktionen: Eine gemeinsame könnte nicht sagen, welches von beiden gemeint ist, und beantwortete die Frage nach dem Makro mit dem Vorhandensein der Variablen.

Fehler (Rückgabecode 16, die Meldung nennt das Makro und die Quelltextzeile des Aufrufs wie bei den übrigen Makrofehlern): ein leeres Argument; ein `$V(...)` im Argument, das sich nicht auswerten lässt; bei `$DEFINED` ein ungültiger Name (Regeln wie in 4.7), eine fehlende oder überzählige Klammer, ein leerer Index, ein `@` vor `name(index)` und ein Index, der sich nicht auswerten lässt (zum Beispiel wegen eines nicht gesetzten `$V(...)` darin); bei `$MDEFINED` ein Name mit Doppelpunkt oder Leerraum. Kein Fehler ist ein Name, den es nicht gibt.

Beispiel 1, „setze nur, wenn noch nicht gesetzt“: `VORGABE` gibt der Variablen `zaehl` den Wert 100, wenn niemand sie vorher gesetzt hat, und erzeugt ihren Wert; `SIEBEN` setzt sie vorher auf 7. Ein Quelltext, der nur `VORGABE` aufruft, erzeugt `dw 100`; ruft er vorher `SIEBEN` auf, erzeugt `VORGABE` `dw 7`. Ein weiterer Aufruf von `VORGABE` erzeugt denselben Wert noch einmal, denn die Variable ist seit dem ersten Aufruf gesetzt:

```
SIEBEN:   MACRO
          $SET zaehl 7
          MEND
VORGABE:  MACRO
          $IF $DEFINED(zaehl) = 0
          $SET zaehl 100
          $ENDIF
          dw $V(zaehl)            ; 100 oder der vorher gesetzte Wert
          MEND
```

Beispiel 2, ein Makro nur aufrufen, wenn es das gibt: `PRUEFUNG` ruft das Makro `HOOK` auf, wenn der Quelltext oder eine Makrodatei es liefert, und erzeugt in jedem Fall das Wort 7. Der Aufruf `HOOK` im nicht erzeugten Zweig wird nicht ausgeführt, der Name muss dort nicht bekannt sein:

```
PRUEFUNG: MACRO
          $IF $MDEFINED(HOOK) = 1
          HOOK                    ; nur, wenn es das Makro HOOK gibt
          $ENDIF
          dw 7
          MEND
```

Beispiel 3, ein Unterprogramm nur einmal erzeugen: Ein Makro, das den Rumpf eines Unterprogramms anlegt (Festlegung 035 im Programmierhandbuch), darf mehrmals im Quelltext stehen, der Rumpf soll aber nur einmal entstehen. Das Makro merkt sich den ersten Aufruf in einer globalen Variablen. `$GLOBAL` vereinbart sie ausdrücklich als global, sodass das Makro mit und ohne `;#MAKROLOKAL=1` (4.31) gleich wirkt, und legt sie mit 0 an; der erste Aufruf setzt sie auf 1, jeder weitere erzeugt nichts:

```
UPEINMAL:  MACRO
           $GLOBAL upda             ; global, anfangs 0
           $IF $V(upda) = 0
           $SET upda 1              ; ab jetzt ist der Rumpf da
           211 wert                 ; Rumpf des UP (nur beim ersten Aufruf)
           240 ablage               ; Rumpf des UP, zweiter Befehl
           $ENDIF
           MEND
```

Die Marke auf der Aufrufzeile steht nur beim ersten Aufruf auf dem Rumpf. Bei einem späteren Aufruf entsteht nichts, die Marke bekäme die Adresse der nächsten erzeugten Zelle (4.1, „Zum Hilfsmakro“). Alle Aufrufe des Unterprogramms gehen deshalb auf die Marke des ersten Aufrufs; spätere Aufrufe des Makros schreibt man ohne Marke. Dieses Muster braucht c82ass ab 1.02-001 (`$GLOBAL`).

Prüfung: Der Positivtest Assembler/Claude/makrodefined (62 Zellen; die Makrodatei `makrodefined.macro` kommt über `;#MACRODATEI1=` hinzu) endet im Interpreter mit „Test wahr“; die Sollwerte sind je Makro von Hand hergeleitet und standen vor dem ersten Lauf im Prüfblock. Er deckt ab: einfache Variable, Feld, `@name` und `name(index)` (letztes Element, Index außerhalb des Feldes, Rechenausdruck als Index; `glob(0)` bei einer einfachen Variablen ergibt 0); lokale Variable und lokales Feld (`$LOCAL`, ohne `$SET` vorhanden), ein aufgerufenes Makro sieht die lokalen des Aufrufers nicht; eine lokale Variable verdeckt die globale und auch ein globales Feld gleichen Namens (`$DEFINED(@fe)` ergibt dann 0) und verschwindet mit `MEND`; `$V(...)` im Argument (`a$V(i)`, `fe($V(i))`) und Klammer um das Argument; `$DEFINED` in `$CALC`, `$IF`, `$ELIF`, `$WHILE` (Schleife, bis das Element fehlt) und `$REPT` (Stand zu Beginn des Aufrufs: ein im selben Aufruf erst gesetzter Name zählt dort noch nicht), in einer Marke und in einem Kommentar; `$MDEFINED` für ein Makro im Quelltext, in der Projektdatei und in `c82ass.makros`, für ein unbekanntes, für ein hinter dem Aufruf definiertes und für eines, das in Quelltext und Projektdatei zugleich steht (zählt einmal, der Quelltext gewinnt), für Variable, Anweisung und Symbol (0), für das laufende Makro (1) und in Kleinschreibung (0); beide

Beispiele aus diesem Abschnitt (Beispiel 1 mit vorherigem `SIEBEN` und ohne, Beispiel 2 mit einem vorhandenen und einem fehlenden Makro). Dazu kommen 15 Negativtests im Regressionstest: `makro_defined_leer`, `_v_undefiniert`, `_name_ziffer`, `_name_lang`, `_klammer_fehlt`, `_klammer_zuviel`, `_index_leer`, `_at_index`, `_index_text`, `_index_v_undefiniert` und `_in_p` sowie `makro_mdefined_leer`, `_doppelpunkt` und `_leerraum` (je Rückgabecode 16 mit der Meldung des Fehlers) und `makro_defined_nicht_erzeugt` (Rückgabecode 0: Fehler in einem nicht erzeugten Zweig bleiben stumm). Der Lauf durch `regressionstest.sh` mit c82ass 1.01-006: Die 257 bisherigen Programme blieben bitgleich zu ihrer Referenz; mit der Referenz für `makrodefined` sind es 258 Programme ohne Abweichung und 321 Negativtests ohne Fehler. `pruefe_alles.sh` einschließlich der Läufe unter AddressSanitizer und UBSan endet mit „ALLES IN ORDNUNG“.

Entschieden (04.11.2026): Name `$DEFINED` für Variablen und Felder, getrennt `$MDEFINED` für Makros (getrennte Namen, siehe oben); Ergebnis 1 oder 0 und keine Art-Zahl wie bei `art` (4.23), damit `= 1` immer „vorhanden“ heißt; `$IF` bleibt dreiteilig, es gibt keine Bedingung aus einem Wort; die Schreibweisen `name`, `@name` und `name(index)` sind die von `$V`; ein Index außerhalb des Feldes ergibt 0 statt eines Fehlers. Ein Vorgabewert in `$V` (etwa `$V(name,vorgabe)`) ist nicht vorgesehen: Er löste nur den Fall einer Variablen und nicht den von Feldern und Makros, und das Komma ist als Trenner nicht eindeutig (es steht schon in `min(a,b)` im Index, 4.8); „setze nur, wenn noch nicht gesetzt“ sind mit `$DEFINED` drei Zeilen (Beispiel 1). Nicht vorgesehen: die Art (Zahl oder Text) einer Variablen abzufragen und zu fragen, ob ein Name lokal oder global ist.

4.26 Benannte Parameter (`$name` statt `$1` bis `$9`)

Zweck: Ein Makro mit mehreren Parametern zwingt den Leser, bei jedem `$1` bis `$9` die Nummer im Kopf in die Bedeutung zurückzuübersetzen. Das Makro `WDHL` der Makrodatei `c82ass.makros` hat fünf Parameter (Löschen, Resultatsbehandlung, Operation, Anzahl, Spur), und sein Rumpf brauchte Kommentare, die sagen, was `$1` bis `$5` bedeuten. Mit benannten Parametern steht die Bedeutung im Namen: `WDHL: MACRO loeschen resultat operation anzahl spur`. Der Aufruf ändert sich nicht (Parameter durch Leerraum getrennt, in der Reihenfolge der Vereinbarung); benannt wird nur im Rumpf. Makros ohne Namen und Quelltexte, die keine benutzen, liefern unverändert dieselben Ergebnisse.

Vereinbarung: Statt der Parameterzahl stehen hinter `MACRO` die Namen, durch Leerraum getrennt, in der Reihenfolge der Aufrufparameter, zum Beispiel `WDHL: MACRO loeschen resultat operation anzahl spur`. Die Zahl der Namen ist die Parameterzahl; eine Zahl zugleich mit Namen (`MACRO 2 anzahl`) ist ein Fehler, ebenso ein doppelter Name. Ein Name beginnt mit einem Buchstaben oder _, danach folgen Buchstaben, Ziffern und _, höchstens 24 Zeichen (wie bei den lesbaren lokalen Labels, Abschnitt 4); Groß- und Kleinschreibung zählen (`$Anzahl` ist ein anderer Parameter als `$anzahl`, getestet im Fall G von makroname). Es sind höchstens 32 Namen zulässig; die Grenze von 9 gilt nur für die Schreibweise mit Zahl. Ein Kommentar mit ; hinter den Namen bleibt erlaubt. Für die variable Parameterzahl (4.13) steht ein + als eigenes Wort hinter den Namen (`MACRO von bis +`): Der Aufruf braucht mindestens so viele Parameter, wie Namen stehen; die weiteren erreicht nur `$P(i)`.

Meldungen bei der Vereinbarung (Rückgabecode 16; wortgetreu aus Läufen, die Prüfung schreibt ohne Umlaute; ein zu langer Name (25 Zeichen) meldet dasselbe wie ein ungültiger). Jede Meldung beginnt mit `Makro <Name>:` und endet mit dem Ort: bei einem Makro im Quelltext „[im Quelltext Zeile n]“ (die Zeile des fehlerhaften Rumpfs, bei Fehlern in der `MACRO`-Zeile deren Zeile), bei einem Makro aus einer Makrodatei `[definiert in <Datei>:<Zeile>]` (immer die Zeile der `MACRO`-Anweisung, nicht der Rumpfzeile). Ein Makro mit fehlerhafter Vereinbarung wird bei mehreren Aufrufen nur einmal gemeldet; die Aufrufe entfallen ohne Folgefehler. Beispiele:

```
Makro M1: Parametername 'a' steht doppelt [im Quelltext Zeile 2]
Makro M1: Parametername 'if' ist ein festes Wort [im Quelltext Zeile 2]
Makro M1: Parametername '1abc' ist ungueltig (Buchstabe oder _, dann Buchstaben, Ziffern oder _, hoechstens 24
    Zeichen) [im Quelltext Zeile 2]
Makro M1: Hinter MACRO stehen Namen und die Zahl 2 [im Quelltext Zeile 2]
Makro M1: Das Zeichen + steht nur als letztes Wort [im Quelltext Zeile 2]
Makro M1: 33 Parameternamen angegeben, zulaessig sind maximal 32 [im Quelltext Zeile 2]
```

Negativtests `makro_name_doppelt`, _fest, _ungueltig, _zu_lang, _zahl_und_name, _plus_mitte und _zu_viele.

Verwendung im Rumpf: `$name` steht überall dort, wo `$1` bis `$9` stehen, und wird zur selben Zeit und auf dieselbe Weise ersetzt: in Operation, Operand und Kommentar der erzeugten Zeilen, in `$CALC(...)`, in den Bedingungen von `$IF`, `$ELIF` und `$WHILE` (`$WHILE $V(i) <= $ende`), hinter `$REPT` (`$REPT $anzahl`), in `$SET` und `$INC` und in den Operanden von `!$wdhl` (4.16). Der Assembler übersetzt nach `MEND` (bei einem Makro im Quelltext nach seiner Definition) jeden Namen in die Nummer: `$name` wird zu `$k`, ab dem zehnten Parameter zu `$P(k)`; `$P(name)` und `$LEN(name)` werden zu `$P(k)` und `$LEN(k)`; `$PR(name)` wird zu `$PRk`. Auch die Behandlung der Zeichenketten-Parameter (4.10) und die Vornull bei vierstelligen Ziffernfolgen (Abschnitt 4) gelten für sie wie für `$n`. Ein Name ist ein ganzes Wort: Er endet am ersten Zeichen, das kein Buchstabe, keine Ziffer und kein _ ist; `$anzahl2` ist also der Parameter anzahl2 und nicht anzahl mit angehängter 2. Wo Text unmittelbar folgen soll, steht `$P(anzahl)2` (bei Parameter 5 ergibt das 52): Die

Form `$P(...)` aus 4.13 nimmt ein einzelnes Wort, das ein vereinbarter Name ist, als Namen und sonst als Ausdruck. Für die Länge eines Zeichenketten-Parameters gibt es `$LEN(name)` (4.10). Die Kommentare (Teil hinter ;) werden mit ersetzt, wie bei `$n`; Zeilen mit # am Anfang werden weder ersetzt noch geprüft (das Makro `WDHL` nutzt das für Kommentare mit `$anzahl`). Wird eine Zeile nach dem Ersetzen länger als 255 Zeichen, meldet der Assembler `Makro LANG: Die Zeile ist nach dem Ersetzen der Parameternamen laenger als 255 Zeichen (Zeile: <die ersten 60 Zeichen>) [im Quelltext Zeile 3]`.

Nummern bleiben gültig: In einem Makro mit Namen bezeichnet `$k` (k = 1 bis 9) denselben Parameter wie der k-te Name; `$P(i)` erreicht alle, und `$#` zählt wie bisher die übergebenen Parameter. Mischen ist erlaubt, aber nicht empfohlen. Die alte Form `!$wdhl $5` (4.16) liest die Parameter 1 bis 5 in der Reihenfolge der Vereinbarung. Ein Parameter `$n` und eine Variable `$V(n)` sind verschieden (`$n` ist der Parameter, `$V(n)` die Variable).

Prüflisten und Vorgabewerte: Statt `$PRx` (Abschnitt 4) steht `$PR(name)`, mit Werteliste (`$PR(loeschen) JA NE`) oder mit Typ (`$PR(spur) $ADRESSE`, `$PR(anzahl) $TYPZAHL 0 5`), und `$DEFAULT` (4.6) nimmt statt der Nummer auch den Namen (`$DEFAULT zweiter 9`). Die Nummernformen bleiben gültig. Ein zweites `$DEFAULT` für denselben Parameter, einmal mit Nummer und einmal mit Namen, ist wie bisher ein Fehler (`$DEFAULT für Parameter $zweiter steht mehrfach`). Mehrere `$PR`-Zeilen für denselben Parameter sind dagegen kein Fehler, auch nicht in früheren Versionen: Alle Zeilen gelten zugleich, der Wert muss jede Liste erfüllen (`$PR(zweiter) $TYPZAHL 0 5` und `$PR2 $TYPZAHL 0 7` mit dem Wert 6 ergibt `Parameter $zweiter: Wert "6" (= 6) liegt nicht im Bereich 0 bis 5`). `$PR0`, `$PR05` und Verwandte gelten nicht als `$PR`-Zeile: Sie erzeugen weder Meldung noch Prüfung und verschwinden im Rumpf. Die Meldung bei einem Wert außerhalb des Bereichs nennt den Namen, und der Kopf nennt ihn wie bei Nummern zweimal: `$anzahl Parameter $anzahl: Wert "9" (= 9) liegt nicht im Bereich 0 bis 5` (bei Nummern `$2 Parameter $2: …`). Ein Parameter ab Nummer 10 lässt sich prüfen: `$PR(zehn) $TYPZAHL 0 5` prüft den zehnten Parameter. Weitere Meldungen:

```
Makro M1: $PR( ) erwartet einen Parameternamen (Zeile: $PR() $TYPADR) [im Quelltext Zeile 3]
Makro M1: $PR(x): kein Parametername dieses Makros (Zeile: $PR(x) $TYPADR) [im Quelltext Zeile 3]
Makro M1: $DEFAULT: Unbekannter Parametername x (Zeile: $DEFAULT x 3) [im Quelltext Zeile 3]
$PR10 $TYPZAHL 0 5 Ungültiger Parameter: $PR10 $TYPZAHL 0 5 (Max. erlaubt: $PR2)
```

Die letzte Meldung gilt für `$PR10` in einem Makro ohne Namen mit nur zwei Parametern. Negativtests `makro_name_pr_unbekannt`, _pr_inhalt, _pr_leer, _pr_null (kein Fehler, rc 0), _pr10_ohne_namen, _default_unbekannt, _default_doppelt und _wert_bereich.

Aufruf mit falscher Parameterzahl: `erwartet 2 Parameter, aber 1 wurden übergeben! (für $zweiter gibt es kein $DEFAULT)`; mit + steht `erwartet mindestens 2 Parameter, aber 1 wurden übergeben! (für $ende gibt es kein $DEFAULT)`. Zu viele Parameter melden wie bisher „erwartet 3 Parameter, aber 4 wurden übergeben!“ (Negativtest `makro_name_aufruf_anzahl`).

Feste Wörter und Tippfehler: Ein Name darf kein festes Wort des Makrosystems sein. In der `MACRO`-Zeile zählt das ohne Rücksicht auf Groß- und Kleinschreibung (im Rumpf mit); fest sind: IF, ELIF, ELSE, ENDIF, REPT, ENDREPT, WHILE, ENDWHILE, EXIT, LOCAL, GLOBAL, SET, INC, DIM, DEFAULT, CALC, CALCO, CALCB, CALCH, V, P, PR, LEN, Z, INCLUDE, NOTE, WARN, ERROR, TRACE, DUMP, DEFINED, MDEFINED, PROBE, AUFRUFNR, SUBSTR, INDEX, UPPER, LOWER, STRCMP, ORD, CHR, VAL, VALO, VALB, VALH, die Typnamen TYPADR, TYPKON, TYPABS, TYPZAHL, TYPSYM, TYPTEXT und ADRESSE, die Namen wdhl und wdhl_adr_e (4.16) und die Namen der dz-Sonderzeichen (Tabelle 2: wrzl, sm, inkb, ingb, irrz, irrs, trans, bm, vorl, irrb, tab, zr, wr), weil `dz $wrzl` im Rumpf weiter das Sonderzeichen meint. Dazu kommen PR mit einer oder zwei Ziffern (`PR1` bis `PR99` und `PR00` sind fest, `PR0` und `PR123` nicht), `LEN1` bis `LEN9` (`LEN0` und `LEN10` sind zulässig), `Z1` bis `Z4` (`Z5` ist zulässig), `CALCO1` bis `CALCO9` (`CALCO0` und `CALCO10` sind zulässig) `CALCB1` bis `CALCB99` (`CALCB0` und `CALCB100` sind zulässig) und `CALCH1` bis `CALCH99` (`CALCH0` und `CALCH100` sind zulässig). „Anzahl“ ist zulässig, „adresse“ nicht (festes Wort ADRESSE; deshalb heißt der Parameter in `WDHL` spur). Maßgeblich ist die Liste `makroFesteWoerter` in `ca82_makros.c`. Ein Verstoß ist ein Fehler bei der Vereinbarung (Negativtest `makro_name_fest`). Gegen Tippfehler: In einem Makro mit Namen ist ein `$wort`, das weder ein vereinbarter Name noch ein festes Wort ist, ein Fehler (`Unbekannter Parametername $dritter (Zeile: $dritter)`, auch im Kommentar: `… (Zeile: ; gleich $zweiter)`) statt unersetzt stehenzubleiben; eine #-Zeile mit unbekanntem `$wort` bleibt ohne Meldung, und in Makros ohne Namen bleibt das bisherige Verhalten. Negativtests `makro_name_unbekannt`, _kommentar und _datei (Makrodatei, Meldung mit `[definiert in makro_name_datei_1.macro:2]`).

Namensräume: Parameter, Variablen (`$V(name)`, 4.7), lokale Variablen (4.20), lokale Marken mit % (Abschnitt 4) und Symbole des Quelltexts (4.18) sind getrennte Namensräume; ein Parameter anzahl und eine Variable anzahl stören sich nicht (`$anzahl` ist der Parameter, `$V(anzahl)` die Variable). Ruft ein Makro ein anderes auf, übergibt es seine Parameter als Werte, wie heute mit `$1`, zum Beispiel `INNEN $ziel $anzahl` (Fall F von makroname).

Beispiel 1: Das Makro `WDHL` mit benannten Parametern, wie es in `c82ass.makros` steht (die Liste hinter `$PR(anzahl)` ist gekürzt; dort reicht sie bis 32). Die Rumpfzeile hat die alte Form (4.16): Der Operand zählt nicht, die Parameter 1 bis 5 liefern die Werte in der Reihenfolge der Vereinbarung.

```
WDHL:  MACRO loeschen resultat operation anzahl spur
$PR(loeschen)   JA NE
$PR(resultat)   LI RE ZL OHNE
$PR(operation)  KON ADD SUB BADD
$PR(anzahl)     1 2 3 4 5 6 7 8
$PR(spur)       $ADRESSE
       !$wdhl  $spur
       MEND
```

Der Aufruf `WDHL JA LI KON 3 004000` erzeugt wie mit den Nummern 605 04031 (4.16; im Lauf mit c82ass 1.01-014 gemessen). Die Fünf-Operanden-Form `!$wdhl $loeschen $resultat $operation $anzahl $spur` ergibt dasselbe 605 04031, ist aber strenger (4.16).

Beispiel 2: Ein Makro mit Rechnung.

```
SUMPAAR:  MACRO erster zweiter
          dw  $erster                  ; erstes Wort
          dw  $CALC($erster+$zweiter)   ; Summe beider Parameter
          MEND
```

Der Aufruf `SUMPAAR 3 4` erzeugt `dw 3` und `dw 7` (gelaufen; Fall A von makroname).

Beispiel 3: Mehr als neun Parameter (gekürzte Fassung von `VIELE` aus makroname): zwölf Namen a1 bis a9, zehn, elf, zwoelf; dazu `$DEFAULT elf 7` und `$DEFAULT zwoelf 8`; der Rumpf enthält `dw $zehn`, `dw $elf`, `dw $zwoelf` und `dw $CALC($zehn+$elf)`. Mit zehn Parametern (der zehnte ist 10) erzeugt der Aufruf dw 10, 7, 8 und 17; mit zwölf Parametern (10, 11, 12) dw 10, 11, 12 und 21. Das Listing zeigt die Vorgaben als `$DEFAULT 11 7` und `$DEFAULT 12 8`.

Festlegungen : (1) Die Schreibweise `$name`, weil `$1` bis `$9` schon so aussehen; die Kollision mit den dz-Sonderzeichen löst die Liste fester Wörter. (2) Die Namen stehen in der `MACRO`-Zeile und nicht in einer eigenen Zeile, damit Parameterzahl und Namen an einer Stelle stehen. (3) Obergrenze 32 Namen. (4) Fehler bei einem unbekannten `$wort` in Makros mit Namen. (5) Der Aufruf bleibt positional; ein Aufruf mit Namen (`WDHL` anzahl=3) ist nicht vorgesehen. Ab c82ass 1.01-025 gilt das nur für Makros ohne Schlüsselwortparameter; Schlüsselwortparameter (`name=vorgabe`) beschreibt 4.30. (6) Von den Makros der Standardmakrodatei `c82ass.makros` ist nur `WDHL` auf Namen umgestellt; die übrigen zählen weiter mit Nummern.

Nicht geprüft: `$LEN(...)` zusammen mit `$DEFINED` im Ausdruck; ein gültiges `$PR10` bis `$PR32` in einem Makro ohne Namen (`MACRO 9+` oder darüber); der Fuzz-Lauf (fuzz.py). Kein Negativtest deckt die Zeilenlänge über 255 Zeichen nach dem Ersetzen ab.

4.27 Binärzahlen im Makroprozessor (`$CALCB`, `$CALCBn`, `0b`, `$VALB`)

Zweck: Die Anweisung `dwb` (3.3.19) schreibt ein Wort als Folge von 1 bis 33 Binärziffern. Der Makroprozessor kannte bisher nur Dezimal- und Oktalzahlen (`$CALC`, `$CALCO`, 4.2 und 4.22); ein Makro, das ein Bitmuster aus einer Rechnung erzeugt, musste die Ziffern von Hand zusammensetzen. Seit c82ass 1.01-022 gibt es dazu drei Ergänzungen, jeweils das Gegenstück zu den Oktalfunktionen: die Ausgabe `$CALCB` und `$CALCBn`, die Eingabe `0b101` im Rechenausdruck und das Lesen eines Textes mit `$VALB`. Alles wirkt nur zur Assemblerzeit; im erzeugten Quelltext steht allein das Ergebnis. Was bisher ging, bleibt bitgleich.

Umsetzungsstand: Umgesetzt in c82ass 1.01-022 (05.11.2026). Gilt nur, wenn `c82ass --version` 1.01-022 oder höher meldet; ein älteres Binary kennt `$CALCB`, `$CALCBn`, `0b` und `$VALB` nicht.

Ausgabe: `$CALCB(<ausdruck>)` wird durch das Ergebnis als Folge von Binärziffern ersetzt, ohne Präfix und ohne führende Nullen; das Ergebnis 0 ergibt `0`. Der Ausdruck ist der aus 4.2 und 4.8. Das Präfix fehlt mit Absicht: `dwb $CALCB(10)` erzeugt das Wort 012 (dezimal 10, binär 1010), weil `dwb` die Ziffernfolge dual liest. `$CALCBn(<ausdruck>)` mit n = 1 bis 64 (eine oder zwei Ziffern) füllt mit führenden Nullen auf genau n Stellen auf. `dwb` nimmt höchstens 33 Stellen (3.3.19); Breiten bis 64 sind für Texte gedacht (`$SET`, siehe unten). Eine Breite 0 oder über 99 ist kein Name: der Text bleibt stehen, und der Assembler meldet ihn als ungültigen Operanden.

```
$CALCB(10)            ergibt  1010
$CALCB(0)             ergibt  0
$CALCB(2+3*4)         ergibt  1110
$CALCB(0b11+0b101)    ergibt  1000
$CALCB8(5)            ergibt  00000101
$CALCB1(1)            ergibt  1
```

Fehler der Ausgabe (Rückgabecode 16; jede Meldung beginnt mit der Quelltextzeile des Aufrufs und „Fehler:“): ein negatives Ergebnis, ein Ergebnis mit mehr Stellen als n, eine Breite über 64 und eine fehlende Klammer. Wortgetreu aus den Negativtests:

```
$CALCB(0-5): negatives Ergebnis -5 ist binär nicht darstellbar
$CALCB4(16): Ergebnis 16 braucht mehr als 4 Binärziffern
$CALCB65(1): Breite 65 liegt außerhalb von 1 bis 64
$CALCB: Klammer ) fehlt
```

Im Text: Wie `$CALCO` (4.22) werden `$CALCB(...)` und `$CALCBn(...)` im Text von `$SET` und in den Textargumenten der Zeichenkettenfunktionen nach dem Auflösen von `$V(...)` ausgewertet. Mit `n` = 1234 setzt `$SET t "$CALCB($V(n))"` den Text `10011010010` (`$V(#t)` = 11), und `$SET t "$CALCB16($V(n))"` setzt `0000010011010010` (16 Zeichen).

Eingabe: Im Rechenausdruck ist `0b` oder `0B`, gefolgt von Ziffern 0 und 1, eine Binärzahl: `0b101` ist 5, `0B101*2` ist 10, `0b1<<20` ist 2^20. Führende Nullen zählen nicht (`0b0000101` ist 5). Der Bereich reicht bis 2^63 - 1, das sind 63 Einsen; davor sind beliebig viele Nullen erlaubt. Ein Minuszeichen davor macht die Zahl negativ (`-0b101+100` ergibt 95). Den Wert liest der Rechner selbst; die Schreibweise gilt deshalb überall, wo ein Rechenausdruck steht: in `$CALC`, `$CALCO` und `$CALCB`, in `$SET` und `$INC`, im `$CALC` einer `$IF`-Bedingung und in den Grenzen von `$TYPZAHL`.

```
            dw      $CALC(0b11111111)       ; 255 = 0377
            dw      $CALC(0b1<<20)          ; 2^20 = 04000000
            dw      $CALC(-0b101+100)       ; 95 = 0137
            $SET a 0b101                    ; a = 5
            $INC a 0b11                     ; a = 8
```

Wo `0b` nicht gilt: im Operanden von `abs` und `equ` und überhaupt in jedem Operanden außerhalb eines Rechenausdrucks, denn der Assembler selbst kennt die Schreibweise dort nicht (`dw 0b101` meldet „Operand '0b101' ist ungültig“; richtig ist `dw $CALC(0b101)` oder `dwb 101`). Ebenso gilt sie nicht im Zahlenvergleich einer `$IF`-Bedingung ohne `$CALC` (4.4): `$IF $V(a) = 0b1000` ist unwahr, auch wenn `a` den Wert 8 hat, während `$IF $V(a) = $CALC(0b1000)` wahr ist.

Fehler der Eingabe (Rückgabecode 16): `0b` ohne Ziffer, eine andere Ziffer als 0 und 1 hinter `0b` und eine Zahl über 2^63 - 1. Wortgetreu aus den Negativtests (die erste Meldung beginnt wie üblich mit der Quelltextzeile):

```
$CALC(0b): ungültige Binärzahl (nach 0b folgen die Ziffern 0 und 1)
$CALC(0b102): ungültige Binärzahl (nach 0b folgen nur die Ziffern 0 und 1)
Zahl zu groß (bei "0b1111111111111111111111111111111111111111")
```

Text lesen: `$VALB <ziel> "<text>"` wandelt den Text in eine binäre ganze Zahl (Ergebnis Zahl), das Gegenstück zu `$VALO` (4.22). Form, Ziel, Textargument, die Auflösung von `$1` bis `$9`, `$V(...)` und `$CALC...` vor dem Lesen und die Wirkung (keine Zeile im erzeugten Quelltext, nur im erzeugten Zweig) sind wie bei `$VAL` und `$VALO` (4.21, 4.22). Erlaubt sind die Ziffern 0 und 1, davor ein einzelnes `-`; führende Nullen sind erlaubt: `101` und `00101` ergeben 5, `-101` ergibt -5, `0` ergibt 0. Keine Zahl sind `102`, `12`, `-`, der leere Text, `+1`, ein Text mit Leerzeichen, ein Text mit Semikolon und ein Text mit dem Präfix `0b` (das `b` ist keine Ziffer). Der Bereich ist der des Ausdrucks: bis 2^63 - 1, das sind 63 Einsen; mit 64 Stellen und dem Wert 2^63 meldet der Assembler „Die Zahl ... liegt außerhalb des Zahlenbereichs“, wobei die Meldung den Text nach 40 Zeichen abschneidet.

Fehler von `$VALB` (Rückgabecode 16, die Meldung nennt das Makro und die Quelltextzeile des Aufrufs): dieselben Fälle wie bei `$VALO` (falsche Argumentzahl, ein Text ohne Anführungszeichen, ungültiges Ziel, ein Text, der keine binäre ganze Zahl ist, eine Zahl außerhalb des Bereichs). Beispiele wortgetreu:

```
M1 $VALB n: Der Text "102" ist keine binäre ganze Zahl (erlaubt sind die Ziffern 0 und 1, davor ein einzelnes -)
M1 $VALB n: Es fehlt ein Argument (Form: $VALB <Ziel> "<Text>")
M1 $VALB n: Argument 1 muss ein Text in Anführungszeichen sein ("..." oder '...'), steht aber: 101
```

Rundreise: `$SET t "$CALCB($V(n))"` und `$VALB r "$V(t)"` ergeben `r` = `n` für jedes `n` von 0 bis 2^63 - 1, auch mit fester Breite (`$CALCBn`). `$VAL` liest denselben Text weiter dezimal (aus `101` wird 101); `017` ist für `$VALB` wegen der Ziffer 7 ein Fehler.

Feste Wörter: `CALCB` (mit Breite `CALCB1` bis `CALCB99`) und `VALB` sind feste Wörter und dürfen kein Parametername sein (4.26).

Beispiel 1: `BINAUS` erzeugt drei Wörter mit `dwb`: 012 (10), 005 (5, mit Breite 8) und 016 (14):

```
BINAUS:    MACRO
           dwb     $CALCB(10)              ; 1010 = 012 oktal
           dwb     $CALCB8(5)              ; 00000101 = 5
           dwb     $CALCB(2+3*4)           ; 1110 = 016 oktal
           MEND
```

Beispiel 2: `RUNDE2` bildet aus 1234 den Binärtext (11 Zeichen) und liest ihn zurück; erzeugt werden die Wörter 013 (11) und 02322 (1234):

```
RUNDE2:    MACRO
           $SET n 1234
           $SET t "$CALCB($V(n))"          ; Text 10011010010
           $VALB r "$V(t)"                 ; zurück: 1234
           dw      $V(#t)                  ; 11 Zeichen = 013
           dw      $V(r)                   ; 1234 = 02322
           MEND
```

Grenzen: Es gibt weiterhin keinen Dezimaltext mit fester Breite (Hexadezimal: 4.28). `0b` gilt weder im Operanden noch im Zahlenvergleich von `$IF` ohne `$CALC` und nicht in `$VAL` und `$VAL0`; wer einen Binärtext als Zahl vergleichen will, liest ihn mit `$VALB` und vergleicht die Zahl.

Stand der Prüfung: Das Programm `Claude/makrobinaer` (28 Prüfzellen; Sollwerte vor dem Lauf von Hand hergeleitet) prüft `$CALCB` und `$CALCBn` mit `dwb` (darunter Breite 33 und 2^33 - 1), Binärzahlen im Ausdruck, `$VALB` (darunter 63 Einsen), die Rundreise für 0 bis 100 mit und ohne feste Breite und `$VAL` unverändert; Test wahr. Im Regressionstest stehen 17 Negativtests `makro_binaer_*`: `_bin_leer`, `_bin_ziffer`, `_bin_gross` (Eingabe), `_calcb_negativ`, `_calcb_breite`, `_calcb_breite65`, `_calcb_klammer` (Ausgabe) und `_valb_ziffer`, `_valb_leer`, `_valb_minus`, `_valb_plus`, `_valb_leerzeichen`, `_valb_semikolon`, `_valb_bereich`, `_valb_argzahl_wenig`, `_valb_argzahl_viel`, `_valb_ohne_anfuehrung` (`$VALB`). Der Regressionstest ohne AddressSanitizer läuft mit c82ass 1.01-022 ohne Abweichung (272 Programme und 448 Negativtests). Zusätzlich von Hand gemessen, aber nicht im Regressionstest: `$SET` und `$INC` mit `0b`, die Grenzen von `$TYPZAHL` mit `0b`, `$IF $V(a) = 0b1000` unwahr, `dw 0b101` ungültig sowie `$CALCB0(...)` und `$CALCB100(...)` als ungültiger Operand. Nicht geprüft: der Lauf mit AddressSanitizer und UBSan (steht vor der Endlieferung an) und das Verhalten unter Windows.

Festlegungen: (1) Die Namen `$CALCB`, `$CALCBn` und `$VALB` und das Präfix `0b` folgen dem Muster von `$CALCO`, `$CALCOn` und `$VAL0`. (2) Die Ausgabe hat kein Präfix, damit `dwb $CALCB(...)` geht. (3) `0b` gilt nur im Rechenausdruck, nicht im Operanden von `abs` und `equ`, weil der Assembler dort die Schreibweise selbst nicht kennt. (4) Die feste Breite reicht bis 64 Stellen; `dwb` nimmt davon höchstens 33. (5) Eine negative Zahl hat keine Binärdarstellung (Fehler); `$VALB` liest dagegen ein einzelnes `-`.

4.28 Hexadezimalzahlen im Makroprozessor (`$CALCH`, `$CALCHn`, `0x`, `$VALH`)

Zweck: Die Anweisung `dwh` (3.3.19a) schreibt ein Wort aus 1 bis 8 Hexadezimalziffern. Wie bei den Binärzahlen (4.27) fehlte dem Makroprozessor bisher die Schreibweise: Er kannte nur Dezimal-, Oktal- und Binärzahlen. Seit c82ass 1.01-023 gibt es dazu drei Ergänzungen, jeweils das Gegenstück zu den Oktal- und Binärfunktionen: die Ausgabe `$CALCH` und `$CALCHn`, die Eingabe `0x1F` im Rechenausdruck und das Lesen eines Textes mit `$VALH`. Alles wirkt nur zur Assemblerzeit; im erzeugten Quelltext steht allein das Ergebnis. Was bisher ging, bleibt bitgleich.

Umsetzungsstand: Umgesetzt in c82ass 1.01-023 (05.11.2026). Gilt nur, wenn `c82ass --version` 1.01-023 oder höher meldet; ein älteres Binary kennt `dwh`, `$CALCH`, `$CALCHn`, `0x` und `$VALH` nicht (`dwh` meldet es als „Operationscode 'dwh' ist ungültig“).

Ausgabe: `$CALCH(<ausdruck>)` wird durch das Ergebnis als Folge von Hexadezimalziffern ersetzt, mit den Großbuchstaben A bis F, ohne Präfix und ohne führende Nullen; das Ergebnis 0 ergibt `0`. Der Ausdruck ist der aus 4.2 und 4.8. Das Präfix fehlt mit Absicht: `dwh $CALCH(255)` erzeugt das Wort 0377, weil `dwh` die Ziffernfolge hexadezimal liest. `$CALCHn(<ausdruck>)` mit n = 1 bis 16 (eine oder zwei Ziffern) füllt mit führenden Nullen auf genau n Stellen auf. `dwh` nimmt höchstens 8 Stellen (3.3.19a); Breiten bis 16 sind für Texte gedacht (`$SET`, siehe unten), denn 2^63 - 1 braucht 16 Hexziffern. Eine Breite 0 oder über 99 ist kein Name: der Text bleibt stehen, und der Assembler meldet ihn als ungültigen Operanden; eine Breite von 17 bis 99 ist ein Fehler der Ausgabe.

```
$CALCH(255)           ergibt  FF
$CALCH(0)             ergibt  0
$CALCH(2+3*4)         ergibt  E
$CALCH(48879)         ergibt  BEEF
$CALCH(0xFF+0x1)      ergibt  100
$CALCH8(255)          ergibt  000000FF
$CALCH1(10)           ergibt  A
```

Fehler der Ausgabe (Rückgabecode 16; jede Meldung beginnt mit der Quelltextzeile des Aufrufs und „Fehler:“): ein negatives Ergebnis, ein Ergebnis mit mehr Stellen als n, eine Breite über 16 und eine fehlende Klammer. Wortgetreu (mit einem Probe-Binary aus dem Quelltext 1.01-023 gemessen):

```
$CALCH(0-5): negatives Ergebnis -5 ist hexadezimal nicht darstellbar
$CALCH2(256): Ergebnis 256 braucht mehr als 2 Hexadezimalziffern
$CALCH17(1): Breite 17 liegt außerhalb von 1 bis 16
$CALCH: Klammer ) fehlt
```

Im Text: Wie `$CALCO` (4.22) und `$CALCB` (4.27) werden `$CALCH(...)` und `$CALCHn(...)` im Text von `$SET` und in den Textargumenten der Zeichenkettenfunktionen nach dem Auflösen von `$V(...)` ausgewertet. Mit `n` = 1234 setzt `$SET t "$CALCH($V(n))"` den Text `4D2` (`$V(#t)` = 3), und `$SET t "$CALCH16($V(n))"` setzt `00000000000004D2` (16 Zeichen).

Eingabe: Im Rechenausdruck ist `0x` oder `0X`, gefolgt von Hexadezimalziffern (0 bis 9, A bis F oder a bis f), eine Hexadezimalzahl: `0xFF` ist 255, `0XaB*2` ist 342, `0x1<<20` ist 2^20. Führende Nullen zählen nicht (`0x0000AB` ist 171). Der Bereich reicht bis 2^63 - 1 (`0x7FFFFFFFFFFFFFFF`); davor sind beliebig viele Nullen erlaubt. Ein Minuszeichen davor macht die Zahl negativ (`-0xA+100` ergibt 90). Die Buchstaben sind hier immer Ziffern: `0x1E+3` ist 30 + 3 = 33 (das `E` ist kein Exponent), und `0xb+0b1` ist 11 + 1 = 12 (`0xb` hexadezimal, `0b1` binär, 4.27). Den Wert liest der Rechner selbst; die Schreibweise gilt deshalb überall, wo ein Rechenausdruck steht: in `$CALC`, `$CALCO`, `$CALCB` und `$CALCH`, in `$SET` und `$INC`, im `$CALC` einer `$IF`-Bedingung und in den Grenzen von `$TYPZAHL`.

```
            dw      $CALC(0xFF)             ; 255 = 0377
            dw      $CALC(0x1<<20)          ; 2^20 = 04000000
            dw      $CALC(-0xA+100)         ; 90 = 0132
            $SET a 0xFF                     ; a = 255
            $INC a 0x10                     ; a = 271
```

Wo `0x` nicht gilt: im Operanden von `abs` und `equ` und überhaupt in jedem Operanden außerhalb eines Rechenausdrucks, denn der Assembler selbst kennt die Schreibweise dort nicht (`dw 0xFF` meldet „Operand '0xFF' ist ungültig“; richtig ist `dw $CALC(0xFF)` oder `dwh FF`). Auch `dwh` nimmt kein Präfix (3.3.19a). Ebenso gilt `0x` nicht im Zahlenvergleich einer `$IF`-Bedingung ohne `$CALC` (4.4): `$IF $V(a) = 0x8` ist unwahr, auch wenn `a` den Wert 8 hat, während `$IF $V(a) = $CALC(0x8)` wahr ist.

Fehler der Eingabe (Rückgabecode 16): `0x` ohne Ziffer, ein Zeichen außer den Hexziffern hinter `0x` und eine Zahl über 2^63 - 1. Wortgetreu aus der Messung (jede Meldung beginnt wie üblich mit der Quelltextzeile):

```
$CALC(0x): ungültige Hexadezimalzahl (nach 0x folgen die Ziffern 0 bis 9 und A bis F) (bei "0x")
$CALC(0xG1): ungültige Hexadezimalzahl (nach 0x folgen nur die Ziffern 0 bis 9 und A bis F) (bei "0xG1")
$CALC(0x8000000000000000): Zahl zu groß (bei "0x8000000000000000")
```

Text lesen: `$VALH <ziel> "<text>"` wandelt den Text in eine hexadezimale ganze Zahl (Ergebnis Zahl), das Gegenstück zu `$VALO` (4.22) und `$VALB` (4.27). Form, Ziel, Textargument, die Auflösung von `$1` bis `$9`, `$V(...)` und `$CALC...` vor dem Lesen und die Wirkung (keine Zeile im erzeugten Quelltext, nur im erzeugten Zweig) sind wie bei `$VAL` und `$VALO` (4.21, 4.22). Erlaubt sind die Ziffern 0 bis 9 und die Buchstaben A bis F oder a bis f, davor ein einzelnes `-`; führende Nullen sind erlaubt: `FF`, `ff` und `00FF` ergeben 255, `-A` ergibt -10, `10` ergibt 16, `0` ergibt 0. Keine Zahl sind `G1`, `-`, der leere Text, `+1`, ein Text mit Leerzeichen, ein Text mit Semikolon und ein Text mit dem Präfix `0x` (das `x` ist keine Ziffer). Der Bereich ist der des Ausdrucks: bis 2^63 - 1 (`7FFFFFFFFFFFFFFF`); `8000000000000000` meldet der Assembler als „Die Zahl ... liegt außerhalb des Zahlenbereichs“, wobei die Meldung den Text nach 40 Zeichen abschneidet.

Fehler von `$VALH` (Rückgabecode 16, die Meldung nennt das Makro und die Quelltextzeile des Aufrufs): dieselben Fälle wie bei `$VALO` (falsche Argumentzahl, ein Text ohne Anführungszeichen, ungültiges Ziel, ein Text, der keine hexadezimale ganze Zahl ist, eine Zahl außerhalb des Bereichs). Beispiele wortgetreu:

```
M1 $VALH n: Der Text "G1" ist keine hexadezimale ganze Zahl (erlaubt sind die Ziffern 0 bis 9 und A bis F oder a
    bis f, davor ein einzelnes -)
M1 $VALH n: Es fehlt ein Argument (Form: $VALH <Ziel> "<Text>")
M1 $VALH n: Argument 1 muss ein Text in Anführungszeichen sein ("..." oder '...'), steht aber: FF
M1 $VALH n: Die Zahl 8000000000000000 liegt außerhalb des Zahlenbereichs
```

Rundreise: `$SET t "$CALCH($V(n))"` und `$VALH r "$V(t)"` ergeben `r` = `n` für jedes `n` von 0 bis 2^63 - 1, auch mit fester Breite (`$CALCHn`). `$VAL` liest denselben Text weiter dezimal (aus `101` wird 101; `FF` ist für `$VAL` ein Fehler).

Falle `dwh $CALC(...)`: `$CALC` erzeugt Dezimalziffern, `dwh` liest jede Ziffernfolge hexadezimal. `dwh $CALC(255)` ergibt deshalb das Wort 0x255 (01125) und keine Fehlermeldung. Für ein Wort mit dem Wert 255 schreibt man `dwh $CALCH(255)` oder `dw $CALC(255)`.

Feste Wörter: `CALCH` (mit Breite `CALCH1` bis `CALCH99`) und `VALH` sind feste Wörter und dürfen kein Parametername sein (4.26).

Beispiel 1: `HEXAUS` erzeugt drei Wörter mit `dwh`: 0377 (255), 0377 (255, mit Breite 8) und 0137357 (48879, `BEEF`):

```
HEXAUS:    MACRO
           dwh     $CALCH(255)            ; FF = 0377 oktal
           dwh     $CALCH8(255)           ; 000000FF = 0377 oktal
           dwh     $CALCH(48879)          ; BEEF = 0137357 oktal
           MEND
```

Beispiel 2: `RUNDE3` bildet aus 1234 den Hexadezimaltext (3 Zeichen) und liest ihn zurück; erzeugt werden die Wörter 03 (3 Zeichen) und 02322 (1234):

```
RUNDE3:    MACRO
           $SET n 1234
           $SET t "$CALCH($V(n))"         ; Text 4D2
           $VALH r "$V(t)"                ; zurück: 1234
           dw      $V(#t)                 ; 3 Zeichen = 03
           dw      $V(r)                  ; 1234 = 02322
           MEND
```

Grenzen: Es gibt weiterhin keinen Dezimaltext mit fester Breite. `0x` gilt weder im Operanden noch im Zahlenvergleich von `$IF` ohne `$CALC` und nicht in `$VAL`, `$VALO` und `$VALB`; wer einen Hexadezimaltext als Zahl vergleichen will, liest ihn mit `$VALH` und vergleicht die Zahl. `dwh` nimmt höchstens 8 Ziffern (32 Bit); ein Wort mit z0 = 1 braucht `dwb` oder `dw`.

Stand der Prüfung: Das Programm `Claude/makrohex` (42 Prüfzellen; Sollwerte vor dem Lauf von Hand hergeleitet, die Oktalwerte gegengerechnet) prüft `dwh` mit 0 bis 8 Ziffern (darunter `DEADBEEF`, `abcdef01` und das Symbol `cafe` als Schutz gegen die Symbolersetzung), `$CALCH` und `$CALCHn` (darunter Breite 8 und 2^32 - 1), Hexzahlen im Ausdruck (darunter `0xb+0b1` und `0x1E+3`), `$VALH` (darunter 2^63 - 1), die Rundreise für 0 bis 100 mit und ohne feste Breite und `$VAL` unverändert; Test wahr. Im Regressionstest (`Assembler/Regressionstest`, Stand 05.11.2026) liegen die Referenz `Claude/makrohex` und 29 Negativtests: acht `dwh_*` und 21 `makro_hex_*`; alle enden mit Rückgabecode 16 außer `dwh_spr` (Warnung SPR, Rückgabecode 0). Zu `dwh`: `dwh_ausdruck`, `dwh_leer`, `dwh_literal`, `dwh_praefix`, `dwh_vorzeichen`, `dwh_zeichen`, `dwh_zu_lang` (9 Stellen) und `dwh_spr`. Zur Eingabe `0x`: `makro_hex_hex_leer`, `makro_hex_hex_ziffer`, `makro_hex_hex_gross` (2^63) und `makro_hex_dw_praefix` (`dw 0xFF`). Zur Ausgabe: `makro_hex_calch_negativ`, `makro_hex_calch_breite` (`$CALCH2(256)`), `makro_hex_calch_breite17`, `makro_hex_calch_breite0`, `makro_hex_calch_breite100` und `makro_hex_calch_klammer`. Zu `$VALH`: `makro_hex_valh_ziffer`, `_leer`, `_minus`, `_plus`, `_leerzeichen`, `_semikolon`, `_praefix`, `_bereich` (2^63), `_argzahl_wenig`, `_argzahl_viel` und `_ohne_anfuehrung`. Die Meldungen entsprechen wortgetreu denen oben. Weiterhin nur von Hand mit einem Probe-Binary aus dem Quelltext 1.01-023 gemessen und nicht im Regressionstest: `$SET` und `$INC` mit `0x`, `$CALCH16`, `$IF $V(a) = 0x8` unwahr und `dwh $CALC(255)` ergibt 01125. Der Lauf mit AddressSanitizer und UBSan (`pruefe_alles.sh`, Linux, Quelltext 1.01-024) ist am 05.11.2026 mit allen neuen Tests gelaufen: 274 Programme und 480 Negativtests ohne Abweichung. Nicht geprüft: das Verhalten unter Windows.

Festlegungen : (1), (2) und die Breite von `dwh` in (3); (4) bis (6) stehen noch zur Bestätigung aus): (1) Die Namen `$CALCH`, `$CALCHn` und `$VALH` und das Präfix `0x` (nur im Rechenausdruck) folgen dem Muster von `$CALCB`, `$CALCBn`, `$VALB` und `0b`. (2) Die Ausgabe hat kein Präfix und verwendet Großbuchstaben, damit `dwh $CALCH(...)` geht. (3) `dwh` nimmt 1 bis 8 Hexziffern (32 Bit, rechtsbündig, z0 bleibt 0), ohne Vorzeichen, Präfix und Ausdruck; ein Wort mit z0 = 1 schreibt man mit `dwb` oder `dw`. : `dwh` nimmt höchstens 8 Ziffern; 9 Ziffern (das ganze 33-Bit-Wort) gibt es nicht. (4) `0x` gilt nur im Rechenausdruck, nicht im Operanden von `abs` und `equ`, weil der Assembler dort die Schreibweise selbst nicht kennt. (5) Die feste Breite reicht bis 16 Stellen; `dwh` nimmt davon höchstens 8. (6) Eine negative Zahl hat keine Hexadezimaldarstellung (Fehler); `$VALH` liest dagegen ein einzelnes `-`.

4.29 Aufrufnummer im Makroprozessor ($AUFRUFNR)

Zweck: Der Makroprozessor zählt jeden Makroaufruf eines Laufs; aus dieser Nummer entstehen die drei Buchstaben der generischen Labels (4.15). `$AUFRUFNR` macht dieselbe Nummer im Rumpf lesbar. So bildet ein Makro eindeutige Marken, Tabellennamen oder Kennungen je Aufruf, ohne eine Variable mit `$SET` und `$INC` zu führen (4.7).

Umsetzungsstand: Umgesetzt in c82ass 1.01-024 (05.11.2026). Gilt nur, wenn `c82ass --version` 1.01-024 oder höher meldet; ein älteres Binary kennt `$AUFRUFNR` nicht (im Operanden meldet es „Operand '$AUFRUFNR' ist ungültig“, Rückgabecode 16).

Form und Wert: `$AUFRUFNR` steht ohne Klammern und ohne Argument. Es wird durch die laufende Nummer des aktuellen Makroaufrufs ersetzt, dezimal und ohne führende Nullen, 1 bis 17000. Gezählt wird über alle Makros des Laufs, nicht je Makro: der erste Aufruf irgendeines Makros hat die Nummer 1, der zweite die Nummer 2 und so fort. Die Reihenfolge ist die der Auflösung: der äußere Aufruf trägt die kleinere Nummer als ein darin verschachtelter Aufruf. Es ist dieselbe Zahl, aus der die Buchstaben der `%`-Labels dieses Aufrufs entstehen (4.15).

Geltungsbereich: Im ganzen Rumpf eines Aufrufs hat `$AUFRUFNR` denselben Wert. Ein verschachtelter Aufruf hat seine eigene Nummer; nach der Rückkehr gilt in der äußeren Zeile wieder die äußere. Übergibt man `$AUFRUFNR` als Parameter an ein inneres Makro, reicht man die Nummer des äußeren Aufrufs weiter, denn die Ersetzung geschieht vor dem Auflösen des inneren Aufrufs. Ein Aufruf, den nur der Probelauf (4.23) erzeugt, verschiebt die Nummern des endgültigen Laufs nicht; sie zählen dort von 1.

Stellung und Zeitpunkt: Wie `$#` (4.6) wird `$AUFRUFNR` überall im Rumpf ersetzt, bevor die übrigen Ersetzungen laufen: in Marke, Operation, Operand und Kommentar, auch mitten im Wort (`tab$AUFRUFNR:` ergibt `tab1:`, `tab2:`), in `$CALC(...)`, in der Bedingung von `$IF` und `$WHILE`, in der Anzahl hinter `$REPT`, in `$SET`, `$INC` und in Zeichenketten (`$SET t "n=$AUFRUFNR"`). In Kommentarzeilen und im Kommentarfeld wird ebenfalls ersetzt; dort meldet der Assembler keinen Fehler.

Wortgrenze: Wie bei `$PROBE` (4.23) darf hinter dem Wort kein Buchstabe, keine Ziffer und kein _ stehen; `$AUFRUFNRX`, `$AUFRUFNR1` und `$AUFRUFNR_` bleiben als Text stehen. Soll unmittelbar hinter der Nummer ein Namenszeichen stehen, schreibt man `$CALC($AUFRUFNR):` `n$CALC($AUFRUFNR)_x:` ergibt `n1_x:`, `n2_x:` (geprüft).

Fehler (Rückgabecode 16): `$AUFRUFNR(` mit Klammer meldet „$AUFRUFNR steht ohne Klammern und ohne Argument“. Im normalen Quelltext außerhalb eines Makroaufrufs (4.1) ist `$AUFRUFNR` wie `$EXIT` und `$LOCAL` ein Fehler: „(Quelltext) $AUFRUFNR steht außerhalb eines Makroaufrufs“ mit der Quelltextzeile. Das Wort wird danach durch 0 ersetzt, damit kein zweiter Fehler „Operand ungültig“ folgt. Die Auswertung des Quelltexts läuft dafür auch dann, wenn keine Zeile mit `$` beginnt.

Feste Wörter: `AUFRUFNR` ist ein festes Wort und darf kein Parametername sein (4.26; „Parametername 'AUFRUFNR' ist ein festes Wort“). Im Rumpf zählt nur die Großschreibung `$AUFRUFNR`.

Was gleich bleibt: Programme ohne `$AUFRUFNR` werden bitgleich übersetzt. Die Makroliste `--makros` (4.11) nennt keine Aufrufnummern.

Beispiel 1: `AUSSEN` ruft `INNEN` auf und übergibt `$AUFRUFNR`. Zweimal `AUSSEN` erzeugt acht Wörter: 1, 1, 2, 1 und 3, 3, 4, 3. Der erste Aufruf von `AUSSEN` hat die Nummer 1, sein `INNEN` die Nummer 2; der zweite `AUSSEN` hat die Nummer 3, sein `INNEN` die Nummer 4:

```
INNEN:          MACRO   aussen
                dw      $aussen                 ; Nummer des äußeren Aufrufs
                dw      $AUFRUFNR               ; eigene Nummer
                MEND
AUSSEN:         MACRO
                dw      $AUFRUFNR               ; eigene Nummer
                INNEN   $AUFRUFNR               ; innerer Aufruf, Parameter ist die äußere Nummer
                dw      $AUFRUFNR               ; wieder die eigene Nummer
                MEND
                AUSSEN                          ; Wörter 1, 1, 2, 1
                AUSSEN                          ; Wörter 3, 3, 4, 3
```

Beispiel 2: `ZEHN` rechnet mit der Nummer. Ist es der fünfte Aufruf des Laufs, entsteht das Wort 50 dezimal (062 oktal):

```
ZEHN:           MACRO
                dw      $CALC($AUFRUFNR*10)     ; Nummer mal 10
                MEND
```

Beispiel 3: Jeder Aufruf legt eine eigene Marke an (`tab1:`, `tab2:`) und wiederholt eine Zeile so oft, wie seine Nummer angibt:

```
TAB:            MACRO
tab$AUFRUFNR:   dw      $AUFRUFNR               ; Marke mit Nummer
                $REPT   $AUFRUFNR
                dw      5                       ; so oft wie die Nummer
                $ENDREPT
                MEND
```

Grenzen: Die Nummer gilt für den ganzen Lauf, nicht für das einzelne Makro; fügt man vor einem Aufruf einen weiteren Makroaufruf ein (auch in einer Makrodatei oder in einem anderen Makro), ändern sich die Nummern der folgenden Aufrufe. Wer eine Zählung je Makro braucht, führt sie mit `$SET` und `$INC` (4.7). Mehr als 17000 Makroaufrufe in einem Lauf bricht der Assembler ab (4.15).

Stand der Prüfung: Das Programm Claude/aufrufnr (Sollwerte vor dem Lauf von Hand hergeleitet: Zellen 1 bis 10 oktal gleich 1, 1, 2, 1, 3, 3, 4, 3 und Zelle 11 gleich 062) endet im Interpreter mit „Test wahr“. Marke, `$IF`, `$REPT`, `$WHILE`, Kommentar, Zeichenkette, Wortgrenze und die Fehler sind in Wegwerfprogrammen geprüft. Regressionstest: Referenz Claude/aufrufnr und die Negativtests makro_aufrufnr_ausserhalb, makro_aufrufnr_klammer und makro_aufrufnr_name; pruefe_alles.sh einschließlich AddressSanitizer und UBSan ohne Befund.

(1) Die Nummer zählt über alle Makros des Laufs, nicht je Makro. (2) Außerhalb eines Makroaufrufs ist `$AUFRUFNR` ein Fehler, nicht der Wert 0. (3) Der Name `$AUFRUFNR` und die Versionsnummer 1.01-024.

4.30 Schlüsselwortparameter im Makro (`name=vorgabe`)

Zweck: Ein Makro mit vielen Parametern, von denen die meisten meist denselben Wert haben, nennt beim Aufruf nur, was abweicht, in beliebiger Reihenfolge und mit lesbarem Namen. Bisher ging das nur mit `$DEFAULT` (4.6), und der Vorgabewert galt nur für die letzten weggelassenen Positionsparameter. Vorbild ist der IBM-Assembler H (`&KEY=DEFAULT`).

Umsetzungsstand: Umgesetzt in c82ass 1.01-025 (05.11.2026). Gilt nur, wenn `c82ass --version` 1.01-025 oder höher meldet; ein älteres Binary kennt keine Schlüsselwortparameter.

Vereinbarung: Hinter `MACRO` steht bei einem Schlüsselwortparameter `name=vorgabe` statt `name`:

```
SCHIEBE:        MACRO   ziel anzahl=1 richtung=LI
```

Hier ist `ziel` ein Positionsparameter, `anzahl` und `richtung` sind Schlüsselwortparameter mit den Vorgaben 1 und LI. Die Vorgabe darf leer sein (`name=` oder `name=""`); steht sie in Anführungszeichen, darf sie Leerzeichen enthalten (`t="x y"`). Die Vorgabe enthält kein Semikolon (Kommentarbeginn), und die `MACRO`-Zeile ist wie bisher höchstens 255 Zeichen lang. Positions- und Schlüsselwortparameter dürfen gemischt stehen. Alle Namen zählen in der Reihenfolge der Vereinbarung durch, wie bisher: `$1` bis `$9`, `$P(k)`, `$LEN(name)` und `$name` bezeichnen den k-ten Namen, auch wenn er ein Schlüsselwortparameter ist; die alte Form `!$wdhl $5` (4.16) bleibt dadurch lesbar. Es bleibt bei höchstens 32 Namen; Namensregeln und feste Wörter wie in 4.26. Das `+` (variable Parameterzahl, 4.13) bleibt das letzte Wort und gilt nur für Positionsparameter; weitere Positionsoperanden hinter den benannten erreicht man mit `$P(k)`.

Aufruf: In einem Makro mit mindestens einem Schlüsselwortparameter ist ein Operand `name=wert` ein Schlüsselwortoperand. Er steht in beliebiger Reihenfolge und an beliebiger Stelle zwischen den Positionsoperanden (`SCHIEBE richtung=RE anzahl=3 04000`). Der Wert reicht bis zum nächsten Leerraum; enthält er Leerzeichen oder ein `=` oder ist er leer, steht er in Anführungszeichen (`text="a=b c"`, `text=""`); die Anführungszeichen gehören nicht zum Wert. Der Wert darf `$CALC(...)` und `$V(...)` enthalten, auch im Aufruf aus einem Makro heraus (`SCHIEBE 5 anzahl=$x`). Die übrigen Operanden sind Positionsoperanden: Sie füllen nur die Positionsparameter in deren Reihenfolge, die Schlüsselwortoperanden zählen dabei nicht mit. Ein weggelassener Schlüsselwortparameter erhält seine Vorgabe. Ein Operand in Anführungszeichen ist immer ein Positionswert.

Verträglichkeit: Die Regel „`name=wert` ist ein Schlüsselwort“ gilt nur für Makros mit mindestens einem Schlüsselwortparameter. In allen anderen Makros bleibt `name=wert` ein gewöhnlicher Positionswert; der Bestand (Makrodateien einschließlich `c82ass.makros`, Programme wie `Claude/makrostrsplit`, das `SPLIT` mit `abc=42` aufruft) wird bitgleich übersetzt. Listing, `.mqo` und `--makros` zeigen wie bisher die Aufrufzeile, wie sie geschrieben steht.

Prüfungen: `$PR(name)` (4.26) gilt für Schlüsselwortparameter wie für Positionsparameter und prüft den Wert, der im Makro ankommt, also auch die Vorgabe; steht sie nicht in der Liste, scheitert der Aufruf („Parameter $w: Wert "9" steht nicht in der Pruefliste“). Eine leere Vorgabe wird nicht geprüft. Ein im Aufruf angegebener leerer Wert

(`text=""`) ist dagegen ein angegebener Wert und wird geprüft; steht die leere Zeichenkette nicht in der Liste, scheitert der Aufruf . `$DEFAULT` (4.6) bleibt für Positionsparameter und gilt neben Schlüsselwortparametern (`$DEFAULT b 8`); für einen Schlüsselwortparameter ist es ein Fehler, denn dessen Vorgabe steht in der `MACRO`-Zeile. `$#` zählt die übergebenen Positionsoperanden, nicht die Schlüsselwortoperanden.

Fehler (Rückgabecode 16, die Meldung nennt das Makro): Im Aufruf: „Makro SCHIEBE: Unbekanntes Schluesselwort 'foo' (bekannt: anzahl, richtung)“; „Schluesselwort 'anzahl' steht doppelt im Aufruf“; „'ziel' ist ein Positionsparameter, kein Schluesselwort“ (ein Positionsparameter ist beim Aufruf nicht mit Namen angebbar); „Operand '=5': vor dem = fehlt der Name des Schluesselworts“; „Schluesselwort 'anzahl' hat keinen Wert (ein leerer Wert wird als anzahl="" geschrieben)“; „Schluesselwort 'anzahl': der Wert darf nicht mit = beginnen (Text mit = steht in Anfuehrungszeichen)“; zu viele oder zu wenige Positionsoperanden wie bisher („erwartet 1 Parameter, aber 2 wurden übergeben!“), wobei die Zahl nur die Positionsparameter zählt. In der Vereinbarung: „Parametername 'w' steht doppelt“; „Parametername '=3' ist ungueltig“; „Parametername 'AUFRUFNR' ist ein festes Wort“; „$DEFAULT fuer den Schluesselwortparameter w ist nicht erlaubt“; „Das Zeichen + steht nur als letztes Wort“ (ein Schlüsselwortparameter hinter dem `+` ist ein Fehler wie jedes Wort hinter dem `+`); „Hinter MACRO stehen Namen und die Zahl 2“; „33 Parameternamen angegeben, zulaessig sind maximal 32“.

Grenzen: Ein Wert mit `$CALC(...)` wird als Text eingesetzt; steht `$name` in einem äußeren `$CALC`, gilt dasselbe wie bei Positionsoperanden (Fehler „unerlaubtes Zeichen“). `WDHL` in `c82ass.makros` bleibt unverändert und wird weiter positional aufgerufen.

Beispiel 1: `SCHIEBE` mit Vorgaben, einem abweichenden Wert und vertauschten Schlüsselwörtern. Jeder Aufruf erzeugt vier Wörter: `ziel`, `anzahl`, die Richtung (LI = 1, RE = 2) und `$#`:

```
SCHIEBE:        MACRO   ziel anzahl=1 richtung=LI
                $PR(richtung) LI RE
                dw      $ziel                   ; Positionsparameter
                dw      $anzahl                 ; Schlüsselwort anzahl
                $IF     $richtung = LI
                dw      1                       ; Richtung LI
                $ELSE
                dw      2                       ; Richtung RE
                $ENDIF
                dw      $#                      ; Zahl der Positionsoperanden
                MEND
                SCHIEBE 7                       ; Wörter 7, 1, 1, 1 (nur Vorgaben)
                SCHIEBE 7 anzahl=3              ; Wörter 7, 3, 1, 1
                SCHIEBE richtung=RE anzahl=4 6  ; Wörter 6, 4, 2, 1 (vertauscht, vor dem Positionsoperanden)
```

Beispiel 2: Leere Vorgabe und Text. `LEER 5` ergibt die Wörter 5, 2, 0, `LEER 5 text=X n=3` ergibt 5, 3, 1 und `LEER 3 text=""` ergibt 3, 2, 0:

```
LEER:           MACRO   a text="" n=2
                dw      $a                      ; Positionsparameter
                dw      $n                      ; Schlüsselwort n
                dw      $LEN(text)              ; Zeichenzahl von text
                MEND
```

Beispiel 3: Aufruf aus einem Makro heraus. `AUSEN 7` ergibt die Wörter 7, 2, 1, 1 und `AUSEN2 4` die Wörter 5, 4, 1, 1 (mit `SCHIEBE` aus Beispiel 1):

```
AUSEN:          MACRO   x
                SCHIEBE $x anzahl=2             ; Positionsoperand aus dem Parameter, Schlüsselwort fest
                MEND
AUSEN2:         MACRO   x
                SCHIEBE 5 anzahl=$x             ; Schlüsselwortwert aus dem Parameter
                MEND
```

Stand der Prüfung: Das Programm Claude/makrokeyword (71 Zellen 0001 bis 0107, Sollwerte vor dem Lauf von Hand hergeleitet) endet im Interpreter mit „Test wahr“. Es deckt ab: nur Vorgaben; ein Schlüsselwort abweichend; vertauschte Reihenfolge; Schlüsselwörter vor, zwischen und hinter den Positionsoperanden; Wert in Anführungszeichen; leere Vorgabe und leerer Wert im Aufruf; `$LEN`; `$PR` (leere Vorgabe ungeprüft, angegebener Wert geprüft); Vorgabe und Wert mit Leerzeichen; `+` mit weiteren Positionsoperanden und `$P(3)`; `$DEFAULT` neben einem Schlüsselwort; verschachtelter Aufruf (Positionsoperand und Schlüsselwortwert aus einem Parameter). Regressionstest: Referenz Claude/makrokeyword und die 16 Negativtests makro_kw_* (unbekannt, doppelt, positionsname, ohnename, ohnewert, gleichgleich, zuviele, zuwenige, default, plus_vorn, name_doppelt, name_leer, festes_wort, zahl, pr_vorgabe, pr_wert); die 274 bisherigen Programme bleiben bitgleich. pruefe_alles.sh mit -Werror, AddressSanitizer und UBSan: 275 Programme und 496 Negativtests ohne Abweichung.

Nicht geprüft: Im Positivtest fehlen `$1` und `$P(k)` auf einen Schlüsselwortparameter, eine Vorgabe mit `$CALC` und die Grenze von 33 Namen; von Hand gemessen gehen `$1` und `$P(3)` auf einen Schlüsselwortparameter, 32 Namen mit einem Schlüsselwort gehen, und 33 melden „33 Parameternamen angegeben, zulaessig sind maximal 32“. Einen eigenen Test für `name=wert` als Positionswert in einem Makro ohne Schlüsselwörter gibt es nicht; er ist durch die Referenz Claude/makrostrsplit gedeckt. Windows und der Fuzz-Lauf (fuzz.py) sind nicht geprüft.

Festlegungen (Annahmen aus der Anforderung, Abschnitt 7: (1) Die Schreibweise ist `name=wert`. (2) Ein Positionsparameter ist beim Aufruf nicht mit Namen angebbar. (3) Die Vorgabe wird beim Aufruf gegen `$PR` geprüft, eine leere Vorgabe nicht; ein angegebener leerer Wert schon. (4) `WDHL` bleibt unverändert. (5) Die Version ist 1.01-025 (ein Build über 1.01-024).

4.31 Makro-Variablen standardmäßig lokal (`;#MAKROLOKAL=1`, `$GLOBAL`)

Zweck: Makro-Variablen und Felder sind ohne weiteres global (4.7, 4.17); lokal wird eine Variable nur durch `$LOCAL` (4.20). Beim IBM-Makroassembler ist es umgekehrt: Eine Variable ist lokal, solange sie nicht mit `GBLA` oder `GBLC` ausdrücklich global vereinbart ist. Die Quelltext-Anweisung `;#MAKROLOKAL=1` schaltet dieses Verhalten ein, die Direktive `$GLOBAL` vereinbart Namen ausdrücklich als global (Gegenstück zu `$LOCAL`). Ohne die Anweisung bleibt alles bitgleich wie bisher.

Umsetzungsstand: Umgesetzt in c82ass 1.02-001 (06.11.2026). Gilt nur, wenn `c82ass --version` 1.02-001 oder höher meldet; ein älteres Binary kennt weder die Anweisung noch `$GLOBAL` (dort ist `$GLOBAL` ein unbekannter Operationscode).

Schalter: `;#MAKROLOKAL=1` steht als Kommentarzeile im Quelltext, wie `;#MACRODATEI1=` und `;#OPSYM=` (eine Anweisung, keine Kommandozeilenoption). Die Zeile beginnt mit `;`; Leerraum zwischen `;` und `#MAKROLOKAL` und um das `=` ist erlaubt, hinter dem Wert darf ein Kommentar mit `;` stehen. Die Anweisung gilt für den ganzen Lauf, auch wenn sie erst hinter der ersten Makrodefinition steht (F2), und damit für alle Makros, gleich woher sie stammen (Quelltext, `c82ass.makros`, Projektdateien `;#MACRODATEI1=` bis `9`). Sie darf in der Hauptdatei oder in einer mit `$INCLUDE` eingebundenen Quelltextdatei stehen. Erlaubt sind die Werte `1` und `0`; `0` schaltet nicht ein, die Zeile zählt aber als Vorkommen. Fehler (Rückgabecode 16, Abbruch nach Pass 0, keine Ausgabedateien): die Anweisung kommt mehr als einmal vor (auch `=0` und danach `=1`), es fehlt das `=`, oder der Wert ist weder `0` noch `1`. `;#MAKROLOKALX` ist keine Anweisung, sondern ein gewöhnlicher Kommentar.

Direktive: `$GLOBAL <name> ...` steht im Rumpf eines Makros und nennt einen oder mehrere Namen, durch Leerzeichen getrennt. Ab dieser Zeile bezeichnet der Name im laufenden Aufruf die globale Variable oder das globale Feld, für alle Verwendungen (`$SET`, `$INC`, `$V(...)`, `$CALC`, `$IF`, `$WHILE`). Gibt es sie noch nicht, legt `$GLOBAL` eine globale Variable mit dem Wert 0 an (wie `GBLA`); ein Feld legt weiterhin nur `$DIM` an (F13). Der Name steht ohne Klammer (eine Anzahl wie bei `$LOCAL name(n)` gibt es nicht); Namensregeln wie in 4.7. Die Zeile steht für sich (ein Kommentar mit `;` dahinter ist erlaubt), erzeugt keinen Code und wirkt nur im erzeugten Zweig. Derselbe Name noch einmal mit `$GLOBAL` im selben Aufruf ist erlaubt. `$GLOBAL` ist ein festes Wort (4.26). `$LOCAL` bleibt mit und ohne Schalter unverändert gültig.

Ohne den Schalter: `$GLOBAL` ist auch dann erlaubt und hat keine sichtbare Wirkung, außer dass es eine fehlende globale Variable mit 0 anlegt (F6). So lassen sich Makros schon vor der Umstellung mit `$GLOBAL` schreiben und laufen mit und ohne Schalter gleich. Sonst ändert sich ohne Schalter nichts (F1): Die 283 früheren Programme und alle Referenzen des Regressionstests bleiben bitgleich.

Regeln mit dem Schalter:

F3 Implizit lokal: Das erste `$SET name ...` oder `$DIM name n` in einem Aufruf, für das der Name weder lokal (durch `$LOCAL` oder ein früheres `$SET` oder `$DIM`) noch mit `$GLOBAL` vereinbart ist, legt eine lokale Variable oder ein lokales Feld an. Sie verdeckt eine gleichnamige globale Variable oder ein globales Feld (die globale bleibt unberührt, wie bei `$LOCAL`), beginnt mit dem gesetzten Wert (`$DIM`: alle Elemente 0) und verschwindet mit dem Ende des Aufrufs, auch bei einem Fehler oder `$EXIT`.

F4 Lesen und `$INC`: Das Lesen (`$V`, `$IF`, `$CALC`, `$REPT`, `$WHILE`) und `$INC` eines Namens, der im Aufruf weder lokal angelegt noch mit `$GLOBAL` vereinbart ist, betrifft die globale Variable, wenn es sie gibt; sonst gilt der Fehler wie bisher („nicht gesetzt“). Ein Makro kann also Werte aus dem Quelltext oder aus anderen Makros lesen, ohne sie zu vereinbaren . `$INC` auf eine nicht lokale, aber vorhandene globale Variable erhöht die globale; `$SET` dagegen legt lokal an (F3). Wer eine globale Variable mit `$SET` schreiben will, vereinbart sie mit `$GLOBAL`.

F5 Konflikte: `$GLOBAL` für einen Namen, der im selben Aufruf schon lokal angelegt oder vereinbart ist, ist ein Fehler, ebenso `$LOCAL` für einen Namen, der schon mit `$GLOBAL` vereinbart ist. `$LOCAL` für einen Namen, der durch `$SET` oder `$DIM` schon lokal angelegt ist, bleibt der Fehler „schon lokal vereinbart“ (4.20).

F7 Verschachtelte Aufrufe: Wie bei `$LOCAL` sieht ein aus einem Aufruf aufgerufenes Makro dessen lokale Variablen nicht; nach der Rückkehr gelten die lokalen des aufrufenden Makros mit unverändertem Stand. Werte gehen über Parameter hinein und über `$GLOBAL`-Variablen heraus.

F8 `$WHILE` und `$REPT`: Im Rumpf von `$WHILE` bleibt eine implizit angelegte lokale Variable über die Durchläufe desselben Aufrufs erhalten (anders als `$LOCAL` im Rumpf, das in jedem Durchlauf neu vereinbart, 4.19). `$REPT` wird vor der Auswertung als Text aufgelöst (4.3, 4.7): In der Anzahl gilt `$V(name)` mit dem Stand zu Beginn des Aufrufs; eine erst im selben Aufruf lokal angelegte Variable ist dort noch nicht da und fällt nach F4 auf die globale zurück oder meldet „nicht gesetzt“.

F9 Grenzen: Je Aufruf höchstens 64 lokale einfache Variablen und 16 lokale Felder (je Feld bis 256 Elemente); die mit `$SET` und `$DIM` implizit angelegten zählen mit. Die Grenzen gelten auch ohne den Schalter für `$LOCAL` (bis 1.01-031 waren es 16 und 4; die Erhöhung ist eine Lockerung und ändert kein vorhandenes Ergebnis). Die Grenzen der globalen Variablen (64) und Felder (16) bleiben davon unabhängig. Mit `$GLOBAL` sind höchstens 80 Namen je Aufruf möglich (sonst „mehr als 80 Namen mit $GLOBAL in einem Makroaufruf“). Die Tabellen sind statisch und auf 32 ineinander aufgerufene Makros bemessen; bei tiefer Verschachtelung entsteht kein Überlauf durch die Bauart.

F10 Quelltext außerhalb von Makros: Im normalen Quelltext (4.1) sind `$SET`, `$INC` und `$DIM` auch mit dem Schalter global, denn dort gibt es keinen Aufruf, dessen Ende die Variable verwerfen könnte. Ein Makro liest eine so gesetzte Variable ohne weiteres (F4) und schreibt sie mit `$GLOBAL`. `$GLOBAL` im Quelltext außerhalb eines Makros ist ein Fehler. Der Quelltext wird nur ausgewertet, wenn mindestens eine Zeile mit `$` beginnt (4.1); ein Programm, das im Quelltext nur `$V(...)` oder `$DEFINED(...)` benutzt, braucht dafür eine `$`-Zeile (die Testprogramme setzen `$SET quelltext 1`).

F11 `$DEFINED` und `$AUFRUFNR`: `$DEFINED` (4.25) fragt die Sicht des laufenden Aufrufs: eine lokale Variable zählt, eine von ihr verdeckte globale nicht. `$AUFRUFNR` (4.29) bleibt unverändert.

F12 bis F16 (in der Anforderung nicht ausdrücklich geregelt, bei der Umsetzung festgelegt): F12 Die Elemente eines Feldes, das im Aufruf nicht lokal vereinbart ist (Lesen und Schreiben, auch `$SET a(i) ...`), bezeichnen das globale Feld, denn `$SET a(i)` legt kein Feld an. F13 `$GLOBAL name` vor `$DIM name n` macht das Feld global: `$DIM` wandelt den nur mit `$GLOBAL` angelegten Namen in ein globales Feld um. F14 Die Ziele der Zeichenkettenfunktionen (4.21, 4.22) legen wie `$SET` lokal an. F15 Ein wiederholtes `$DIM` desselben lokalen Namens im selben Aufruf setzt wie beim globalen Feld alle Elemente auf 0; mit anderer Anzahl ist es ein Fehler. F16 Die Fehler zu `$GLOBAL` und `$LOCAL` für denselben Namen und die erlaubte Wiederholung von `$GLOBAL` gelten mit und ohne Schalter gleich.

Spur (4.24): `$TRACE` zeigt `$GLOBAL g` als „global vereinbart: g“, ein `$SET` auf die mit `$GLOBAL` vereinbarte Variable als „g: 0 -> 5“ und das erste `$SET` auf einen neuen lokalen Namen als „x: (neu) -> 5 (lokal)“; eine gleichnamige verdeckte globale Variable erscheint dabei nicht als Altwert.

Fehler (Rückgabecode 16, keine Ausgabedateien; die Meldung beginnt mit Dateiname und Zeile, bei den Schalterfehlern mit „Zeile n:“, und trägt eine Kennung G1 bis G9 oder L6 bis L9): „$GLOBAL erwartet mindestens einen Namen: $GLOBAL <Name> [<Name>] ...“; „$GLOBAL q(3): Der Name steht ohne Klammer (ein Feld legt $DIM an)“; „Variablenname 1x ungültig“; „$GLOBAL a: Der Name ist in diesem Makroaufruf schon lokal angelegt oder vereinbart“; „$LOCAL a: Der Name ist in diesem Makroaufruf schon mit $GLOBAL als global vereinbart“; „$LOCAL a: Der Name ist in diesem Makroaufruf schon lokal vereinbart“; „$DIM a: Anzahl 4, früher vereinbart mit 3“; „$DIM a: Der Name ist schon eine einfache Variable“; „Variable v65: mehr als 64 lokale Variablen in einem Makroaufruf (auch die mit $SET angelegten zählen; global mit $GLOBAL)“; „$DIM f17: mehr als 16 lokale Felder in einem Makroaufruf (auch die mit $LOCAL angelegten zählen; global mit $GLOBAL)“; im Quelltext „(Quelltext) $GLOBAL steht außerhalb eines Makroaufrufs“; zum Schalter „;#MAKROLOKAL steht schon in Zeile 2 (die Anweisung darf nur einmal vorkommen).“, „;#MAKROLOKAL ohne '=' (Syntax ;#MAKROLOKAL=1).“ und „;#MAKROLOKAL=2 ist ungueltig (erlaubt sind ;#MAKROLOKAL=1 und ;#MAKROLOKAL=0).“. Unverändert bleiben die Meldungen zu einer nicht gesetzten Variablen, zu Feldern ohne `$DIM`, zum Feldindex außerhalb und zu einem Namen, der Variable und Feld zugleich wäre.

Beispiel: SUMME bildet die Summe 1 bis $1 in den lokalen Variablen i und s (ohne $LOCAL), erzeugt sie als Wort und gibt sie über die globale Variable gesamt nach außen. i und s sind in jedem Aufruf neu und stören ein aufrufendes Makro nicht; gesamt bleibt nach dem Aufruf erhalten. SUMME 4 erzeugt das Wort 10 und setzt gesamt auf 10. Ohne die erste Zeile würde $SET i 1 die globale Variable i setzen, und SUMME zerstörte den Zähler eines aufrufenden Makros (4.20):

```
;#MAKROLOKAL=1
SUMME:     MACRO 1
           $GLOBAL gesamt            ; Ergebnis geht nach außen
           $SET i 1                  ; lokal, ohne $LOCAL
           $SET s 0                  ; lokal
           $WHILE $V(i) <= $1
           $INC s $V(i)
           $INC i
           $ENDWHILE
           $SET gesamt $V(s)         ; global, weil mit $GLOBAL vereinbart
           dw $V(s)                  ; Summe 1 bis $1
           MEND
```

Umstellung vorhandener Makros: Wer ein Makro auf den Schalter umstellt, schreibt für jede Variable, die über das Ende des Aufrufs hinaus leben soll (Zähler über mehrere Aufrufe, Nummern für Marken, Ergebnisse), eine Zeile $GLOBAL <name> an den Anfang; alle anderen Variablen brauchen nichts mehr. Variablen, die vorher nur im Quelltext außerhalb von Makros gesetzt wurden, bleiben global (F10), und ein Makro, das sie nur liest, muss nichts ändern (F4). Die Standardmakros brauchen keine Anpassung: Am Stand 1.01-031 kommen $SET, $INC und $DIM in c82ass.makros und den *.macro-Dateien des Baums nur in Claude/makroinclude/macro_inc.macro vor (zwei Zeilen), $LOCAL nirgends.

Stand der Prüfung: Sieben Programme unter Assembler/Claude/ enden im Interpreter mit „Test wahr“ (Sollwerte vor dem Lauf von Hand hergeleitet): makrolokal_vorgabe ($SET ohne $LOCAL ist lokal; ein Zähler des aufrufenden Makros bleibt erhalten), makrolokal_global (Werte über $GLOBAL nach außen und über mehrere Aufrufe), makrolokal_feld ($DIM lokal und globales Feld mit $GLOBAL), makrolokal_while (implizit lokale Variable über die Durchläufe, F8), makrolokal_verschachtelt (F7), makrolokal_quelltext (Variable aus dem Quelltext: Lesen ohne und Schreiben mit $GLOBAL, F4 und F10) und makrolokal_ohne (dasselbe Quelltextmuster ohne Schalter: Verhalten wie bisher, $GLOBAL ohne Wirkung, F6). Regressionstest: die Referenzen dieser sieben Programme und die sechzehn neuen Negativtests makro_global_klammer, makro_global_nach_local, makro_global_nach_set, makro_global_name_ungueltig, makro_global_ohne_name, makro_local_dim_zuviele, makro_local_nach_global, makro_local_nach_set, makro_local_set_zuviele, makro_lokal_schalter_doppelt, makro_lokal_schalter_null_eins, makro_lokal_schalter_ohne_gleich, makro_lokal_schalter_wert, makro_dim_lokal_anzahl, makro_set_dim_lokal_art und quelltext_global; geändert sind makro_local_zuviele und makro_local_felder_zuviele (Grenzen 64 und 16). pruefe_alles.sh ohne und mit AddressSanitizer und UBSan (-Werror): 290 Programme (die 283 früheren ohne Schalter bitgleich) und 524 Negativtests ohne Abweichung.

Nicht geprüft: Windows und der Fuzz-Lauf (fuzz.py); ein Lauf mit 32 ineinander aufgerufenen Makros; ein Negativtest für die Grenze von 80 $GLOBAL-Namen; die Spur mit Schalter (nur von Hand an einem Wegwerfprogramm gesehen, nicht im Regressionstest); die Bestandsaufnahme, welche vorhandenen Programme mit dem Schalter ein $GLOBAL bräuchten (sie laufen im Regressionstest ohne Schalter); compute_all_tests.sh (der Interpreter ist unberührt).

Festlegungen: (1) ;#MAKROLOKAL=0 zählt als Vorkommen: Eine spätere Zeile ;#MAKROLOKAL=1 ist der Fehler „steht schon in Zeile n“ (Test makro_lokal_schalter_null_eins). Die Anforderung sagt nur, 0 sei gleichbedeutend mit fehlender Zeile und zweimal sei ein Fehler; diese Auslegung ist eine Annahme und wird bei Widerspruch geändert. (2) Ein Kommentar hinter dem Wert ist erlaubt. (3) $INC auf eine vorhandene, nicht lokale globale Variable erhöht die globale (F4; die Begründung in der Anforderung, wer eine globale Variable „schreiben oder erhöhen“ wolle, vereinbare sie mit $GLOBAL, widerspricht dieser Regel; umgesetzt ist die Regel). (4) F12 bis F16 sind bei der Umsetzung festgelegt. (5) Aus den offenen Fragen der Anforderung gelten die Vorschläge: $GLOBAL auf einen unbekannten Namen legt die Variable mit 0 an, ein wiederholtes $GLOBAL desselben Namens ist erlaubt, $GLOBAL im Quelltext ist ein Fehler. (6) Die Unterversion ist 1.02 (Build 001).

4.32 Spezifikationszeile und gemeinsame Blöcke im Makro ($SPEC, $BLOCK)

Zweck: Ein Makro kann in einer Spezifikationszeile Eigenschaften angeben, die das Makro als Ganzes betreffen (seine Art, seine Klasse und eine Aktion), nicht einen einzelnen Aufruf. Die Zeile ist eine Direktive des Makroprozessors wie die übrigen $-Direktiven (4.1).

Umsetzungsstand: Umgesetzt in c82ass 1.03-003 (09.11.2026): `$SPEC` (Spezifikationszeile mit Prüfungen und Warnung) und `$BLOCK`/`$ENDBLOCK` (gemeinsame Blöcke). `TYP=Eingebettet` (Kurzform `E`) gilt ab 1.03-004; ein älteres Binary lehnt `TYP=E` als ungültig ab. Gilt nur, wenn `c82ass --version` 1.03-003 oder höher meldet; 1.03-002 kennt `$SPEC`, aber noch kein `$BLOCK`, ein älteres Binary kennt beides nicht. Die Wirkung von `TYP=LoadMakro` und `ACTION=Load` ist noch offen: beide werden geprüft und gespeichert, haben aber keine Wirkung (siehe „Offene Punkte“ unten).

Vereinbarung: Jedes Makro kann eine Spezifikationszeile enthalten:

```
$SPEC  [TYP={Macro|Aufruf|LoadMakro|Eingebettet}]  [CLASS=<Zeichenkette in Großbuchstaben>]  [ACTION={None|Load}]
```

Alle drei Angaben sind wahlfrei (eckige Klammern). `TYP` nimmt einen der Werte `Macro`, `Aufruf`, `LoadMakro` und `Eingebettet`; `ACTION` nimmt `None` oder `Load`; `CLASS` ist eine frei wählbare Zeichenkette aus Großbuchstaben (GB). Die Angaben dürfen in beliebiger Reihenfolge stehen, jede höchstens einmal. Die Schlüsselwörter (`TYP`, `CLASS`, `ACTION`) und die Werte stehen genau in der hier gezeigten Schreibweise; Groß- und Kleinschreibung wird unterschieden. Kurzformen von `TYP`: `M` für `Macro`, `A` für `Aufruf` und `L` für `LoadMakro` und `E` für `Eingebettet`.

Vorgabe: Fehlt die Spezifikationszeile, oder fehlt in ihr eine Angabe, so gilt `TYP=M` (die Art Macro) und `ACTION=None`. Für `CLASS` gibt es keine Vorgabe: ohne `CLASS` hat das Makro keine Klasse und kann `$BLOCK` nicht benutzen.

Stellung und Zahl: Je Makro darf genau eine Spezifikationszeile auftreten. Sie steht als zweite Zeile des Makros, unmittelbar hinter der `MACRO`-Zeile, die mit dem einleitenden Schlüsselwort `MACRO` beginnt. Kommentar- und Leerzeilen zwischen der `MACRO`-Zeile und der Spezifikationszeile zählen nicht als Zeilen. Eine zweite Spezifikationszeile im selben Makro und eine Spezifikationszeile, die nicht unmittelbar hinter der `MACRO`-Zeile steht, sind ein Fehler.

Einmalerzeugung bei `TYP=M`: Jedes Makro, das eine Spezifikationszeile mit `TYP=M` hat, erzeugt seinen Rumpf (die Zellen des Unterprogramms) in einem Lauf nur einmal, auch wenn es mehrfach aufgerufen wird. Dazu legt es einen globalen Merker an (`$GLOBAL`, 4.31): Der erste Aufruf setzt ihn und erzeugt den Rumpf, jeder spätere Aufruf findet den Merker gesetzt und erzeugt nichts. Das Muster steht in 4.25, Beispiel 3 („Unterprogramm nur einmal erzeugen“). Die Regel gilt nur für Makros mit Spezifikationszeile; ein Makro ohne Spezifikationszeile hat zwar die Vorgabe `TYP=M`, ist aber nicht daran gebunden und darf bei jedem Aufruf neuen Code erzeugen. Der Merker liegt im Namensraum des Makros (Name = Makroname klein, mit angehängtem `_da` oder einem festen Präfix je Makro, wie in der Standardmakrodatei), damit er nicht mit Variablen des Programms zusammenstößt.

Einmalerzeugung über ein gewöhnliches Symbol (zurückgestellt): Die Idee, die Einmaligkeit statt über einen globalen Merker über ein gewöhnliches Symbol des Programms zu entscheiden (ein Label mit festem Namen im Rumpf, Abfrage mit `def(name)`, 4.23), wird vorerst nicht umgesetzt (Entscheidung vom 09.11.2026). Sie bräuchte eine Erweiterung von `def`: Heute sucht `def` nur in den Zeilen der Quelldatei ohne Makrodefinitionen und sieht ein erst beim Aufruf erzeugtes Symbol nicht; für gemeinsame Teile mehrerer Makros gibt es außerdem die Blöcke (unten). Für `TYP=M` gilt der globale Merker.

Mehrfachauflösung bei `TYP=A`: Ein Makro mit `TYP=A` (Aufruf) darf nie eine Mehrfachauflösung verhindern. Es erzeugt bei jedem Aufruf seinen Text von neuem; es setzt keinen Merker, der einen späteren Aufruf unterdrückt, und fragt keines der oben genannten Symbole ab. Das gilt für jeden Aufruf in jedem Lauf, ohne Ausnahme.

Eingebettete Makros bei `TYP=E` (Eingebettet, ab c82ass 1.03-004; Entscheidung des Inhabers vom 09.11.2026): Ein Makro mit `TYP=E` ist kein wirklich mehrfach verwendetes Makro, sondern ein Assembler-Hilfskonstrukt zur Verringerung der Komplexität des Quelltexts. Der Assembler behandelt es wie `TYP=A` bei der Prüfung: keine Prüfung der Einmalerzeugung und keine Warnung, auch wenn der Rumpf weder `$GLOBAL` noch `$BLOCK` enthält. `CLASS` und `$BLOCK` gelten nach den bestehenden Regeln; die Sonderregel von `TYP=A` (Blöcke bei jedem Aufruf erzeugen) gilt für `TYP=E` nicht.

Verlegung durch den Optimierer (ab c82ass 1.03-007, siehe 5.6; Auftrag des Inhabers vom 10.11.2026): Der Rumpf eines Makros mit ausdrücklicher Spezifikationszeile `TYP=M` ist ein Unterprogramm (Aufruf mit 025, Rücksprung vorhanden; niemand läuft hinein oder heraus). Der Optimierer darf diese Rümpfe hinter das Programmende verlegen und dicht packen, wenn dadurch die Endadresse kleiner wird; ein Programm, das sich auf die Lage eines Rumpfs im Quelltext verlässt (feste Adressen, Hineinlaufen, Herauslaufen), verliert das. Makros ohne Spezifikationszeile und Makros mit `TYP=A`, `TYP=E` oder `TYP=L` bleiben an ihrer Stelle. Abschaltbar mit `;#MAKROUMORDNEN=NO` und mit `--no-opt`.

Prüfung durch den Assembler: Bei einem Makro mit `TYP=M` und Spezifikationszeile meldet der Assembler eine Warnung, wenn der Rumpf weder `$GLOBAL` noch `$BLOCK` enthält, also keine Einmalerzeugung erkennbar ist. Bei `TYP=A` und `TYP=E` prüft er nichts, denn ob ein Merker eine Mehrfachauflösung verhindert, lässt sich nicht zuverlässig erkennen; dafür steht der Autor des Makros ein.

Gemeinsame Blöcke mit Namen, Identität über einen Hash (Idee des Inhabers vom 09.11.2026, Direktivennamen am selben Tag bestätigt, umgesetzt in c82ass 1.03-003): Mehrere Unterprogramm-Makros, meist derselben Klasse, aber nicht zwingend, enthalten oft dieselbe Folge von Definitionen (Zellen mit festen Namen), etwa Hilfsroutinen, Tabellen oder Arbeitszellen. Diese Folge soll in einem Lauf nur einmal entstehen; die Makros greifen über die gleichen, scheinbar doppelten Namen auf dieselben Zellen zu. Der Assembler verwaltet das. Festlegungen: (1) Ein Makro kennzeichnet eine solche Folge ausdrücklich mit `$BLOCK name` am Anfang und `$ENDBLOCK` am Ende; der Assembler erkennt gleiche Folgen nicht selbständig. (2) Jeder Block hat einen Namen (Pflicht; Namensregeln wie bei Symbolen, 3.2). Blöcke, die zusammengefasst werden dürfen, tragen denselben Namen, auch wenn sie in verschiedenen Makros stehen; Blöcke mit verschiedenen Namen werden nie zusammengefasst. Der Name steht nur hinter `$BLOCK`; `$ENDBLOCK` hat keinen Operanden, weil Blöcke nicht verschachtelt sind. Der Namensraum der Blocknamen ist die Klasse des Makros (`CLASS` der Spezifikationszeile): Ein Blockname gilt innerhalb einer Klasse, auch über Makros und Makrodateien hinweg; derselbe Blockname in zwei Klassen bezeichnet zwei verschiedene Blöcke, die weder zusammengefasst werden noch einander stören. (3) Die Identität des Inhalts ist ein Hash über den ausgeschriebenen Text des Blocks, also nach dem Einsetzen der Parameter und Variablen, nicht über den Makrotext. Bei gleichem Namen und gleichem Hash vergleicht der Assembler zusätzlich den vollen Text, so dass eine Hash-Kollision keinen falschen Block ergibt. (4) Der erste Block eines Namens erzeugt den Text; der Assembler merkt sich Name, Hash und Ort. Jeder spätere Block mit demselben Namen und demselben Hash erzeugt nichts, seine Namen gelten für die schon vorhandenen Zellen. (5) Derselbe Blockname in derselben Klasse mit anderem Hash (anderem Text) ist ein Fehler; die Meldung nennt den Blocknamen, die Klasse und die beiden Makros. Verschiedene Blocknamen mit gleichem Inhalt (gleicher Hash und gleicher Text) erzeugen einen Hinweis, der beide Blocknamen und Makros nennt und anregt, denselben Namen zu vergeben; die Blöcke werden nicht zusammengefasst, und definieren sie dieselben Symbole, folgt zusätzlich der gewöhnliche Fehler „Symbol doppelt“. (6) Im Block stehen feste Namen, keine generischen Marken (`ML0001%`), denn die sind je Aufruf verschieden. (7) Blöcke dürfen nicht verschachtelt sein: Ein `$BLOCK` innerhalb eines Blocks, ein `$ENDBLOCK` ohne offenen Block und ein am Makroende offener Block sind Fehler. (8) Der Block liegt dort, wo ihn der erste Aufrufer erzeugt; seine Adressen hängen von der Aufrufreihenfolge ab. (9) Die Regel zur Einmalerzeugung bei `TYP=M` (globaler Merker oder Symbol) gilt weiter für das Makro als Ganzes; der Block regelt zusätzlich die Teile, die mehrere Makros gemeinsam haben. Ein Makro mit `TYP=A` darf auch mit `$BLOCK` nie eine Mehrfachauflösung verhindern: seine Blöcke werden bei jedem Aufruf erzeugt. (10) `CLASS` ist der Namensraum der Blocknamen (Festlegung des Inhabers vom 09.11.2026); weitere Auswertungen von `CLASS` sind noch offen (offener Punkt 1). Ein Makro ohne `CLASS` hat keinen definierten Namensraum; `$BLOCK` ist dort nicht anwendbar und ein Fehler. `CLASS` und `TYP=M` sind voneinander unabhängig: `CLASS` ist kein Namensraum der Merker der Einmalerzeugung (Festlegung des Inhabers vom 09.11.2026).

Beispiel (nur die Schreibweise; die Wirkung der übrigen Werte ist noch nicht festgelegt):

```
KOPIE:          MACRO   quelle ziel
                $SPEC   TYP=Macro CLASS=TRANSPORT ACTION=None
                ...                             ; Rumpf des Makros
                MEND
```

Offene Punkte (Rückfragen an den Inhaber): (1) Was bewirken der Wert `LoadMakro` von `TYP` und der Wert `Load` von `ACTION`? Das bleibt offen, bis der Inhaber es festlegt. Alle übrigen früheren Fragen hat der Assistent am 09.11.2026 im Auftrag des Inhabers entschieden (siehe Festlegungen (11)).

Festlegungen (Annahmen aus der Vorgabe des Inhabers vom 09.11.2026, zu bestätigen): (1) `M` in „Standard ist TYP=M“ ist die Kurzform von `Macro`, `A` in „TYP=A“ die von `Aufruf`. (2) „GB“ bedeutet Großbuchstaben. (3) Die Spezifikationszeile gehört zu genau einem Makro und steht innerhalb seiner Definition, zwischen `MACRO` und `MEND`. (4) Die Nummer 4.32 war frei (der frühere Abschnitt 4.32 ist mit 1.03-003 entfernt); die Nummern der übrigen Abschnitte bleiben. (5) Die Einmalerzeugung bindet nur Makros mit Spezifikationszeile und `TYP=M`, nicht solche mit der bloßen Vorgabe. (6) „Mehrfachauflösung“ heißt: mehrfaches Erzeugen des Rumpfes bei mehreren Aufrufen. (7) Gemeinsame Blöcke: Direktiven `$BLOCK name`/`$ENDBLOCK` (vom Inhaber bestätigt), Blockname Pflicht und gleich für zusammenfassbare Blöcke, keine Verschachtelung (Inhaber), Hash über den ausgeschriebenen Text (Vorschlag des Assistenten, vom Inhaber angenommen). (8) Verschiedene Blocknamen werden nie zusammengefasst, auch bei gleichem Inhalt; das erzeugt einen Hinweis (Inhaber). Definieren sie dieselben Symbole, folgt zusätzlich der gewöhnliche Fehler „Symbol doppelt“ (Annahme). (9) Der Name steht nur hinter `$BLOCK`, nicht hinter `$ENDBLOCK` (Inhaber). (10) `CLASS` ist der Namensraum der Blocknamen (Inhaber); ohne `CLASS` ist `$BLOCK` nicht anwendbar (Inhaber); `CLASS` und `TYP=M` sind verschieden, `CLASS` ist nicht der Namensraum der Merker (Inhaber); die Stufe des Hinweises ist freigestellt (Inhaber). (11) Auf Anweisung des Inhabers vom 09.11.2026 vom Assistenten entschieden: `CLASS` hat keine Vorgabe und wird sonst nur als Namensraum der Blocknamen und zur Kennzeichnung benutzt; Reihenfolge der Angaben beliebig, Schreibweise genau wie gezeigt (Groß- und Kleinschreibung wird unterschieden), Kurzform `L` für `LoadMakro`; eine zweite oder falsch platzierte Spezifikationszeile ist ein Fehler,

Kommentar- und Leerzeilen zählen nicht mit; Warnung bei TYP=M ohne $GLOBAL und ohne $BLOCK, keine Prüfung bei TYP=A; die Symbolvariante ist zurückgestellt. (12) Umsetzung der gemeinsamen Blöcke in c82ass 1.03-003 (vom Inhaber bestätigt): Der Text eines Blocks sind die Zeilen zwischen $BLOCK und $ENDBLOCK nach allen Ersetzungen (Parameter, Variablen, $CALC, $IF, $WHILE), ohne Kommentarzeilen und ohne Zeilenkommentare; Identität über den Hash FNV-1a (64 Bit) und den Vergleich des vollen Textes. Der Hinweis bei gleichem Inhalt unter anderem Blocknamen gilt nur innerhalb derselben Klasse und erscheint auf der Meldungsausgabe. Der Blockname beginnt mit einem Buchstaben oder _, besteht aus Buchstaben, Ziffern und _ und hat höchstens 63 Zeichen; hinter dem Namen darf nur ein Kommentar stehen. Auf ein fehlerhaftes $BLOCK folgt keine zweite Meldung für sein $ENDBLOCK. Bei TYP=Aufruf bleiben die Blockzeilen bei jedem Aufruf stehen, nur die Markenzeilen entfallen. Die Tabelle der Blöcke gilt für einen Assemblerlauf. Die Version ist 1.03-003 (Build +1, die Unterversion bleibt 1.03). Tests: negativ/blk_* im Regressionstest. (13) Umsetzung von TYP=Eingebettet in c82ass 1.03-004 (vom Inhaber am 09.11.2026 bestätigt): Langname Eingebettet, Kurzform E; Wirkung wie bei TYP=A ohne Prüfung der Einmalerzeugung und ohne Warnung; CLASS und $BLOCK nach den bestehenden Regeln (Annahme: keine Sonderregel für Blöcke). Die Version ist 1.03-004 (Build +1). Tests: negativ/spec_e_*. (14) Verlegung der Rümpfe von TYP=M-Makros durch den Optimierer (c82ass 1.03-007): Umsetzung und Festlegungen in 5.6.

5. Der Optimierer des Assemblers

Status: festgelegt am 28.10.2026, ergänzt am 28.10.2026 (Schalter, equ10/equ40/equ100 nur mit *; dw-Zeilen werden nie verlegt) und am 06.11.2026 (Blockverschiebung mit begin RELOC-EQU und FILL); umgesetzt ab c82ass 1.00-040 (Quelltext ca82_opt.c; ab 1.00-042 nutzt die Zellenwahl alle freien Zellen eines Sprungbereichs, siehe 5.1; ab 1.00-043 verlegt der Optimierer keine dw-Zeilen mehr, siehe 5.2; ab 1.01-030 die Blockverschiebung, siehe 5.4; ab 1.03-007 die Umordnung der Makro-Rümpfe, siehe 5.6). Mit --no-opt stehen alle Literale hinter dem ersten end (3.3.11), kein Block wird verschoben und kein Makro-Rumpf umgeordnet.

Der Optimierer arbeitet, wenn nach Pass 2 alle Adressen bekannt sind (die Blockverschiebung davor, nach Pass 1, siehe 5.4) und bevor eine Ausgabedatei geschrieben wird. Er verändert den Wortlaut keiner Zeile, die der Programmierer geschrieben hat (Ausnahme: die equ-Zeile eines verschobenen Blocks, 5.4), und gibt weder eine Meldung noch einen Protokolleintrag aus. In der jetzigen Stufe hat er drei Aufgaben. Erstens verlegt er die vom Assembler selbst erzeugten Literale (#<werteausdruck>, @<adressausdruck> und !<symbolname>, siehe 3.2) in freie Zellen des Programms, damit der Literalbereich und das Programmende kürzer werden oder der Literalbereich ganz entfällt. Zweitens verschiebt er auf ausdrücklichen Wunsch (begin RELOC-EQU, 3.3.12) ganze Blöcke an das obere Ende des Programmbereichs und füllt auf Wunsch (FILL) deren Lücken mit Literalen (5.4). Drittens verlegt er die Rümpfe von Makros mit $SPEC TYP=M (4.32) hinter das Programmende und packt sie dicht, wenn dadurch die Endadresse kleiner wird (5.6). Einzelne Zeilen des Programmierers verlegt er nie, auch keine dw-Zeile (5.2). Er ist standardmäßig eingeschaltet und mit dem Schalter --no-opt abschaltbar (5.3); die Umordnung der Makro-Rümpfe lässt sich auch einzeln mit ;#MAKROUMORDNEN=NO abschalten (5.6).

5.1 Freie Zellen

- Belegt ist jede Zelle, die eine Anweisung beschreibt oder reserviert: Maschinenbefehle, dw, dwb, dwh, dwf, dwg, da, dbn, dt, dto, dtm, dz, kat4, kat5, kat6 und db. Anders als die Belegungsprüfung (Pass 3) zählt der Optimierer auch db als belegt. Wer eine Zelle für Laufzeitwerte freihalten will, schreibt db 1 und schützt sie so vor dem Optimierer.
- Freie Zellen gibt es nur im Sprungbereich einer der Anweisungen equ10 *, equ40 *, equ100 * und spa. Der Sprungbereich sind die Zellen vom Zählerstand a vor der Anweisung bis n−1, wenn der Zähler dabei vorwärts auf den Zählerstand n springt (a < n). Steht der Zähler schon auf der verlangten Grenze oder springt spa zurück (3.3.10), ist der Sprungbereich leer. Er hat höchstens 077 Zellen. Es sind die Zellen, die ein Programm wegen der Ausrichtung auf 010-, 040- oder 0100-Grenzen oder wegen eines Sektorsprungs ohnehin unbenutzt lässt (Handbuch 5.8, Festlegungen 002 und 013 bis 015).
- Eine Zelle des Sprungbereichs ist frei, wenn sie von keiner Anweisung des ganzen Programms belegt wird, gleich an welcher Stelle des Quelltextes diese steht. Springt ein späteres equ zurück und belegt die Zelle, ist sie nicht frei.
- Alle anderen Sprünge liefern keine freien Zellen: equ mit Zahl oder Symbol. Dort legt der Programmierer ausdrücklich fest, wohin der Zähler springt; der Bereich dazwischen bleibt unberührt. equ10, equ40 und equ100 mit einer Zahl als Operand gibt es nicht; sie sind unzulässig (3.3.8).
- Es zählen nur Sprungbereiche, die im Quelltext vor dem Literalbereich liegen (vor dem ersten end, ohne end vor dem Quelltextende). Was dahinter liegt, hängt von der Größe des Literalbereichs ab und wird nicht benutzt. Zellen, die der Literalbereich selbst belegt, gelten als belegt.

- Ab c82ass 1.00-042 sind alle freien Zellen eines Sprungbereichs nutzbar, auch solche hinter einer belegten Zelle (zum Beispiel durch ein späteres rückspringendes equ). Der Optimierer setzt dann vor die verlegte Zeile ein equ mit der Zieladresse ein; im Objektcode kommt dadurch eine Adressmarke hinzu, das Lochband kann länger werden. Das Ergebnis im Speicher ist davon nicht berührt.
- Zeilen zwischen begin und bend sind nie Sprungbereiche: Die Zeile equ100 * bzw. equ40 * eines Blocks mit RELOC-EQU liefert keine freien Zellen (nach der Verschiebung steht an ihrer Stelle ein equ mit der Anfangsadresse des Blocks).
- Bei einem verschobenen Block mit FILL ist zusätzlich seine Lücke frei (5.4): die Zellen hinter dem Blockende bis zur Bereichsgrenze, gekürzt vor der ersten dort belegten Zelle. Ohne FILL ist die Lücke nie frei. Die Regel „nur Sprungbereiche vor dem Literalbereich“ gilt für den vorderen Teil weiter.

Beispiel (Zahlen oktal):

```
     211     #5   ; Literal l0001
     210     #7   ; Literal l0002
     ...          ; weitere Befehle, enden unterhalb 0100
     equ100  *    ; Zähler auf 0100
w1:  dw      3    ; 0100
w2:  dw      4    ; 0101
puf: db      020  ; 0102 bis 0121 reserviert (belegt)
     equ40   *    ; Zähler springt von 0122 auf 0140
     end
```

Der Sprungbereich von equ40 * umfasst 0122 bis 0137, das sind 016 (oktal) = 14 Zellen. l0001 erhält die Adresse 0122, l0002 die Adresse 0123. Hinter end steht kein Literalbereich mehr; 0124 bis 0137 bleiben unbenutzt.

5.2 Umordnung der Literale

- Der Optimierer geht die Literale in der Reihenfolge der Literaltabelle durch (Reihenfolge des ersten Auftretens im Quelltext; gleiche Werte teilen sich ein Literal) und die freien Zellen in aufsteigender logischer Adresse (Adressnummerierung; nicht nach der Trommelzeit). Das erste Literal erhält die niedrigste freie Zelle, das zweite die nächste, und so fort.
- Literale, für die keine freie Zelle mehr übrig ist, bleiben hinter dem ersten end, in derselben Reihenfolge und ohne Lücke (wie bisher). Sind alle Literale verlegt, entfällt der Literalbereich.
- Ein verlegtes Literal hat eine neue Adresse. Sein Literalsymbol (l0001, ...) gilt mit dieser Adresse im ganzen Programm: Alle Befehle und da-Zeilen, die es verwenden, erhalten sie als Adressteil, mit dem Kennzeichen K1K2 wie bei jeder relativen Adresse (3.3.8). Das Ergebnis ist dasselbe, als hätte der Programmierer das Literal selbst als dw- bzw. da-Zeile mit seinem Symbol an die neue Stelle geschrieben.
- Zellen, die der Programmierer selbst schreibt (Befehle, dw, da, db, dt usw.), und alle Marken behalten ihre Adresse; dw-Zeilen verlegt der Optimierer nie (siehe unten). Die Adressen vor dem Literalbereich ändern sich nicht. Hinter dem Literalbereich rücken nur Zellen nach unten, die sich ohne equ oder spa unmittelbar an den Literalbereich anschließen (Code hinter end, 3.3.11), und zwar um die Zahl der verlegten Literale.
- Symbol-Literale (`!<symbolname>`, 3.2) gehen mit den anderen Literalen in der Tabellenreihenfolge durch, verlegt wird die Zelle `<symbolname> dbn 1` (Inhalt 0) samt ihrem Namen; die Zelle zählt nicht bei den Literalsymbolen l0001, ... Der Optimierer ordnet auch dann um, wenn der Name in einem Rechenausdruck steht (der Assembler warnt, 3.2); Reihenfolge und Nachbarschaft solcher Zellen verantwortet der Programmierer.
- Kommt ein Literalsymbol im Quelltext selbst vor (zum Beispiel in dw l0002-l0001, 3.3.3), ordnet der Optimierer in diesem Programm nichts um, denn solche Ausdrücke setzen Reihenfolge und Nachbarschaft der Literale voraus.
- Wer ein Literal an einer bestimmten Stelle haben will, etwa wegen der Trommelzeit, schreibt statt des Literals eine dw-Zeile mit Marke an diese Stelle; dann rührt der Optimierer sie nicht an, denn jede dw-Zeile bleibt, wo sie steht.
- Ein Programm darf sich nicht darauf verlassen, dass eine übersprungene Zelle den Inhalt 0 hat. Braucht es die 0, schreibt es dbn 1 oder dw 0; will es die Zelle nur freihalten, schreibt es db 1.
- dw-Zeilen werden nie verlegt (Festlegung vom 28.10.2026): Der Optimierer lässt jede dw-Zeile des Programmierers an ihrer Adresse, auch wenn sie eine Marke hat, die nur als ganzer Adressteil benutzt wird, und auch außerhalb von begin und bend. Nur die vom Assembler erzeugten Literale (3.2) wandern. Bis c82ass 1.00-042 durfte der Optimierer unter bestimmten Bedingungen dw-Zeilen mit Marke nach unten verlegen; das ist ab 1.00-043 entfallen. Zellen für Rücksprünge, berechnete Befehle und ähnliches dürfen daher auch mit dw definiert werden; db und dbn bleiben ebenfalls unberührt.

- dwb-, dwh-, dwf- und dwg-Zeilen (3.3.19 bis 3.3.21) werden wie dw-Zeilen nie verlegt.
- Zusammengehörige Worte (Tabellen, Felder, Folgen, die ein Programm als Nachbarn voraussetzt) schreibt der Programmierer zwischen begin und bend (5.4); sie bleiben unberührt.
- Reihenfolge der Zellenwahl mit Blockverschiebung: zuerst die freien Zellen der Sprungbereiche in aufsteigender Adresse, danach die Lücken der verschobenen Blöcke mit FILL in aufsteigender Adresse. Ein Literal in der Lücke eines Blocks steht im Quelltext vor der bend-Zeile dieses Blocks. Literale, für die auch dort keine Zelle bleibt, stehen wie bisher hinter dem ersten end im vorderen Bereich.

5.3 Wirkung auf Ausgaben und Laufzeit

- Alle Ausgaben (.obj, .dmp, .sym, .test und das Listing auf STDOUT) zeigen die endgültigen Adressen; Listing und .dmp zeigen die verlegten Literale an ihrer neuen Stelle. Die Zahl der Adressmarken (Kennzeichen 20, 3.3.8) im Objektcode kann sich ändern; die Länge des Lochbandes ist kein Ziel des Optimierers.
- Die Belegungsprüfung (Pass 3) läuft unverändert über das Ergebnis. Der Optimierer verursacht keinen Belegungskonflikt, weil er nur freie Zellen benutzt.
- Die Lage eines Literals auf der Trommel bestimmt, wie lange ein Befehl auf seinen Operanden wartet (Zugriffsfolge der Sektoren, Programmierhandbuch Anhang A). Der Optimierer berücksichtigt das nicht (Speicher geht vor Laufzeit, Festlegung 011); die Laufzeit eines Programms kann sich dadurch verkürzen oder verlängern.
- Der Optimierer ist standardmäßig eingeschaltet. Der Schalter --no-opt (auch --NOOPT) schaltet ihn aus: Literale stehen dann hinter dem ersten end, wie bei c82ass 1.00-040. dw-Zeilen bleiben mit und ohne Schalter an ihrer Adresse. Mit --no-opt findet auch keine Blockverschiebung statt (5.4): Ein Block mit begin RELOC-EQU bleibt dann an seiner Stelle, und seine Zeile equ100 * bzw. equ40 * rundet den Zähler wie sonst; ebenso findet keine Umordnung der Makro-Rümpfe statt (5.6). Er dient dazu, historische Programme mit ihren ursprünglichen Literaladressen zu reassemblieren und Regressionstests unverändert zu lassen; ohne ihn ändern sich die Literaladressen vieler Bestandsprogramme.

5.4 Blöcke begin – bend und Blockverschiebung (RELOC-EQU, RELOC-ALL, FILL)

Zellen zwischen begin und bend (3.3.12, 3.3.13) bleiben an ihrer Adresse, außer der Block beginnt mit begin RELOC-EQU (ab c82ass 1.01-030; festgelegt vom Inhaber am 06.11.2026) oder mit begin RELOC-ALL (ab c82ass 1.03-011; festgelegt vom Inhaber am 10.11.2026). Einzelne Zeilen eines Blocks verlegt der Optimierer nie, auch keine dw-Zeile (5.2) und keine Befehle (ab 1.00-041 erlaubt). Ein Block ohne RELOC-EQU und RELOC-ALL wird nicht ausgewertet; er hält zum Beispiel eine Tabelle im Quelltext als Ganzes sichtbar zusammen, und der Assembler prüft seine Paarung. Mit RELOC-EQU oder RELOC-ALL legt der Assembler den Block als Ganzes dicht unter die Adresse 07600 und lässt bei FILL die Lücke dahinter mit Literalen füllen.

Verschiebung:

- Bedingung: Der Block beginnt mit begin RELOC-EQU oder begin RELOC-ALL, und seine erste Anweisung ist equ100 * (Gruppe 1) oder equ40 * (Gruppe 2); Kommentar- und Leerzeilen davor zählen nicht. Sonst bleibt der Block bei RELOC-EQU, wo er ist, ohne Meldung; bei RELOC-ALL wird er als Gruppe 4 ohne Ausrichtung verschoben (Bullet „Gruppe 4“ unten).
- Zeitpunkt: Die Verschiebung läuft nach Pass 1 und vor dem ersten Lauf von Pass 2, in jedem Probelauf für adr und abstand (4.23) und nur bei eingeschaltetem Optimierer. Mit --no-opt (5.3) bleiben alle Blöcke.
- Größe: N ist die Zellenzahl des Blocks, gezählt wie bei zellen(von,bis) (4.23): db und dbn mit ihrer Zahl, jeder Befehl und jedes dw, dwb, dwh, dwf, dwg, da und dz eine Zelle, kat5 (3.3.15, ab c82ass 1.03-001) eine Zelle, dt, dto, dtm, kat4 und kat6 mit der Zellenzahl, die Pass 2 für ihren Text ergibt (ab c82ass 1.02-003); Kommentarzeilen, die equ-Zeile und eine Marke allein zählen 0. Bei dt, dto und dtm hängt die Zahl vom Zustand KB/GB der Textzählung ab (3.3.5). Der Optimierer zählt in der Reihenfolge, in der Pass 2 nach der Verschiebung läuft: zuerst alle Zeilen außerhalb der verschobenen Blöcke in Quelltextreihenfolge (die Zählung beginnt bei GB), dann die Blöcke der Gruppe 1, dann die der Gruppe 2, dann die der Gruppe 4, je in Quelltextreihenfolge. Ein verschobener Block kann daher eine andere Zellenzahl haben, als sein Text an der Quelltextstelle ergäbe (Test begin_reloc_text). Ein Block, dessen einziger Inhalt ein leeres dt ist, hat 0 Zellen und bleibt (Test begin_reloc_dtleer). Ein Block, den ein Makro erzeugt hat, zählt nach Pass 0 wie ein geschriebener (Test begin_reloc_makro). R ist N aufgerundet auf ein Vielfaches von 0100 (Gruppe 1) bzw. 040 (Gruppe 2); bei Gruppe 4 (nur RELOC-ALL) ist R = N.
- Anfänge: Die Blöcke der Gruppe 1 liegen in Quelltextreihenfolge von 07600 aus nach unten: Anfang des ersten Blocks = 07600 − R, jeder weitere = Anfang des Vorgängers − R. Die Gruppe 2 setzt darunter nach demselben

Muster fort (gibt es keine Gruppe 1, bei 07600). Alle Anfänge sind Vielfache von 0100 bzw. 040. 07600 ist eine Relativadresse wie jedes equ (3.3.8) und entspricht der absoluten Adresse 7600, wenn Leit- und Ladeadresse 0 sind.

- Quelltext: Danach stehen die Zeilen der verschobenen Blöcke hinter allen übrigen Zeilen (Gruppe 1, dann Gruppe 2, je in Quelltextreihenfolge), und die Zeile equ100 * bzw. equ40 * ist durch equ <Anfangsadresse> ersetzt; Marke und Kommentar bleiben. Die Zeilen des Blocks bleiben beieinander und in ihrer Reihenfolge. Listing, .dmp, .sym und .obj zeigen die endgültigen Adressen.
- Lücke und FILL: Die Lücke eines Blocks sind die Zellen hinter dem Blockende bis zur Bereichsgrenze (höchstens 077 Zellen bei Gruppe 1, 037 bei Gruppe 2), gekürzt vor der ersten dort schon belegten Zelle. Nur mit FILL füllt der Optimierer sie mit Literalen (Reihenfolge der Zellenwahl in 5.2); ohne FILL bleibt sie leer. FILL allein, ohne RELOC-EQU oder RELOC-ALL, hat keine Wirkung.
- Gruppe 4 (nur begin RELOC-ALL, ab c82ass 1.03-011): Ein Block, dessen erste Anweisung kein equ100 * und kein equ40 * ist, wird ohne Ausrichtung verschoben (R = N); er darf wie jeder Block kein equ, spa, abs, start oder end enthalten. Die Blöcke der Gruppe 4 werden der Größe nach gesetzt, der größte zuerst (bei Gleichheit der erste im Quelltext). Jeder kommt in die erste Lücke (Zellen N bis R) eines ausgerichteten Blocks der Gruppe 1 oder 2 ohne FILL, in die er passt; sonst liegt er dicht unter den bisherigen Blöcken (von 07600 abwärts: Gruppe 1, Gruppe 2, dann Gruppe 4). Die Zeilen stehen hinter den Blöcken der Gruppe 2 in Quelltextreihenfolge, und vor das begin setzt der Assembler eine Zeile equ <Anfangsadresse> (Kommentar „; RELOC-ALL: Block verschoben“); deshalb behalten Marken auf der begin-Zeile und vor der ersten Anweisung die Blockadresse. FILL hat bei Gruppe 4 keine Wirkung, weil keine Lücke entsteht. Der Block wird immer verschoben (kein Vergleich der Endadressen). Passt er nicht unter 07600, ist es derselbe Fehler wie bei den übrigen Gruppen.

Beispiel (Zahlen oktal; Programm Claude/begin_reloc): Gruppe 1 hat den Block A mit N = 0150 Zellen (mit FILL) und B mit N = 040, Gruppe 2 hat C mit N = 020 und D mit N = 050 (alle ohne FILL), in der Quelltextreihenfolge A, B, C, D. Es ist R = 0200, 0100, 040 und 0100, und die Anfänge sind A 07400 (07600 − 0200), B 07300, C 07240, D 07140; die vier Bereiche schließen lückenlos aneinander an. Die Lücke von A, 07550 bis 07577, nimmt die drei Literale des vorderen Codes auf (07550 bis 07552); die Lücken von B bis D bleiben leer. Dasselbe Programm mit --no-opt (Test begin_reloc_noopt) lässt alle Blöcke an ihrer Stelle.

Beispiel Gruppe 4 (Programm Claude/begin_reloc_all, oktal): Block A (RELOC-ALL mit equ100 *, N = 3, R = 0100) liegt auf 07500; seine Lücke 07503 bis 07577 hat 075 Zellen. Die Blöcke ohne equ sind D (N = 0100), B (N = 5, Marke auf der begin-Zeile) und C (N = 2, FILL ohne Wirkung). Größter zuerst: D passt nicht in die Lücke und liegt dicht unter A auf 07400 (07500 − 0100); B kommt auf 07503, C dahinter auf 07510; die Marke von B zeigt auf 07503.

Fälle, in denen ein Block bleibt, und Fehler:

- Ohne Meldung bleibt ein Block, wenn RELOC-EQU (nicht RELOC-ALL) ohne equ100/equ40 als erste Anweisung steht, wenn er 0 Zellen hat (seine Zeile equ100 * wirkt dann am Ort) oder wenn seine Größe in Pass 1 nicht feststeht (zum Beispiel db 3+1; dabei meldet die normale Operandenprüfung von db den Fehler, Rückgabecode 16). Bis c82ass 1.02-002 blieb auch ein Block mit dt, dto, dtm, kat4, kat5 oder kat6 ohne Meldung an seiner Stelle; ab 1.02-003 wird er verschoben.
- Fehler (Rückgabecode 16): ein unbekanntes Schlüsselwort hinter begin (erlaubt sind RELOC-EQU, RELOC-ALL und FILL), ein doppeltes Schlüsselwort, RELOC-EQU und RELOC-ALL zusammen, FILL vor RELOC-EQU oder RELOC-ALL; equ100 * oder equ40 * als erste Anweisung ohne RELOC-EQU oder RELOC-ALL, auch mit --no-opt; equ100 oder equ40 nicht als erste Anweisung und jedes sonstige equ im Block; eine Marke auf der begin-Zeile oder allein vor der equ-Zeile eines Blocks mit RELOC-EQU oder RELOC-ALL und equ100/equ40 vorn; die Blöcke passen nicht unter 07600 („Die Blöcke mit begin RELOC-EQU oder RELOC-ALL passen nicht unter 07600“). Die Fehlertexte stehen in den Negativtests Assembler/Regressionstest/negativ/begin_reloc_*.erwartet und dwb_begin_equ.erwartet.
- Überlappung: Stößt ein verschobener Block an Zellen des vorderen Teils oder an einen anderen Block, meldet das die Belegungsprüfung (Pass 3, „Adresse … doppelt belegt“), nicht der Optimierer, und nur, wenn beide Zellen Inhalt haben; reine db-Reservierungen überschneiden sich ohne Meldung, wie bei jedem equ.
- Eintritt: Ein Block mit equ100 * bzw. equ40 * vorn wird von davorstehendem Code nie durchlaufen, denn der Zähler springt (außer er steht schon auf der Grenze). Wer dennoch in den Block hineinläuft, verliert das durch die Verschiebung; das Programm hat das selbst sicherzustellen. Bei RELOC-ALL ohne equ gibt es diese Sicherung nicht: Der Zähler springt nicht, der Code davor kann in den Block hinein und der Block an seinem Ende in den Code dahinter laufen; verschiebt der Assembler den Block, bricht das Programm dann still, ohne Meldung. Deshalb ist RELOC-ALL nur für Blöcke gedacht, die weder durchlaufen noch betreten werden, sondern

nur per Sprung erreicht und per Sprung verlassen werden (Rückkehrwort am Ende sequentiell erreicht: kein RELOC-ALL).

Tests: Claude/begin_reloc, begin_reloc_noopt, begin_reloc_g2 (nur Gruppe 2 mit FILL), begin_reloc_leer, begin_reloc_adr, begin_reloc_makro, begin_reloc_text (Texte im Block, KB/GB-Zustand), begin_reloc_dtleer (leeres dt) und begin_reloc_dtbs (dtm mit Backslash-Text und FILL) Claude/begin_reloc_all (RELOC-ALL, Gruppe 1 und 4, ab c82ass 1.03-011) sowie Claude/dt_text_lauf2 (Fehlerbehebung: dtm mit Backslash-Text und Literal in einer equ10-Lücke, zweiter Pass-2-Lauf des Optimierers) und die Negativtests Regressionstest/negativ/begin_reloc_* und begin_reloc_all_*.

5.5 Noch nicht festgelegt

- Festgelegt am 28.10.2026 (früher hier offen): Schalter zum Ausschalten, Optimierer standardmäßig an (5.3); equ10, equ40, equ100 nur mit * (3.3.8); Wahl der Zelle nach aufsteigender logischer Adresse, nicht nach Trommelzeit (5.2); begin–bend-Blöcke werden nur mit begin RELOC-EQU verlegt (5.4; ab 1.01-030, vorher gar nicht); dw-Zeilen werden nie verlegt (5.2, 5.4; vom Inhaber am 28.10.2026 festgelegt, ersetzt die frühere Regel „verlegbare dw-Zeilen“). Der Name des Schalters (--no-opt, auch --NOOPT) hat der Inhaber am 28.10.2026 bestätigt.
- Festgelegt am 06.11.2026 (Blockverschiebung, 5.4): begin RELOC-EQU [FILL] nach den Festlegungen F1 bis F10 der Anforderung des Inhabers (die Datei ist nicht mehr Teil des Projekts); die Reihenfolge der Blöcke legt F10 fest. Vom Inhaber ausdrücklich mit ja bestätigt: Die Lücke liegt hinter dem Block, und nur sie ist füllbar (A2); die verschobenen Blöcke stehen hinter allen übrigen Zeilen (A3); RELOC-EQU ohne equ100/equ40 bleibt ohne Meldung (A5); eine Marke auf begin ist ein Fehler (A8). Als Annahme der Umsetzung, vom Inhaber nicht ausdrücklich bestätigt: Zählung der Blockgröße nach zellen(von,bis) (A4); FILL allein ohne Wirkung und die Reihenfolge der Schlüsselwörter (A6); equ100 */equ40 * nur bei RELOC-EQU zulässig, auch mit --no-opt (A7); 07600 als Relativadresse (A9); Eintritt in den Block (A10); ferner: Ein Block mit 0 Zellen bleibt, und eine Marke allein vor der equ-Zeile ist ein Fehler. Die Unterversion von c82ass wurde nicht erhöht (das entscheidet der Inhaber).
- Festgelegt am 10.11.2026 (RELOC-ALL, 5.4; c82ass 1.03-011): begin RELOC-ALL als Alternative zu RELOC-EQU verschiebt auch Blöcke ohne equ100/equ40 (Gruppe 4); vom Inhaber im Gespräch festgelegt. Als Annahmen der Umsetzung, vom Inhaber nicht ausdrücklich bestätigt: die Packregel (größter Block zuerst, Lücken ausgerichteter Blöcke ohne FILL zuerst, sonst dicht unter 07600 abwärts); das equ vor dem begin statt einer Marken-Sperre; Gruppe 4 wird immer verschoben; FILL ohne Wirkung bei Gruppe 4. Die Unterversion von c82ass wurde nicht erhöht.
- Festgelegt am 10.11.2026 (Umordnung der Makro-Rümpfe, 5.6): Festlegungen und Annahmen der Umsetzung stehen dort.
- Sprungbereiche hinter dem Literalbereich gibt es nicht: Hinter dem ersten end steht nur noch der Literalbereich (Festlegung des Inhabers am 28.10.2026); ein zweiter Durchlauf entfällt.
- Näherungen der Umsetzung: keine bekannt. (Die Näherung „nur zusammenhängende freie Zellen ab Bereichsanfang“ aus 1.00-040/041 ist ab 1.00-042 entfallen, der Platzhalter „db 1“ einer verlegten dw-Zeile ab 1.00-043, weil dw-Zeilen nicht mehr verlegt werden; Entscheidungen des Inhabers am 28.10.2026.)

5.6 Umordnung der Makro-Rümpfe (`TYP=M`, `;#MAKROUMORDNEN`)

Der Optimierer kann die Rümpfe von Makros mit der Spezifikationszeile `$SPEC TYP=M` (4.32) hinter das Programmende verlegen und dort dicht packen (ab c82ass 1.03-007; Auftrag des Inhabers vom 10.11.2026). Ein solcher Rumpf ist ein Unterprogramm, das der Makroprozessor an der Stelle des ersten Aufrufs mitten im Programm erzeugt. Beginnt er mit `equ100 *` oder `equ40 *`, lässt er dort eine Lücke bis zur nächsten Grenze; hinter dem Programmende lassen sich die Rümpfe so ordnen, dass andere Rümpfe diese Lücken füllen und die Endadresse kleiner wird. Verändert wird nur die Reihenfolge der Zeilen: Es entsteht kein `equ`, und kein Wortlaut ändert sich. Die Umordnung gilt nur, wenn sie die Endadresse verkleinert (Messung unten); sonst bleibt alles, wie es war.

Verlegung:

- Block: Ein Block ist ein zusammenhängender Lauf von Zeilen, die der Aufruf eines Makros mit ausdrücklicher Spezifikationszeile `TYP=M` erzeugt hat (4.32). Makros ohne Spezifikationszeile (die bloße Vorgabe `TYP=M`) und Makros mit `TYP=A`, `TYP=E` oder `TYP=L` werden nie umgeordnet. Ruft ein Rumpf selbst ein weiteres `TYP=M`-Makro auf (Makro im Makro, zum Beispiel GK_SIN mit GK_MATH), gehört der innere Rumpf zum Block des äußersten Aufrufs und wandert mit ihm. Ein späterer Aufruf, der wegen der Einmalerzeugung (4.32) keinen Rumpf erzeugt, hat

keine belegende Zeile und bleibt. Der Assembler vermerkt dazu in jeder Zeile des Makroergebnisses die Nummer des äußersten TYP=M-Aufrufs (Feld makroInst in lineElements, Quelltext ca82_typen.h).

- Gruppen: Maßgebend ist die erste Anweisung des Rumpfs; Kommentarzeilen, eine Marke allein, abs und die vom Makroaufruf erzeugte Zeile <Marke> equ * zählen dabei nicht. Gruppe 1: Die erste Anweisung ist equ100 *. Gruppe 2: Sie ist equ40 *. Gruppe 4: Der Rumpf hat keines von beiden als erste Anweisung. Gruppe 3 bleibt an ihrer Stelle: Der Rumpf hat equ100 *, equ40 * oder spa hinter der ersten Anweisung (zum Beispiel die Zeichentabelle von KAT4EINGABE, KAT5EINGABE und KAT6EINGABE). Nach dem Quelltext von c82ass.makros (Stand 1.03-008), durch Assemblerläufe mit Programmen bestätigt, die sie aufrufen, gehören von den Standardmakros mit TYP=M KAT4EINGABE, KAT5EINGABE und KAT6EINGABE zur Gruppe 3, KAT5AUSGABE bleibt wegen dto (siehe unten), und die übrigen sieben (KAT4AUSGABE, KAT5SORT, KAT5SORT2, KAT6AUSGABE, GK_MATH, GK_SIN und FK_MATH) gehören zur Gruppe 4. Von den Makros der Projektdatei macro_zt_loader.macro hat nur NEUANL die Spezifikation TYP=M (Gruppe 4); ZTKONST, TYP10EINGABE, KAT4AUSGABE_G, ZTINIT, PNAME und HSCHL haben TYP=E und werden nie umgeordnet. Gruppen 1 und 2 gibt es bisher nur für neue Makros (Tests Claude/makro_umord_g1 und Claude/makro_umord_g2, siehe unten).
- Block bleibt: Ohne Meldung bleibt ein Block an seiner Stelle, wenn sein Rumpf begin, bend, start, end oder spa enthält; wenn er ein equ mit einem anderen Operanden als * enthält (eine Zahl oder ein Symbol; der Zähler wäre dann absolut gesetzt und das Packen unmöglich); wenn equ100 oder equ40 hinter der ersten Anweisung steht (Gruppe 3); wenn er dt, dto oder dtm enthält (sie führen den Zustand KB/GB der Textzählung, 3.3.5, den eine Umordnung für die übrigen Zeilen ändern würde); wenn er zwischen begin und bend steht oder hinter dem ersten end; wenn er keine belegende Zeile hat; wenn dieselbe Aufrufnummer in zwei getrennten Läufen vorkommt. Die nach 5.4 verschobenen Blöcke (RELOC-EQU) fasst die Umordnung nicht an. equ10 * und abs im Rumpf sind kein Hindernis; bei Gruppe 4 ist die Größe dann eine Schätzung, die der zweite Pass-2-Lauf überprüft.
- Größe: n ist die Zahl der Zellen vom Anfang des Rumpfs bis zur letzten belegenden Zeile, gemessen am ersten Pass-2-Lauf (nicht statisch gezählt wie in 5.4, damit der Zustand KB/GB und kat4, kat5, kat6 stimmen). Anfang ist bei Gruppe 1 und 2 die Adresse hinter dem equ100 * bzw. equ40 * (also die aufgerundete), bei Gruppe 4 die Adresse der ersten Zeile, die nicht durchsichtig ist. r ist n aufgerundet auf ein Vielfaches von 0100 (Gruppe 1) bzw. 040 (Gruppe 2); bei Gruppe 4 ist r gleich n.
- Einfügestelle: Die Blöcke stehen danach unmittelbar hinter dem ersten end (ohne end hinter dem Quelltextende) und vor den Literalen, die Pass 1 angehängt hat (5.2). Gibt es keine Literale, stehen sie vor den nach 5.4 verschobenen Blöcken, sonst am Ende des Quelltexts. Makroaufrufe hinter dem ersten end bleiben.
- Reihenfolge (Packen): Der Optimierer schätzt den Zählerstand an der Einfügestelle (Stand dort im ersten Pass-2-Lauf, vermindert um die Zellen der herausgenommenen Blöcke) und ordnet die Blöcke so: Phase A, die Blöcke der Gruppe 1 nach r absteigend (bei Gleichstand in Quelltextreihenfolge); vor jedem von ihnen füllt er die Lücke bis zur nächsten 0100-Grenze mit dem größten Block der Gruppe 2 oder 4, der noch hineinpasst (Verbrauch bei Gruppe 4 gleich n, bei Gruppe 2 die Aufrundung des Zählers auf 040 plus n), und wiederholt das, bis nichts mehr passt; das gilt auch vor dem ersten Block, also für die Lücke zwischen dem Programmende und der ersten 0100-Grenze. Phase B, die übrigen Blöcke der Gruppe 2 nach r absteigend, jeweils mit der Lücke bis zur 040-Grenze von Blöcken der Gruppe 4 gefüllt. Phase C, die übrigen Blöcke der Gruppe 4 nach n absteigend, dicht hintereinander. Hinter dem letzten Gruppe-1-Block gibt es keine eigene Füllung.
- Messung und Verwerfen: Die Umordnung läuft nach der Blockverschiebung (5.4) und dem ersten Pass-2-Lauf, vor der Umordnung der Literale (5.2). Der Optimierer misst die Endadresse E0 des Originals: die höchste belegte Zelle plus 1 aller belegenden Zeilen vor den nach 5.4 verschobenen Blöcken (Zeilen ohne Zellenbelegung wie equ, spa, abs, end und Kommentarzeilen zählen nicht, die Literale hinter end zählen mit, die spätere Literalumordnung nicht). Danach ordnet er die Zeilen um und lässt Pass 2 ein weiteres Mal laufen; dieser Probelauf schreibt keine Zeilenausgabe und kein Listing, und Warnungen erscheinen nicht doppelt. Die Meldung „Startadresse: 0000“ (mit der Startadresse in vier Oktalziffern; Operationscode start) wird dagegen nicht unterdrückt: Jeder Pass-2-Lauf gibt sie aus, sie erscheint also auch ohne Umordnung mehrmals, wenn Pass 2 mehrmals läuft (Literalumordnung, Probeläufe für adr und abstand). Die Umordnung fügt bis zu drei Läufe hinzu (Probelauf 2, der Neulauf mit dem genauen Zählerstand und nach dem Verwerfen der erneute Lauf auf dem Original); die Meldung erscheint entsprechend öfter (Annahme (I)). Weicht der Zählerstand an der Einfügestelle von der Schätzung ab, packt der Optimierer einmal mit dem gemessenen Wert neu und läuft ein drittes Mal. Die Umordnung gilt nur, wenn der letzte Probelauf ohne Fehler endet und die neue Endadresse E1 kleiner ist als E0; bei Gleichstand, bei Vergrößerung und bei einem Fehler kommt der ursprüngliche Text zurück, und Pass 2 läuft darauf erneut. Es gibt weder eine Meldung noch eine Wirkung auf den Rückgabecode.
- Folgen der Endadresse-Regel: Hat ein Programm feste Sprünge (equ <Zahl>, zum Beispiel einen Datenbereich auf fester Adresse), liegen die verlegten Rümpfe hinter dem Datenbereich; die Endadresse wächst dann fast immer, und die Umordnung wird verworfen. Gewinn gibt es vor allem bei Programmen ohne feste Sprünge. Die

Lücken, die ein Rumpf mitten im Programm gelassen hätte, stehen den Literalen nach der Verlegung nicht mehr zur Verfügung, und Lücken hinter `end` benutzt die Literalumordnung nicht (5.1); der Vergleich misst dies nicht (Entscheidung des Inhabers).

- Schalter: `;#MAKROUMORDNEN=YES` und `;#MAKROUMORDNEN=NO` stehen wie `;#MAKROLOKAL=1` (4.31) als Kommentarzeile im Quelltext (eine Anweisung, keine Kommandozeilenoption), in der Hauptdatei oder in einer mit `$INCLUDE` eingebundenen Datei. Die Zeile beginnt mit `;`; Leerraum zwischen `;` und `#MAKROUMORDNEN` und um das `=` ist erlaubt, hinter dem Wert darf ein Kommentar mit `;` stehen. Der Wert `YES` oder `NO` gilt ohne Unterscheidung von Groß- und Kleinschreibung. Ohne die Anweisung gilt die Vorgabe: Die Umordnung ist an (Entscheidung des Inhabers); `NO` schaltet sie ab, `YES` ändert dann nichts. Fehler (Rückgabecode 16, Abbruch nach Pass 0): die Anweisung kommt mehr als einmal vor („;#MAKROUMORDNEN steht schon in Zeile n (die Anweisung darf nur einmal vorkommen).“), es fehlt das `=` („;#MAKROUMORDNEN ohne '=' (Syntax ;#MAKROUMORDNEN=YES oder ;#MAKROUMORDNEN=NO).“), oder der Wert ist weder `YES` noch `NO` („;#MAKROUMORDNEN=x ist ungueltig (erlaubt sind ;#MAKROUMORDNEN=YES und ;#MAKROUMORDNEN=NO).“); jede Meldung beginnt mit „Zeile n:“. `;#MAKROUMORDNENX` ist keine Anweisung, sondern ein gewöhnlicher Kommentar. Mit `--no-opt` (5.3) findet keine Umordnung statt, gleich was die Anweisung sagt.
- Wirkung: Die Adressen der verlegten Rümpfe und aller Zellen dahinter ändern sich; Marken behalten ihre Namen. Listing, `.dmp`, `.sym` und `.obj` zeigen die endgültigen Adressen (5.3). Ein Programm, das Adressen von Rümpfen oder Zellen dahinter fest einträgt (etwa die Startadresse im BIS-Eingabeband oder Sollwerte im `;#PRUEFE`-Block), ist nach der Umordnung neu zu prüfen; mit `;#MAKROUMORDNEN=NO` oder `--no-opt` bleibt alles wie bisher. Die Probeläufe für `adr` und `abstand` (4.23) sind eigene Assemblerläufe und ordnen ebenso um: `adr` und `abstand` liefern die Adressen nach der Umordnung, und die Konvergenz bleibt, weil die Umordnung deterministisch ist.
- Marke am Aufruf: Hat der Aufruf eines Makros der Gruppe 1 oder 2 eine Marke, steht sie auf der vom Makroaufruf erzeugten Zeile `<Marke> equ *` vor dem `equ100 *` bzw. `equ40 *` und hat den Zählerstand vor dem Aufrunden (bisheriges Verhalten). Die Zeile wandert mit dem Block.
- Eintritt: Rümpfe von `TYP=M`-Makros sind Unterprogramme (Aufruf mit 025, Rücksprung vorhanden); niemand läuft in sie hinein oder aus ihnen hinaus. Wer es dennoch tut, verliert das durch die Verlegung; das Programm hat das selbst sicherzustellen (wie bei 5.4).
- Überlappung: Die nach 5.4 verschobenen Blöcke liegen unterhalb 07600, die umgeordneten Rümpfe hinter `end`; die Umordnung betrachtet und misst die Blöcke nicht und prüft nicht auf Überlappung. Wird ein sehr großes Programm zu groß, meldet Pass 3 es wie sonst bei jedem zu großen Programm.

Festlegungen des Inhabers vom 10.11.2026: (1) Nur Makros mit ausdrücklichem `$SPEC TYP=M` werden beachtet, Makros ohne `$SPEC` nicht. (2) Das neue Feld in `lineElements` trägt zugleich die Kennzeichnung „TYP=M“ und die Nummer der Makroauflösung. (3) Der Größenvergleich geschieht ohne die spätere Literalumordnung (nur Schätzung). (4) Die Folgen der Endadresse-Regel werden in Kauf genommen. (5) Mit `--no-opt` bleibt alles wie bisher. (6) `TYP=M`-Makros sind Unterprogramme; niemand läuft hinein oder heraus. (7) Die Umordnung wird nur übernommen, wenn die Endadresse kleiner wird. (8) Sie ist standardmäßig an und mit `;#MAKROUMORDNEN=NO` abschaltbar. (9) Gruppe 3 ist `equ100`, `equ40` oder `spa` im Rumpf; alles andere ist unschädlich. (10) Die Lücke zwischen dem Programmende und der ersten 0100-Grenze wird mit Rümpfen der Gruppe 2 und 4 gefüllt. (11) Die RELOC-Blöcke (5.4) werden bei der Umordnung weder betrachtet noch gemessen.

Annahmen der Umsetzung (vom Inhaber nicht ausdrücklich bestätigt, soweit nicht anders vermerkt): (a) Bei Gleichstand der Endadresse bleibt das Original. (b) Das Feld nennt den äußersten `TYP=M`-Aufruf, und ein innerer Aufruf wandert mit dem äußeren (der Inhaber hat mit „mache weiter“ fortgefahren; als Bestätigung gewertet). (c) Die Umordnung läuft nach der Blockverschiebung und dem ersten Pass-2-Lauf, nicht davor, weil ein Probelauf im unverschobenen Layout einen Überlauf melden könnte. (d) Über Gruppe 3 hinaus bleiben Blöcke mit `begin`, `bend`, `start`, `end`, einem `equ` mit anderem Operanden als `*` sowie `dt`, `dto` oder `dtm`. (e) `<Marke> equ *` und `abs` zählen nicht als erste Anweisung; ein `equ` mit Operand `*+n` zählt als Anweisung. (f) `equ100` oder `equ40` hinter der ersten Anweisung lässt auch einen Block mit `equ100 *` oder `equ40 *` vorn an seiner Stelle (strenge Lesart des Auftrags). (g) Innerhalb einer Gruppe gilt: größte zuerst, bei Gleichstand Quelltextreihenfolge. (h) Hinter dem letzten Gruppe-1-Block gibt es keine eigene Füllung. (i) Die Endadresse zählt nur belegende Zeilen vor den nach 5.4 verschobenen Blöcken. (j) Es gibt höchstens zwei Packversuche (Schätzung, dann der genaue Zählerstand aus dem Probelauf). (k) Ohne `end` ist das Programmende das Quelltextende vor den Literalen. (l) Die Meldung „Startadresse“ bleibt, wie sie bei jedem Pass-2-Lauf ist, und wird bei den Probeläufen der Umordnung nicht unterdrückt; sie zu unterdrücken hieße auch die heutige Mehrfachausgabe bei Literalumordnung und Probeläufen zu ändern. Der Inhaber hat das nicht ausdrücklich entschieden. (m) Hat sich die Zeilenzahl nach dem ersten Pass-2-Lauf geändert, unterbleibt die Umordnung (Sicherung).

Tests und Bestand: Die Tests in `Assembler/Claude/` (Sollwerte von Hand hergeleitet, alle „Test wahr“) sind `makro_umord_g1` (Gruppe 1 mitten im Programm mit Marke am Aufruf, Übernahme) und die Gegenprobe `makro_umord_g1_nein` (`;#MAKROUMORDNEN=NO`), `makro_umord_pack` (Packen mit Lücke, Phase A), `makro_umord_g2` (Gruppe 2, Phase B), `makro_umord_vergroessert` (Verwerfen, weil ein fester Sprung die Endadresse vergrößert), `makro_umord_gleich` (Gleichstand, das Original bleibt), `makro_umord_mim` (Makro im Makro, Übernahme) und `makro_umord_zweiter` (der zweite Packversuch mit dem genauen Zählerstand). Nicht auslösen ließ sich ein Fehler im zweiten Probelauf (Pass 2 meldet kaum Fehler, Belegung und Überlauf prüft Pass 3); die Rückkehr zum Original ist mit Gleichstand und Vergrößerung erprobt. Im Bestand des Regressionstests (Stand 10.11.2026) ändert die Umordnung als Vorgabe nur die Ausgaben von `kat_einmal` (KAT4AUSGABE und KAT6AUSGABE wandern hinter die Marke `emark`) und von `neuanlage` (Projekt_ZT_LOADER); bei allen übrigen Programmen, auch bei denen mit eingeteilten, aber verworfenen Blöcken, sind `.obj`, `.sym`, `.dmp`, `.test`, `.log` und `.kep` unverändert.

6. Compilierungsbeispiel

```
c82ass.sh -p Projekt_1/bsp_5.16 -s bsp_5_16
```

6.1 Quelltext bsp_5_16.asm und Assemblerlauf

Hinweis: Die Datei `Projekt_1/bsp_5.16/bsp_5_16.asm` ist eine veränderte Fassung des hier abgedruckten Beispiels: Sie beginnt mit dem Startsprung `000 go` mit `equ 02000` und der Marke `go:`, enthält vor dem Halt den Debug-Befehl `164 0001`, und der Datenbereich beginnt mit `equ 02100` statt `equ 0100`. Ihr Objektcode weicht deshalb von 5.2 ab; der abgedruckte Quelltext ergibt genau den Objektcode in 5.2.

```
Projektverzeichnis: Projekt_1/bsp_5.16
Benutze für -s die Datei: bsp_5_16
Startadresse: 0000
C8205 Assembler C82aas Version 1.03-011

Pass 0
Verarbeite 'c82ass.makros', Erzeuge Macroersetzungen

Pass 1
Löse Literale auf

Pass 2
Löse Symbole auf nach Projekt_1/bsp_5.16/bsp_5_16.log und Projekt_1/bsp_5.16/bsp_5_16.sym

Pass 3
Erzeuge Kennzeichen-Befehle in Objektdatei Projekt_1/bsp_5.16/bsp_5_16.obj
```

Quelltext (`; C8205 Heft 2 Programmierung in Maschinensprache; Beispiel: 5.16 Umspeichern eines Vektors`):

```
                start
                000     beginn
aqv:            da      qvektor    ; Adresse der ersten Komponente Quellvektor
dqv:            dw      05000      ; Dimension von qvektor gezählt in Adresseinheiten
azv:            da      zvektor    ; Adresse der ersten Komponente Zielvektor
adreinheit:     dw      01000      ; Adresseinheit
spbef:          dw      0240       ; Speicherbefehl
i:              db      1          ; Arbeitszelle für i
beginn:         241     i          ; I:=0
schleife:       210     azv        ; ac:=(azv+i)
                210     spbef      ; ac:=(azv+i)240
                240     genbef     ; Arbeitszelle genbef:= (qzv+i) 240
                211     i          ; ac:=<i>
                210     aqv        ; ac:=qvektor+i
; 251: Laden des Inhaltes einer Speicherstelle, deren Adresse im AC steht, in den AC.
                251     0          ; ac:=<qvektor+i>
genbef:         db      1          ; hier steht der Befehl 240 (qzv+i) Speichern
                211     i          ; ac:=<i>
                210     adreinheit ; ac:=i+1
                240     i          ; i:=ac
                330     dqv        ; vergleicher 3, falls i == <dqv>
                170     halt       ; sprung bei gleich
                000     schleife
halt:           020     0          ; halt
qvektor:        dw      1
                dw      2
                dw      3
                dw      4
                dw      5
                equ     0100
zvektor:        db      5
ende:           end
```

Nach dem Listing meldet der Assembler:

```
Ausgabe als binaer codierte Objektdatei (R300-Lochstreifen) nach: Projekt_1/bsp_5.16/bsp_5_16.obj
Ausgabe der formatierten Speicherbelegung nach: Projekt_1/bsp_5.16/bsp_5_16.dmp
Ausgabe der Testdatei (Spalten 1,3,4,5 fuer K1=00/01) nach: Projekt_1/bsp_5.16/bsp_5_16.test
```

6.2 Durch c82ass erzeugter Objektcode (K1K2-Programmcode) für die Eingabe mit Kennzeichen (Format des Minimalprogramms)

Das Beispiel zeigt das Listing mit Adressspalte und Leerzeichen zur besseren Lesbarkeit; die Datei `.obj` enthält die Zeilen ohne Adressspalte und ohne Leerzeichen (zum Beispiel `010007000.`), jedes Zeichen als R300-Lochbild aus acht Ziffern 0/1.

```
0000: 01 0007 000.
0001: 01 26000.
0002: 00 00000005000.
0003: 01 100000.
0004: 00 00000001000.
0005: 00 00000000240.
0006: 20 0007.
0007: 01 0006 241.
0010: 01 0003 210.
0011: 01 0005 210.
0012: 01 0016 240.
0013: 01 0006 211.
0014: 01 0001 210.
0015: 00 0000 251.
0016: 20 0017.
0017: 01 0006 211.
0020: 01 0004 210.
0021: 01 0006 240.
0022: 01 0002 330.
0023: 01 0025 170.
0024: 01 0010 000.
0025: 01 0000 020.
0026: 00 00000000001.
0027: 00 00000000002.
0030: 00 00000000003.
0031: 00 00000000004.
0032: 00 00000000005.
      20 0000 0100.
0100: 20 0105.
```